

Hyper-Connections to go-mHC

Several leftover copies of the same thought

A deep network has a habit. At every layer it takes what it currently believes about the input, computes a new version, and — if it is well designed — keeps a copy of the old version so the new one cannot erase everything useful.

These notes start there: one leftover copy, added back in. Then they ask what happens if the network is allowed to keep several leftover copies, and to learn how each layer should read them, write them, and mix them.

The research papers give that family of ideas names: Hyper-Connections, mHC, mHC-lite, KromHC, go-mHC. We will earn those names. On page one they are just a pile of letters.

You do not need residual networks, stochastic matrices, manifolds, or anything with a German mathematician's name on it. If you can multiply a small matrix and read a few lines of Julia, you can start.

How to read this

There are three honest routes.

Just understand the idea. Read Chapters 1 through 8, then 15, 16, and 20. You should finish able to say, in ordinary language, what the extra copies are, how a layer reads and writes them, and why unconstrained mixing can explode.

Build one. Read Chapters 1 through 13, then 15, 19, and 20. Add 14 and 18 before you change block sizes or brag about coverage. You should finish able to implement the update and test that the mixer does what the paper claims, not what a blog post hoped.

Audit the papers. Read 1 through 6 for the nouns, then 7 through 20 in order. Do not skip Chapter 3. That is where “stream” stops meaning “whatever the last four layers spat out.”

Do the tiny numerical example in each chapter before the next abstraction. The exercises are not homework theatre. They are how the shapes stay attached to the symbols.

The road

1. A sentence becomes a table of numbers.
2. A layer can add a correction instead of replacing the table.
3. Several tables can be carried at once.
4. A layer can read a mixture, compute once, write back, and remix what remains.
5. Unconstrained remixing can blow up. Constrained remixing is the whole subject after that.
6. Different papers constrain the mixer differently: repeated normalization, permutation mixtures, Kronecker products, or a geometric construction that starts from a skew matrix.
7. Then we compare cost, expressivity, spectra, and what the papers actually measured.

A table of symbols appears when the symbols exist. It does not appear here. Chapter 1 has three tokens and two numbers each. That is enough furniture.

A promise about language

I will not mention Sinkhorn, Birkhoff, or Cayley until you have a problem those names solve. I will not call a row of the token table a “stream.” I will not treat a paper-reported accuracy bump as a theorem.

When a statement is a definition, a derivation, a measurement from a named paper, or a guess, the surrounding sentences will say which. The papers themselves are recent preprints. Treat their numbers as “this is what that paper reported,” not as the laws of deep learning.

- Hyper-Connections: Zhu et al., arXiv:2409.19606
- mHC: Xie et al., arXiv:2512.24880
- mHC-lite: Yang and Gao, arXiv:2601.05732
- KromHC: Zhou et al., arXiv:2601.21579
- go-mHC: Dandachi and Diggs-Galligan, arXiv:2604.02309

Chapter 1 — Three words and a spreadsheet

Take the sentence *red fox runs*. A language model does not keep those words as words. It turns each one into a short list of numbers and then stirs the lists together.

Before we invent clever ways to stir, we need to see the lists. This chapter is a tiny spreadsheet, one multiplication, and a habit: never lose track of what a row means.

Words become rows

A **token** is one discrete chunk the model is willing to look at. It might be a word, a scrap of a word, a byte, or a patch of an image. Here the three tokens are **red**, **fox**, and **runs**. Their order in the sentence is their **position**.

Each token is replaced by a short list of numbers. Each number in that list is a **feature**. We will not pretend the two features here are “redness” and “foxness.” Learned features almost never have names that polite.

Write the three lists as the rows of a table:

$$X = \begin{bmatrix} 1 & -2 \\ 0 & 3 \\ 2 & 1 \end{bmatrix}.$$

Row 1 is **red**. Row 2 is **fox**. Row 3 is **runs**. Column 1 is the first feature of every token; column 2 is the second.

If there are m tokens and n features each, the table is m by n :

$$X \in \mathbb{R}^{m \times n}.$$

In this chapter, $m = 3$ and $n = 2$. The first index always picks a token. The second always picks a feature. Swap those meanings halfway through a calculation and the arithmetic may still run. The model will be talking to the wrong house.

Hence X is a spreadsheet only in the sense that a row is one word’s whole representation and a column is one measurement watched across the sentence. After that, the object is the matrix.

A layer is a reusable stir

A **layer** is a recipe the network applies again and again. Abstractly,

$$Y = F(X; W, b).$$

X is the current table. W and b are **parameters**: numbers the model keeps after this sentence is gone, and adjusts when it trains. Y is an **activation**: a table that exists only while this sentence is being read. Tomorrow’s sentence gets a new Y . It reuses the same W .

Our recipe is almost the simplest one. Multiply by a weight matrix, add a bias to every row, then clip the negative numbers:

$$Z = XW + \mathbf{1}_3 b, \quad Y = \text{ReLU}(Z).$$

That is the whole layer. We will compute it by hand so the shapes have nowhere to hide.

The arithmetic, slowly

The same six parameter numbers hit every token:

$$W = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

W is 2×2 : its first index is an *input* feature, its second an *output* feature. So XW contracts the feature axis of X with the matching axis of W :

$$(3 \text{ tokens} \times 2 \text{ input features}) \times (2 \text{ input features} \times 2 \text{ output features}) = 3 \text{ tokens} \times 2 \text{ output features}.$$

The bias has one value per output feature. It is added to every row. Writing $\mathbf{1}_3$ for a column of three ones makes that broadcasting visible:

$$\mathbf{1}_3 b = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

Now the pre-activation:

$$\begin{aligned} Z &= XW + \mathbf{1}_3 b \\ &= \begin{bmatrix} 1 & -2 \\ 0 & 3 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 3 & 4 \\ 5 & 0 \end{bmatrix}. \end{aligned}$$

One entry, so you can see the contract: the second output of **red** is

$$Z_{1,2} = (1)(-1) + (-2)(1) + 1 = -2.$$

ReLU keeps a number if it is positive and writes zero otherwise. Entry by entry:

$$Y = \begin{bmatrix} 0 & 0 \\ 3 & 4 \\ 5 & 0 \end{bmatrix}.$$

Hence six learned numbers did that: four in W , two in b . The tables X , Z , and Y also have six entries each. Those six are not parameters. They belong to *this* sentence.

This particular layer does not mix positions. The output row for **fox** depends only on the input row for **fox**. It does mix features, through W . Later, attention will let one token look at others. Both kinds of block can still accept and return an $m \times n$ table.

Drop the axis names and you can still multiply two arrays of size 2×2 . You will not know whether you mixed features or tokens.

Names for the sides of the table

name	what it indexes	size here
m	tokens, in order	3
n	input features	2
p	output features	2
B	how many sentences in a batch	we omitted it
l	which layer in a stack	one layer

A batch of B sentences would be

$$X \in \mathbb{R}^{B \times m \times n} \quad \text{with axes} \quad [\text{batch}, \text{token}, \text{feature}].$$

The same multiply still acts on the last axis:

$$[B, m, n] @ [n, p] \longrightarrow [B, m, p].$$

Batch and token ride along. They are not summed away.

In scalars, the layer is

$$Z_{ik} = \sum_{j=1}^n X_{ij} W_{jk} + b_k.$$

The repeated index j disappears. Token i and output feature k remain. An elementwise ReLU does not create or destroy an axis, so Y has the same shape as Z .

Before you multiply anything:

1. Write each shape with the axis names, not just the numbers.
2. Mark which axes are about to be contracted.
3. Check that those axes have the same size *and the same meaning*.
4. List what survives.
5. Check that additions match, or are supposed to broadcast.
6. Ask whether a nonlinearity is elementwise.

“Both matrices are 2×2 ” is not a reason to add them. Shape records extent. It does not record meaning.

The same numbers in code

```
# Input: (tokens=3, input_features=2)
X = [1.0  -2.0;
      0.0   3.0;
      2.0   1.0]

# Parameters: (input_features=2, output_features=2) and (output_features=2)
W = [2.0  -1.0;
      1.0   1.0]
b = [0.0, 1.0]

Z = X * W .+ b'           # (tokens=3, output_features=2)
Y = max.(Z, 0)           # (tokens=3, output_features=2)
```

```

@assert size(X) == (3, 2)
@assert size(W) == (2, 2)
@assert size(Z) == size(Y) == (3, 2)
@assert Z == [0.0 -2.0; 3.0 4.0; 5.0 0.0]
@assert Y == [0.0 0.0; 3.0 4.0; 5.0 0.0]

```

Julia broadcasts the row b' across tokens. In the math, $\mathbf{1}_3 b$ showed the same copy without hiding it.

What this chapter is not

This chapter has one table:

$$X \in \mathbb{R}^{m \times n} = [\text{token}, \text{feature}].$$

A row is one position. Three rows are not three leftover copies of the whole sentence. Later we will carry several complete copies of a table like X through a deep stack. That will add a *new* axis. It will not rename this one.

If you remember only one boundary from Chapter 1, remember this: a token row is one word. It is not a spare copy of the sentence.

Ways to get this wrong

- Writing WX instead of XW . Under the row-token convention that either crashes or, worse, quietly does a different job when m happens to equal n .
- Giving the bias one value per token. Then the layer owns the sentence length, and variable-length text becomes a mess.
- Saying this layer “mixes tokens.” It does not. It mixes features.
- Counting the entries of Y as learned parameters. Parameter count would then grow with the sentence.
- Calling columns tokens. Some books use column vectors. Either convention works. Changing midway does not.
- Confusing the batch axis with the token axis. Eight sentences of three tokens is $[8, 3, n]$, not one sentence of twenty-four tokens.
- Inferring meaning from a size. If someday $m = n = 4$, you still have two different axes. Four is just being popular.

Try it

1. Read the table

For the example X , what is $X_{2,1}$? What is the row for runs? What is the column of feature 2? Give each shape.

Answer

$X_{2,1} = 0$, a scalar. runs is row 3, $[2, 1]$. Feature 2 is $[-2, 3, 1]^\top$, shape 3.

2. Widen the output

Keep X as 3×2 , but let the layer produce four features. What shapes do W , b , Z , and Y have? How many scalar parameters?

Answer

W is 2×4 , b is 1×4 (or just length 4 in a library), Z and Y are 3×4 . Twelve parameters: $2 \cdot 4 + 4$.

3. Another layer by hand

$$X' = \begin{bmatrix} 1 & 2 \\ -1 & 3 \\ 0 & -2 \end{bmatrix}, \quad W' = \begin{bmatrix} 1 & 2 \\ -1 & 0 \end{bmatrix}, \quad b' = [1 \quad -1].$$

Compute $Z' = X'W' + \mathbf{1}_3b'$ and $Y' = \text{ReLU}(Z')$.

Answer

$$X'W' = \begin{bmatrix} -1 & 2 \\ -4 & -2 \\ 2 & 0 \end{bmatrix}, \quad Z' = \begin{bmatrix} 0 & 1 \\ -3 & -3 \\ 3 & -1 \end{bmatrix}, \quad Y' = \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 3 & 0 \end{bmatrix}.$$

4. Diagnose the multiply

Someone has five tokens with eight features, so X is 5×8 , and W is 8×16 . They write $W @ X$. What breaks, and what shape did they want?

Answer

$W @ X$ tries to pair 16 with 5. The intended product is $X @ W$, which is 5×16 : five tokens, sixteen output features.

5. A batch, later

A batch has two sentences, three tokens, two features. Later, a model will carry four complete leftover copies of each sentence table. What is the ordinary batched shape? What is the later shape? Does that make the sentence twelve tokens long?

Answer

Ordinary: $[2, 3, 2]$. Later: $[2, 4, 3, 2]$ if we put the leftover-copy axis second. Each sentence still has three positions. Twelve is four copies of three, not a longer sentence.

Check yourself

- Say the shape of $X \in \mathbb{R}^{m \times n}$ aloud.**
 m rows are token positions, n columns are features.
- Show that an affine layer preserves the token axis.**
It does $[m, n] @ [n, p] + [p] \rightarrow [m, p]$. The contracted index is the feature index.
- What fails if you count the entries of Y as parameters?**
Parameter count would then grow with the sentence. W and b stay; X , Z , and Y come and go.
- What fails if this layer is said to mix tokens?**
Each output row depends only on the matching input row. Features mix; positions do not.

Next we stop replacing the table outright. We will keep the old table and add a note in the margin. That note is a residual connection, and it is the one leftover copy everything later multiplies.

Sources

- [R010] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems* 30, 2017. arXiv:1706.03762. <https://arxiv.org/abs/1706.03762>
- [R012] Julia documentation, “Broadcasting,” *The Julia Language Manual*. <https://docs.julialang.org/en/v1/manual/arrays/>
- [R006] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of CVPR*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.
- [R011] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 6. <https://www.deeplearningbook.org/>

Chapter 2 — Keep the old page

A layer is supposed to change a table. The cheapest change is no change at all: copy the table and stop. A plain layer is not allowed that copy. It must rebuild every number, even the ones that were already right.

If the next table should look almost like this one, the learned map still has to invent “copy everything, then tweak a little.” Copying is the easiest map in the world, and the network cannot use it.

So we will not throw the table away. We keep it. Next to it we compute a correction of the same shape. Then we add the two, number by number.

That sum is the whole idea. The rest of the chapter is that sum, once slowly, once with two numbers, and once backwards.

The table plus a correction

Call the current hidden state x_l . The subscript l is which layer we are on. For one hidden list with n features,

$$x_l \in \mathbb{R}^n.$$

A plain layer asks a learned function G to produce the entire next state:

$$x_{l+1} = G(x_l, W_l).$$

W_l is the bundle of numbers the layer keeps after this sentence is gone. If the useful next state is almost x_l , then G still has to reconstruct x_l and only then change it.

Write the desired next state as the old state plus a correction. The correction is whatever G would have written, minus the state you already have:

$$F(x_l, W_l) := G(x_l, W_l) - x_l.$$

F has the same shape as x_l . Substitute, and the next state is

$$\boxed{x_{l+1} = x_l + F(x_l, W_l)}.$$

That leftover correction is a **residual**. The name is worth keeping because the ordinary English word suggests leftover debris; here it is the entire learned change, and it may be larger than the state it edits.

Addition is coordinate by coordinate. For each feature index j ,

$$(x_{l+1})_j = (x_l)_j + F_j(x_l, W_l), \quad j = 1, \dots, n.$$

Nothing here mixes one feature into another. If F wants to mix features, it has to do that *inside* F , before the plus sign.

Hence a residual block is not a new kind of computation. It is a new way to *use* a computation: keep x_l , and let F propose an edit of the same shape.

Drop the requirement that F have the same shape as x_l , and the plus sign is illegal. The edit can still say “delete everything,” so the wiring is not a safety proof.

What a zero correction does

Suppose x_l already has something useful in it. The branch output $F(x_l, W_l)$ is only a proposal. Three cases are worth separating.

1. If $F(x_l, W_l) = 0$, then $x_{l+1} = x_l$ exactly.
2. If F is small compared with x_l , the block makes a small update.
3. If F is large, or points the other way, the output can be wildly different. It can even be zero.

The copy of x_l that rides around the branch is a **shortcut**. It does not go through the learned recipe. After the addition, the correction is still allowed to cancel the copy. The shortcut is a route, not a lock.

For a run of these blocks from index l to a later index L , just keep substituting. Write $F_i := F(x_i, W_i)$ for the correction at layer i . Then

$$\begin{aligned} x_{l+2} &= x_{l+1} + F_{l+1} \\ &= x_l + F_l + F_{l+1}, \end{aligned}$$

and in general

$$x_L = x_l + \sum_{i=l}^{L-1} F(x_i, W_i).$$

Hence the earlier state appears, by name, as one term in the later sum. That is the precise sense in which the architecture *supports* keeping a representation.

The corrections may still overwrite it by cancellation. Support is not a promise that the term survives.

The same sum, read backwards

Training asks a slightly different question: if I nudge the old state, how does the new state move?

A tiny change in x_l becomes the same tiny change, plus whatever F does to that change. The matrix of those tiny changes, entry by entry, is

$$[J_{F_l}]_{ij} = \frac{\partial F_i(x_l, W_l)}{\partial (x_l)_j}.$$

That matrix $J_{F_l}(x_l) \in \mathbb{R}^{n \times n}$ is the **Jacobian** of the correction. The Jacobian of “just copy x_l ” is simpler: it is the identity matrix I_n .

Differentiate the residual update and you get

$$\frac{\partial x_{l+1}}{\partial x_l} = I_n + J_{F_l}(x_l).$$

Let $\mathcal{L}$ be a single number we are trying to make small. Write $g_l := \nabla_{x_l} \mathcal{L}$ for the column of “how much that number cares about each entry of x_l .” The vector chain rule splits the care into two routes:

$$g_l = (I_n + J_{F_l})^\top g_{l+1} = g_{l+1} + J_{F_l}^\top g_{l+1}.$$

The first summand is the shortcut's contribution. It reaches the previous state without multiplication by a learned weight matrix. The second summand is the correction's contribution.

Hence a direct gradient term exists. That is weaker than “the total gradient cannot vanish.” The two terms are added. They can partially or exactly cancel.

If $J_{F_l} = -I_n$, then the block Jacobian is zero even though a shortcut is sitting right there in the source code.

Across many blocks the local Jacobians multiply. The rightmost factor acts first:

$$g_l = \left[\prod_{i=l}^{L-1} (I_n + J_{F_i})^\top \right] g_L.$$

Equivalently, differentiating the earlier sum exposes an additive identity term:

$$\frac{\partial x_L}{\partial x_l} = I_n + \frac{\partial}{\partial x_l} \sum_{i=l}^{L-1} F(x_i, W_i).$$

Both views are correct. The sum view makes the direct route visible. The product view makes amplification, shrinking, rotation, and cancellation visible. Keep both, or the algebra will let you get optimistic.

Two numbers through the block

Take one hidden vector with two features:

$$x_l = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \in \mathbb{R}^2.$$

Let the correction be the simplest interesting recipe: multiply by a matrix, then add a bias. Define

$$W_l = \begin{bmatrix} 0.5 & 0 \\ -0.25 & 0.5 \end{bmatrix}, \quad b_l = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}, \quad F(x_l, W_l, b_l) = W_l x_l + b_l.$$

Forward

First compute the correction:

$$F(x_l, W_l, b_l) = \begin{bmatrix} 0.5 & 0 \\ -0.25 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -1.05 \end{bmatrix} \in \mathbb{R}^2.$$

Then add the old state:

$$x_{l+1} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0.6 \\ -1.05 \end{bmatrix} = \begin{bmatrix} 1.6 \\ -3.05 \end{bmatrix} \in \mathbb{R}^2.$$

Hence the block did not invent $[1.6, -3.05]^\top$ from a blank sheet. The learned correction proposed $[0.6, -1.05]^\top$. The shortcut supplied $[1, -2]^\top$.

Backward

Choose a target

$$t = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \in \mathbb{R}^2$$

and the squared-error score

$$\mathcal{L} = \frac{1}{2} \|x_{l+1} - t\|_2^2 \in \mathbb{R}.$$

That score is a **loss**: a single number training tries to make smaller. Every derivative in this example is a derivative of $\mathcal{L}$.

The output gradient is just the error vector:

$$g_{l+1} = \nabla_{x_{l+1}} \mathcal{L} = x_{l+1} - t = \begin{bmatrix} 0.6 \\ -1.05 \end{bmatrix} \in \mathbb{R}^2.$$

Because this particular correction is affine, its Jacobian is the matrix W_l itself. The block Jacobian is therefore

$$I_2 + W_l = \begin{bmatrix} 1.5 & 0 \\ -0.25 & 1.5 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Send g_{l+1} back through that matrix:

$$\begin{aligned} g_l &= (I_2 + W_l)^\top g_{l+1} \\ &= \begin{bmatrix} 1.5 & -0.25 \\ 0 & 1.5 \end{bmatrix} \begin{bmatrix} 0.6 \\ -1.05 \end{bmatrix} \\ &= \begin{bmatrix} 1.1625 \\ -1.575 \end{bmatrix} \in \mathbb{R}^2. \end{aligned}$$

Now split the same result into its two routes:

$$\underbrace{g_{l+1}}_{\text{shortcut}} + \underbrace{W_l^\top g_{l+1}}_{\text{correction}} = \begin{bmatrix} 0.6 \\ -1.05 \end{bmatrix} + \begin{bmatrix} 0.5625 \\ -0.525 \end{bmatrix} = \begin{bmatrix} 1.1625 \\ -1.575 \end{bmatrix}.$$

A plain affine map $x_{l+1} = W_l x_l + b_l$ would have sent back only $W_l^\top g_{l+1}$. Hence this example shows the extra direct term. It does not say the residual gradient always has a larger size. Addition is perfectly capable of subtraction when the signs disagree.

The parameter gradients still come only through the learned correction:

$$\nabla_{W_l} \mathcal{L} = g_{l+1} x_l^\top = \begin{bmatrix} 0.6 & -1.2 \\ -1.05 & 2.1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}, \quad \nabla_{b_l} \mathcal{L} = g_{l+1} \in \mathbb{R}^2.$$

The shortcut has no parameters to update. It changes the graph through which gradients reach x_l . In this affine example it does not change the formula for the correction's own weight gradient.

The same sum for a whole table

Chapter 1's object was a table, not a single column. For a batch of sentences the residual state is

$$X_l \in \mathbb{R}^{B \times m \times n}.$$

B is how many sentences travel together, m is how many tokens sit in each sentence, and n is how many features each token has. A legal correction must come back the same shape:

$$F(X_l, W_l) \in \mathbb{R}^{B \times m \times n}.$$

Then

$$X_{l+1} = X_l + F(X_l, W_l) \in \mathbb{R}^{B \times m \times n}.$$

The plus sign pairs batch b , token position p , and feature q with the same (b, p, q) entry. A feature projection or an attention block *inside* F may mix features or tokens. The final correction still has to match the leftover table. Addition never invents a new axis and never quietly sums one away.

If the widths do not match, you can multiply the old table by a matrix so the addition becomes legal. That matrix is a **projection shortcut**. The copy is then no longer a pure copy. Its derivative is the projection, not I , and the clean identity-path derivation no longer applies exactly [R006]; [R007].

Here is the same idea in code. It is executable. A production Transformer block also adds normalization, dropout, and specialized kernels.

```

"""Input/output: x (B, m, n) → y (B, m, n)."""
struct ResidualMLP
    W1::Matrix{Float64}
    b1::Vector{Float64}
    W2::Matrix{Float64}
    b2::Vector{Float64}
end

function ResidualMLP(n::Int)
    ResidualMLP(randn(n, 4n), zeros(4n), randn(4n, n), zeros(n))
end

gelu(x) = 0.5 .* x .* (1 .+ tanh.(sqrt(2/pi) .* (x .+ 0.044715 .* x.^3)))

function (m::ResidualMLP)(x)
    B, tokens, n = size(x)
    h = gelu.(reshape(x, B * tokens, n) * m.W1 .+ m.b1')
    delta = reshape(h * m.W2 .+ m.b2', B, tokens, n)
    return x .+ delta           # (B, m, n)
end

```

Batch, token, and feature ride through the plus sign together. The code does not grow a fourth axis, and it does not flatten the sentence into one long token list.

What the plus sign actually buys

An identity shortcut is a parameter-free map $h(x) = x$ added to a correction of the same shape. That wiring gives two exact algebraic facts, and then an experimental one.

Algebra first.

1. The block can represent identity whenever its correction can represent zero.
2. Its local input Jacobian is $I + J_F$, so the backward pass has an additive shortcut term.

Then the experiment. In the original residual-network comparisons, residual networks reduced *training* error more successfully than the matching plain networks as depth increased [R006]. Adding layers to a plain network could raise training error even though an identity solution existed on paper — that was the degradation problem those experiments were aimed at. In a later study, units designed to keep the post-addition route clean — **pre-activation** units, with the normalization and nonlinearity sitting on the correction rather than after the sum — were easier to train in the reported CIFAR and ImageNet runs [R007]. Those findings concern specific architectures, initializations, normalizations, optimizers, and datasets.

Hence the residual writing often parks a useful near-identity solution next to a small correction, and it gives derivatives a shorter route. That can improve the conditioning that gradient descent actually meets. “Can” and

“often” are doing honest work here.

The experiments do not promote the plus sign to a theorem about every architecture.

Three ways the plus sign fails

Write $J_i := J_{F_i}(x_i)$ for the correction Jacobian at depth i . The total Jacobian from depth l to L is the product

$$P_{l \rightarrow L} = \prod_{i=l}^{L-1} (I_n + J_i) \in \mathbb{R}^{n \times n}.$$

Residual wiring alone places no bound on J_i , and therefore no bound on $\|P_{l \rightarrow L}\|$. Three scalar corrections make the limitation painfully concrete.

- If $F(x) = 9x$, one block multiplies activations and gradients by 10. L identical blocks multiply by 10^L .
- If $F(x) = -0.9x$, one block multiplies by 0.1. L blocks multiply by 0.1^L .
- If $F(x) = -x$, one block outputs zero and has zero derivative. The correction exactly cancels the shortcut.

Even a uniform local bound needs care. The 2-norm $\|\cdot\|_2$ asks how much a matrix can stretch a vector. The smallest **singular value** $\sigma_{\min}$ asks the opposite: how little it can stretch. If $\|J_i\|_2 \leq \varepsilon < 1$, then

$$(1 - \varepsilon)^{L-l} \leq \sigma_{\min}(P_{l \rightarrow L}) \leq \|P_{l \rightarrow L}\|_2 \leq (1 + \varepsilon)^{L-l}.$$

The lower bound follows by multiplying $\sigma_{\min}(I + J_i) \geq 1 - \varepsilon$. The upper end can still grow exponentially. The lower end can still decay exponentially. Hence small per-block nudges are not, by themselves, a depth-independent promise.

A depth-aware scaling is more informative as a *sufficient* bound. Suppose there are $D = L - l$ blocks and $\|J_i\|_2 \leq c/D$ with $c < D$. Then

$$\|P_{l \rightarrow L}\|_2 \leq (1 + c/D)^D \leq e^c,$$

while

$$\sigma_{\min}(P_{l \rightarrow L}) \geq (1 - c/D)^D.$$

For fixed c , that lower expression approaches e^{-c} as D grows. The bound can be loose, and it says nothing about training actually finding such Jacobians. This is a conditional mathematical bound, not a property supplied by the plus sign alone.

The eigenvalues of each $I + J_i$ do not fully determine a product of changing, possibly crooked matrices. Singular values and how the blocks line up across depth both matter. We will study that distinction later rather than asking one number to do several jobs.

Normalization placement, branch scaling, initialization, activation functions, optimizer settings, finite precision, and data all affect the Jacobians you actually get. A residual connection guarantees neither convergence to a good minimum, bounded activations, nonvanishing total gradients, generalization, nor immunity to rounding error. It changes the parameterization and creates a direct route. That is substantial. It is not sorcery.

If we throw the table away

Remove the shortcut and the block becomes

$$x_{l+1} = F(x_l, W_l).$$

Four useful properties disappear.

First, identity is no longer obtained by making the correction blank. In fact $F = 0$ now destroys the state. The branch must learn identity explicitly if identity is wanted.

Second, the old state is no longer an explicit summand of the next state. Every feature must pass through the branch. Saturating activations, rank-deficient maps, and skinny bottlenecks can delete information with no bypass.

Third, the input Jacobian changes from $I + J_F$ to J_F . The additive direct gradient term disappears. Across depth, gradients are multiplied only by branch Jacobians.

Fourth, building a deeper model that behaves like a trained shallower one becomes less direct. With residual blocks, newly added shape-matching corrections can in principle be set to zero, leaving the earlier mapping unchanged. That establishes existence of an identity extension. It does not promise that an optimizer will find or keep it.

None of this says a plain network cannot train. Plain networks can and do train. The claim is narrower: omitting the shortcut removes a convenient way to keep a representation, and it removes the direct derivative route that came with it.

Ways to get this wrong

- Saying the skip makes the whole block identity. Only the shortcut is identity. The block is $x + F(x)$, which is identity only when $F(x) = 0$.
- Saying the gradient bypasses the branch, so it cannot vanish. The two contributions add. $J_F^\top g$ can cancel g , and products across blocks can shrink it.
- Saying residual always means a small update. The architecture does not bound $\|F(x)\|$. Small updates may come from initialization, scaling, normalization, regularization, or training — not from the plus sign.
- Adding when the shapes differ. For a clean identity shortcut, both tensors have the same shape. Silent broadcasting is usually a bug wearing comfortable shoes.
- Calling a token a leftover copy of the sentence. In $X_l \in \mathbb{R}^{B \times m \times n}$, m still indexes token positions. The entire table is the ordinary leftover page. Several complete leftover copies, when we meet them, will be a new axis. They will not make the sentence longer.
- Saying residual networks are unconditionally stable. The three scalar corrections $F(x) = 9x$, $F(x) = -0.9x$, and $F(x) = -x$ already kill explosion, vanishing, and cancellation guarantees.
- Treating the original ResNet result as a universal theorem. It is strong empirical evidence in the reported vision settings [R006]. Architecture, normalization, scaling, and optimization details remain relevant.

What this chapter is not

This chapter has one leftover table. For a vector that table is $x_l \in \mathbb{R}^n$. For a batch of sentences it is $X_l \in \mathbb{R}^{B \times m \times n}$.

A token row is still one word. It is not a spare copy of the sentence. Later we will carry several complete leftover copies of a table like X_l . That will add a *new* axis. It will not rename this one, and it will not replace the plus sign.

Try it

1. One forward update

Let $x = [2, -1]^\top \in \mathbb{R}^2$ and let the note be $F(x) = [-0.5, 3]^\top \in \mathbb{R}^2$. What is the residual output? What note would leave x exactly as it is?

Answer

$x + F(x) = [1.5, 2]^\top$. Exact preservation requires $F(x) = [0, 0]^\top$.

2. Separate the two gradient routes

For the linear note $F(x) = Wx$, let

$$W = \begin{bmatrix} 0 & 2 \\ -1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}, \quad g_{l+1} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \in \mathbb{R}^2.$$

Compute the shortcut contribution, the note contribution, and the total g_l .

Answer

The identity contribution is $[3, 4]^\top$. The branch contribution is $W^\top g_{l+1} = [-4, 6]^\top$. Therefore $g_l = [-1, 10]^\top$.

3. Build exact cancellation

Find a 2×2 matrix W such that the linear residual block $y = x + Wx$ sends every $x \in \mathbb{R}^2$ to zero. What is its input Jacobian?

Answer

Choose $W = -I_2$. Then $y = (I_2 + W)x = 0$, and the input Jacobian is $I_2 + W = 0_{2 \times 2}$. Both activations and gradients are canceled.

4. Depth changes the verdict

Consider D scalar residual blocks with $F_l(x) = ax$, the same scalar a in every block. Write x_D and $\partial x_D / \partial x_0$. Evaluate them for $a = 0.02$ and $D = 1000$ to one significant figure.

Answer

Each block multiplies by $1 + a$, so $x_D = (1 + a)^D x_0$ and $\partial x_D / \partial x_0 = (1 + a)^D$. For $a = 0.02$, $1.02^{1000} \approx 4 \times 10^8$. A two-percent local change is not small after one thousand aligned repetitions.

5. A conditional bound

There are D residual blocks satisfying $\|J_l\|_2 \leq c/D$, with $0 < c < D$. Give an upper bound for the end-to-end Jacobian norm and a lower bound for its smallest singular value. Name one thing these bounds do not guarantee.

Answer

Submultiplicativity gives $\|\prod_l (I + J_l)\|_2 \leq (1 + c/D)^D \leq e^c$. Also $\sigma_{\min}(I + J_l) \geq 1 - c/D$, so $\sigma_{\min}(\prod_l (I + J_l)) \geq (1 - c/D)^D$. The assumptions are not enforced by residual wiring, and the bounds do not guarantee that training converges or generalizes.

Check yourself

1. **Show that identity is recovered when the correction is blank.**

If $F(x_l, W_l) = 0$, then $x_{l+1} = x_l + 0 = x_l$.

2. **What fails if you drop the shortcut?**

The update becomes $x_{l+1} = F(x_l, W_l)$. Identity now requires the branch to learn a copy, the old state is no longer a summand, and the backward term g_{l+1} disappears.

3. **Show that a shortcut in the source code does not prove the gradient cannot vanish.**

The two backward terms add: $g_l = g_{l+1} + J_F^\top g_{l+1}$. If $J_F = -I$, they cancel and the block Jacobian is zero.

4. **In the two-feature example, the correction proposed $[0.6, -1.05]^\top$. What did the shortcut supply, and what was the next state?**

Shortcut $[1, -2]^\top$; next state $[1.6, -3.05]^\top$.

5. **What fails if residual wiring is called unconditionally stable?**

The three scalar maps $F(x) = 9x$, $F(x) = -0.9x$, and $F(x) = -x$ already give explosion, vanishing, and exact cancellation. The plus sign is a parameterization, not a bound.

One leftover table is the workspace this chapter built. The next question is whether one workspace is enough.

Sources

- [R006] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *CVPR 2016*, pp. 770–778. arXiv:1512.03385. <https://arxiv.org/abs/1512.03385>. DOI: 10.1109/CVPR.2016.90.
- [R007] K. He, X. Zhang, S. Ren, and J. Sun, “Identity Mappings in Deep Residual Networks,” *ECCV 2016*, pp. 630–645. arXiv:1603.05027. <https://arxiv.org/abs/1603.05027>. DOI: 10.1007/978-3-319-46493-0_38.
- [R008] Y. Bengio, P. Simard, and P. Frasconi, “Learning Long-Term Dependencies with Gradient Descent Is Difficult,” *IEEE Transactions on Neural Networks* 5(2), 1994, pp. 157–166. DOI: 10.1109/72.279181.

Chapter 3 — Four notebooks, same sentence

Open four notebooks. On the first page of each, write the same two-word sentence, already turned into a table of numbers:

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}.$$

The four pages now match. We will carry all four through every station of a deep stack. At a station we may glance at a blend of the pages, run one tool once, scribble different amounts of the result into each notebook, and walk on with all four still in hand.

Last chapter we had one leftover copy of a table and we added a correction. Today that leftover copy has friends. The friends start equal. They are allowed to stop being equal.

Name the notebooks

Stack the four pages. Do not add them. Do not slice the table into four thinner tables. Copy it:

$$x_0 = \begin{bmatrix} X \\ X \\ X \\ X \end{bmatrix} \in \mathbb{R}^{4 \times 2 \times 2}.$$

Each page is one **stream**: a leftover slot the size of the whole sentence table, carried from one layer door to the next. The name is worth keeping because the *slot* survives at every depth. The ink inside can change, vanish, or flip sign.

When the sentence has m tokens and n features each, one stream is still m by n . All d of them together are

$$x_l \in \mathbb{R}^{d \times m \times n}.$$

For this chapter, $d = 4$. Stream i at the door of layer l is written $x_l^{(i)}$. The first index picks a notebook. The second picks a token. The third picks a feature. The original table is still $X \in \mathbb{R}^{m \times n}$.

Hence a stream is not a token, not a feature, and not a layer. It is a new axis: another complete copy of the sentence table, living at the same depth as its siblings.

Four identical starting pages are a convenience. Later mixing can make them diverge without four different embedding modules.

A model may have four streams and 32 layers, or four streams and 80, or four streams and 1000. At every layer door it still carries four current pages. A four-layer network could just as well carry one stream or four. Depth answers “which station?” The new axis answers “which notebook?” Those are two different coordinates.

If we drop the extra axis, d returns to 1 and we are back in Chapter 2. The friends are what we are adding.

One station, by hand

The sequence here has $m = 2$ tokens and $n = 2$ features, so X is the table we already wrote. Copied start gives

$$x_0 = \begin{bmatrix} X \\ X \\ X \\ X \end{bmatrix} \in \mathbb{R}^{4 \times 2 \times 2}.$$

Suppose the station reads the average of the four pages. Call that temporary blend z_0 :

$$z_0 = \frac{1}{4}(x_0^{(1)} + x_0^{(2)} + x_0^{(3)} + x_0^{(4)}) = X \in \mathbb{R}^{2 \times 2}.$$

For a deliberately boring tool, take $F_0(A) = 2A$ for every 2×2 table A . Run it once:

$$y_0 = F_0(z_0) = 2X = \begin{pmatrix} 2 & 4 \\ 6 & 8 \end{pmatrix} \in \mathbb{R}^{2 \times 2}.$$

Now keep each old page and write a different multiple of that one result:

$$\begin{aligned} x_1^{(1)} &= x_0^{(1)} + 1 y_0 = 3X, \\ x_1^{(2)} &= x_0^{(2)} + 0 y_0 = X, \\ x_1^{(3)} &= x_0^{(3)} + \frac{1}{2} y_0 = 2X, \\ x_1^{(4)} &= x_0^{(4)} - 1 y_0 = -X. \end{aligned}$$

Therefore

$$x_1 = [3X, X, 2X, -X] \in \mathbb{R}^{4 \times 2 \times 2}.$$

Four facts sit in that one step, and none of them is optional:

- there are still four pages in x_1 ;
- there is one temporary tool output, y_0 ;
- every page and y_0 has the full 2×2 token-feature shape;
- F_0 was called once, not four times.

Hence one station can split four equal pages into four unequal pages without multiplying the tool. The negative write is a teaching choice, not a recommendation. Later chapters will show why unconstrained mixing can blow up, and how a later constraint changes which leftover-routing matrices are allowed.

Not a tape of the last four stations

A tempting misreading: the four notebooks are just the outputs of the last four layers, a sliding window. A video recorder would keep “output from layer $l - 3$, then $l - 2$, then $l - 1$, then l .” We are not holding a video recorder.

Suppose they were. Then at the door of layer 0 the four slots would have to be four previous tool outputs $y_{-4}, \dots, y_{-1}$. But the state

$$x_0 = [X, X, X, X]$$

already contains four streams, and there are no previous tool outputs to file. The identification fails before the first station opens.

One might try to rescue it after the first step. The numbers refuse. We produced

$$x_1 = [3X, X, 2X, -X]$$

from exactly one tool output, $y_0 = 2X$. All four current pages exist at once. None of them is required to equal a distinct historical output. In particular, $x_1^{(2)} = X$ is copied state, while $x_1^{(4)} = -X$ mixes copied state with the current output. A sliding history of four distinct previous layer outputs cannot be assembled from one existing output.

A tape also has a forced shift: three slots copy their neighbors, and one slot receives the newest result. Our notebooks allow every next page to depend on every current page and on the current tool output. An allowed step already violates the forced shift.

Hence the two machines are not the same machine. The four streams are not the previous four layers.

A learned network could still *choose* a history-like mixing pattern. That would be a routing choice. It is not the definition of the extra axis.

Four streams are also not four different token sequences, and they are not four mandatory runs of the tool. Those mix-ups produce matrices that look plausible and mean the wrong house.

The name on the paper

We now have several leftover copies of a sentence table, a way to blend them into one tool input, and a way to write the tool's answer back in different amounts. Zhu et al. called that idea **Hyper-Connections** in a 2024 preprint, later revised in 2025 [R001]. The new connections sit *around* an ordinary leftover path: they decide how the notebooks talk, not what the expensive tool computes.

The paper framed Hyper-Connections as another way to generalize residual connections, with learnable “depth-connections” and “width-connections.” It reported language-model and vision experiments, and it studied both a static version and an input-dependent dynamic version (DHC). Those are results from a recent paper, not settled architectural law.

The original paper uses n for its expansion rate and d for hidden-vector width in part of its notation [R001]. These notes do the opposite: d is the stream count and n is the feature width. Under our letters, the paper's expansion rate is $d = 4$, and one sequence-shaped hidden state still has per-stream shape $m \times n$.

The useful split is capacity. Four full streams carry $4mn$ scalars per unbatched example. The tool can still eat and return mn scalars. “Wider leftover state” and “four copies of every layer computation” are different claims. Do not let them share a coat.

If you meet the paper's letters first, match meanings and shapes, not just names. The expansion-rate letter is not ours.

Not four thinner columns

Here is a different wrong picture. Cut the feature list of X into four pieces and call each piece a stream.

If 4 divides n , that cut is a **feature partition**: four blocks $P^{(i)} \in \mathbb{R}^{m \times (n/4)}$ whose concatenation along features rebuilds X ,

$$X = \text{concat}_{\text{feature}}(P^{(1)}, P^{(2)}, P^{(3)}, P^{(4)}).$$

Our copied start is not that cut. Each stream is the whole table: $x_0^{(i)} = X \in \mathbb{R}^{m \times n}$ for every i . Glue the four streams together along features and you get shape $m \times 4n$, not $m \times n$.

The hand example makes the dodge impossible. Here $n = 2$, which cannot be split into four nonempty equal-width feature blocks. Four full streams are still perfectly well-defined.

Hence a stream is not a feature shard. If an implementation stores the four pages as a flattened table of shape $m \times (4n)$, that is a storage view of four full-width groups. It does not turn one n -wide representation into four $n/4$ -wide scraps.

“Expanded leftover width” describes total carried capacity. It does not mean each stream has width n/d . In the formulation used here, each stream has width n .

One tool, four pages

Let $F_l(\cdot, W_l)$ be the computational branch at layer l : one $m \times n$ table in, one $m \times n$ table out. The usual walk is:

```
input:  x_l, shape [4, m, n]
read:   z_l = combine_streams(x_l), shape [m, n]
compute: y_l = F_l(z_l, W_l), shape [m, n]      # one evaluation
write:  x_{l+1} = route_and_write(x_l, y_l), shape [4, m, n]
output: x_{l+1}, shape [4, m, n]
```

The blender and the writer can each own four coefficients without causing four calls to F_l . Coefficient count, notebook count, and tool-evaluation count are three different numbers.

An extended design could intentionally use several branches $F_l^{(1)}, \dots, F_l^{(q)}$. That would create q evaluations because the design asked for q branches — not because $d = q$, and not because streams are feature slices.

Without a batch axis the state is

$$\underbrace{4}_{\text{streams}} \times \underbrace{m}_{\text{tokens}} \times \underbrace{n}_{\text{features}}.$$

With a batch of B examples, a practical layout may be $B \times 4 \times m \times n$ or $4 \times B \times m \times n$. The convention may change. The meanings may not. Mixing notebooks must not accidentally mix sentences, token positions, or feature coordinates.

For a fixed token t and feature c , the slice

$$x_l[:, t, c] \in \mathbb{R}^4$$

is that same coordinate across all four pages. A 4×4 notebook-routing matrix acts naturally on this vector. By contrast,

$$x_l[i, t, :] \in \mathbb{R}^n$$

is the full feature list for token t in stream i . Those two slices answer different questions.

Hence routing along the stream axis is not routing along the token axis or the feature axis. If you drop the axis names, a 4×4 multiply still runs. You will not know which house it entered.

A later paper keeps the notebooks

The same four pages, the same one-tool walk, and the same leftover copies show up again in a 2025 preprint [R002]. What changes is not the notebooks. What changes is which mixing matrices are allowed when one page is written into another.

That constrained version is **mHC**. The streams stay streams; only the leftover-mixing rule gets tighter. The paper still writes a stream-to-branch read, a branch-to-stream write, and a leftover map among streams. It does not turn streams into a history tape or into feature shards.

The paper reports large-scale training stability and efficiency numbers for that constrained form [R002]. Those are recent-preprint numbers. Read them in the setting they report.

A different way to write the mixing

A later construction starts from the same d -stream state. With batch, sequence, and hidden axes included, its paper writes $x_l \in \mathbb{R}^{d \times B \times L \times D}$ [R005]. Drop the batch axis and rename $L \rightarrow m$, $D \rightarrow n$, and you have this chapter's $d \times m \times n$ shape. It also initializes x_0 with d copies of the input and keeps one branch evaluation in its central recurrence.

That construction is **go-mHC**. It is a different way to parameterize a constrained mixing matrix, not a different meaning of “stream.” Neither later name means “memory of the last d layers.”

The paper reports synthetic mixing tests and 30M-parameter GPT-style experiments [R005]. Again, a recent preprint. Again, read the claim in its reported setting.

What goes missing

If we drop the extra axis, d returns to 1. The state is again $m \times n$, and notebook-to-notebook routing collapses to a scalar. We lose the chance to carry several routed leftover trajectories through depth at once.

If we keep four tables but reread them as the last four outputs, the update becomes a shift register. Three steps turn into hard-coded copies, and the architecture cannot freely mix all current pages into all next pages. That is a different model.

If we replace streams with feature partitions, each branch sees only $n/4$ features unless extra concatenation or projection restores the width. The shapes and parameters of F_l change. That may be a useful architecture. It is not the one being studied.

If we assume one F_l evaluation per stream, branch compute is multiplied by four and the central efficiency idea is hidden. The usual single-branch walk is one read, one tool call, and one distributed write per block.

If we forget the copied start, the model needs another specified rule for building four first pages. Leave that rule implicit and the first layer is undefined. Four zeros, four learned states, and four copies of X are not interchangeable opening conditions.

Ways to get this wrong

- Calling streams “layers.” A layer index l selects depth. A stream index i selects a notebook at that depth.
- Writing $x_l^{(i)} \in \mathbb{R}^{m \times (n/4)}$. Here each stream is $m \times n$. The total state is $4 \times m \times n$.
- Treating copied start as four embedding evaluations. The value X is formed once and repeated along the stream axis. A framework may materialize that repetition or only view it. The mathematical start is the same.
- Assuming copied streams stay equal. Unequal writes can split them after one layer. The hand example already did.
- Equating persistence with immutability. A persistent slot stays part of the recurrent state. Its contents can be replaced by routed mixtures at every layer.
- Counting four calls to F_l . The usual equation reduces streams before F_l and distributes after it. One layer can therefore contain one F_l evaluation.
- Flattening and forgetting. A tensor stored as $[m, 4n]$ can encode four full streams, but stream routing must respect the grouping. An arbitrary feature operation is not automatically stream routing.
- Reading paper letters without a legend. In the original Hyper-Connections paper, the expansion-rate letter is not ours. Match meanings and shapes, not just names.

Try it

1. Shape ledger

Let $m = 7$, $n = 16$, and $d = 4$. Give the shapes of X , one stream $x_l^{(3)}$, the complete state x_l , a one-branch input z_l , and branch output y_l .

Answer

$X, x_l^{(3)}, z_l, y_l \in \mathbb{R}^{7 \times 16}$. The complete state is $x_l \in \mathbb{R}^{4 \times 7 \times 16}$.

2. Copy, do not partition

Let

$$X = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 3 & 4 \end{pmatrix} \in \mathbb{R}^{2 \times 3}.$$

Write $x_0^{(4)}$, state the shape of x_0 , and explain why a four-way equal feature partition is impossible.

Answer

$x_0^{(4)} = X$. Stacking four copies gives $x_0 \in \mathbb{R}^{4 \times 2 \times 3}$. An equal four-way feature partition would require pieces of width $3/4$, which is not an integer; streams remain valid because each has the full width 3.

3. One evaluation, four writes

Assume four scalar-shaped streams $x_0 = [2, 2, 2, 2]$, read their average, and use $F(z) = z + 1$. Preserve every stream and write $[1, 0, -1, 2]$ times the branch output. Compute y_0 and x_1 . How many times is F evaluated?

Answer

The read gives $z_0 = 2$, so one evaluation gives $y_0 = 3$. The next streams are $[2+3, 2+0, 2-3, 2+6] = [5, 2, -1, 8]$. F is evaluated once.

4. Refute the history claim

A classmate says that at the entrance to layer 1, the four streams must be $[y_{-3}, y_{-2}, y_{-1}, y_0]$, the four most recent outputs. Give two independent contradictions using copied initialization and simultaneous updates.

Answer

First, copied $x_0 = [X, X, X, X]$ exists before any y_l , so negative-index historical outputs are unavailable. Second, one transition may update all four streams with different mixtures of current streams and the single y_0 ; it need not shift three historical values and append one new value. The proposed rule describes a different state machine.

5. Separate the three counts

An experimental block carries $d = 4$ streams, partitions each token vector into $p = 2$ feature groups inside its branch, and explicitly evaluates $q = 3$ different branch functions. State the stream count, feature-partition count, and branch-evaluation count. Does any equality among them follow from Hyper-Connections?

Answer

The counts are $d = 4$, $p = 2$, and $q = 3$. No equality follows. Stream slots describe persistent state, partitions describe internal feature organization, and q counts explicit branch calls.

Check yourself

Read

$$x_l \in \mathbb{R}^{4 \times m \times n}$$

as “four current, full-width, sequence-shaped leftover states at depth l .” The start is

$$x_0 = [X, X, X, X],$$

not a feature split and not a layer-history buffer.

1. Show that the four streams cannot be the previous four layer outputs.

At the door of layer 0 the state is already $x_0 = [X, X, X, X]$, and no tool outputs $y_{-4}, \dots, y_{-1}$ exist. After one step, $x_1 = [3X, X, 2X, -X]$ is built from a single $y_0 = 2X$, so it cannot be four distinct historical outputs.

2. What fails if a stream is written $x_l^{(i)} \in \mathbb{R}^{m \times (n/4)}$?

Each stream here is $m \times n$. The hand example has $n = 2$, which cannot be split into four nonempty equal-width feature blocks, yet four streams are well-defined.

3. What is the shape of one usual branch output?

$m \times n$. The tool sees one blended table, not four.

4. Does $d = 4$ force four branch evaluations?

No. The usual walk reduces streams, calls F_l once, and writes back.

5. What fails if persistence is read as immutability?

A persistent slot stays part of the recurrent state. Its contents can be replaced by routed mixtures at every layer. The hand example already split four equal pages.

Next chapter turns “read,” “route,” and “write” into explicit tensors: a 1×4 pre-routing row, one computation F_l , a 1×4 post-routing row, and a 4×4 leftover-routing matrix. Keep today’s boundary lines visible. The next equation is compact enough to hide them all at once.

Sources

- [R006] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *CVPR*, 2016. DOI: 10.1109/CVPR.2016.90; arXiv:1512.03385.
- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024; revised 2025. <https://arxiv.org/abs/2409.19606>.
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025; revised 2026. <https://arxiv.org/abs/2512.24880>.
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309v1>.
- [R009] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” *CVPR*, 2017. DOI: 10.1109/CVPR.2017.243; arXiv:1608.06993.

Chapter 4 — Four drawers, one tool

Four leftover copies of the same two-word table sit on a workbench. Someone wants to run an expensive tool on them — attention, a small network, whatever we will later call F . The budget did not approve four tool calls.

So we cheat, politely.

We blend the four copies into one table. We run the tool once. We write scaled copies of that one answer back into the four drawers. Meanwhile we remix the old drawer contents so yesterday’s work does not vanish. Then we add the remix to the write.

That is the whole layer. Four verbs: read, do the work once, write back, remix leftovers.

The spell, then the four verbs

Papers pack those four verbs into one line [R001]; [R002]:

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l).$$

It looks like a compressed spell. Read it from the inside out, after the picture.

The workbench is x_l . Each leftover copy is one stream: a full table of tokens by features that rides from this layer to the next. Chapter 3 already stacked four of them. A stream names a slot, not a token and not a layer.

Innermost: $H_l^{\text{pre}} x_l$ blends the drawers into one ordinary table. That blend is the **read**.

Next: $F(\cdot, W_l)$ is the expensive tool, run once on that blend. W_l is every number the tool itself owns.

Then $(H_l^{\text{post}})^\top$ takes the one tool answer and fans it back into four drawers. That fan-out is the **write**.

Meanwhile $H_l^{\text{res}} x_l$ remixes the old drawers without asking F . That remix is the leftover path: persist, then add.

Hence two questions should now have answers you can shout: which axis is being mixed, and how many times is the expensive tool called? Stream axis. Once.

If you drop the leftover term, every drawer becomes a scaled copy of the fresh answer and yesterday vanishes. If you drop the write, the answer has no agreed home. The four maps are not ornaments on one another.

Blend the drawers

Let b be how many sentences ride together, d the number of leftover copies, m how many tokens, and n how many features each. The workbench is

$$x_l \in \mathbb{R}^{b \times d \times m \times n}.$$

Copy i is the slice $x_l^{(i)} \in \mathbb{R}^{b \times m \times n}$. The stream axis is not the batch, not the token list, and not the feature list. Routing stirs streams. The other three axes stay put.

For one sentence, with the same blend used at every token, the read is a row of d numbers:

$$H_l^{\text{pre}} = [p_1, \dots, p_d] \in \mathbb{R}^{1 \times d}.$$

It makes one table

$$z_l = H_l^{\text{pre}} x_l = \sum_{j=1}^d p_j x_l^{(j)} \in \mathbb{R}^{b \times m \times n}.$$

The stream axis is gone. The tool now sees the same kind of shape an ordinary residual block sees.

Hence the read is a contraction along streams, not along tokens or features. A row of length m would mix positions. That is a different map.

Do the work once

The tool returns another ordinary table:

$$y_l = F(z_l, W_l) \in \mathbb{R}^{b \times m \times n}.$$

F might be attention, an MLP, a state-space block, a convolution, or any other learned block that keeps the token-feature shape. The routing algebra does not care which, so long as the answer can be written back.

Call F the **branch**. The name is worth keeping because the ordinary English word could mean “one copy per stream.” Here it is the expensive side path, run once, not the leftover remix.

There is one z_l , one y_l , and one call. We do not compute $F(x_l^{(1)}, W_l), \dots, F(x_l^{(d)}, W_l)$. That different program would cost about d times as much and would not be this equation.

If you assume one evaluation per stream, you have written a different architecture and hidden the central efficiency claim.

Put copies back

The write is another row of d numbers:

$$H_l^{\text{post}} = [q_1, \dots, q_d] \in \mathbb{R}^{1 \times d}.$$

Stand it up: $(H_l^{\text{post}})^\top \in \mathbb{R}^{d \times 1}$. One answer becomes d scaled copies:

$$w_l = (H_l^{\text{post}})^\top y_l = \begin{bmatrix} q_1 y_l \\ \vdots \\ q_d y_l \end{bmatrix} \in \mathbb{R}^{b \times d \times m \times n}.$$

Each q_i multiplies the whole table y_l . It does not pick a token or a feature.

Hence the write restores the stream axis that the read removed. Until that axis is back, y_l cannot be added to a d -stream workbench.

The coefficients need not be probabilities. Nothing in the equation forces them to be nonnegative or to add to one.

Remix leftovers, then add

The leftover remix is a $d \times d$ table

$$H_l^{\text{res}} = [r_{ij}]_{i,j=1}^d \in \mathbb{R}^{d \times d}.$$

Row i is the destination drawer. Column j is the source. Old copy j contributes to new copy i by

$$s_l^{(i)} = (H_l^{\text{res}} x_l)^{(i)} = \sum_{j=1}^d r_{ij} x_l^{(j)}.$$

That remix never looks at y_l . Read and write touch the branch. This map only moves old state around it.

The layer output is $x_{l+1} = s_l + w_l$.

Hence the leftover path is the old residual idea — modify rather than replace — except the identity has become a learned stream-to-stream map. If $H_l^{\text{res}} = I_d$ and the write is a single one in one drawer, we are almost back in Chapter 2, with extra silent passengers.

If you reverse the indices of r_{ij} , destinations and sources swap and the remix is a different matrix.

Four streams, two tokens, two features

We now run the full layer by hand. One sentence, no batch axis. Set $d = 4$, $m = 2$, $n = 2$. Then

$$x_l \in \mathbb{R}^{4 \times 2 \times 2}.$$

The four leftover copies are

$$\begin{aligned} x_l^{(1)} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, & x_l^{(2)} &= \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}, \\ x_l^{(3)} &= \begin{bmatrix} 0 & 2 \\ -1 & 1 \end{bmatrix}, & x_l^{(4)} &= \begin{bmatrix} 1 & -1 \\ 2 & 2 \end{bmatrix}. \end{aligned}$$

Each row is a token. Each column is a feature. Same two positions, four different leftover tables.

Pull a blend

Use the read

$$H_l^{\text{pre}} = [0.6, 0.2, 0.1, 0.1] \in \mathbb{R}^{1 \times 4}.$$

The blend is a weighted sum over drawers:

$$z_l = 0.6x_l^{(1)} + 0.2x_l^{(2)} + 0.1x_l^{(3)} + 0.1x_l^{(4)}.$$

Entry by entry:

$$z_l = \begin{bmatrix} 0.6(1) + 0.2(2) + 0.1(0) + 0.1(1) & 0.6(0) + 0.2(1) + 0.1(2) + 0.1(-1) \\ 0.6(0) + 0.2(1) + 0.1(-1) + 0.1(2) & 0.6(1) + 0.2(0) + 0.1(1) + 0.1(2) \end{bmatrix} = \begin{bmatrix} 1.1 & 0.3 \\ 0.3 & 0.9 \end{bmatrix}.$$

Shape 2×2 , not $4 \times 2 \times 2$. All four drawers contributed. The branch still gets one table.

One multiply

For arithmetic rather than drama, let the branch be a feature multiply

$$F(z_l, W_l) = z_l W_l, \quad W_l = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Then

$$y_l = F(z_l, W_l) = \begin{bmatrix} 1.1 & 0.3 \\ 0.3 & 0.9 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 1.1 & 1.9 \\ 0.3 & -0.3 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Exactly one multiply by W_l . Exactly one call to F .

Fan the answer back

Use the write

$$H_l^{\text{post}} = [0.1, 0.8, 0.3, 0.2] \in \mathbb{R}^{1 \times 4}.$$

Four scaled copies of the one answer:

$$w_l = (H_l^{\text{post}})^{\top} y_l = \begin{bmatrix} 0.1y_l \\ 0.8y_l \\ 0.3y_l \\ 0.2y_l \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 0.11 & 0.19 \\ 0.03 & -0.03 \\ 0.88 & 1.52 \\ 0.24 & -0.24 \end{bmatrix} \\ \begin{bmatrix} 0.33 & 0.57 \\ 0.09 & -0.09 \\ 0.22 & 0.38 \\ 0.06 & -0.06 \end{bmatrix} \end{bmatrix} \in \mathbb{R}^{4 \times 2 \times 2}.$$

These four numbers sum to 1.4. They are gains, not probabilities. Unconstrained Hyper-Connections do not force a write row to add to one [R001]. Calling every coefficient row a probability distribution would invent a constraint the equation does not have.

Remix, then add

Choose the leftover map

$$H_l^{\text{res}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 0.25 & 0.75 & 0 \\ 0 & 0 & 0.2 & 0.8 \end{bmatrix} \in \mathbb{R}^{4 \times 4}.$$

Drawer 1 keeps its old table. The others mix neighbors. The remixed leftovers are

$$s_l = H_l^{\text{res}} x_l = \begin{bmatrix} x_l^{(1)} \\ 0.5x_l^{(1)} + 0.5x_l^{(2)} \\ 0.25x_l^{(2)} + 0.75x_l^{(3)} \\ 0.2x_l^{(3)} + 0.8x_l^{(4)} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix} \\ \begin{bmatrix} 0.5 & 1.75 \\ -0.5 & 0.75 \end{bmatrix} \\ \begin{bmatrix} 0.8 & -0.4 \\ 1.4 & 1.8 \end{bmatrix} \end{bmatrix}.$$

Now add matching shap:

$$x_{l+1} = s_l + w_l = \begin{bmatrix} \begin{bmatrix} 1.11 & 0.19 \\ 0.03 & 0.97 \end{bmatrix} \\ \begin{bmatrix} 2.38 & 2.02 \\ 0.74 & 0.26 \end{bmatrix} \\ \begin{bmatrix} 0.83 & 2.32 \\ -0.41 & 0.66 \end{bmatrix} \\ \begin{bmatrix} 1.02 & -0.02 \\ 1.46 & 1.74 \end{bmatrix} \end{bmatrix} \in \mathbb{R}^{4 \times 2 \times 2}.$$

The four outputs differ because they inherited different remixed histories and received different multiples of the shared branch answer. That is the complete update.

Hence one layer can split four drawers without four tool calls, and without mixing tokens into features.

Names for every shape

Put the batch axis back. One set of routing numbers per sentence, with singleton axes so they broadcast across tokens and features. Static coefficients are the special case where every sentence shares the same numbers.

Object	Meaning	Shape
x_l	leftover workbench	$b \times d \times m \times n$
H_l^{pre}	read numbers	$b \times 1 \times d$
$z_l = H_l^{\text{pre}} x_l$	one branch input, streams gone	$b \times m \times n$
W_l	every parameter inside F	branch-dependent
$y_l = F(z_l, W_l)$	one branch answer	$b \times m \times n$
H_l^{post}	write numbers	$b \times 1 \times d$
$(H_l^{\text{post}})^\top$	write column on the stream axes	$b \times d \times 1$
w_l	written copies	$b \times d \times m \times n$
H_l^{res}	leftover remix	$b \times d \times d$
$s_l = H_l^{\text{res}} x_l$	remixed leftovers	$b \times d \times m \times n$
$x_{l+1} = s_l + w_l$	next workbench	$b \times d \times m \times n$

The superscript $\top$ on H^{post} flips only the conceptual $1 \times d$ routing row. It does not flip batch, token, or feature. In code, named axes or einsum subscripts are safer than a generic transpose.

With indices, the same layer is

$$\begin{aligned} z_{a,t,c} &= \sum_{j=1}^d p_{a,j} x_{a,j,t,c}, \\ y_{a,t,c} &= F(z, W_l)_{a,t,c}, \\ s_{a,i,t,c} &= \sum_{j=1}^d r_{a,i,j} x_{a,j,t,c}, \\ w_{a,i,t,c} &= q_{a,i} y_{a,t,c}, \\ x'_{a,i,t,c} &= s_{a,i,t,c} + w_{a,i,t,c}. \end{aligned}$$

Here a picks a sentence, i and j pick streams, t picks a token, and c picks a feature. Only the first and third lines sum over streams. Neither line sums over tokens or features.

A clear teaching implementation is:

```

# x: (batch, streams, tokens, features)
# h_pre, h_post: (batch, streams)
# h_res: (batch, out_stream, in_stream)
# F maps (batch, tokens, features) → (batch, tokens, features)
B, d, tokens, features = size(x)
z = dropdims(sum(reshape(h_pre, B, d, 1, 1) .* x; dims=2); dims=2)
y = F(z)                                # exactly one branch call
persisted = similar(x)
for b in 1:B
    persisted[b, :, :, :] = reshape(
        h_res[b, :, :] * reshape(x[b, :, :, :], d, tokens * features),
        d, tokens, features)
end
written = reshape(h_post, B, d, 1, 1) .* reshape(y, B, 1, tokens, features)
x_next = persisted .+ written            # (batch, streams, tokens, features)

```

This code explains the algebra. It is not a claim about a production kernel. Real code may fuse the routing and may grow the coefficients from the input. The primary papers say how they actually build those numbers [R001]; [R002].

Why the order is not optional

Read must come before compute, because F expects no stream axis. Compute must come before write, because every drawer receives a scaled copy of the computed answer. The leftover remix can run beside compute, because it depends on x_l , not on y_l . The two paths meet only at the last addition.

```

x_l --read--> z_l --one F--> y_l --write-->
\                                     +--> x_{l+1}
+-----persist-----+

```

The upper path proposes a new edit. The lower path carries existing state. That is the old residual idea — modify rather than replace [R006] — except the identity path has become a learned stream-to-stream map.

Suppose you add y_l to $H_l^{\text{res}}x_l$ before the write. The shapes disagree: $b \times m \times n$ against $b \times d \times m \times n$. The meeting point is addition of matching workbenches, not hope.

What later variants still do

Hyper-Connections leave the three maps learnable. In the unconstrained form, the leftover map is not forced to be an averaging matrix [R001]; [R002].

mHC keeps the same four verbs. What changes is how at least the leftover map is made: it is pushed toward matrices whose rows and columns each add to one, using a finite number of Sinkhorn normalizations [R002]. Later chapters will name that set and treat the finite-iteration caveat carefully. The central equation stays. The recipe for H_l^{res} changes.

go-mHC is another recent recipe for the leftover map. Its skew and Cayley construction, and its block-norm checks, generate routing numbers. They do not replace x_l , run F , or invent the leftover copies by themselves [R005]. Chapters 12–14 build that recipe. For now the interface is enough:

$$H_l^{\text{res}} \in \mathbb{R}^{d \times d} \quad \text{enters the same leftover slot.}$$

Different variants are different ways of building or constraining routers around one shared skeleton. That is a comparison of dataflow, not a claim that they optimize the same way or can reach the same maps.

What disappears if you drop a piece

Drop the read, and there is no agreed way to shrink $b \times d \times m \times n$ down to the ordinary $b \times m \times n$ the branch expects. Picking one fixed drawer would fit the shapes, but the branch could not learn to combine leftovers through this mechanism.

Drop the branch, and the layer only remixes old tables. It can rearrange or average what it already has. It cannot apply the intended attention, MLP, convolution, or other learned change.

Drop the write, and one $b \times m \times n$ answer has no agreed home in a $b \times d \times m \times n$ workbench. Copying it equally is a legal fixed policy. It deletes learned per-drawer write strengths.

Drop the leftover remix and set that term to zero. Then

$$x_{l+1}^{(i)} = q_i F(z_l, W_l).$$

Every output drawer is a scalar times the same fresh branch answer. Earlier drawer-specific contents vanish in one step. The tensors still exist. “Persistent” has become decoration.

Drop the last addition, and one path replaces the other. Addition is the meeting point that keeps old state and new computation both available.

Ways to get this wrong

“There are d streams, so call F d times.” Not in this equation. The read contracts the stream axis first. One z_l , one y_l , one call.

1. Mixing the token axis. H^{pre} has length d , not m . It combines leftover tables at matching token and feature coordinates.
2. Forgetting the write transpose. A $1 \times d$ row reads from d drawers. A $d \times 1$ column writes one answer to d drawers.
3. Treating writes as four independent numbers. Each write is a whole $m \times n$ table times one coefficient.
4. Assuming the gates are probabilities. The write above sums to 1.4. Unconstrained Hyper-Connection coefficients need not be nonnegative or normalized. Constraints belong to later variants.
5. Reversing leftover indices. In r_{ij} , row i is the destination and column j is the source.
6. Adding before shapes agree. y_l cannot be added straight to $H^{\text{res}} x_l$. The write must restore the stream axis first.
7. Confusing leftover copies with recent layers. Every $x_l^{(i)}$ is state at layer l . The leftover map updates all of them together.
8. Overclaiming how fine the routing is. A compact equation hides batch and broadcast details. Do not announce that a router is token-wise, sequence-wise, or global without checking the particular code.

Try it

1. Read four numbers

Four leftover scalars sit in the drawers: $[2, 6, -2, 10]$. The read is $H^{\text{pre}} = [0.5, 0.25, 0.25, 0]$. What does the branch see? What is the shape, and what is the shape after you restore a batch of three sentences?

Answer

The read is $0.5(2) + 0.25(6) + 0.25(-2) + 0(10) = 2$. For one unbatched scalar example its shape is scalar (or 1×1 if token and feature axes are retained). With batch size three and one token/feature, it is $3 \times 1 \times 1$.

2. Write one answer into four drawers

Let $y = \begin{bmatrix} 2 & -1 \end{bmatrix} \in \mathbb{R}^{1 \times 2}$ and $H^{\text{post}} = [0, 1, -0.5, 2]$. Write the four leftover copies and give the shape.

Answer

$$(H^{\text{post}})^{\top} y = \begin{bmatrix} [0, 0] \\ [2, -1] \\ [-1, 0.5] \\ [4, -2] \end{bmatrix} \in \mathbb{R}^{4 \times 1 \times 2}.$$

Negative and greater-than-one coefficients are algebraically valid for an unconstrained write gate.

3. Remix leftovers

Scalar leftovers $x = [1, 2, 4, 8]^{\top}$ and

$$H^{\text{res}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0.5 & 0.5 \\ 0.25 & 0.25 & 0.25 & 0.25 \end{bmatrix}.$$

Compute $H^{\text{res}} x$. Which old drawers feed each new one?

Answer

The result is $[2, 1, 6, 3.75]^{\top}$. Destination 1 copies source 2; destination 2 copies source 1; destination 3 averages sources 3 and 4; destination 4 averages all four sources.

4. One whole scalar layer

Keep the leftovers and leftover map from exercise 3. Let $H^{\text{pre}} = [1, 0, 0, 0]$, $F(z) = 3z$, and $H^{\text{post}} = [1, 0.5, 0, -1]$. Compute x_{l+1} . How many times is F evaluated?

Answer

The read gives $z = 1$. One evaluation gives $y = F(1) = 3$. The write is $[3, 1.5, 0, -3]^{\top}$. Adding it to the persistent result $[2, 1, 6, 3.75]^{\top}$ gives

$$x_{l+1} = [5, 2.5, 6, 0.75]^{\top}.$$

F is evaluated once, not four times.

5. Catch the wrong program

A programmer writes:

```
y = stack([F(x[:, i, :, :]) for i in 1:d]; dims=2)
x_next = persisted .+ reshape(h_post, size(h_post)..., 1, 1) .* y
```

What is computationally wrong, and what is semantically wrong, with the central equation? After `persisted` is already in hand, what are the correct two lines?

Answer

The loop evaluates F d times on separate streams. The equation first forms one routed mixture and evaluates F once, so the wrong program computes a different function and multiplies branch cost by roughly d . Correct code is:

```
z = dropdims(sum(reshape(h_pre, size(h_pre)..., 1, 1) .* x; dims=2); dims=2)
y = F(z)
x_next = persisted .+ reshape(h_post, size(h_post)..., 1, 1) .*
    reshape(y, size(y, 1), 1, size(y, 2), size(y, 3))
```

Check yourself

Read the central line from the inside out:

$$\underbrace{x_{l+1}}_{b \times d \times m \times n} = \underbrace{H_l^{\text{res}} x_l}_{\text{persist: } b \times d \times m \times n} + \underbrace{(H_l^{\text{post}})^\top}_{\text{write: } d \times 1} \underbrace{F\left(\underbrace{H_l^{\text{pre}} x_l, W_l}_{\text{read: } b \times m \times n}\right)}_{\text{one compute result: } b \times m \times n}.$$

1. **Show that the read removes the stream axis.**

H_l^{pre} is $1 \times d$ (per sentence). It contracts $x_l \in \mathbb{R}^{b \times d \times m \times n}$ to $z_l \in \mathbb{R}^{b \times m \times n}$. That is the shape F sees.

2. **What fails if you call F once per stream?**

You evaluate a different function and multiply branch cost by about d . The equation forms one mixture first.

3. **Show that y_l cannot be added to $H_l^{\text{res}} x_l$ until the write.**

y_l is $b \times m \times n$. The leftover remix is $b \times d \times m \times n$. The write restores the stream axis.

4. **Does the leftover map look at y_l ?**

No. It remixes x_l only.

5. **What fails if you drop the last addition?**

One path replaces the other. Old state and new computation are no longer both available.

If those five sentences are clear, the equation is no longer hiding anything essential.

At the next layer, the four tables in our numerical x_{l+1} become the four leftover inputs. A new read can blend them differently. A new branch can propose another edit. New write numbers can make the drawers diverge further. Chapter 5 follows that recurrence across four layers. The verbs stay the same. The layer index will stop sitting quietly in the corner.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” *arXiv:2409.19606*, 2024; revised 2025. <https://arxiv.org/abs/2409.19606>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” *arXiv:2512.24880*, 2025; revised 2026. <https://arxiv.org/abs/2512.24880>
- [R006] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of CVPR*, 2016, pp. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC,” *arXiv:2604.02309v1*, 2026. <https://arxiv.org/abs/2604.02309>
- [R010] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems* 30, 2017. <https://arxiv.org/abs/1706.03762>

Chapter 5 — Four notebooks through four stations

Four notebooks lie open. The number 2 is written in each of them.

Those four 2s are leftover copies of one one-word sentence, not four different words and not the last four answers a network produced. We will send the same four pages through four stations. At a station we form one peek, compute once, write unequal shares of that one answer, and maybe reorder the pages. We never grow a fifth page.

When we stop, the pages hold $15/2$, 8, 6, and $9/2$. Each of those numbers is a sum. That is the fact this chapter exists to make unavoidable.

Four copies of two

One token. One feature. Four leftover copies. Every number we write is therefore a single scalar, but it still lives on the leftover-copy axis.

Display the four copies as a column, only to save ink:

$$x_0 = \begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}.$$

The two missing axes are still there, both of size one: one token, one feature. In a real sentence each entry would be a whole table of tokens and features. Mixing leftover copies would still not mix word positions.

The true shape is $x_0 \in \mathbb{R}^{4 \times 1 \times 1}$. We will keep writing the short column. Do not let the shorthand talk you into treating stream, token, and feature as the same axis that got lucky.

If those three axes were interchangeable, a mix of leftover copies would also mix word positions. It does not. The column is a view, not a license.

The first write that is not fair

At station 0 we peek at all four notebooks equally:

$$q_0 = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{bmatrix}.$$

The peek is

$$z_0 = \frac{1}{4}(2 + 2 + 2 + 2) = 2.$$

The station's one calculation is “do nothing”: $F_0(z) = z$. So the answer is $y_0 = 2$.

We write the whole answer into notebook 1, half of it into notebook 3, and nothing into the other two:

$$p_0 = \begin{bmatrix} 1 \\ 0 \\ \frac{1}{2} \\ 0 \end{bmatrix}, \quad p_0 y_0 = \begin{bmatrix} 2 \\ 0 \\ 1 \\ 0 \end{bmatrix}.$$

Meanwhile the old numbers stay put. Adding the two routes:

$$x_1 = \begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \boxed{\begin{bmatrix} 4 \\ 2 \\ 3 \\ 2 \end{bmatrix}}.$$

That is the whole station: shuffle the old notebooks (here, not at all), write different amounts of the one new answer, add. The compact rule is

$$x_{l+1} = R_l x_l + p_l y_l. \quad (5.1)$$

Here R_0 is the 4×4 identity, so $R_0 x_0 = x_0$. The shapes fit because

$$(4 \times 4)(4 \times 1) + (4 \times 1)(1 \times 1) = 4 \times 1.$$

There is still only one evaluation of F_0 . The four numbers in p_0 are how that one answer is shared.

Hence x_1 differs from x_0 only by an unequal write. Suppose instead every peek whose four numbers add to one. Then, because the notebooks are still identical, every such peek would have read 2. A peek cannot prefer useful content that is not yet different. Drop the unfair write and the tie never breaks: notebook 1 now holds 4, notebook 3 holds 3, and the other two still hold 2.

Now one notebook is actually different

Station 1 can finally pick a favorite. Peek only at notebook 1, which now holds 4:

$$q_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}.$$

This station halves what it sees: $F_1(z) = z/2$. So

$$z_1 = 4, \quad y_1 = 2.$$

Write the whole answer into notebook 2 and half of it into notebook 4:

$$p_1 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \frac{1}{2} \end{bmatrix}, \quad p_1 y_1 = \begin{bmatrix} 0 \\ 2 \\ 0 \\ 1 \end{bmatrix}.$$

At the same time, swap notebooks 1 and 2, and leave the other two alone:

$$R_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_1 x_1 = \begin{bmatrix} 2 \\ 4 \\ 3 \\ 2 \end{bmatrix}.$$

That matrix has a single 1 in every row and every column, and zeros elsewhere. Multiplying by it only reorders the notebooks. It does not change their values. That kind of matrix is a **permutation**.

Add the two routes:

$$x_2 = \begin{bmatrix} 2 \\ 4 \\ 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 2 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 6 \\ 3 \\ 3 \end{bmatrix}.$$

Notice the two independent decisions. R_1 sends the old value 4 from notebook 1 to notebook 2. Then p_1 adds the new value 2 to that same seat. Matrix addition is doing the bookkeeping. A hand-drawn arrow diagram is welcome to try.

Hence the leftover shuffle and the write can land on the same notebook without being the same decision. All four leftover-copy shuffles in this chapter will be permutations, so the arithmetic stays transparent. Drop that restriction and R_l can stretch or cancel values as it reorders them; the routes are still not trivial even when it does not.

Mix two, then cycle four

Station 2 peeks at notebooks 1 and 3 as they sit now, half and half:

$$q_2 = \begin{bmatrix} \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix}.$$

Those current values are 2 and 3. Double what you see: $F_2(z) = 2z$. Then

$$z_2 = \frac{1}{2}(2) + \frac{1}{2}(3) = \frac{5}{2}, \quad y_2 = 5.$$

Write none of that 5 into notebook 1, one fifth into notebook 2, two fifths into notebook 3, three fifths into notebook 4:

$$p_2 = \begin{bmatrix} 0 \\ -\frac{1}{5} \\ \frac{2}{5} \\ \frac{3}{5} \end{bmatrix}, \quad p_2 y_2 = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}.$$

Meanwhile cycle the old notebooks: the last becomes first, and the others shift down one:

$$R_2 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

In other words $R_2[v_1, v_2, v_3, v_4]^\top = [v_4, v_1, v_2, v_3]^\top$, so

$$R_2 x_2 = \begin{bmatrix} 3 \\ 2 \\ 6 \\ 3 \end{bmatrix}.$$

Add:

$$x_3 = \begin{bmatrix} 3 \\ 2 \\ 6 \\ 3 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix} = \boxed{\begin{bmatrix} 3 \\ 3 \\ 8 \\ 6 \end{bmatrix}}.$$

The peek used notebooks 1 and 3 as they existed in x_2 . It did not peek at the already-cycled R_2x_2 . Both terms on the right of (5.1) are computed from the incoming column.

Hence order is part of the definition, not a clerical preference. Suppose the peek were taken after the cycle. Then q_2 would see 3 and 6, not 2 and 3, and y_2 would be a different number. The two summands in (5.1) are parallel ingredients of the addition. They are not a pipeline.

Reverse the stack

Station 3 peeks at notebooks 2 and 4, half and half:

$$q_3 = \begin{bmatrix} 0 & \frac{1}{2} & 0 & \frac{1}{2} \end{bmatrix}.$$

Those current values are 3 and 6. Subtract $3/2$: $F_3(z) = z - \frac{3}{2}$. Then

$$z_3 = \frac{1}{2}(3) + \frac{1}{2}(6) = \frac{9}{2}, \quad y_3 = 3.$$

Write half of that 3 into notebooks 1 and 4, none into notebook 2, and the whole 3 into notebook 3:

$$p_3 = \begin{bmatrix} \frac{1}{2} \\ 0 \\ 1 \\ \frac{1}{2} \end{bmatrix}, \quad p_3 y_3 = \begin{bmatrix} \frac{3}{2} \\ 0 \\ 3 \\ \frac{3}{2} \end{bmatrix}.$$

Meanwhile reverse the stack:

$$R_3 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad R_3 x_3 = \begin{bmatrix} 6 \\ 8 \\ 3 \\ 3 \end{bmatrix}.$$

Finally

$$x_4 = \begin{bmatrix} 6 \\ 8 \\ 3 \\ 3 \end{bmatrix} + \begin{bmatrix} \frac{3}{2} \\ 0 \\ 3 \\ \frac{3}{2} \end{bmatrix} = \boxed{\begin{bmatrix} \frac{15}{2} \\ 8 \\ 6 \\ \frac{9}{2} \end{bmatrix}}.$$

The whole walk is therefore

$$\begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix} \rightarrow \begin{bmatrix} 4 \\ 2 \\ 3 \\ 2 \end{bmatrix} \rightarrow \begin{bmatrix} 2 \\ 6 \\ 3 \\ 3 \end{bmatrix} \rightarrow \begin{bmatrix} 3 \\ 3 \\ 8 \\ 6 \end{bmatrix} \rightarrow \boxed{\begin{bmatrix} \frac{15}{2} \\ 8 \\ 6 \\ \frac{9}{2} \end{bmatrix}}.$$

Four leftover copies started identical and did not stay that way. That is the whole point of an unfair write.

If F_3 had been the identity, y_3 would have been $9/2$ rather than 3 , and none of those four finals would match. The walk is not a property of the shuffles alone.

Where each number came from

Notebook 1 now holds $15/2$. Who paid?

We can chase each injection forward through every later shuffle. The original four 2s never change value under a permutation, so they arrive still uniform:

$$R_3 R_2 R_1 R_0 x_0 = \begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}.$$

The station-0 write began as $p_0 y_0 = [2, 0, 1, 0]^\top$. Then

$$\begin{bmatrix} 2 \\ 0 \\ 1 \\ 0 \end{bmatrix} \xrightarrow{R_1} \begin{bmatrix} 0 \\ 2 \\ 1 \\ 0 \end{bmatrix} \xrightarrow{R_2} \begin{bmatrix} 0 \\ 0 \\ 2 \\ 1 \end{bmatrix} \xrightarrow{R_3} \begin{bmatrix} 1 \\ 2 \\ 0 \\ 0 \end{bmatrix}.$$

The station-1 write began as $p_1 y_1 = [0, 2, 0, 1]^\top$:

$$\begin{bmatrix} 0 \\ 2 \\ 0 \\ 1 \end{bmatrix} \xrightarrow{R_2} \begin{bmatrix} 1 \\ 0 \\ 2 \\ 0 \end{bmatrix} \xrightarrow{R_3} \begin{bmatrix} 0 \\ 2 \\ 0 \\ 1 \end{bmatrix}.$$

The station-2 write began as $p_2 y_2 = [0, 1, 2, 3]^\top$:

$$\begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix} \xrightarrow{R_3} \begin{bmatrix} 3 \\ 2 \\ 1 \\ 0 \end{bmatrix}.$$

The station-3 write lands where it is written. There is no later shuffle:

$$p_3 y_3 = \begin{bmatrix} \frac{3}{2} \\ 0 \\ 3 \\ \frac{3}{2} \end{bmatrix}.$$

Add the five origins:

$$\begin{aligned} x_4 &= \underbrace{\begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}}_{x_0} + \underbrace{\begin{bmatrix} 1 \\ 2 \\ 0 \\ 0 \end{bmatrix}}_{y_0} + \underbrace{\begin{bmatrix} 0 \\ 2 \\ 0 \\ 1 \end{bmatrix}}_{y_1} + \underbrace{\begin{bmatrix} 3 \\ 2 \\ 1 \\ 0 \end{bmatrix}}_{y_2} + \underbrace{\begin{bmatrix} \frac{3}{2} \\ 0 \\ 3 \\ \frac{3}{2} \end{bmatrix}}_{y_3} \\ &= \begin{bmatrix} 15 \\ 2 \\ 8 \\ 6 \\ \frac{9}{2} \end{bmatrix}. \end{aligned}$$

That is also a destination-by-origin ledger:

final notebook	from x_0	from y_0	from y_1	from y_2	from y_3	total
1	2	1	0	3	3/2	15/2
2	2	2	2	2	0	8
3	2	0	0	1	3	6
4	2	0	1	0	3/2	9/2

Each row is a sum of leftovers from several depths. That is the fact that kills the sliding-window picture. Notebook 2 is not “whatever station 2 said.” It holds 2 from the start, 2 from station 0, 2 from station 1, and 2 from station 2.

The compact way to write the chase we just did is to substitute (5.1) into itself:

$$\begin{aligned} x_1 &= R_0 x_0 + p_0 y_0, \\ x_2 &= R_1 R_0 x_0 + R_1 p_0 y_0 + p_1 y_1, \\ x_3 &= R_2 R_1 R_0 x_0 + R_2 R_1 p_0 y_0 + R_2 p_1 y_1 + p_2 y_2. \end{aligned}$$

One more substitution is the four-station expansion:

$$x_4 = R_3 R_2 R_1 R_0 x_0 + R_3 R_2 R_1 p_0 y_0 + R_3 R_2 p_1 y_1 + R_3 p_2 y_2 + p_3 y_3. \quad (5.2)$$

Every term is in $\mathbb{R}^{4 \times 1}$. A contribution written at station i is multiplied by every later leftover-copy shuffle, right to left, oldest shuffle first. The newest write $p_3 y_3$ has no later shuffle. The initial column crosses all four.

Follow one number through that product. Station 0 wrote coefficient 1 into notebook 1. That piece then travels

$$1 \xrightarrow{R_1} 2 \xrightarrow{R_2} 3 \xrightarrow{R_3} 2.$$

So $y_0 = 2$ contributes 2 to final notebook 2, which is the ledger entry 2 in that cell. The half-strength station-0 write started in notebook 3 and ends in final notebook 1. A learned route is not the slogan “layer 0 connects to layer 4.” It names an entry strength and every intervening seat change.

That chain of seats is a **path**. The name is worth keeping because the write chooses where an answer first enters, and the later R matrices choose where that leftover travels. In this chapter each path is a single wire: a permutation keeps exactly one continuation alive from any starting notebook.

With denser shuffles a write can split at every later station, so “path” means a weighted family of routes rather than one wire. Let j_i be a notebook that p_i first writes into, and let $j_{i+1}, \dots, j_3$ be later seats. One individual route into final notebook a has coefficient

$$(R_3)_{aj_3} (R_2)_{j_3 j_2} \cdots (R_{i+1})_{j_{i+1} j_i} (p_i)_{j_i}.$$

The total coefficient is the sum of this product over all intermediate seats. That sum is exactly the a -th entry of $R_3 \cdots R_{i+1} p_i$. In our permutation example each product is either zero or the original write coefficient. With dense routers several products survive and add.

Equation (5.2) groups those already-computed answers $y_0, \dots, y_3$. It does not claim the answers are independent. Each later peek is $y_i = F_i(q_i x_i)$, so a later calculation can depend on earlier writes. If some F_i is nonlinear, you generally cannot expand that dependence into a linear sum of the earlier y ’s. The leftover-copy ledger remains exact anyway: it says where each already-computed answer was injected and how it was shuffled.

A gradient walks the same graph backwards. These forward routes are therefore also routes by which a later loss can reach earlier notebooks and earlier answers. That is a fact about the wiring. It is not a promise that every route has a healthy size.

The later papers write this same read–compute–write recurrence and then expand it across depth [R002]. Reading the expansion is useful. Executing one is what makes the indices stop being decorative.

Hence (5.2) is an identity about already-computed injections, not a claim that the four answers are free variables. Drop the later shuffles from an early write and you misplace that write: y_0 would not land as $[1, 2, 0, 0]^\top$.

When the swaps become splits

Every R_l in this chapter is a permutation, so every entry is nonnegative and every row and every column adds to one. Any matrix with those two properties is **doubly stochastic**.

Hyper-Connections allow a learned leftover-copy shuffle R_l with no such promise [R001]. mHC keeps the same recurrence and asks R_l to stay inside the set of nonnegative doubly stochastic matrices — people call that set the Birkhoff polytope [R002]. go-mHC keeps the same constraint and proposes a different exact way to build those matrices [R005]. These are recent architecture proposals, not settled design law.

Our four permutations already satisfy the constraint. That was a teaching choice: permutation routes make the fate of each leftover visible. It is not a claim that trained routers usually become exact permutations, nor that every parameterization reaches every boundary point in the same way.

A soft doubly stochastic shuffle replaces each crisp arrow above with conservative splitting and recombination across notebooks. The algebraic expansion (5.2) does not change. Only the allowed coefficients do.

A trained model might use different notebooks to keep or combine leftovers from different depths. The Hyper-Connections paper reports pictures of learned connection patterns [R001]. Calling our four seats “specialized streams” is a story we told about this trace. It is not a measurement unless a particular trained model is probed.

If we drop nonnegativity, a leftover can cancel another leftover instead of only sharing it. If we drop the unit row-and-column sums, a shuffle can create or destroy aggregate leftover mass. The walk we just did would still be defined. It would no longer be a conservative split.

What we would lose

If every write is fair, identical notebooks can stay identical under a symmetric shuffle. Four copies then store no more variety than one copy. Station 1’s proud peek at notebook 1 would still see the same number as the other three.

If we freeze the peek, every station receives a fixed blend. Once the notebooks diverge, the block cannot change which leftover it consumes.

If we freeze the write, every answer enters the same seats in the same amounts. The model loses control over where new work lands.

If we freeze the leftover-copy shuffle, old contents cannot change seats between stations. Writes may still differ, but the wiring of old leftovers becomes much thinner.

If we never unroll (5.1), we miss the central fact: an early write is acted on by every later shuffle. A list of four local updates does not by itself explain why y_0 reaches final notebooks 1 and 2 while y_2 reaches final notebooks 1, 2, and 3 in this trace.

Ways to get this wrong

Multiplying in the wrong order is the classic one. A station-0 write reaches station 4 as $R_3 R_2 R_1 p_0 y_0$, because R_1 acts first. Matrix products read right to left.

- Calling the notebooks “the last four layers.” The ledger is four sums, not a sliding window.
- Peeking at $R_l x_l$. The branch input is $q_l x_l$. The shuffle and the peek are parallel ingredients of the final addition.

- Running F_l four times. Here $q_l x_l$ produces one input, hence one y_l . The four-vector p_l shares it.
- Forgetting the transpose on the write. The write row is 1×4 ; standing it up makes a 4×1 column that can land in four notebooks.
- Treating every peek as a probability distribution without looking. Our q_l rows happen to be nonnegative and add to one. The core equation alone does not force that.
- Reading (5.2) as a claim that the four answers are independent. Later y_l values depend on earlier columns. The expansion groups injection origins. It does not cut the computational graph.
- Dropping the singleton axes in your head. The toy column is shorthand for $4 \times 1 \times 1$, not evidence that stream, token, and feature are interchangeable.

Try it

1. A different peek

At the entrance to station 2 the notebooks hold $[2, 6, 3, 3]^\top$. If we peeked with $q_2 = [0, \frac{1}{4}, \frac{3}{4}, 0]$ instead, what would z_2 be? Keep $F_2(z) = 2z$ and find y_2 .

Answer

$$z_2 = \frac{1}{4}(6) + \frac{3}{4}(3) = \frac{15}{4}. \text{ Therefore } y_2 = 2z_2 = \frac{15}{2}.$$

2. A fair write that fails

Keep station 0 the same except write equally: $p_0 = [\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}]^\top$. What is x_1 ? When station 1 peeks at only notebook 1 with $q_1 = [1, 0, 0, 0]$, is it seeing something the others do not have?

Answer

Since $y_0 = 2$, the write is $[1, 1, 1, 1]^\top$. With $R_0 = I$, $x_1 = [3, 3, 3, 3]^\top$. The peek selects notebook 1, but its content is not unique; every notebook contains 3.

3. Chase one 1

Suppose a hypothetical station-1 write is $e_2 = [0, 1, 0, 0]^\top$. Using this chapter's R_2 and R_3 , calculate $R_3 R_2 e_2$. Which final notebook receives it?

Answer

$R_2 e_2 = e_3 = [0, 0, 1, 0]^\top$, then $R_3 e_3 = e_2 = [0, 1, 0, 0]^\top$. It ends in final notebook 2. The intermediate and final seat numbers happen to differ and then return; calculate, do not guess from the endpoints.

4. A missing shuffle

A classmate writes

$$x_4 = R_3 R_2 R_1 R_0 x_0 + R_3 R_2 p_0 y_0 + R_3 R_2 p_1 y_1 + R_3 p_2 y_2 + p_3 y_3.$$

What did they drop? Repair the term.

Answer

The station-0 write must cross R_1 , R_2 , and R_3 . Its term is $R_3 R_2 R_1 p_0 y_0$. All other terms shown are correct.

5. Any number of stations

For L stations numbered 0 through $L-1$, write x_L as the initial notebooks plus every write. What is the empty product when a write has no later shuffle?

Answer

With later station indices placed on the left,

$$x_L = R_{L-1} \cdots R_0 x_0 + \sum_{i=0}^{L-1} (R_{L-1} \cdots R_{i+1}) p_i y_i.$$

For $i = L - 1$, the product contains no matrices and is defined as the identity I_4 , leaving $p_{L-1}y_{L-1}$.

Check yourself

1. **Show that x_1 is no longer $[2, 2, 2, 2]^\top$.** p_0 wrote 2 into notebook 1 and 1 into notebook 3. The first unfair write is what makes a later peek able to prefer anything.
2. **What fails if station 2 peeks at the already-cycled notebooks?** It would compute $q_2(R_2x_2)$ instead of q_2x_2 . The two summands in (5.1) are parallel. They are not a pipeline.
3. **How many times does each station run F_l ?** Once. Four write numbers share that one answer.
4. **Show that y_0 contributes $[1, 2, 0, 0]^\top$ to x_4 .** It is written as $[2, 0, 1, 0]^\top$, then shuffled by R_1 , R_2 , and R_3 . That answer names both the injection and the path.
5. **What fails if the four notebooks are called the last four station outputs?** The ledger is four sums. Notebook 2's 8 is $2 + 2 + 2 + 2 + 0$ from five different origins.

We picked every peek, write, and shuffle by hand. Next we stop doing that. Whatever mechanism produces q_l , p_l , and R_l , the resulting numbers still have to survive this same walk.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024; revised 2025. <https://arxiv.org/abs/2409.19606>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025; revised 2026. <https://arxiv.org/abs/2512.24880>
- [R006] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *CVPR*, 2016, pp. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>

Chapter 6 — A small opinion about the notebooks

For four chapters we mixed the notebooks ourselves. We wrote a read row, a write column, and a leftover shuffle, then watched those numbers travel through a stack. That was the right way to learn the arithmetic. It is a terrible way to run a model.

Nobody is going to sit inside layer 47 with a pencil. The network needs a cheap opinion about the mix — an opinion that is allowed to change when the page changes.

Look at the notebooks at this position. Squeeze them into a short list. Multiply by a learned matrix. You now have a handful of scores. Those scores are not yet a legal read, a legal write, or a legal leftover mix. A second machine, usually with no extra learned weights, turns the scores into operators the rest of the layer is allowed to use.

That whole front end is the **router**. It is a tiny linear layer plus a construction that refuses illegal matrices. It is not a second attention head.

The finished pipeline is four arrows:

$$\boxed{x_l \longrightarrow \hat{x}_l \longrightarrow u_l \longrightarrow H_l}.$$

Activation. Normalize and shrink. Learned projection. Then a map that is allowed to produce an operator. Keeping the last two arrows separate is the whole lesson of this chapter.

The pencil and the rulebook

Until now the routing numbers arrived by hand. A real model has to obtain them somehow.

If the numbers are just stored parameters, every sentence at that layer follows the same mix. If they depend on the current activation, the mix can change with the input. That second choice is the “dynamic” in dynamic Hyper-Connections.

A router is easy to describe as one blob. It is three choices:

1. what summary of the page it is allowed to see;
2. how learned numbers turn that summary into scores;
3. how those scores become a permitted operator.

The papers in this family use a small linear layer for step 2, not an obligatory deep network [R001]; [R002]; [R003]; [R004]; [R005]. After that layer you may see tanh, sigmoid, softmax, a few Sinkhorn steps, a Kronecker product, or a Cayley construction. That later work belongs to the rulebook. It does not secretly turn the linear layer into a deep predictor.

Call the scores **coordinates**. Coordinates describe a construction. They are not automatically probabilities, and they are not automatically the entries of H .

For a layer index l , write

- B for how many sentences ride together;
- T for how many tokens sit in one sentence;
- d for how many notebooks we carry;
- C for how wide each notebook is — the feature count we used to call n ;
- $x_l \in \mathbb{R}^{B \times T \times d \times C}$ for the layer input;
- G for how many separate routing decisions we will make;
- r for how wide the squeezed summary is.

A general router first builds

$$\hat{x}_l = R(N(x_l)) \in \mathbb{R}^{G \times r}.$$

N is a normalization. R is a reshape or a reduction. “Compact” means compact relative to the full state the expensive branch will see, not “a single scalar.” For one decision per token, $G = B \times T$, and a common squeeze flattens the notebook axis with the feature axis, so $r = dC$. For one decision per sentence, R would have to reduce the T axis, for example by pooling. For a static layer-level router there need not be an activation input at all.

Those are different architectures. A silent mean over the sentence is not clerical tidying. It changes who gets their own mix.

Hence a router is not one matrix. It is a summary, a learned score, and a license. Drop the third choice and the linear output is treated as a finished operator. Drop the first and “dynamic” has nothing to look at.

Squeeze the notebooks first

A layer whose features have grown large should not automatically shout louder about routing. The HC papers normalize before the dynamic linear step, and the mHC-family papers use RMSNorm on the feature list they feed the router [R001]; [R002]; [R003]; [R004]; [R005].

RMSNorm divides by a root-mean-square along a specified last axis [R013]. The axis is the whole point. Normalize the wrong axis and you have averaged the wrong story.

For a token-local squeeze that only glues notebooks to features, one explicit shape ledger is

$$\begin{aligned} x_l &\in \mathbb{R}^{B \times T \times d \times C}, \\ \text{flatten}_{d,C}(x_l) &\in \mathbb{R}^{B \times T \times dC}, \\ \hat{x}_l &= \text{RMSNorm}(\text{flatten}_{d,C}(x_l)) \in \mathbb{R}^{B \times T \times dC}. \end{aligned}$$

This flattening does not average tokens together. It makes one feature vector from the d notebook vectors that share a batch index and a token index. Conversely,

$$\hat{x}_l = \frac{1}{T} \sum_{t=1}^T N(x_{l, :, t, :, :})$$

would be a sentence-level reduction. It would produce a different router. Neither choice is universally correct. Only an implementation can settle which one it uses.

Hence the squeeze decides the routing grain before any learned matrix speaks. Suppose we flattened T with d and C . Then one sentence would own one mix, and two tokens in that sentence could no longer disagree. The arithmetic of the later multiply would still run. The architecture would not be the same.

A linear guess

Now the squeezed list has to become scores. For one operator type $a \in \{\text{pre}, \text{post}, \text{res}\}$, keep four objects:

- $W_l^a \in \mathbb{R}^{r \times q_a}$, a matrix of learned weights;

- $b_l^a \in \mathbb{R}^{q_a}$, a bias;
- $\alpha_l^a \in \mathbb{R}$, a learned loudness;
- $u_l^a \in \mathbb{R}^{G \times q_a}$, the coordinates.

The linear step is

$$u_l^a = \alpha_l^a \hat{x}_l W_l^a + b_l^a.$$

That multiply is the **projection**. It is a map from the squeezed width r onto a coordinate width q_a . It is not yet a routing matrix.

Some HC writeups wrap the projected activation in $\tanh$ before the small loudness and a static term [R001]; [R002]. That changes derivatives and bounds the dynamic correction. It does not merge the pencil with the rulebook.

Read and write routers often need $q_{\text{pre}} = q_{\text{post}} = d$: one number per notebook. Residual routers differ by variant:

- HC can form unconstrained dynamic mixing coefficients directly [R001].
- mHC produces candidate residual logits and then runs a finite Sinkhorn–Knopp cleanup [R002].
- mHC-lite produces $d!$ mixture logits, softmaxes them, and mixes all $d \times d$ permutations [R003].
- KromHC emits mixture coordinates for smaller permutation families, builds doubly stochastic factors, and takes their Kronecker product [R004].
- go-mHC emits $ds(ds-1)/2$ coordinates for a skew-symmetric $ds \times ds$ matrix before its Cayley and block-norm construction [R005].

Those are architectural reports from recent papers, not a ranking. The coordinate width need not equal the number of entries in H_l^a . go-mHC with $N = ds$ emits $q_{\text{res}} = N(N-1)/2$ skew coordinates and still returns a $d \times d$ leftover matrix [R005].

Hence the projection aims. It does not license. Suppose q_a equalled d^2 and someone reshaped u into a $d \times d$ table. That table would still be a proposal. Rows might fail to sum to one. Entries might be negative. The next section is what refuses those proposals.

Two scores that are not yet a matrix

Take one routing group, two notebooks, and two features each: $G = 1$, $d = 2$, and $C = 2$. After normalization and flattening, suppose

$$\hat{x} = [1, -1, 0.5, -0.5] \in \mathbb{R}^{1 \times 4}.$$

We want two coordinates, so choose

$$W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ -1 & 1 \end{pmatrix} \in \mathbb{R}^{4 \times 2}, \quad \alpha = 0.1, \quad b = [0, 0] \in \mathbb{R}^2.$$

The projection gives

$$\hat{x}W = [2, -1], \quad u = 0.1[2, -1] = [0.2, -0.1] \in \mathbb{R}^{1 \times 2}.$$

These two numbers are not a 2×2 leftover matrix. Use the mHC-lite-style rulebook for $d = 2$. There are two permutation matrices,

$$P_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad P_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Softmax turns the coordinates into mixing weights:

$$a = \text{softmax}(u) \approx [0.5744, 0.4256].$$

The rulebook then builds

$$H^{\text{res}} := a_1 P_1 + a_2 P_2 = \begin{pmatrix} 0.5744 & 0.4256 \\ 0.4256 & 0.5744 \end{pmatrix}.$$

Every entry is nonnegative. Every row and every column sums to one. Those facts come from softmax plus the permutation mixture, not from the linear multiply.

If the two notebook vectors are $x^{(1)} = [2, 0]$ and $x^{(2)} = [0, 4]$, both in $\mathbb{R}^2$, leftover routing gives

$$H^{\text{res}} \begin{bmatrix} x^{(1)} \\ x^{(2)} \end{bmatrix} = \begin{bmatrix} [1.1489, 1.7022] \\ [0.8511, 2.2978] \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Change W , b , or α , and the proposal changes. Keep the same rulebook, and the row-and-column sums stay exact.

Hence the linear multiply did not produce the doubly stochastic matrix. Softmax and the two permutations did. If we treated $u = [0.2, -0.1]$ itself as a leftover mix, we would not even have four entries, let alone unit row and column sums.

The rulebook

The projection has spoken. Something still has to decide which operators exist.

Let $\mathcal{C}_a$ be the variant-specific map. Then

$$H_l^a = \mathcal{C}_a(u_l^a).$$

That map is the **constraint map**. The name is worth keeping because it is a separate machine from W . The projection learns where to aim. The constraint map decides which targets exist.

The mHC paper maps candidate read and write scores with

$$H_l^{\text{pre}} = \sigma(\tilde{H}_l^{\text{pre}}), \quad H_l^{\text{post}} = 2\sigma(\tilde{H}_l^{\text{post}}),$$

and maps the candidate leftover matrix with a finite number of Sinkhorn–Knopp iterations [R002]. Sigmoid puts each read number in $(0, 1)$. Twice-sigmoid puts each write number in $(0, 2)$. Those maps do not make the read or write vectors into probability distributions. Their entries need not sum to one.

The leftover operator has a different contract in each variant. Finite Sinkhorn aims at a matrix whose rows and columns sum to one, but a finite loop leaves a little numerical dirt. Permutation mixtures and KromHC are exact by construction in exact arithmetic. go-mHC maps skew coordinates to an orthogonal matrix with a Cayley transform, cuts that matrix into $s \times s$ blocks, and sets each routing entry to a scaled squared block size; that construction proves nonnegativity and unit row and column sums [R005]. Later chapters derive each map. Here the important fact is that none of them is “the linear layer.”

Hence $H = \mathcal{C}(u)$ is a function of coordinates, not a reshape of them. Suppose we skipped $\mathcal{C}$ and used the leftover coordinates as matrix entries. Then nothing would force nonnegativity or unit sums, and Chapter 7 would apply immediately. The constraint map can have no learned parameters and still be the whole difference between a legal mixer and an arbitrary one.

Teaching the pencil

Training has to move both stages, even when only the first stage owns parameters.

Let $\mathcal{L}$ be the training loss and drop the layer and operator labels for a minute. Define

$$g_u = \frac{\partial \mathcal{L}}{\partial u} = J_{\mathcal{C}}(u)^\top \frac{\partial \mathcal{L}}{\partial H},$$

where $J_{\mathcal{C}}(u)$ is the Jacobian of the constraint map — the grid of how each output entry moves when each coordinate moves. This grid is the bridge from “which routing matrix would reduce the loss?” to “how should the coordinates move?”

For $u = \alpha \hat{x}W + b$, summing over routing groups,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W} &= \alpha \hat{x}^\top g_u, & \frac{\partial \mathcal{L}}{\partial b} &= \sum_G g_u, \\ \frac{\partial \mathcal{L}}{\partial \alpha} &= \sum_G \langle g_u, \hat{x}W \rangle, & \frac{\partial \mathcal{L}}{\partial \hat{x}} &= \alpha g_u W^\top. \end{aligned}$$

Backpropagation therefore trains W , b , and α , and also sends a gradient through the normalization into the current activation. The map itself may have no learned parameters and still affect every upstream gradient. Softmax, sigmoid, a fixed number of Sinkhorn steps, linear solves in a Cayley transform, and squared block sizes are differentiable on their regular domains.

Constraint maps can also muffle gradients. Saturated sigmoid or softmax coordinates have small derivatives. A finite Sinkhorn loop has a long normalization Jacobian. The Cayley route differentiates through a solve. “Exact constraint” describes the forward output. It does not automatically mean easy optimization.

Hence the constraint map is on the backward path even when it owns no weights. If $J_{\mathcal{C}}$ vanishes, g_u vanishes, and the projection cannot move. Exactness of H and usefulness of $\partial H / \partial u$ are different properties.

Start almost idle

The dynamic correction should not vandalize the leftover highway before seeing one minibatch. The cited implementations therefore combine small loudness with biases or static terms chosen near ordinary residual behavior [R001]; [R002]; [R003]; [R004]; [R005].

HC initializes dynamic projection weights to zero and dynamic scales to 0.01, while keeping static routing terms [R001]. The mHC experiments report a gating-factor initialization of 0.01 [R002]. mHC-lite and KromHC initialize projection weights to zero, loudness to 0.01, and leftover biases so softmax weight piles onto identity permutations [R003]; [R004]. Finite logits still leak a little weight onto the alternatives, so “near identity” is the honest description.

That loudness number is the **gain**. It is not a third copy of the bias. Bias sets the input-independent default route. Gain controls how loudly the activation-dependent correction speaks. Projection weights decide what evidence changes that route.

Why not initialize both $W = 0$ and $\alpha = 0$? Look at the gradient above. $\partial \mathcal{L} / \partial W$ is multiplied by α . If both start at zero, W receives no first-step gradient. Worse, $\partial \mathcal{L} / \partial \alpha$ is zero while $W = 0$. A small nonzero gain lets the projection begin learning while keeping its first effect small.

Hence $\alpha = 0.01$ with $W = 0$ is a start that can move. Suppose both start at zero. Then the first minibatch cannot teach the projection anything through this path. The dead start is a dropped-hypothesis fact about those two gradients, not a claim that every other initialization is wise.

What the papers actually flatten

For one routing item, the mHC paper writes the notebook state as $x_l \in \mathbb{R}^{d \times C}$, then flattens it:

$$v_l = \text{vec}(x_l) \in \mathbb{R}^{1 \times dC}, \quad v'_l = \text{RMSNorm}(v_l).$$

The normalized dC -wide vector feeds three learned projections. The pre- and post-router projections use weights in $\mathbb{R}^{dC \times d}$. The leftover projection uses weights in $\mathbb{R}^{dC \times d^2}$ and then reshapes its output to $d \times d$. Sigmoid maps constrain the read and write numbers. Finite Sinkhorn constrains the leftover candidate [R002]. So the primary mHC construction does flatten the per-item notebook and feature axes. mHC-lite and KromHC keep this dC -wide routing input and change only the leftover constraint map [R003]; [R004]. Across the family, the trainable projection and the constraint map remain separate stages.

For go-mHC leftover routing, let s be the block size and $N = ds$. The pipeline is

$$\hat{x}_l \xrightarrow{\text{linear projection}} u_l \in \mathbb{R}^{N(N-1)/2} \xrightarrow{\text{fill skew } A_l} A_l \in \mathbb{R}^{N \times N} \xrightarrow{\text{Cayley}} Q_l \in O(N) \xrightarrow{\text{block norms}} H_l^{\text{res}} \in \mathbb{R}^{d \times d}.$$

The projection learns coordinates. Skew filling, the Cayley transform, and the block norms enforce the geometry. The Cayley transform does not “learn” anything by itself.

How fine is the decision? The papers and their public code do not all say the same thing, and one of them does not quite say.

- HC’s published Algorithm 2 uses a hyper-hidden tensor of shape $B \times L \times d \times C$. Its dynamic projections keep the B and L axes, so that language-model implementation predicts a mix per batch item and token position [R001].
- mHC discusses memory access per token, and its pictures average token-level leftover matrices over a chosen sentence. Its displayed single-item equations use $x_l \in \mathbb{R}^{d \times C}$, which matches token-level application in the reported language-model implementation [R002].
- mHC-lite explicitly calls its measured leftover matrices “token-level” and keeps the activation-dependent mHC structure while changing the leftover constraint map [R003].
- KromHC’s public implementation reshapes $(b \ s) \dots d$ to $b \dots s \ d$ and keeps every leading $\dots$ position through the projection. The published code is therefore per item at those leading positions, including token positions when the tensor is a sequence [R004].
- go-mHC writes the full state as $d \times B \times L \times D$, but its displayed leftover-router equation introduces a reduced $\hat{x}'_l$ without saying which axes were reduced [R005]. The paper supports “activation-dependent and layer-specific.” The equation alone does **not** justify a token-wise or sentence-wise label. A concrete implementation has to state the reduction first.

A layer index on the parameters means each layer owns a router. It does not mean there is only one routing decision per layer. Likewise, “dynamic” means input-dependent. It does not automatically mean token-wise.

Hence granularity is an axis claim, not a synonym. If a paper omits the reduction, do not invent one. Token-wise, sentence-wise, and layer-wise routers can share the same constraint map and still be different machines.

What we lose if we skip a piece

Skip the normalization, and router size can track activation size rather than useful direction. Deep layers may then change their mix merely because their features grew larger.

Skip the learned projection, and an activation-dependent router has no trainable way to pick coordinates. A fixed bias can still give a static mix. It cannot answer the page.

Skip the bias, and the model loses an easy input-independent default. Initialization must then force the desired baseline entirely through activation-dependent terms, which is fragile when those terms start near zero.

Skip the gain, and a randomly initialized or rapidly learned projection can shove the leftover path at full strength from the first update. A gain is not the only stabilizer, but it is a direct knob.

Skip the constraint map, and leftover coordinates become unconstrained mixing entries. Repeated unconstrained leftover matrices can grow or shrink both the forward signal and the backward gradient. Chapter 7 is that failure.

Conversely, skip the projection but keep the constraint map, and the matrix can be valid yet deaf — excellent paperwork, little intelligence.

Ways to get this wrong

- Calling coordinates probabilities. Logits become probabilities only after a map such as softmax.
- Calling read numbers a normalized distribution. Elementwise sigmoid bounds entries. It does not make their sum one.
- Treating b as noise. It encodes the static default route and often carries the near-identity start.
- Setting both gain and projection to zero. That can block the projection's first gradient.
- Flattening every axis. Flattening d and C per token is not the same as flattening T, d, C per sentence.
- Assuming a deep multilayer predictor. The cited router front ends are small linear projections. Later nonlinear constraints do not retroactively deepen that projection.
- Claiming token-wise routing from the word “dynamic.” Granularity needs explicit axes or implementation evidence.
- Confusing an exact constraint with exact identity. A matrix can satisfy the leftover contract exactly and still only sit near the identity at initialization.

Try it

1. Shape the projection

Let $B = 3$, $T = 8$, $d = 4$, and $C = 64$. A token-level router flattens only notebook and feature axes and emits leftover logits for a dense 4×4 candidate. Give the shapes of $\hat{x}$, W , b , and u .

Answer

Here $r = dC = 256$ and $q = d^2 = 16$. Thus $\hat{x} \in \mathbb{R}^{3 \times 8 \times 256}$, $W \in \mathbb{R}^{256 \times 16}$, $b \in \mathbb{R}^{16}$, and $u \in \mathbb{R}^{3 \times 8 \times 16}$.

2. Separate the guess from the license

Using the numerical example, replace $u = [0.2, -0.1]$ with $u = [5, -2]$. Does the projection output sum to one? Does the softmax output? Is the resulting permutation mixture doubly stochastic?

Answer

The coordinate sum is 3, so the raw projection is not normalized. Softmax produces positive weights summing to one (approximately $[0.9991, 0.0009]$). A convex combination of the two permutation matrices is doubly stochastic, independent of the raw coordinate sum.

3. Diagnose a dead start

Suppose $u = \alpha \hat{x}W + b$, with $W = 0$, $\alpha = 0$, and a loss gradient $g_u \neq 0$. Compute the first-step gradients for W and α .

Answer

$\partial \mathcal{L} / \partial W = \alpha \hat{x}^\top g_u = 0$. Also $\partial \mathcal{L} / \partial \alpha = \langle g_u, \hat{x}W \rangle = 0$. The dynamic path cannot start moving on that step. A small nonzero gain avoids this particular dead start.

4. Audit the axes

A function accepts x with shape $[B, T, d, C]$, computes $h = \text{rmsnorm}(x.\text{reshape}(B, T, d \times C))$, and applies $h @ W$. Is routing token-level, sequence-level, or layer-level? What single change would make it sequence-level?

Answer

The B and T axes survive, so it is token-level. Reducing over T , for example $h = h.\text{mean}(\text{dim}=1)$ to shape $[B, d \times C]$, would produce one routing feature per sequence. Whether mean pooling is a good design is a separate question.

5. Count go-mHC coordinates

For $d = 4$ notebooks and block size $s = 2$, find N , the number of skew coordinates, and the final leftover-matrix shape.

Answer

$N = ds = 8$. A real 8×8 skew-symmetric matrix has $8 \cdot 7/2 = 28$ independent coordinates. The constraint construction returns $H^{\text{res}} \in \mathbb{R}^{4 \times 4}$.

Check yourself

1. **Show that the last two arrows of the pipeline are different machines.** The projection $u = \alpha \hat{x}W + b$ learned the aim. The constraint map $H = \mathcal{C}(u)$ issued the license. Coordinates need not be matrix entries.
2. **What fails if “dynamic router” is taken to mean every token gets its own mix?** Dynamic means input-dependent. Token-wise, sentence-wise, and layer-wise are extra claims. They need axes or code.
3. **What fails if both α and W start at zero?** $\partial \mathcal{L} / \partial W$ is multiplied by α , and $\partial \mathcal{L} / \partial \alpha$ vanishes while $W = 0$. The pencil cannot start moving on that path.
4. **After sigmoid, a read row has entries in $(0, 1)$. Is it a probability distribution?** Not unless the entries happen to sum to one. Sigmoid bounds. It does not normalize.
5. **Show that go-mHC’s $N(N - 1)/2$ numbers are not flattened leftover entries.** They fill a skew matrix that a later construction turns into a $d \times d$ matrix H^{res} .

The compact picture is

normalized activation $\rightarrow$ linear coordinates $\rightarrow$ valid routing operator.

Why bother with the last arrow? Leftover routing is applied again and again through depth. Even a modest growth factor compounds: a direction multiplied by 1.1 across 100 layers grows by 1.1^{100} . That is less a highway and more a launch vehicle. Chapter 7 is that calculation.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606v3, 2025. <https://arxiv.org/abs/2409.19606>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880v2, 2026. <https://arxiv.org/abs/2512.24880>
- [R003] Y. Yang and J. Gao, “mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations,” arXiv:2601.05732, 2026. <https://arxiv.org/abs/2601.05732>
- [R004] W. Zhou et al., “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices,” arXiv:2601.21579v2, 2026. <https://arxiv.org/abs/2601.21579>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R013] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” *NeurIPS 2019*, arXiv:1910.07467. <https://arxiv.org/abs/1910.07467>

Chapter 7 — Stretching a number until it explodes

Take the number 1.1. Multiply it by itself. Do it again. After a hundred multiplications you have about 13,780. A ten-percent raise looks polite at one layer. Depth is an excellent accountant and a terrible negotiator. That is the whole chapter in one number. Later we will mix leftover copies of a sentence with a matrix, then do it again, then again. The matrix can stretch some directions and squash others. Before we let it, we need to see what repeated stretching does.

One number, many layers

Suppose each leftover copy is just a single number u_l , and each layer multiplies it by the same factor h :

$$u_{l+1} = hu_l, \quad u_L = h^L u_0.$$

Everything depends on how big $|h|$ is.

h	h^L behavior	Meaning
1.1	grows without bound	explosion
0.8	tends to 0	decay
1	remains 1	persistence
-1	alternates between 1 and -1	persistent sign alternation
-0.8	alternates while shrinking	oscillatory decay

If the factor is allowed to change from layer to layer, write h_l for the factor at layer l . Then

$$u_L = \left(\prod_{l=0}^{L-1} h_l \right) u_0, \quad \log \left| \frac{u_L}{u_0} \right| = \sum_{l=0}^{L-1} \log |h_l|,$$

provided no h_l is zero. No single layer is in charge. The sum of the log-gains is.

Hence explosion or decay is a product, not a local courtesy. Drop the hypothesis that every $|h_l|$ equals one, and a polite 1.1 becomes a launch vehicle. The sign of h only decides whether the number also alternates. Magnitude decides whether it survives.

Two stretches at once

One leftover copy is too small. Take two numbers and stretch them differently:

$$D = \begin{bmatrix} 1.2 & 0 \\ 0 & 0.5 \end{bmatrix} \in \mathbb{R}^{2 \times 2}, \quad u_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in \mathbb{R}^2.$$

The first coordinate is multiplied by 1.2 every time. The second is multiplied by 0.5. After L layers,

$$D^L = \begin{bmatrix} 1.2^L & 0 \\ 0 & 0.5^L \end{bmatrix}, \quad u_L = \begin{bmatrix} 1.2^L \\ 0.5^L \end{bmatrix}.$$

The first four steps are

l	$u_l^{(1)}$	$u_l^{(2)}$
0	1	1
1	1.2	0.5
2	1.44	0.25
3	1.728	0.125
4	2.0736	0.0625

One direction is already more than twice as loud. The other is almost gone. A single length for the whole pair — “the norm looks moderate” — can hide both facts.

Those two stretch factors, 1.2 and 0.5, have a name. Each is an **eigenvalue** of D . The word still works when the special directions are not the coordinate axes: it is the factor the matrix uses when it stretches a direction without tilting it.

Hence a pair can explode and decay at once. If we judged D by one average length, we would miss both facts. The two numbers 1.2 and 0.5 are not two different matrices. They are two different fates inside one matrix.

A direction that only stretches

A nonzero list q is special for a square matrix H when

$$Hq = \lambda q.$$

The matrix does not scramble the orientation of q . It only rescales it, maybe flips the sign, maybe twists it in the complex plane. That special list is an **eigenvector**. Along it, the matrix problem is the scalar problem we already solved:

$$H^L q = \lambda^L q.$$

If a general starting list can be built from these special directions, each piece just rides its own stretch factor. That is the whole reason we bother finding them.

Hence repeated application along an eigenvector is ordinary powering. Suppose no such direction exists in the real plane, or the special directions fail to span. Then the scalar picture is incomplete, and a later section will show a bump that eigenvalues alone do not predict.

The biggest stretch

Look back at D . One eigenvalue is 1.2. The other is 0.5. After many layers the 1.2 piece dominates. The eventual explosion or decay of a fixed mixer is decided by the largest of those magnitudes.

That largest magnitude is the **spectral radius**:

$$\rho(H) = \max_i |\lambda_i|,$$

where λ_i are the eigenvalues of H . It is one number, read off the eigenvalues, that answers the first stability question for a *fixed* matrix: will repeated application eventually explode?

For D , $\rho(D) = 1.2 > 1$. For a mixer whose every eigenvalue sits inside the unit circle, $\rho < 1$, and the long-run story is decay. For a fixed matrix that can be written as $H = V\Lambda V^{-1}$, spectral radius gives the correct asymptotic exponential rate. It is an excellent first question. It is not the last one.

Hence $\rho > 1$ is an eventual explosion for powers of one fixed mixer, and $\rho < 1$ is an eventual decay. Neither claim is a bound on every finite product, and neither claim is a bound on a sequence of *different* mixers. Those two dropped hypotheses occupy the rest of the chapter.

A turn that does not grow

Not every mixer only stretches. Some turn.

$$R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

This rotates real pairs counterclockwise by 90° . Starting from $u_0 = (1, 0)^\top$,

$$u_1 = (0, 1)^\top, \quad u_2 = (-1, 0)^\top, \quad u_3 = (0, -1)^\top, \quad u_4 = (1, 0)^\top.$$

After four layers we are back where we started. The Euclidean length never changed:

$$\|Ru\|_2 = \|u\|_2$$

for every real pair u , because $R^\top R = I_2$.

The eigenvalues of R are i and $-i$. Both have magnitude one. They are complex, but the network state stays real. A real 2×2 rotation just needs a conjugate pair to record a turn. The angle is the phase. The magnitude is still the radial gain.

If we use $1.05R$ instead, the same turn is accompanied by growth 1.05^L . Phase describes cycling. Magnitude decides explosion or decay.

Hence a complex eigenvalue is not an explosion. It is a turn whose radius is $|\lambda|$. Drop the unit-magnitude hypothesis and the same turn becomes a growing spiral. The sign of a real eigenvalue is the same story in one dimension: it alternates; it does not, by itself, explode.

The leftover highway

We have been stretching abstract pairs. In the network those pairs are leftover copies of a sentence table.

The running state is

$$x_l \in \mathbb{R}^{d \times m \times n},$$

where d is how many leftover copies we carry — the persistent residual streams — m is how many tokens sit in the sentence, and n is how many features each token has. A routing matrix

$$H_l^{\text{res}} \in \mathbb{R}^{d \times d}$$

acts only on the stream axis. The same stream-to-stream map hits every token and every feature.

Freeze one token and one feature. Their d stream values form a column

$$u_l \in \mathbb{R}^d.$$

That is the pair we have been stretching. We have not pretended that a token row is itself a leftover copy.

The Hyper-Connections update is [R001]; [R002]

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l).$$

Here $H_l^{\text{pre}} \in \mathbb{R}^{1 \times d}$ reads from the streams, F maps an $m \times n$ table to another of that shape, and $H_l^{\text{post}} \in \mathbb{R}^{1 \times d}$ writes the result back. The first term is different: H_l^{res} transports *all* leftover state to the next layer.

If we temporarily silence the computed branch, the stream column obeys

$$u_{l+1} = H_l^{\text{res}} u_l.$$

After L layers,

$$u_L = H_{L-1}^{\text{res}} H_{L-2}^{\text{res}} \cdots H_0^{\text{res}} u_0.$$

The rightmost matrix acts first. Reversing that order is harmless only when the matrices commute, which dynamic routers generally need not do.

This product is the architecture's leftover-only highway. If it magnifies some stream direction by a huge factor, activations can explode before the computed branches have their say. If it crushes a direction, information carried only there becomes hard to recover. The transpose of the same product appears when we send gradients back down the highway.

The mHC paper treats the loss of the ordinary residual identity map as a scalability risk and reports that unconstrained Hyper-Connections training became unstable at larger scale [R002]. That is a recent paper's motivation and a measured result in the regimes they tried. It is not a theorem that every unconstrained model must fail.

Hence H_l^{res} is a map on leftover copies, not on tokens and not on features. If we mixed the token axis with the same $d \times d$ table, we would be mixing word positions. We are not. The highway claim is also only a leftover-only claim: silence the branch and the product is exact; keep the branch and the product is still the carrier of every injection.

Every injection rides the same road

The computed branch still happens. Write its contribution as

$$b_l = (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l) \in \mathbb{R}^{d \times m \times n}.$$

Then

$$x_{l+1} = H_l^{\text{res}} x_l + b_l.$$

For indices $b > a$, define the ordered product

$$P_{b:a} = H_{b-1}^{\text{res}} H_{b-2}^{\text{res}} \cdots H_a^{\text{res}} \in \mathbb{R}^{d \times d},$$

and let $P_{a:a} = I_d$. Unrolling the recurrence gives the exact identity

$$x_L = P_{L:0} x_0 + \sum_{i=0}^{L-1} P_{L:i+1} b_i.$$

The initial state is carried by every leftover mixer. The injection b_i is created after $H_i^{\text{res}} x_i$, so it is carried only by layers $i + 1$ through $L - 1$. That is the same product the mHC analysis highlights [R002].

A large $\|P_{L:i+1}\|$ can therefore amplify not only the start but also a branch output inserted much later. Strong contraction can erase older injections. This statement is about the leftover transport factor. The injection b_i itself depends on x_i , so the full nonlinear path still needs the branch.

Hence every write from Chapter 5 is riding this product. Drop later mixers from an early b_i and you misplace that write, just as a missing R_1 misplaced y_0 . The identity does not say the injections are independent of the leftover state. It says where an already-computed injection then travels.

Gradients walk the road backward

Let a scalar loss $\mathcal{L}$ depend on x_L . Isolate the derivative through the leftover-only path. For the frozen token-feature column,

$$g_l = \frac{\partial \mathcal{L}}{\partial u_l} \in \mathbb{R}^d.$$

On that path $u_{l+1} = H_l^{\text{res}} u_l$, so

$$g_l = (H_l^{\text{res}})^\top g_{l+1}, \quad g_0 = P_{L:0}^\top g_L.$$

Forward stretch and backward stretch are partners. What a matrix does to lengths, its transpose does to gradients.

Eigenvalues do not directly answer “how much can one step stretch any vector?” That one-step worst case is

$$\sigma_{\max}(H) = \max_{v \neq 0} \frac{\|Hv\|_2}{\|v\|_2}.$$

That number is the largest **singular value**. Unlike an eigenvalue, it is defined for every real matrix and it talks about Euclidean length, not about a special direction. It is the honest one-step gain, with no “if you happen to start on an eigenvector” attached.

The leftover product cannot stretch more than the product of the one-step worst cases:

$$\|P_{L:0}\|_2 \leq \prod_{l=0}^{L-1} \|H_l^{\text{res}}\|_2 = \prod_{l=0}^{L-1} \sigma_{\max}(H_l^{\text{res}}).$$

The bound can be loose: the direction one layer stretches most may not be the direction the next layer stretches most. It is still a valid alarm.

The full network Jacobian is not merely $P_{L:0}$. Differentiating b_l brings in H_l^{pre} , H_l^{post} , any input dependence they have, and F . The transpose product diagnoses one important highway, not every gradient.

Hence singular values bound one Euclidean step; eigenvalues describe special directions. If we used $\rho(H)$ as a one-step length bound, we would be wrong whenever the worst vector is not an eigenvector. In general $\rho(H) \leq \|H\|_2$, often strictly.

Tame numbers, a wild start

Eigenvalues can look calm while a short product still grows.

A real matrix A is called normal when $A^\top A = AA^\top$. Symmetric matrices and rotation matrices are normal: their special directions sit at right angles. Drop that, and the special directions can lean on each other — or fail to span the space at all.

Here is a matrix that does both jobs badly:

$$A = \begin{bmatrix} 0.8 & 0.6 \\ 0 & 0.8 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Both eigenvalues equal 0.8, so $\rho(A) = 0.8 < 1$. The long-run story is decay. But A is a repeated stretch with a sideways shove. For $L \geq 1$,

$$A^L = \begin{bmatrix} 0.8^L & 0.6L 0.8^{L-1} \\ 0 & 0.8^L \end{bmatrix}.$$

Start from $u_0 = (0, 1)^\top$:

$$u_L = \begin{bmatrix} 0.6L 0.8^{L-1} \\ 0.8^L \end{bmatrix}.$$

At $L = 5$ this is $(1.2288, 0.32768)^\top$, whose Euclidean length is about 1.272 — larger than the starting length 1. Eventually 0.8^L wins and the state dies. First it grows. For deeper or more strongly sheared systems, the bump can be much larger.

The off-diagonal term is a polynomial L times a decaying exponential. Spectral radius describes the eventual rate of powers of this *fixed* matrix. It does not bound every finite-depth response by $\rho(A)^L$.

A matrix that fails $A^\top A = AA^\top$ is **nonnormal**. The name is worth keeping because it marks exactly this gap: calm eigenvalues, rude transients [R032]; [R033].

If a fixed matrix can be written $A = V\Lambda V^{-1}$, then

$$\|A^L\|_2 \leq \|V\|_2 \|V^{-1}\|_2 \rho(A)^L.$$

The factor $\kappa_2(V) = \|V\|_2 \|V^{-1}\|_2$ measures how tilted the special directions are. Nearly parallel eigenvectors can make it huge. Eigenvalues may be calm while the eigenvectors are quietly stacking chairs against the exit.

Hence $\rho < 1$ is not a finite-depth safety proof. Suppose we dropped the off-diagonal 0.6. Then A would be $0.8I$, every length would decay at once, and the bump would vanish. The hypothesis that failed was normality, not the eigenvalues.

Mixers that change their minds

Now let the matrix vary with depth. There may be no shared special directions and no identity such as

$$\text{spec}(H_{L-1} \cdots H_0) = \prod_l \text{spec}(H_l).$$

That displayed relation is generally false.

A sharp counterexample uses

$$A = \begin{bmatrix} 0 & 2 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 \\ 2 & 0 \end{bmatrix}.$$

Each matrix, applied twice, gives zero. Every eigenvalue of each matrix is zero. Alternate them, with $H_0 = A$, $H_1 = B$, $H_2 = A$, and so on. Then

$$BA = \begin{bmatrix} 0 & 0 \\ 0 & 4 \end{bmatrix}.$$

For $u_0 = (0, 1)^\top$,

$$u_{2k} = (BA)^k u_0 = 4^k u_0.$$

Every individual spectral radius is zero. The two-layer product explodes. The matrices hand the state back and forth between directions that each one amplifies. Checking $\rho(H_i^{\text{res}}) \leq 1$ independently at every layer is not a complete stability certificate for dynamic routing.

For a set of possible routing matrices, the worst asymptotic growth over all products drawn from the set has a name: the **joint spectral radius** [R034]. It is the right question for a *family* of mixers, not one mixer. It can be painful to compute. In practice, architectures often pick constraints with simple invariants and closure under multiplication, rather than solving a worst-case product problem during training.

Hence per-layer eigenvalues do not multiply. Suppose the mixers shared the same special directions and were applied in any order. Then the product of eigenvalues would be an eigenvalue of the product, and the shortcut would work. Dynamic leftover routers generally do not offer that courtesy.

What a useful rule would do

In the notation of the mHC paper, $H_i^{\text{res}} \in \mathbb{R}^{d \times d}$ is the leftover-copy mixer in the Hyper-Connections update [R002]. It acts along the d -stream axis of $x_i \in \mathbb{R}^{d \times m \times n}$. Original Hyper-Connections permits learned static or input-dependent connection weights [R001]. Input dependence makes the product trajectory-dependent: different examples can induce different sequences of mixers.

The examples above are the design pressure. A useful constraint should ideally give properties that survive products — a conserved direction, a norm bound — while still letting information move among leftover copies.

The 2025–2026 mHC work reports instability for unconstrained Hyper-Connections at increased training scale and motivates restricting leftover mixers to doubly stochastic matrices [R002]. That constraint and its exact guarantees are the next chapter. The go-mHC paper keeps constrained leftover routing but replaces the constraint map with a generalized-orthostochastic construction [R005]. This chapter does not infer that construction from the stability motivation alone.

Constraining H_i^{res} cannot by itself bound the entire nonlinear network. The read gate, the branch F , the write gate, normalization, the optimizer, finite precision, and derivatives of input-dependent routing all remain. The narrower claim is valuable enough: constrain a repeatedly multiplied structural path instead of leaving it arbitrary.

Hence a leftover-mixer constraint is a claim about one highway. If we advertised it as a theorem that the whole model is stable, we would have smuggled in every other moving part. The next chapter keeps the highway and stops it from freely creating or destroying aggregate signal.

Ways to get this wrong

If leftover mixers are entirely unconstrained, several failures are possible rather than inevitable:

1. **Exploding transport.** A direction with repeated gain above one can dominate activations. A branch injection can also be magnified by every later mixer.
2. **Vanishing transport.** Contracted directions lose old information and provide a weak leftover-only gradient path.
3. **Anisotropy.** One direction can grow while another decays, as in the diagonal example. A single average statistic may miss both.
4. **Transient amplification.** Nonnormal matrices can create large finite-depth responses even when their eigenvalues predict eventual decay.
5. **Sequence-dependent surprises.** Dynamic matrices can align amplifying directions across layers. Safe-looking per-layer eigenvalues do not imply a safe product.
6. **Oscillation.** Negative or complex eigenvalues can alternate or rotate state. Oscillation is not itself a failure — the 90° rotation preserved length — but uncontrolled scale attached to it can be.

Large transient gains may make optimization more sensitive to precision, initialization, and batch-specific routing. That is a plausible consequence of the product analysis, not a universal measured result for every HyperConnections variant.

The response is not “ban matrices.” The response is to choose a routing family whose algebraic properties match the conservation and stability properties the architecture needs.

- “Every eigenvalue has magnitude below one, so every intermediate activation shrinks.” False for nonnormal matrices. That conclusion is asymptotic for powers of a fixed matrix, not a finite-depth norm bound.
- Confusing spectral radius with the one-step stretch $\|A\|_2$. In general, $\rho(A) \leq \|A\|_2$, often strictly. Singular values govern worst-case one-step Euclidean gain.
- Multiplying per-layer eigenvalues. Eigenvalues of products do not generally equal pairwise products of eigenvalues. This shortcut needs special structure, such as the matrices sharing the same special directions.
- Ignoring product order. H_1H_0 and H_0H_1 can differ.
- Calling every negative eigenvalue unstable. The magnitude controls growth. The sign adds alternation.
- Calling every complex eigenvalue explosive. Complex phase gives rotation; modulus gives radial gain.
- Assuming $\lambda = 1$ preserves all state. It preserves components along its associated eigenvector. Other directions may behave differently.
- Mixing axes. $H_l^{\text{res}} \in \mathbb{R}^{d \times d}$ mixes leftover copies, not the m token positions or n feature coordinates in this equation.
- Claiming a routing constraint stabilizes the whole model. It controls specified properties of one structural path; the branch and dynamic-router derivatives still matter.

Try it

1. One number, four layers

Let $u_0 = 3$ and $u_{l+1} = -0.5u_l$. What are u_1 , u_2 , and u_4 ? Across those four layers, how large is the leftover-only gradient multiplier?

Answer

$u_1 = -1.5$, $u_2 = 0.75$, and $u_4 = (-0.5)^4(3) = 0.1875$. The scalar backward multiplier has magnitude $|-0.5|^4 = 0.0625$.

2. Two directions at once

For $D = \text{diag}(-1, 0.9)$ and $u_0 = (2, 3)^\top$, compute u_3 . Which coordinate persists and alternates? Which decays?

Answer

$D^3 = \text{diag}(-1, 0.9^3)$, so $u_3 = (-2, 2.187)^\top$. The first coordinate persists in magnitude and alternates sign. The second decays without sign alternation.

3. A turn, then a growing turn

Let

$$R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Show that $R^4 = I_2$. Then give the length of $(1.1R)^{10}u_0$ when $\|u_0\|_2 = 2$.

Answer

Two applications give $R^2 = -I_2$, hence $R^4 = I_2$. Since R preserves Euclidean length, $(1.1R)^{10}$ scales length by 1.1^{10} . The answer is $2(1.1)^{10} \approx 5.187$.

4. A bump before the decay

For

$$A = \begin{bmatrix} a & c \\ 0 & a \end{bmatrix},$$

verify that

$$A^L = \begin{bmatrix} a^L & Lc a^{L-1} \\ 0 & a^L \end{bmatrix}$$

for $L \geq 1$. Why can $|a| < 1$ still grow for a while?

Answer

Write $A = aI + N$, where $N = \begin{bmatrix} 0 & c \\ 0 & 0 \end{bmatrix}$ and $N^2 = 0$. The binomial expansion leaves only $(aI)^L + L(aI)^{L-1}N$, giving the formula. Although a^L eventually decays, the off-diagonal term contains Lca^{L-1} , whose polynomial factor can grow for a while and can be large when c is large.

5. Dead-looking layers, live pair

Using the matrices A and B from the changing-mixer example, compute AB as well as BA . Starting from $e_1 = (1, 0)^\top$, which order amplifies it after two layers?

Answer

$AB = \text{diag}(4, 0)$ and $BA = \text{diag}(0, 4)$. With the convention that the rightmost matrix acts first, $ABe_1 = 4e_1$ amplifies e_1 , while $BAe_1 = 0$. Order changes the result even though both individual matrices have only zero eigenvalues.

Check yourself

1. **What does an eigenvalue tell you?** Along its special direction, the matrix is just a number. Apply it L times and that number is raised to the L .
2. **Show that $\rho < 1$ is not a finite-depth safety proof.** A nonnormal matrix can grow first and decay later. Spectral radius describes the long run of a *fixed* matrix, not every short product.
3. **What fails if each layer's eigenvalues are checked independently?** Changing mixers can hand the state between amplifying directions. Every individual spectral radius can be zero while the product explodes.
4. **What do singular values answer that eigenvalues do not?** The worst one-step stretch of Euclidean length, for every real matrix, not just along a special direction.
5. **What fails if a leftover-mixer constraint is advertised as stabilizing the whole network?** It can give a property that survives products — a conserved direction or a norm bound — without controlling the branch, the gates, or the derivatives of input-dependent routing.

You should now be able to read

$$P_{L:0} = H_{L-1}^{\text{res}} \cdots H_0^{\text{res}}$$

as a depth-wise transport operator on the stream axis. Stability is not one binary property. Ask exactly what is conserved, what length is bounded, over which matrices, for how many layers, and whether the claim concerns the leftover-only path or the full network.

Next we let the leftover mixers move information, but we stop them from freely creating or destroying aggregate signal. The next chapter introduces nonnegative matrices whose rows and columns each sum to one.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” *arXiv:2409.19606*, 2024; revised 2025. <https://arxiv.org/abs/2409.19606>

-
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” *arXiv:2512.24880*, 2025; revised 2026. <https://arxiv.org/abs/2512.24880>
 - [R032] L. N. Trefethen and M. Embree, *Spectra and Pseudospectra: The Behavior of Nonnormal Matrices and Operators*, Princeton University Press, 2005. <https://doi.org/10.1515/9780691213101>
 - [R033] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2012. <https://doi.org/10.1017/CBO9781139020411>
 - [R034] G.-C. Rota and G. Strang, “A note on the joint spectral radius,” *Indagationes Mathematicae*, 22, 379–381, 1960. [https://doi.org/10.1016/S1385-7258\(60\)50046-1](https://doi.org/10.1016/S1385-7258(60)50046-1)
 - [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” *arXiv:2604.02309v1*, 2026. <https://arxiv.org/abs/2604.02309>

Chapter 8 — Recipes that keep the total

Three cups sit on the table. The first holds 9, the second holds 3, the third is empty. We are going to remix them. No extra liquid. Nothing down the drain.

A recipe for a new cup is a short list of nonnegative fractions that add to one. Then the new cup is a weighted average of the old ones: it cannot come out bigger than 9 or smaller than 0.

One honest recipe is not enough. If every new cup copies the first old cup, each recipe looks legal and we have invented two extra nines. The second honesty check is about the old cups: each one must be spent exactly once across all the new recipes.

After that picture we drop the kitchen. The object is a square matrix of mixing weights.

One complete recipe per cup

Write three output recipes as the three rows of a table. Here is one we will keep using:

$$B = \begin{pmatrix} 0.5 & 0.3 & 0.2 \\ 0.2 & 0.4 & 0.4 \\ 0.3 & 0.3 & 0.4 \end{pmatrix}.$$

Row 1 says: the first new value is half of old value 1, three tenths of old value 2, two tenths of old value 3. The three row sums are

$$0.5 + 0.3 + 0.2 = 1, \quad 0.2 + 0.4 + 0.4 = 1, \quad 0.3 + 0.3 + 0.4 = 1.$$

Every entry is at least zero. Each row is a complete recipe.

A square matrix with nonnegative entries whose rows each sum to one is **row-stochastic**. The name is worth keeping because “stochastic” here means only that each row is a finished mixing recipe, not that the matrix was drawn at random.

If $\mathbf{1}_3$ is a column of three ones, the three row sums are the single matrix-vector line

$$B\mathbf{1}_3 = \mathbf{1}_3.$$

Hence B fixes the all-ones column. Some probability books stand their recipes on end and call the transpose stochastic. We will always say which way the recipes face.

Drop nonnegativity and the same row sums can hide cancellation. That matrix is not row-stochastic, even if $A\mathbf{1} = \mathbf{1}$.

Spend each cup once

Rows do not watch the budget of the old values. This matrix has three perfect output recipes:

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}.$$

Each new value copies old value 1. Pour the original $(9, 3, 0)$ through it and you get $(9, 9, 9)$. The total jumps from 12 to 27. Value 1 was counted three times. Values 2 and 3 were ignored.

The missing check is down the columns. Column j is the list of fractions that use old value j . If those fractions are nonnegative and add to one, that old value is spent exactly once.

For our matrix B ,

$$0.5 + 0.2 + 0.3 = 1, \quad 0.3 + 0.4 + 0.3 = 1, \quad 0.2 + 0.4 + 0.4 = 1.$$

A square matrix with nonnegative entries whose columns each sum to one is **column-stochastic**. Now each *input* has total weight one.

In matrix form that is $B^\top \mathbf{1}_3 = \mathbf{1}_3$, or, written as a sum, $\sum_i B_{ij} = 1$ for every column j .

Hence column honesty is a statement about sources, not about destinations. R fails it: its first column sums to 3 and the others sum to 0.

Honest both ways

Ask for both books at once. Every row is a complete recipe. Every column is a complete spend. The entries stay nonnegative.

A square matrix that passes both checks is **doubly stochastic**. *Doubly* means the two families of sums, not that anyone squared the matrix. We take this as the definition.

Our B already passed both checks, so

$$B\mathbf{1}_3 = \mathbf{1}_3, \quad B^\top \mathbf{1}_3 = \mathbf{1}_3,$$

and every $B_{ij} \geq 0$. That is the whole definition.

Later we will sit this matrix on leftover copies of a sentence — d streams, not three kitchen cups. The same two checks still mean the same thing. The mixer may move mass among streams. It may not invent a new total, and it may not throw the total away.

A softmax over each row gives a row-stochastic matrix. It does not, in general, give a doubly stochastic one. The second family of sums is a separate demand.

Remix the three numbers

Freeze one token and one feature. The three stream values at that spot are the column

$$v = \begin{pmatrix} 9 \\ 3 \\ 0 \end{pmatrix}.$$

Each output is one row recipe applied to v :

$$Bv = \begin{pmatrix} 0.5(9) + 0.3(3) + 0.2(0) \\ 0.2(9) + 0.4(3) + 0.4(0) \\ 0.3(9) + 0.3(3) + 0.4(0) \end{pmatrix} = \begin{pmatrix} 5.4 \\ 3.0 \\ 3.6 \end{pmatrix}.$$

Each new number sits between 0 and 9, because each row is a convex average. The total does not move:

$$9 + 3 + 0 = 12 = 5.4 + 3.0 + 3.6.$$

That is not a coincidence of these particular digits. Suppose every column of a $d \times d$ matrix sums to one. Then for any column v ,

$$\mathbf{1}_d^\top (Bv) = (\mathbf{1}_d^\top B)v = \mathbf{1}_d^\top v.$$

Hence the sum across streams is conserved. So is the mean $d^{-1}\mathbf{1}_d^\top v$.

In the network the leftover copies form a table x_l of shape $d \times m \times n$: d streams, m token positions, n features. The same $d \times d$ mixer is reused on every token-feature slice. Conservation holds separately at each of the mn slices. It does not say that token sums or feature sums are being mixed. Those axes are passengers.

Row honesty gives a second fact. If every stream already holds the same number, routing leaves them equal and unchanged:

$$B\mathbf{1}_d = \mathbf{1}_d.$$

The all-ones column is a right eigenvector with eigenvalue 1. Column honesty makes $\mathbf{1}_d^\top$ a left eigenvector with the same eigenvalue. Equal streams stay equal. The total stays the total.

Conservation of the *sum* is not preservation of each stream. Individual coordinates of Bv can move a long way.

Averages, not extrapolations

Take two legal mixers A and C . For a number t between 0 and 1, the blend $tA + (1-t)C$ is still nonnegative, and every row and column still sums to one. You can average more than two of them the same way.

Given points $q_1, \dots, q_k$, a **convex combination** is

$$q = \sum_{r=1}^k \alpha_r q_r, \quad \alpha_r \geq 0, \quad \sum_{r=1}^k \alpha_r = 1.$$

The coefficients are a recipe. Negative coefficients would extrapolate past an endpoint. That is a different sport.

For two points, convex combinations fill the segment between them. For several points, they fill the region you can reach by weighted averaging and never by backing up past a corner.

A set is convex when the segment between any two of its points stays inside. The set of all doubly stochastic $d \times d$ matrices is convex, and each entry is trapped in $[0, 1]$, so the set is bounded.

Hence $\mathcal{B}_d$, once we name it, will be a convex body, not an open cone of free weights. Drop $\alpha_r \geq 0$ and the combination can leave the set even when the α_r still sum to one.

Just swap the cups

The bluntest legal mixer does not blend at all. It just reorders.

The identity leaves every value where it is:

$$P_{123} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

This one sends new value 1 to old value 1, new value 2 to old value 3, new value 3 to old value 2:

$$P_{132} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

A **permutation matrix** has exactly one 1 in every row and every column, and 0 everywhere else. Multiplying Pv only reorders the coordinates of v .

Every permutation matrix is doubly stochastic: each row recipe is “take this one value,” and each old value is used once. There are $d!$ of them. For three streams that is six hard rewirings.

Hence the hard rewirings sit inside the same set as the soft averages. They will turn out to be the corners of the whole set.

A permutation can cycle streams forever. Double stochasticity does not force a mixer to blend, or to settle.

The region those swaps span

How many free numbers are in a doubly stochastic $d \times d$ matrix? There are d^2 entries. Nonnegativity is inequality, not an equation. The row sums and column sums look like $2d$ equations, but they overcount: the grand total is the sum of the row sums and also the sum of the column sums, so one equation is redundant. That leaves $2d - 1$ independent equalities, and

$$d^2 - (2d - 1) = (d - 1)^2$$

free parameters.

For $d = 2$ the set is a line segment. For $d = 3$ it is four-dimensional, even though we write nine entries. Sketching instincts should accept defeat.

The set of all such matrices is the **Birkhoff polytope**

$$\mathcal{B}_d = \{B \in \mathbb{R}^{d \times d} : B_{ij} \geq 0, B\mathbf{1}_d = \mathbf{1}_d, B^\top \mathbf{1}_d = \mathbf{1}_d\}.$$

A polytope is a bounded convex region cut out by finitely many linear equalities and inequalities — or, equivalently, the filled region of finitely many corners.

Theorem. Birkhoff and von Neumann proved that the corners are exactly the $d!$ permutation matrices. Equivalently, every doubly stochastic matrix is a convex combination of permutation matrices [R019]; [R020].

Soft routing sits between hard rewirings. The theorem says a decomposition exists. It does not say the decomposition is unique.

Our running matrix has one verified writing

$$B = 0.4 P_{123} + 0.1 P_{132} + 0.3 P_{231} + 0.2 P_{312},$$

where the subscript lists which old value each new value takes, and the other two corners are

$$P_{231} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad P_{312} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

The four coefficients are nonnegative and add to one. Adding the matrices entrywise recovers

$$\begin{pmatrix} 0.5 & 0.3 & 0.2 \\ 0.2 & 0.4 & 0.4 \\ 0.3 & 0.3 & 0.4 \end{pmatrix}.$$

Hence B is a point inside the polytope, not a new kind of object. Interior points usually have many writings. The corners are fixed; the coordinates are not.

Multiply two recipes

If A and B are both doubly stochastic, so is the product AB .

Nonnegative times nonnegative stays nonnegative. The all-ones column is still a fixed point:

$$(AB)\mathbf{1}_d = A(B\mathbf{1}_d) = A\mathbf{1}_d = \mathbf{1}_d,$$

$$(AB)^\top \mathbf{1}_d = B^\top (A^\top \mathbf{1}_d) = B^\top \mathbf{1}_d = \mathbf{1}_d.$$

Hence $AB \in \mathcal{B}_d$. A stack of leftover-copy mixers is still a leftover-copy mixer. The product still conserves the stream sum and still leaves equal streams equal.

The same argument does not need the factors to commute. Dynamic routers generally need not.

Allowed to shrink, not to grow

How much can one of these mixers stretch a vector?

The 1-norm of a matrix is its largest absolute column sum. The ∞ -norm is its largest absolute row sum. For any B in $\mathcal{B}_d$ both of those are exactly 1, because every row sum and every column sum is already 1. The spectral norm then obeys

$$\|B\|_1 = \|B\|_\infty = 1, \quad \|B\|_2 \leq \sqrt{\|B\|_1 \|B\|_\infty} = 1.$$

So $\|Bv\|_2 \leq \|v\|_2$. The Birkhoff writing gives the same bound another way: a permutation matrix only reorders, so it preserves Euclidean length, and a convex average of such maps cannot have operator norm bigger than 1.

Hence no matrix in $\mathcal{B}_d$ can expand Euclidean length. That property is **non-expansion**.

The inequality can be strict. Let J_d be the all-ones matrix and $U = J_d/d$ the uniform averager. Every row and column of U is $1/d$, so U is doubly stochastic. Any vector whose entries add to zero is sent to zero in one step. Calling this “norm preservation” would be wrong.

The honest general property is: lengths do not grow, the mean mode is kept exactly, and disagreement among streams is allowed to shrink, even to nothing.

Unconstrained leftover-copy mixers do not promise this. Chapter 7 multiplied them across depth and watched directions explode or vanish. Double stochasticity is the bookkeeping rule that takes explosion off the table for this one product. It does not take vanishing off the table.

The leftover-copy mixer

The Hyper-Connections update is

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l),$$

with $x_l \in \mathbb{R}^{d \times m \times n}$, a $d \times d$ leftover-copy mixer H_l^{res} , read and write gates $H_l^{\text{pre}}, H_l^{\text{post}} \in \mathbb{R}^{1 \times d}$, and a branch output $F(\cdot) \in \mathbb{R}^{m \times n}$. The first term reroutes what is already there. The second writes something new.

mHC aims to put each leftover-copy mixer in the Birkhoff polytope,

$$H_l^{\text{res}} \in \mathcal{B}_d,$$

by a finite Sinkhorn cleanup of learned, input-dependent coefficients [R002]. Chapter 9 will look at why finite iterations and finite precision make the implemented sums approximate rather than symbolic.

On the persistence-only path, each layer then redistributes stream state by convex averaging. The product stays doubly stochastic, keeps the common-stream mode, keeps the stream mean at every token-feature coordinate, and has spectral norm at most one. The transpose used on the way back is also doubly stochastic, so it obeys the same non-expansion bound. This matrix product, by itself, cannot have gain greater than one.

Empirical result. Xie et al. report that, in their 27B-model experiments, mHC avoided the loss and gradient-norm instability they saw in unconstrained Hyper-Connections, and that propagation-gain deviations were much smaller [R002]. Those are paper-specific observations. They are not a theorem that every mHC-like model trains stably.

The later go-mHC construction also aims at doubly stochastic leftover-copy routing, but it gets there with an orthogonal block construction rather than finite Sinkhorn steps. The conservation and non-expansion facts in this chapter depend on the resulting matrix sitting in $\mathcal{B}_d$, not on how the numbers were born. How much of the polytope a finite-parameter family can cover is a separate question.

A constraint on H_l^{res} is not a bound on x_{l+1} . The additive branch can increase the norm, and its Jacobian also contributes during backpropagation.

Recipes that cheat

Use only row honesty, and R above is legal. For $v = (9, 3, 0)^\top$,

$$Rv = (9, 9, 9)^\top,$$

and the total jumps from 12 to 27. Row honesty still fixes a constant input, because $R\mathbf{1}_3 = \mathbf{1}_3$. It does not conserve total mass.

Use only column honesty. Totals are conserved, but output rows need not be averages. The matrix

$$C = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

has unit column sums, yet $Cv = (12, 0, 0)^\top$. All the mass piles into one stream, and $C\mathbf{1}_3 = (3, 0, 0)^\top$: equal streams are not stationary.

Drop nonnegativity and the sums can still equal one while cancellation hides large coefficients. For

$$A_t = \begin{pmatrix} 1+t & -t \\ -t & 1+t \end{pmatrix},$$

every row and every column sums to one, but the eigenvalue along $(1, -1)^\top$ is $1 + 2t$. Take $t > 0$ and the mixer amplifies. The inequalities in the definition are doing real work.

Hence each of the three clauses — nonnegative, rows sum to one, columns sum to one — is necessary. Drop any one and a counterexample sits on the table.

What we did not prove

Double stochasticity of H_l^{res} gives nonnegative unit row and column sums, a stationary all-ones stream mode, conservation of the stream sum, closure under products, and non-expansion of the persistence-only linear map in the usual 1, 2, and ∞ norms. It does **not** give any of the following.

- **Identity routing.** A permutation can swap streams. An interior point can blend them. Only I_d leaves every stream separately unchanged.
- **Orthogonality or exact norm preservation.** J_d/d is doubly stochastic and rank one. It erases all stream disagreement.
- **Invertibility or a lower singular-value bound.** Gradients and signals in non-mean directions may contract to zero. The constraint blocks explosion from this mixer. It does not block every form of vanishing.
- **Unit-modulus eigenvalues.** All eigenvalues satisfy $|\lambda| \leq 1$, and 1 is present, but other eigenvalues can sit strictly inside the unit disk or be zero.
- **Convergence to uniform mixing.** A permutation can cycle forever. A changing product of mixers need not behave like repeated powers of one primitive Markov matrix.
- **Preservation of each stream, token, feature, variance, covariance, or information content.** It preserves one linear quantity — the sum across streams — for each fixed token-feature coordinate. Averaging may discard recoverable distinctions.
- **Stability of the full nonlinear network or its full Jacobian.** The branch F , read and write gates, normalization, additive injections, parameter updates, numerical errors, and cross-layer interactions remain. Their gains are not bounded merely because $H_l^{\text{res}} \in \mathcal{B}_d$.
- **Exact constraint satisfaction by an approximate parameterization.** Finite Sinkhorn steps and finite precision can leave row or column residuals. That is an implementation issue, not a revision of the definition.
- **Good accuracy, optimal routing, stream specialization, or successful optimization.** Those are empirical outcomes, not consequences of sitting in a polytope.

“The spectral norm is at most one” is a statement about a particular linear routing term. Replacing it with “the model is stable” drops most of the model from the sentence.

Ways to get this wrong

- Checking rows and forgetting columns. A softmax over each row gives a row-stochastic matrix, not generally a doubly stochastic one.
- Checking sums and forgetting signs. Unit row and column sums without nonnegativity do not define $\mathcal{B}_d$ and do not give convex averaging.
- Mixing up axes. $H^{\text{res}} \in \mathbb{R}^{d \times d}$ acts on streams in $x \in \mathbb{R}^{d \times m \times n}$, not on the m tokens.
- Treating conservation as per-stream preservation. The total across streams is fixed. Individual stream values can change a lot.
- Calling every interior mixer a probability transition without saying which way the recipes face. Row-vector and column-vector Markov conventions transpose one another.
- Calling the whole Birkhoff polytope a smooth manifold. It is a convex polytope with faces, edges, and corners. Its relative interior is smooth inside its affine hull. Its boundary is not. “Manifold-constrained” is the architecture’s name, not permission to sand off the corners.
- Assuming the Birkhoff decomposition is unique. The corners are fixed. Interior points may have several convex writings.

Try it

1. Check both books

Is

$$A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$$

row-stochastic, column-stochastic, both, or neither?

Answer

All entries are nonnegative and both rows sum to 1, so A is row-stochastic. Its column sums are 1.1 and 0.9, so it is not column-stochastic and therefore not doubly stochastic.

2. Pour a different column

Using this chapter's 3×3 matrix B , route $v = (2, -1, 5)^\top$. Check that the total is conserved and that each output sits between the smallest and largest inputs.

Answer

$$Bv = (1.7, 2.0, 2.3)^\top.$$

Both totals equal 6. Each output lies in the input range $[-1, 5]$. Negative *state values* are allowed; nonnegativity constrains the routing weights.

3. Products stay inside

Let $A, B \in \mathcal{B}_d$. Show that AB has nonnegative entries and unit row and column sums.

Answer

Each $(AB)_{ij} = \sum_k A_{ik} B_{kj} \geq 0$. Also $(AB)\mathbf{1} = A(B\mathbf{1}) = A\mathbf{1} = \mathbf{1}$, and $(AB)^\top \mathbf{1} = B^\top (A^\top \mathbf{1}) = \mathbf{1}$. Hence $AB \in \mathcal{B}_d$.

4. A mixer that erases disagreement

Let $U = J_3/3$, the 3×3 matrix with every entry $1/3$. Show that U is doubly stochastic but not orthogonal. What happens to $w = (1, -1, 0)^\top$?

Answer

Every row and column sums to one, so $U \in \mathcal{B}_3$. But $Uw = 0$, even though $w \neq 0$; therefore U is singular and cannot be orthogonal. This demonstrates that double stochasticity does not preserve all norms or prevent vanishing disagreement modes.

5. The two-stream picture

Solve the row- and column-sum equations for a general 2×2 doubly stochastic matrix. Name the two corners.

Answer

Every such matrix has the form

$$B(a) = \begin{pmatrix} a & 1-a \\ 1-a & a \end{pmatrix}, \quad 0 \leq a \leq 1.$$

Thus $\mathcal{B}_2$ is a line segment. Its endpoints are the swap matrix at $a = 0$ and the identity at $a = 1$, exactly the two 2×2 permutation matrices.

6. The whole update, not one term

Assume $H_l^{\text{res}} \in \mathcal{B}_d$. Does it follow that $\|x_{l+1}\|_2 \leq \|x_l\|_2$ for the full Hyper-Connections update? Why or why not?

Answer

No. The bound applies to $H_l^{\text{res}} x_l$. The additive branch $(H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l)$ can increase the norm, and its Jacobian also contributes during backpropagation. A constraint on one term is not a bound on their sum.

Check yourself

1. **Show that column stochasticity conserves the stream sum.**

If $B^\top \mathbf{1} = \mathbf{1}$, then $\mathbf{1}^\top (Bv) = (\mathbf{1}^\top B)v = \mathbf{1}^\top v$.

2. **What fails if nonnegativity is dropped?**

Rows and columns can still sum to one while large opposite signs amplify a disagreement mode, as A_t does along $(1, -1)^\top$.

3. **Which axis of $x \in \mathbb{R}^{d \times m \times n}$ does H^{res} mix?**

The stream axis of size d . Token and feature axes ride along.

4. **Show that $\mathbf{1}_d$ is stationary.**

Row sums equal to one are exactly $B\mathbf{1}_d = \mathbf{1}_d$.

5. **What fails if we call the Birkhoff polytope a smooth manifold?**

The set has faces, edges, and corners. Only its relative interior is smooth in its affine hull.

6. **Can the mixer still contract, erase, swap, or cycle stream directions?**

Yes. Non-expansion forbids growth, not vanishing, permutation, or a cycle.

Doubly stochastic means convex redistribution, stream-sum conservation, and bounded linear gain. It does not mean the whole network is stable.

A network does not emit a point of $\mathcal{B}_d$ by accident. Raw learned numbers can be negative and can have any row or column total. Next we take those numbers and alternately fix the rows and the columns — the Sinkhorn procedure — and ask how close a finite number of passes gets us to the polytope.

Sources

- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880v2, 2026. <https://arxiv.org/abs/2512.24880>
- [R019] G. Birkhoff, “Tres observaciones sobre el álgebra lineal,” *Universidad Nacional de Tucumán, Revista, Serie A* 5 (1946), 147–151, MR0020547. <https://cir.nii.ac.jp/crid/1570572699525842816?lang=en>
- [R020] J. von Neumann, “A Certain Zero-Sum Two-Person Game Equivalent to the Optimal Assignment Problem,” in *Contributions to the Theory of Games II*, Princeton University Press, 1953, pp. 5–12. <https://doi.org/10.1515/9781400881970-002>
- [R021] R. Sinkhorn and P. Knopp, “Concerning Nonnegative Matrices and Doubly Stochastic Matrices,” *Pacific Journal of Mathematics* 21(2), 1967, 343–348. <https://doi.org/10.2140/pjm.1967.21.343>
- [R023] A. W. Marshall, I. Olkin, and B. C. Arnold, *Inequalities: Theory of Majorization and Its Applications*, 2nd ed., Springer, 2011. <https://doi.org/10.1007/978-0-387-68276-1>

Chapter 9 — Rows, then columns, then again

You have a small table of mixing weights. You want every row to add to one, so each destination is a complete recipe. You also want every column to add to one, so no source is counted twice.

Those two wishes fight. Divide each row by its sum and the columns go crooked. Divide each column by its sum and the rows go crooked.

The fix is stubborn, not clever. Make the rows sum to one, then the columns, then repeat. Or start with the columns. Either way you are bargaining, not solving both sums at once.

Chapter 8 asked for both families of sums together: a doubly stochastic router. This chapter is the cheapest chase after that matrix, one repair at a time, and then an honest leftover: how much is still wrong when you stop.

Start with scores, not weights

The model does not hand us a polite table. It hands us unconstrained numbers, one for each source-to-destination pair. Call those raw scores **logits**.

Here is a 2×2 toy:

$$Z = \begin{pmatrix} \log 1 & \log 2 \\ \log 3 & \log 4 \end{pmatrix}.$$

We need every entry positive. Then later divisions never hit zero, and every weight can be read as a share. Exponentiate each entry. Do not take a matrix exponential. A matrix exponential would mix powers of Z . We only want positivity:

$$M^{(0)} = \exp(Z) = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}.$$

Read entry (i, j) as: how strongly source stream j contributes to destination stream i . A row is one destination's recipe. A column is one source's outgoing budget.

The mHC paper writes columns first, then rows. We will follow that order so the leftovers match their numbers. The fight is the same if you swap the order. The finite table is not.

Hence $\exp(Z)$ is an entrywise map. It is not $\sum_k Z^k/k!$. Confusing the two turns a positivity trick into a completely different matrix.

Fix the columns, ruin the rows

The column sums of $M^{(0)}$ are $(4, 6)$. Divide each column by its own sum:

$$\begin{pmatrix} 1/4 & 2/6 \\ 3/4 & 4/6 \end{pmatrix} = \begin{pmatrix} 0.250000 & 0.333333 \\ 0.750000 & 0.666667 \end{pmatrix}.$$

Those columns now add to one. The row sums are 7/12 and 17/12. Divide each row by *its* sum:

$$M^{(1)} = \begin{pmatrix} 3/7 & 4/7 \\ 9/17 & 8/17 \end{pmatrix} \approx \begin{pmatrix} 0.428571 & 0.571429 \\ 0.529412 & 0.470588 \end{pmatrix}.$$

The row sums are exactly (1, 1). The column sums are not:

$$(0.957983, 1.042017).$$

The first repair did its job. The second undid part of it. That leftover is the whole point of the chapter.

Score it by looking at every row sum and every column sum, taking the distance from one, and keeping the worst. That worst leftover is the **constraint error**

$$\varepsilon_{\infty}(D) = \max \left\{ \max_i \left| \sum_j D_{ij} - 1 \right|, \max_j \left| \sum_i D_{ij} - 1 \right| \right\}.$$

The name is just a ruler: one number that refuses to ignore whichever side you repaired last. Here

$$\varepsilon_{\infty}(M^{(1)}) = 0.0420168.$$

Hence one full pass can leave a visible column error even though the rows look perfect. If you only checked the rows you would report zero and miss the unfinished work.

Do it again

Take $M^{(1)}$. Repair the columns, then the rows:

$$\begin{pmatrix} 0.447368 & 0.548387 \\ 0.552632 & 0.451613 \end{pmatrix}, \quad M^{(2)} \approx \begin{pmatrix} 0.449275 & 0.550725 \\ 0.550296 & 0.449704 \end{pmatrix}.$$

Rows are again (1, 1). Columns are now

$$(0.99957122, 1.00042878).$$

So

$$\varepsilon_{\infty}(M^{(2)}) \approx 4.2878 \times 10^{-4}.$$

Two passes are much better than one for this matrix. “Much better” is still not “done.”

The same matrix, the same two maps, a smaller leftover. Nothing in the arithmetic turned the leftover into zero. A second pass is not a proof of exactness.

The name of the loop

That stubborn alternation has a name. **Sinkhorn** — Sinkhorn–Knopp, in the form we use — is exactly this procedure: start from a positive table and keep repairing columns, then rows, until you stop.

The name is worth keeping because the theorem people quote is about the infinite run, and a training kernel is a finite one. Same last name. Different objects.

Write the two repairs as maps. For a positive matrix A ,

$$[\mathcal{T}_c(A)]_{ij} = \frac{A_{ij}}{\sum_{k=1}^d A_{kj}}, \quad [\mathcal{T}_r(A)]_{ij} = \frac{A_{ij}}{\sum_{k=1}^d A_{ik}}.$$

Here i picks a row, j a column, d is the number of streams, and k is the index being summed away. One full iteration, in the paper's order, is

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})).$$

In exact arithmetic every completed iteration ends with row sums exactly one. Its column sums need not be one. Reverse the order and you reverse which side is exact at the end of a step. You never repair both sides in the same instant.

Two people straighten the same picture from opposite sides. Under the right conditions the adjustments shrink toward agreement. Stop after either person acts and their side looks better.

Hence the finite object always has a preferred exact side. Hiding the order hides which leftover you chose to keep.

One more pass, and the table it is chasing

One more pair of repairs:

$$\begin{pmatrix} 0.449468 & 0.550489 \\ 0.550532 & 0.449511 \end{pmatrix}, \quad M^{(3)} \approx \begin{pmatrix} 0.449488 & 0.550512 \\ 0.550508 & 0.449492 \end{pmatrix}.$$

Column sums about

$$(0.99999562, 1.00000438),$$

so

$$\varepsilon_\infty(M^{(3)}) \approx 4.3758 \times 10^{-6}.$$

For this 2×2 example the table the loop is chasing is

$$M^{(\infty)} = \begin{pmatrix} a & 1-a \\ 1-a & a \end{pmatrix}, \quad a = \frac{\sqrt{2}}{\sqrt{2} + \sqrt{3}} \approx 0.449490.$$

That limit is exactly doubly stochastic as a mathematical object. A stored floating-point approximation is not exact in the real-number sense, even when its displayed decimals look cooperative.

Three iterations put ε_∞ near 4×10^{-6} for this tiny matrix. That is a calculation, not a license to call $M^{(3)}$ a point of $\mathcal{B}_2$.

Finite is not exact

For logits $Z \in \mathbb{R}^{d \times d}$, the finite T -step map we actually run is

$$S_T(Z) = M^{(T)}, \quad M^{(0)} = \exp(Z), \quad M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})).$$

Elementwise exponentiation makes every entry positive for finite real logits in exact arithmetic. It also keeps every denominator off zero.

Theorem. For a strictly positive square matrix, alternating row and column scaling converges to a doubly stochastic diagonal scaling of that matrix [R021]; [R022]. More general nonnegative matrices need support conditions. Positivity ducks those edge cases. The theorem is about $T \rightarrow \infty$, not about an arbitrary finite T .

Each repair only rescales rows or columns. In matrix language, the iterates stay of the form $D_1 M^{(0)} D_2$ with positive diagonal D_1 and D_2 . The loop is hunting scale factors that hit the desired sums. That is also why adding one constant to every logit does not change the converged table: it multiplies $M^{(0)}$ by one positive scalar, and the scalings swallow it.

A fixed budget chooses T before seeing the matrix. A tolerance-based loop instead stops at the first t with, for a declared tolerance $\tau > 0$,

$$\varepsilon_\infty(M^{(t)}) \leq \tau.$$

Those policies answer different engineering questions:

- fixed T gives predictable work and a static differentiation graph;
- tolerance stopping gives an input-dependent error certificate, up to floating-point evaluation of the certificate;
- a maximum-iteration cap is still required, because some matrices crawl.

Reserve “exact” for a construction whose algebra enforces both constraints at finite parameter values, apart from floating-point roundoff. Truncated Sinkhorn is approximate in a stronger sense: even exact real arithmetic generally leaves one family of sums off target. A tolerance such as 10^{-5} certifies a bound under the chosen ruler. It does not turn approximation into equality.

A useful test reports three quantities separately:

$$e_r = \max_i |(D\mathbf{1})_i - 1|, \quad e_c = \max_j |(D^\top \mathbf{1})_j - 1|, \quad e_- = \max(0, -\min_{ij} D_{ij}).$$

The first two expose order-dependent leftovers. The last catches a negative entry. For an exponential start, negatives should not arise mathematically. Overflow, invalid arithmetic, or a faulty kernel deserves a less philosophical response than “the theorem said it was fine.”

The mHC paper itself reports a small single-layer deviation and a larger deviation in composed mappings when using 20 iterations [R002]. “Converges to a doubly stochastic matrix” and “returns a doubly stochastic matrix after 20 steps” are different sentences. The first is about a limit. The second is generally false in exact arithmetic, except for special inputs, and floating point adds another error source.

Gradients through a stopped loop

For a fixed finite T , S_T is a composition of smooth operations wherever every denominator is positive. Automatic differentiation can therefore push a loss gradient from $H_l^{\text{res}} = S_T(Z_l)$ back to the logits Z_l through every exponentiation, sum, and division.

This is the gradient of the *truncated* algorithm. It is not the gradient of the exact infinite-iteration limit. The two may approach each other as the leftovers shrink. They are not the same object by definition.

Naive reverse-mode autodiff may save the intermediate matrix from all T iterations, so $O(Td^2)$ saved values per routed item. Recomputation trades arithmetic for activation memory. The mHC paper reports a custom backward kernel that recomputes Sinkhorn intermediates on chip and walks the full iteration [R002].

Tolerance-based early stopping adds a discrete choice of how many steps to take. Each fixed-count branch is differentiable. The choice of branch is not a smooth function at the stopping boundaries. The original mHC experiments used the fixed value $T = 20$ [R002].

Hence a training run that stops the loop at a tolerance is differentiating a piecewise object. Fixed T avoids that seam. It does not make the forward map exact.

Numbers that explode

Exponentials overflow for large positive logits and underflow for large negative ones. Subtract a common maximum before exponentiating,

$$M^{(0)} = \exp(Z - \max_{ij} Z_{ij}).$$

That changes only a global positive scale, so it does not change any later normalized matrix in exact arithmetic. Log-domain updates are another standard option when the numbers get ugly [R024]. Neither trick makes a finite loop exact.

Adding a small ϵ to every denominator can prevent division by zero, but it also changes the equation: the resulting sum is $s/(s + \epsilon)$, not one. With finite logits and a stable exponential, denominators should be positive. Production code should diagnose non-finite values rather than quietly redefine the constraint.

The mHC paper's coefficient pipeline uses mixed precision but lists the final pre, post, and residual mappings in float32, and it implements the Sinkhorn loop in a fused kernel [R002]. That is a report about one optimized implementation, not a theorem that float32 or 20 steps is universally enough.

A silent ϵ in the denominator is not a stabilizer of the original problem. It is a different problem.

What it costs, and a loop you can type

Let b be batch size, m sequence length, d the number of leftover residual streams, and c the feature width. A full residual state has shape

$$x_l \in \mathbb{R}^{b \times m \times d \times c}.$$

At each routed position the residual router is a $d \times d$ table

$$H_l^{\text{res}} \in \mathbb{R}^{d \times d}.$$

The two d -axes are source stream and destination stream. Neither is a token axis. With dynamic routing over a batch and a sequence, the collection of tables is $H_l^{\text{res}} \in \mathbb{R}^{b \times m \times d \times d}$.

Suppose $q = bm$ routed positions each own a $d \times d$ logit matrix. One column pass and one row pass each touch $\Theta(qd^2)$ entries. For T full iterations the normalization costs

$$\Theta(Tqd^2)$$

arithmetic and memory traffic. It stores $\Theta(qd^2)$ current values. Naive autodiff can additionally retain $\Theta(Tqd^2)$ intermediates. Applying each finished router to d streams of width c costs $\Theta(qd^2c)$, separate from the normalization itself.

For the paper's $d = 4$ and $T = 20$, each matrix is tiny. Tiny kernels are not automatically free: reductions, launches, and repeatedly moving small tensors can dominate their modest flop count. That is why fusion and a custom backward exist. It is not a claim that big-O notation paid the electricity bill.

The following is pedagogical Julia, not the paper's production kernel:

```
"""Input/output: logits/router (... , d, d) → (... , d, d)."""
function finite_sinkhorn(logits; steps=20)
    shifted = logits .- maximum(logits; dims=(ndims(logits) - 1, ndims(logits)))
```

```

matrix = exp.(shifted)
for _ in 1:steps
    matrix = matrix ./ sum(matrix; dims=ndims(matrix) - 1) # columns
    matrix = matrix ./ sum(matrix; dims=ndims(matrix))      # rows
end
return matrix
end

"""Input (... , d, d); outputs e_row, e_col, e_negative over last axes."""
function constraint_errors(matrix)
    rdim, cdim = ndims(matrix) - 1, ndims(matrix)
    e_row = dropdims(maximum(abs.(sum(matrix; dims=cdim) .- 1); dims=cdim); dims=cdim)
    e_col = dropdims(maximum(abs.(sum(matrix; dims=rdim) .- 1); dims=rdim); dims=rdim)
    e_negative = dropdims(max.((-).(minimum(matrix; dims=(rdim, cdim))), 0); dims=(rdim, cdim))
    return e_row, e_col, e_negative
end

```

The leading axes may be $[b, m]$. No explicit inverse is used. A production implementation should define dtype policy, finite-value checks, backward memory strategy, and acceptance tolerances, then benchmark the fused routed layer rather than this loop in isolation.

Batch and sequence ride along. They are not summed away by Sinkhorn. The two contracted axes are the two stream axes of the router.

Where the paper puts the loop

Original mHC first builds unconstrained dynamic-plus-static coordinates $\tilde{H}_l^{\text{res}} \in \mathbb{R}^{d \times d}$ from the normalized, flattened d -stream hidden state. It then sets [R002]

$$H_l^{\text{res}} = \text{SinkhornKnopp}(\tilde{H}_l^{\text{res}}).$$

The paper separately maps its pre-routing coordinates through a sigmoid and its post-routing coordinates through twice a sigmoid. Sinkhorn specifically constrains the residual state-to-state router. The routed layer remains

$$x_{l+1} := H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l).$$

For one position, $x_l \in \mathbb{R}^{d \times c}$, $H_l^{\text{res}} x_l \in \mathbb{R}^{d \times c}$, $H_l^{\text{pre}} \in \mathbb{R}^{1 \times d}$, and $H_l^{\text{post}} \in \mathbb{R}^{1 \times d}$. The paper’s parameterization derives mappings from the flattened per-position dc -dimensional state. Its mapping visualization averages matrices over tokens in a selected sequence [R002].

Empirical result. Xie et al. used $d = 4$ for HC and mHC and fixed the Sinkhorn maximum at 20 iterations in their reported 3B, 9B, and 27B experiments [R002]. In the 27B comparison, the paper reports that unconstrained HC had a loss surge around step 12,000 correlated with gradient-norm instability, whereas mHC’s loss and gradient-norm profiles were stable relative to that HC run. It reports a final training-loss reduction of 0.021 for mHC versus its baseline.

Empirical result. On the paper’s eight reported downstream benchmarks for the 27B models, mHC exceeded the baseline on all eight and exceeded HC on seven of eight; MATH was the exception (26.0 for mHC versus 26.4 for HC). These are paper-reported results for its models, data, and evaluation settings — not evidence that mHC dominates every HC configuration.

Empirical result. The paper explicitly observes that 20 iterations only approximate double stochasticity. Its Figure 7 reports slight single-layer backward-gain deviation from one and a maximum composite gain of about 1.6, while HC’s maximum gain was nearly 3000 in the compared analysis [R002]. This supports improved

propagation behavior in that experiment. It does not establish exact conservation for finite Sinkhorn or prove stability for all depths and logits.

Empirical result. With its kernel fusion, custom recomputing backward, activation recomputation, and pipeline scheduling, the paper reports 6.7% additional training time for mHC with $d = 4$ in its in-house large-scale setting [R002]. That number measures the whole optimized system in a stated regime. It should not be relabeled as the isolated cost of Sinkhorn, nor assumed for an unfused framework loop.

Interpretation. Finite Sinkhorn gave the paper a practical compromise: residual routers close enough to the Birkhoff constraints to markedly reduce measured gain drift, while retaining differentiability and bounded, predictable work. The evidence is promising recent research, not settled architectural law.

A theorem about $T \rightarrow \infty$ does not earn those four measurements. The measurements do not earn the theorem.

Skip a repair

If we skip positivity, negative routing weights can cancel signals, and alternating sum-normalization no longer supplies a convex-combination reading. Zero patterns can also violate the support conditions needed for matrix scaling.

If we normalize only rows, forward destination mixtures are convex, but source totals need not balance. In backpropagation through $y = Hx$, the transpose $H^\top$ appears. Uncontrolled column sums then matter directly to aggregate gradient gain.

If we normalize only columns, the mirror-image problem remains: outgoing source weight is balanced, but each destination's incoming total is not fixed.

If we omit repeated alternation, the second normalization damages the first constraint and leaves a potentially large leftover. In our example, one full iteration left $\varepsilon_\infty \approx 0.042$; three left about 4.38×10^{-6} .

If we omit error measurement, "20 iterations" becomes a ritual rather than evidence. Different logit ranges and dtypes can converge at different rates. A fixed budget is valid, but its observed errors should be characterized in the actual training regime.

Finally, omitting Sinkhorn does not mathematically guarantee failure. It returns us to unconstrained HC, whose router can learn useful mappings but lacks the double-stochastic constraint. The 27B instability is an empirical observation reported for the paper's comparison, not a theorem that every unconstrained HC run must diverge.

Hence each skipped hypothesis fails a different promise. The last one fails no theorem at all. It fails a measured comparison in one paper.

Ways to get this wrong

Calling the elementwise $\exp(Z)$ a matrix exponential. The matrix exponential mixes powers of Z . Sinkhorn needs entrywise positivity.

1. Claiming exactness after a fixed number of steps. One final row normalization makes rows exact only in ideal arithmetic. Columns remain approximate in general.
2. Checking only the last-normalized axis. Always compute both e_r and e_c .
3. Hiding the normalization order. Column-then-row and row-then-column have different finite outputs.
4. Treating tolerance as equality. $\varepsilon_\infty \leq 10^{-5}$ means a bounded violation in one ruler, not membership proved by exact algebra.
5. Ignoring finite precision. Stabilize exponentiation and inspect non-finite values. An epsilon denominator changes the constraint and should be disclosed.
6. Stopping gradients through the router. Unless you are deliberately ablating, gradients must walk the finite Sinkhorn operations so the logits can learn.
7. Quoting 6.7% as a universal overhead. It is the paper's optimized end-to-end measurement at $d = 4$, not a portable constant.
8. Attributing every mHC system cost to Sinkhorn. Widened-stream I/O, routing application, saved activations, recomputation, and communication also matter.

Try it

1. One repair damages another

Apply column normalization and then row normalization to

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}.$$

Report the two row sums and two column sums after the full iteration.

Answer

Column normalization gives $\begin{pmatrix} 2/3 & 1/2 \\ 1/3 & 1/2 \end{pmatrix}$. Its row sums are $7/6, 5/6$. Row normalization gives $\begin{pmatrix} 4/7 & 3/7 \\ 2/5 & 3/5 \end{pmatrix}$. Row sums are $(1, 1)$; column sums are $(34/35, 36/35)$.

2. Measure the leftover

For

$$D = \begin{pmatrix} 0.51 & 0.49 \\ 0.48 & 0.52 \end{pmatrix},$$

compute e_r , e_c , and ε_∞ .

Answer

Both row sums equal one, so $e_r = 0$. Column sums are 0.99 and 1.01, so $e_c = 0.01$. Therefore $\varepsilon_\infty = 0.01$.

3. Decide whether to stop

Suppose a tolerance-based implementation uses $\tau = 10^{-4}$. Using the errors from the worked example, at which of the first three iterations may it first stop? Does that make the output exact?

Answer

The errors are approximately 4.20×10^{-2} , 4.29×10^{-4} , and 4.38×10^{-6} . The first acceptable iteration is 3. It satisfies the selected tolerance but is not exactly doubly stochastic.

4. Count the entries

Let $b = 8$, $m = 2048$, $d = 4$, and $T = 20$. What is the shape of a per-position logit tensor? How many scalar matrix entries are processed per normalization pass, and what is the asymptotic cost of all full iterations?

Answer

The logits have shape $[8, 2048, 4, 4]$. There are $8 \cdot 2048 \cdot 16 = 262,144$ entries per row or column pass. There are two passes per full iteration. The asymptotic normalization cost is $\Theta(Tbmd^2)$; counting entry visits in this simple model gives $2 \cdot 20 \cdot 262,144 = 10,485,760$, excluding reductions' implementation details and backward work.

5. Differentiate one row

Let one row be $u = (u_1, u_2)$ with $u_i > 0$, and let $r_1 = u_1/(u_1 + u_2)$. Compute $\partial r_1/\partial u_1$ and $\partial r_1/\partial u_2$. What numerical danger appears when the denominator is tiny?

Answer

Writing $s = u_1 + u_2$, $\partial r_1/\partial u_1 = u_2/s^2$ and $\partial r_1/\partial u_2 = -u_1/s^2$. A tiny s can make derivatives large and divisions inaccurate. Stable exponentiation or log-domain updates address the scale problem more honestly than silently declaring the denominator to be $s + \epsilon$.

Check yourself

1. What is Sinkhorn, in this chapter?

Start from a positive table and alternately repair column sums and row sums. The name comes after the procedure, not before it.

2. Show that a finished column-then-row step has exact row sums.

$\mathcal{T}_r$ divides each row by that row's sum. In exact arithmetic the new row sums are one. The columns generally are not.

3. What fails if you report only the last-normalized axis?

You can print $e_r = 0$ and miss a column leftover of size ε_∞ .

4. Is $T = 20$ doubly stochastic?

No. It is a finite approximation. The infinite limit is the doubly stochastic object. The mHC paper itself reports leftover gain after 20 steps.

5. What did the 27B comparison actually show?

In that run, unconstrained HC surged around step 12,000; mHC did not, and finished 0.021 lower in training loss. Downstream, mHC beat the baseline on eight of eight reported tasks and beat HC on seven of eight. That is paper-reported evidence, not a theorem.

6. Does the 6.7% figure measure Sinkhorn alone?

No. It is the paper's optimized end-to-end extra training time at $d = 4$.

Next we ask whether we can build a router that is already fair by algebra, with no leftover error to bargain down. That route starts from permutation matrices. Numerical approximation leaves the room. Combinatorics is waiting in the corridor.

Sources

- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880v2, 2026. <https://arxiv.org/abs/2512.24880>
- [R021] R. Sinkhorn and P. Knopp, “Concerning Nonnegative Matrices and Doubly Stochastic Matrices,” *Pacific Journal of Mathematics* 21(2), 343–348, 1967. <https://doi.org/10.2140/pjm.1967.21.343>
- [R022] P. A. Knight, “The Sinkhorn–Knopp Algorithm: Convergence and Applications,” *SIAM Journal on Matrix Analysis and Applications* 30(1), 261–275, 2008. <https://doi.org/10.1137/060659624>
- [R024] G. Peyré and M. Cuturi, “Computational Optimal Transport,” *Foundations and Trends in Machine Learning* 11(5–6), 355–607, 2019; arXiv:1803.00567. <https://doi.org/10.1561/22000000073>
- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024. <https://arxiv.org/abs/2409.19606>

Chapter 10 — Mixing a few seat shuffles

Three people walk into a room with three labeled chairs. A shuffle is a complete reassignment: Alice sits where Bob sat, Bob where Carol sat, Carol where Alice sat. Nobody gets two chairs. No chair is left empty.

That is already a wiring diagram. Each destination takes exactly one source, and every source is used once.

Now mix two shuffles. With weight 0.7, leave everyone put. With weight 0.3, swap the first two chairs. Alice's new seat is seven-tenths her old one and three-tenths Bob's. The bookkeeping still balances, because the same two numbers scale the whole seating chart, not one chair at a time.

Chapter 9 repaired a table by alternately fixing its rows and columns. After finitely many repairs, one side can still be wrong. This chapter writes the table so that every output is already balanced.

Two chairs, two shuffles

Two people, two chairs. There are only two seating charts: leave them put, or swap.

Write a 1 where an output chair reads an input chair, and a 0 everywhere else:

$$P_{\text{id}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad P_{\text{swap}} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Each of those matrices has one 1 per row and one 1 per column. Mix them with a weight a between 0 and 1:

$$H = aP_{\text{id}} + (1-a)P_{\text{swap}} = \begin{pmatrix} a & 1-a \\ 1-a & a \end{pmatrix}.$$

Every entry is nonnegative. Each row adds to 1. Each column adds to 1. Every nonnegative 2×2 table with those sums has exactly this form.

That matrix of zeros and a single 1 in each row and column is a **permutation matrix**. It is a complete seating chart, not a bag of independent wires. One number will later scale the whole chart.

Hence $\mathcal{B}_2$ is this line segment, and its two corners are the two charts. Drop $0 \leq a \leq 1$ and H leaves the segment, even if the two diagonal entries still match.

Three chairs by hand

Take three streams and three seating charts. The same convention: $P_{ij} = 1$ means output stream i reads input stream j .

$$P_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad P_2 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad P_3 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

P_1 leaves everyone put. P_2 is the cycle “1 reads 2, 2 reads 3, 3 reads 1.” P_3 cycles the other way.

Give the three charts the weights

$$a = (0.5, 0.3, 0.2) \in \mathbb{R}^3, \quad a_k \geq 0, \quad \sum_{k=1}^3 a_k = 1.$$

The mix is

$$H = 0.5P_1 + 0.3P_2 + 0.2P_3 = \begin{pmatrix} 0.5 & 0.3 & 0.2 \\ 0.2 & 0.5 & 0.3 \\ 0.3 & 0.2 & 0.5 \end{pmatrix} \in \mathbb{R}^{3 \times 3}.$$

Every entry is nonnegative. The row sums and the column sums are all one:

$$H\mathbf{1}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad H^\top \mathbf{1}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

So H sits in $\mathcal{B}_3$, the set of all 3×3 nonnegative matrices with those sums. Chapter 8 called that set the Birkhoff polytope. We reached a point inside it by mixing seating charts, not by repairing a table.

Now hide batch, token, and feature axes and give each stream one number:

$$x = (10, 20, 40)^\top \in \mathbb{R}^3.$$

Then

$$Hx = \begin{pmatrix} 19 \\ 24 \\ 27 \end{pmatrix}.$$

The total is still $19 + 24 + 27 = 70 = 10 + 20 + 40$. Column sums keep the total. Row sums say something else: if every stream already holds the same value, $x = q\mathbf{1}_3$, the mix leaves that constant alone.

A weighted average of seating charts with nonnegative weights that add to one is a **convex mixture**. The same two rules — nonnegative, add to one — are what make the output balanced. One coefficient still belongs to a whole chart, not to a single chair.

Three charts are not all six of $\mathcal{B}_3$. This H is one point on a triangle inside a four-dimensional body. Exact balance is not full coverage.

From chairs to leftover copies

The chairs are the leftover copies of a sentence table that earlier chapters called residual streams. Let b be how many sentences sit in a batch, m the number of tokens, d the number of those leftover copies, and c the feature width. A hidden state has shape

$$X_l \in \mathbb{R}^{b \times d \times m \times c}.$$

A residual router acts only on the stream axis. It does not shuffle the batch, the tokens, or the features. When the router can change at each token, write

$$H_l^{\text{res}} \in \mathbb{R}^{b \times m \times d \times d}.$$

For batch index β , output stream i , token t , and feature r , the routed state is

$$\tilde{X}_{l,\beta,i,t,r} = \sum_{j=1}^d H_{l,\beta,t,i,j}^{\text{res}} X_{l,\beta,j,t,r}.$$

The paper’s language-model experiments look at those token-level residual matrices [R003]. The seating-chart construction itself does not care how often a model writes a new router. The same mix works for one matrix or for a stack of them.

The contracted index is the source stream j . Token t and feature r ride along. Swap those meanings and the arithmetic may still run. The model will be talking to the wrong house.

One 1 in each row and column

A **permutation** of $\{1, \dots, d\}$ is a one-to-one reassignment of those labels: a seating chart. Call it σ . Its matrix $P_\sigma \in \{0, 1\}^{d \times d}$ is

$$(P_\sigma)_{ij} = \begin{cases} 1, & j = \sigma(i), \\ 0, & \text{otherwise.} \end{cases}$$

Each row contains one 1 because each output chooses one input. Each column contains one 1 because no input is chosen twice. Therefore

$$P_\sigma \mathbf{1}_d = \mathbf{1}_d, \quad P_\sigma^\top \mathbf{1}_d = \mathbf{1}_d, \quad P_\sigma \geq 0.$$

Every permutation matrix is already balanced. Some texts flip the convention and put a 1 at $i = \sigma(j)$. Either rule is fine. Changing it halfway through a calculation is not.

Hence $P_\sigma \in \mathcal{B}_d$ with no further work. The finite Sinkhorn leftover of Chapter 9 does not appear, because there is nothing left to repair.

Mixing always balances

Take any list of permutation matrices and any nonnegative weights that add to one. The mix

$$B = \sum_k a_k P_k$$

cannot go negative, because neither the weights nor the charts do. Linearity does the rest:

$$B \mathbf{1}_d = \sum_k a_k P_k \mathbf{1}_d = \left(\sum_k a_k \right) \mathbf{1}_d = \mathbf{1}_d,$$

and the same argument on the transpose gives $B^\top \mathbf{1}_d = \mathbf{1}_d$.

That is the exactness argument. Every convex mixture of seating charts is doubly stochastic. The argument never asked how long the list was. One chart is already enough. A short list is enough. The full list is enough.

“Exact” describes the algebra. Floating-point additions still round. The construction does not promise a bit-for-bit row sum of 1.0 on every device.

If the weights can go negative, row and column sums can still equal one while some routing weights go below zero. The matrix leaves $\mathcal{B}_d$. If they do not add to one, the all-ones mode is scaled instead of left alone.

Every balanced table is some mix

The other direction is older than any language model. Birkhoff and von Neumann proved that a matrix $B \in \mathbb{R}^{d \times d}$ is doubly stochastic if and only if it is a convex mixture of $d \times d$ permutation matrices [R019]; [R020]. If $P_1, \dots, P_{d!}$ list every seating chart, then

$$B \in \mathcal{B}_d \iff B = \sum_{k=1}^{d!} a_k P_k$$

for some $a \in \mathbb{R}^{d!}$ with

$$a_k \geq 0, \quad \sum_{k=1}^{d!} a_k = 1.$$

In geometry: the permutation matrices are the corners of the Birkhoff polytope, and the polytope is their convex hull,

$$\mathcal{B}_d = \text{conv}\{P_1, \dots, P_{d!}\}.$$

The name is worth keeping because the **Birkhoff–von Neumann theorem** is two claims glued together: every mix is balanced, and every balanced table is some mix. The first claim is cheap. The second claim, through this explicit list, is not.

The easy half is the last section. Here is the other half, as a peeling argument.

Start with $B \in \mathcal{B}_d$. Draw a bipartite graph: output chairs on the left, input chairs on the right, and an edge (i, j) whenever $B_{ij} > 0$. Any set S of outputs currently sends total mass $|S|$. Each neighboring input can take total mass at most 1, so S must touch at least $|S|$ inputs. Hall’s matching theorem then supplies a perfect matching sitting on the positive entries. That matching is a permutation matrix P .

Let ε be the smallest B_{ij} on that matching. Then $B - \varepsilon P$ is still nonnegative, and every row and column now sums to $1 - \varepsilon$. If $\varepsilon < 1$, renormalize the remainder:

$$B' = \frac{B - \varepsilon P}{1 - \varepsilon} \in \mathcal{B}_d, \quad B = \varepsilon P + (1 - \varepsilon)B'.$$

At least one positive entry becomes zero. Repeat. The process stops after finitely many steps and writes B as a convex mixture. The writing is not unique. Interior points usually have many writings.

Hence membership in $\mathcal{B}_d$ is exactly “being some mix of the $d!$ charts.” Existence of a writing is not uniqueness of a writing, and it is not a promise that one fixed short list will serve every target.

A short mix is enough for one table

A $d \times d$ table has d^2 entries. The row-sum and column-sum rules are not independent: if you know every row sum and all but one column sum, the last column sum is forced. That is $2d - 1$ independent equations, so the balanced tables live in a flat of dimension $(d - 1)^2$.

Carathéodory’s theorem then says any one point in that flat is a mix of at most

$$(d - 1)^2 + 1 = d^2 - 2d + 2$$

corners [R025]. So a single target table never needs the whole list.

That existence result does **not** hand you one fixed short list that can represent every table. Different targets may need different corners. A router that must move from target to target either keeps a rich fixed list or solves a discrete selection problem as the target changes.

A sparse decomposition of one matrix is therefore not a scalable parameterization of all of $\mathcal{B}_d$. Confusing the two is how a $d^2 - 2d + 2$ bound gets quoted as a K -parameter architecture.

A vote among seating charts

The network still has to pick the weights. Flatten the d stream vectors of width c at one token into

$$\hat{x}_l \in \mathbb{R}^{dc}.$$

Following the paper's full-list construction [R003], a learned affine map writes one score per seating chart. With normalized features $\hat{x}_l' \in \mathbb{R}^{dc}$, projection weights $W_l^{\text{res}} \in \mathbb{R}^{dc \times d!}$, bias $b_l^{\text{res}} \in \mathbb{R}^{d!}$, and a learned scalar gain $\alpha_l^{\text{res}} \in \mathbb{R}$,

$$u_l = \alpha_l^{\text{res}} \hat{x}_l' W_l^{\text{res}} + b_l^{\text{res}} \in \mathbb{R}^{d!}.$$

Turn those scores into positive weights that add to one, then mix:

$$a_l = \text{softmax}(u_l) \in \mathbb{R}^{d!}, \quad H_l^{\text{res}} = \sum_{k=1}^{d!} a_{l,k} P_k \in \mathbb{R}^{d \times d}.$$

Softmax is that map from raw scores to a convex mixture. Finite scores never quite reach a corner: every chart gets a little weight. An exact permutation is only a limit. The closed-simplex theorem covers the whole polytope. Softmax covers the interior. Exact double stochasticity is not the thing that fails. The slogan “represents every matrix” is, if you say it literally.

For all $b \times m$ routing sites the scores and weights have shape $b \times m \times d!$, and the router has shape $b \times m \times d \times d$. The paper implements the last sum as one matrix multiply: flatten the constant list from shape $d! \times d \times d$ to $d! \times d^2$, multiply, then reshape [R003].

The initialization reported in that paper piles weight on the identity chart. Finite biases still leak a little mass onto every other chart. You do not get an exact one-hot vector.

This construction — mix seating charts with a softmax, and keep the rest of the mHC layer — is **mHC-lite** [R003]. It names a coordinate system, not a new kind of leftover copy. The streams were already there.

The paper is a January 2026 preprint [R003]. The algebra above is a theorem. The training comparisons later in the chapter are recent measurements, not settled consensus.

How many seating charts?

Two chairs: $2! = 2$. Three chairs: $3! = 6$. The first output has d possible sources, the next has $d - 1$, and so on:

$$d(d-1) \cdots 2 \cdot 1 = d!.$$

That product is the **factorial**. It is just the number of seating charts. It is not a claim that every algorithm for a balanced table must list them all.

The numbers get rude quickly:

$$4! = 24, \quad 8! = 40,320, \quad 16! = 20,922,789,888,000 \approx 2.09 \times 10^{13}.$$

The projection alone has $(dc)d! + d!$ weights and biases per layer. The fixed 0/1 list has $d!d^2$ entries. A direct mix costs on the order of $d!d^2$ arithmetic operations per routing site, before anyone pays to produce the scores.

For $d = 4$, the paper's experimental setting, 24 is a small constant [R003]. For $d = 8$, even with $c = 64$, the projection has shape $512 \times 40,320$, or 20,643,840 weights before its bias. At $d = 16$, merely writing down the coefficients is already hopeless. Full coverage by listing every chart is not a scalable parameterization.

The factorial is a count of corners, not a recommended layer width.

Keep only a few

The same paper suggests keeping only K sampled charts when d gets larger [R003]. Fix a list

$$\mathcal{S} = \{P_{s_1}, \dots, P_{s_K}\}, \quad K \ll d!,$$

produce $a_l \in \mathbb{R}^K$ by softmax, and set

$$H_l^{\text{res}} = \sum_{k=1}^K a_{l,k} P_{s_k}.$$

Now $W_l^{\text{res}} \in \mathbb{R}^{d \times K}$, the list has shape $K \times d \times d$, and the direct mix costs $O(Kd^2)$ per routing site. Every output is still exactly doubly stochastic. The easy half of the theorem never needed the full list.

The reachable region is only

$$\text{conv}(\mathcal{S}) \subseteq \mathcal{B}_d,$$

and the inclusion is usually strict. K is a knob for compute, parameters, and coverage. That is the distinction that survives at large d :

Construction	Coefficients	Doubly stochastic?	Birkhoff coverage	Scaling
All charts, closed simplex	$d!$	exact	full	factorial
All charts, finite-logit softmax	$d!$	exact	relative interior; full in the closure	factorial
Fixed subset	K	exact	generally only $\text{conv}(\mathcal{S})$	tunable in K

Call the stored list a **permutation bank**. The bank is a tensor of whole charts, shape $K \times d \times d$, not a bag of independent edges. Combining scores of shape $b \times m \times K$ with that bank produces routers of shape $b \times m \times d \times d$. No batch, token, or feature axis is being shuffled.

A bank that holds only the identity and one swap cannot write a three-cycle. Exactness survives. Coverage does not.

The rest of the layer stays

mHC-lite changes how the persistent-state router H_l^{res} is written. It keeps the surrounding read, branch, and write structure [R003]:

$$X_{l+1} = H_l^{\text{res}} X_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} X_l, W_l).$$

Here H_l^{pre} reads from the d streams, F computes one branch value at each routing site, and H_l^{post} writes that value back. The seating-chart mix governs only the leftover copies that persist. The theorem does not constrain the branch, and it does not invent streams that were not already there.

Empirical result. Yang and Gao report experiments with $d = 4$ and with 6-, 12-, and 24-layer language models on OpenWebText and FineWeb-Edu. Their final-loss table shows mHC-lite broadly comparable to mHC in that setup. Their measured gradient-norm curves fluctuate less than their mHC reimplementations [R003]. Those

measurements support the proposal in the tested regimes. They do not establish universal superiority. They do not test factorial-scale stream counts.

The construction trades iterative repair for an overcomplete convex coordinate system. Later chapters will write other exact routers that refuse to list every global chart. That comparison has to be earned, not smuggled in under a large coat.

A theorem about $\mathcal{B}_d$ does not train a 24-layer model. A 24-layer run does not prove the theorem.

What breaks if we cheat

If the mixture weights can go negative, row and column sums can still equal one while some routing weights go below zero. The table leaves the Birkhoff polytope.

If the weights do not add to one, then

$$H\mathbf{1}_d = \left(\sum_k a_k \right) \mathbf{1}_d,$$

so the all-ones mode is scaled instead of left alone.

If the basis matrices are arbitrary 0/1 tables rather than seating charts, a source may be copied or dropped. Column or row conservation fails.

If we drop charts from a fixed bank, exact double stochasticity does **not** break. Coverage can. A bank that holds only the identity and one swap cannot write a three-cycle.

If we advertise the full $d!$ bank as scalable, memory breaks first and patience follows. The short bank repairs the scaling only by accepting a smaller polytope.

Hence exactness and coverage are different promises. Cheating the weights breaks the first. Cheating the bank breaks the second.

Ways to get this wrong

- Treating each 1 in a permutation matrix as an independently weighted edge. One coefficient weights the whole chart.
- Confusing exactness with coverage. A two-chart mix can be exactly balanced and still only reach a line segment inside $\mathcal{B}_d$.
- Calling the decomposition unique. Interior points usually have several writings.
- Reversing the convention silently. Say whether rows select sources or columns select destinations.
- Forgetting the factorial router parameters. The constant bank is not the only cost; the dynamic projection emits $d!$ scores.
- Claiming softmax emits a permutation exactly. Finite scores give positive weights. A one-hot vector is a limit.
- Thinking a sparse decomposition solves dynamic scaling. A short writing for each target is not one universal short bank.
- Equating doubly stochastic with orthogonal. A permutation is orthogonal. An average of permutations generally is not.

Try it

1. Read a seating chart

Let

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad x = (4, 7, 9)^\top.$$

Compute Px and check both kinds of sums for P .

Answer

$Px = (7, 9, 4)^\top$. Every row and every column contains one 1, so $P\mathbf{1}_3 = \mathbf{1}_3$ and $P^\top \mathbf{1}_3 = \mathbf{1}_3$.

2. Unmix a 2×2 router

Write

$$B = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$$

as a convex mixture of the two 2×2 permutation matrices.

Answer

$$B = 0.7 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + 0.3 \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Both coefficients are nonnegative and sum to one.

3. A short list is still balanced

Let $P_{s_1}, \dots, P_{s_K} \in \mathbb{R}^{d \times d}$ be any fixed permutation matrices and $a = \text{softmax}(u) \in \mathbb{R}^K$. Show that $H = \sum_k a_k P_{s_k}$ is doubly stochastic. Does the argument need $K = d$?

Answer

Softmax gives $a_k > 0$ and $\sum_k a_k = 1$. Hence $H \geq 0$, $H\mathbf{1} = \sum_k a_k P_k \mathbf{1} = \mathbf{1}$, and similarly $H^\top \mathbf{1} = \mathbf{1}$. The argument does not need every permutation; $K = 1$ already preserves the constraint.

4. Count the burden

For $d = 8$ and $c = 64$, find the shape and number of entries of W_l^{res} in the full-bank construction. How many entries are in the flattened permutation bank?

Answer

Here $dc = 512$ and $d! = 40,320$. Thus W has shape $512 \times 40,320$ and 20,643,840 entries. The bank has $d!d^2 = 40,320 \times 64 = 2,580,480$ entries.

5. Find the missing chart

For $d = 3$, take a bank containing only the identity and the permutation that swaps streams 1 and 2. Show that no mixture in this bank equals the three-cycle P_2 from the worked example.

Answer

Both bank matrices leave stream 3 fixed, so every mixture has entry $H_{33} = 1$. The three-cycle has $(P_2)_{33} = 0$. It sits outside this bank's convex hull, although every matrix inside the hull remains doubly stochastic.

6. Corners and softmax

Why can finite scores not produce the identity matrix exactly when the bank holds every permutation? Can they approach it arbitrarily closely?

Answer

Finite-logit softmax assigns every nonidentity permutation positive weight, so off-diagonal entries receive positive mass. Sending the identity score above all others without bound drives its weight to one and the rest to zero, so the identity is reached in the limit.

Check yourself

1. **What is a permutation matrix doing to the streams?**

A one-to-one rewiring. Each output reads exactly one input. Each input is used once.

2. **Show that a mix of those matrices is already balanced.**

Nonnegative weights that add to one, times nonnegative charts that already have row and column sums equal to one, stay nonnegative and keep those sums.

3. **What extra claim does Birkhoff–von Neumann add?**

Every doubly stochastic matrix is some such mix, if the bank holds all $d!$ charts and the weights may sit on the closed simplex.

4. **What fails if you list every chart at large d ?**

$4! = 24$ is fine. $8! = 40,320$ already inflates the projection. $16! \approx 2.09 \times 10^{13}$ is not a parameterization.

5. **What do you keep, and what do you lose, if you keep only K charts?**

You keep exact double stochasticity. You generally lose the rest of the Birkhoff polytope.

6. **Given scores of shape $b \times m \times K$ and a bank of shape $K \times d \times d$, what comes out?**

A router of shape $b \times m \times d \times d$. The mix is over streams. Batch, token, and feature axes ride along.

Sources

- [R003] Y. Yang and J. Gao, “mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations,” arXiv:2601.05732v1, 2026. <https://arxiv.org/abs/2601.05732>
- [R019] G. Birkhoff, “Tres observaciones sobre el algebra lineal,” *Universidad Nacional de Tucumán, Revista A*, vol. 5, pp. 147–151, 1946. MathSciNet MR0020547: <https://mathscinet.ams.org/mathscinet-getitem?mr=20547>
- [R020] J. von Neumann, “A Certain Zero-Sum Two-Person Game Equivalent to the Optimal Assignment Problem,” in *Contributions to the Theory of Games II*, pp. 5–12, 1953. <https://doi.org/10.1515/9781400881970-002>
- [R025] R. T. Rockafellar, *Convex Analysis*, Princeton University Press, 1970. ISBN 978-0-691-01586-6. <https://press.princeton.edu/books/paperback/9780691015866/convex-analysis>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R021] R. Sinkhorn and P. Knopp, “Concerning Nonnegative Matrices and Doubly Stochastic Matrices,” *Pacific Journal of Mathematics*, vol. 21, no. 2, pp. 343–348, 1967. <https://doi.org/10.2140/pjm.1967.21.343>

Chapter 11 — Small mixers, tiled

A two-by-two mixer already knows how to share mass between two leftover copies of a sentence. Four copies ask for a four-by-four table. Sixteen independent entries would then have to be repaired, afterward, into nonnegativity and unit row and column sums.

Copy the small mixer four times instead. Scale each copy by one entry of a second two-by-two mixer, and glue the copies into one square. The square is still a mixer: every entry stays nonnegative, every row still adds to one, every column still adds to one. You never wrote sixteen free numbers.

That is the whole construction. It is cheap. It is exact in real arithmetic. It also refuses some mixings a free four-by-four mixer can do, and we will see exactly which jobs it turns down.

Four copies of a tile

Call the first small mixer A and the second B :

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Replace every entry of A by a whole copy of B , scaled by that entry:

$$\begin{pmatrix} aB & bB \\ cB & dB \end{pmatrix} = \begin{pmatrix} ae & af & be & bf \\ ag & ah & bg & bh \\ ce & cf & de & df \\ cg & ch & dg & dh \end{pmatrix}.$$

Read the first block. The number a does not sit alone; it becomes the whole tile aB . The number b becomes the tile bB to its right. The output is 4×4 , not 2×2 . This is not ordinary matrix multiplication. The side length multiplies.

That glued matrix is the **Kronecker product** $A \otimes B$. The name is worth keeping because the tile rule is not “multiply A by B .” It is “pave a bigger floor with scaled copies of B .”

In general, if A is $p \times p$ and B is $q \times q$, then $A \otimes B$ is $pq \times pq$:

$$A \otimes B = \begin{pmatrix} a_{11}B & \cdots & a_{1p}B \\ \vdots & \ddots & \vdots \\ a_{p1}B & \cdots & a_{pp}B \end{pmatrix}.$$

Suppose you multiply A by B in the ordinary way. Then you get a 2×2 table whose entries are sums of products, not a 4×4 table whose entries sit in scaled tiles. Drop the distinction and the shapes stop matching the leftover-copy axis.

Two bits instead of one index

The four leftover copies — the four **streams** — can be labeled by a pair of bits instead of a single integer 0, 1, 2, 3:

$$(0, 0), (0, 1), (1, 0), (1, 1).$$

One two-by-two mixer stirs the first bit. The other stirs the second bit. The tiled product does both jobs at once. The rules repeat across the floor. That is why the construction is cheap. It is also why a diagonal shortcut on the floor is illegal: each factor only sees its own coordinate.

Original mHC builds a large mixer by writing a free matrix and then repairing its rows and columns with a finite Sinkhorn loop. The repair is approximate. mHC-lite instead mixes permutation matrices and lands exactly on a mixer whose rows and columns sum to one, but listing every rearrangement of d streams costs $d!$ slots. KromHC asks for a middle path: glue small exact mixers into one large exact mixer, and do not pay for every large permutation. [R004]

Choose integers $i_1, \dots, i_K \geq 2$ whose product is the stream count d :

$$d = \prod_{k=1}^K i_k.$$

Learn one small mixer per factor,

$$U_l^k \in \mathbb{R}^{i_k \times i_k}, \quad k = 1, \dots, K,$$

each already a mixer with nonnegative entries and unit row and column sums. The large residual mixer is the tiled product

$$H_l^{\text{res}} = U_l^K \otimes U_l^{K-1} \otimes \dots \otimes U_l^1 \in \mathbb{R}^{d \times d}.$$

That residual mixer is a **KromHC** router. The paper parameterizes each small factor as a convex mix of its own permutation matrices. The read and write routers stay in the ordinary mHC style. Only the residual mixer is tiled. [R004]

The paper calls the stream width n . Here that letter is d . Batch size is B , sequence length is m , feature width is c , so the state is

$$x_l \in \mathbb{R}^{B \times d \times m \times c}.$$

The residual mixer acts only on the second axis. It does not secretly reshuffle tokens.

Suppose the factorization is dropped and a free $d \times d$ table is written instead. Then you are back to asking a repair loop, or a list of $d!$ permutations, to finish the job. The bits are not decoration. They are the coordinates the tiles can see.

Two honest tiles, one big square

Take two mixers that already behave:

$$A = \begin{pmatrix} 0.75 & 0.25 \\ 0.25 & 0.75 \end{pmatrix}, \quad B = \begin{pmatrix} 0.60 & 0.40 \\ 0.40 & 0.60 \end{pmatrix}.$$

Glue them:

$$H = A \otimes B = \begin{pmatrix} 0.45 & 0.30 & 0.15 & 0.10 \\ 0.30 & 0.45 & 0.10 & 0.15 \\ 0.15 & 0.10 & 0.45 & 0.30 \\ 0.10 & 0.15 & 0.30 & 0.45 \end{pmatrix}.$$

Check one tile by hand. The upper-left block is $0.75 B$:

$$0.75 \begin{pmatrix} 0.60 & 0.40 \\ 0.40 & 0.60 \end{pmatrix} = \begin{pmatrix} 0.45 & 0.30 \\ 0.30 & 0.45 \end{pmatrix}.$$

Every entry is a product of nonnegative numbers, so every entry is nonnegative. The first row sums to

$$0.45 + 0.30 + 0.15 + 0.10 = 1,$$

and the other rows and columns do the same. Hence this particular H is already a mixer. One hand-checked table is not a proof for every pair of tiles. It is the arithmetic we are about to do without picking numbers.

To see the mixer move mass, ignore batch, token, and feature for a moment and keep one number per stream:

$$x = \begin{pmatrix} 8 \\ 4 \\ 0 \\ 0 \end{pmatrix}.$$

Then

$$Hx = \begin{pmatrix} 4.8 \\ 4.2 \\ 1.6 \\ 1.4 \end{pmatrix}.$$

The total is still twelve: $8 + 4 = 4.8 + 4.2 + 1.6 + 1.4$. Nothing was invented. Nothing was thrown away.

For any one fixed mixer H , those same sixteen coefficients hit every batch item, every token, and every feature of $x_l \in \mathbb{R}^{B \times 4 \times m \times c}$. How a dynamic implementation chooses a different H at a different place is a separate question. The tiling is about streams.

Drop the nonnegativity of a single factor entry and a negative product appears in H . The row sums can still be one. The table is then no longer a mixer.

Honest tiles stay honest

A square matrix M is **doubly stochastic** when its entries are nonnegative and every row and every column sums to one:

$$M \geq 0, \quad M \mathbf{1}_r = \mathbf{1}_r, \quad M^\top \mathbf{1}_r = \mathbf{1}_r.$$

Here $\mathbf{1}_r$ is a column of r ones. You already met this bookkeeping in Chapter 8. The new fact is that tiling preserves it.

Suppose A is $p \times p$ and doubly stochastic, and B is $q \times q$ and doubly stochastic. Then $A \otimes B$ is doubly stochastic. The same holds for any finite stack of tiles. [R004]; [R018]

The entries of $A \otimes B$ look like $a_{ij}b_{rs}$, so they stay nonnegative. The all-ones vector of length pq itself tiles:

$$\mathbf{1}_{pq} = \mathbf{1}_p \otimes \mathbf{1}_q.$$

Kronecker products act factor by factor, $(A \otimes B)(u \otimes v) = (Au) \otimes (Bv)$, so

$$(A \otimes B)\mathbf{1}_{pq} = (A\mathbf{1}_p) \otimes (B\mathbf{1}_q) = \mathbf{1}_p \otimes \mathbf{1}_q = \mathbf{1}_{pq}.$$

The transpose splits the same way, $(A \otimes B)^\top = A^\top \otimes B^\top$, and the column sums follow. Repeat the two-factor argument to cover more factors. Hence a tiled product of doubly stochastic tiles is doubly stochastic.

That is exactness in real arithmetic. A floating-point chip can still round. “Exact by construction” means there is no leftover Sinkhorn gap from stopping a repair loop early. It does not mean IEEE arithmetic has resigned.

KromHC makes each small tile exact as well. Let $P_1, \dots, P_{i_k!}$ be the permutation matrices of size i_k , and set

$$a_l^k = \text{softmax}(g_l^k) \in \mathbb{R}^{i_k!}, \quad U_l^k = \sum_{r=1}^{i_k!} a_l^k(r) P_r.$$

The weights are nonnegative and add to one. Every permutation matrix is doubly stochastic, so each U_l^k is too, and the tiled product inherits the property. [R004]

Suppose a single factor is allowed to have a row summing to two. Then that factor fails $A\mathbf{1}_p = \mathbf{1}_p$, and the matching row sums of $A \otimes B$ fail with it. One bad tile is enough. Nonnegativity of the other factor does not repair the sum.

Stretch factors multiply

If $Au = \lambda u$ and $Bv = \mu v$, the tiled product stretches the tiled vector by the product of the two stretch factors:

$$(A \otimes B)(u \otimes v) = (Au) \otimes (Bv) = (\lambda u) \otimes (\mu v) = \lambda\mu (u \otimes v).$$

So every pair of factor eigenvalues becomes one eigenvalue of the big mixer. Counting algebraic multiplicity, including for matrices that are short of a full set of eigenvectors,

$$\text{spec}(A \otimes B) = \{\lambda_i(A)\mu_j(B) : 1 \leq i \leq p, 1 \leq j \leq q\}.$$

That set of stretch factors is the **spectrum**. For more tiles, pick one eigenvalue from each factor and multiply:

$$\text{spec}\left(\bigotimes_{k=1}^K U^k\right) = \left\{ \prod_{k=1}^K \lambda_{j_k}(U^k) \right\}_{j_1, \dots, j_K}.$$

Every two-by-two doubly stochastic mixer has one free number:

$$U(t) = \begin{pmatrix} t & 1-t \\ 1-t & t \end{pmatrix}, \quad 0 \leq t \leq 1.$$

The vector $(1, 1)^\top$ stretches by 1. The vector $(1, -1)^\top$ stretches by $2t - 1$. For the numerical tiles above,

$$\text{spec}(A) = \{1, 0.5\}, \quad \text{spec}(B) = \{1, 0.2\},$$

so

$$\text{spec}(A \otimes B) = \{1, 0.5, 0.2, 0.1\}.$$

Hence the spectrum of a tiled mixer is a product set, not a free list of d numbers.

Two uses. First, repeated mixing is predictable: each mode is scaled by a power of its eigenvalue. Second, the limitation is now visible. With only two-by-two doubly stochastic tiles, every eigenvalue is real. The big mixer cannot itself carry the complex conjugate pair that a genuine three-stream rotation needs.

Double stochasticity does bound the spectral norm by one. It does not force every non-stationary eigenvalue to be positive. A tile with $t < 1/2$ has $2t - 1 < 0$, so that mode flips sign each time it is applied. Stability and expressivity are different questions.

What the floor cannot cycle

At a corner of the Birkhoff polytope the mixer is a permutation: a complete wiring diagram. With two-by-two factors, each corner is either the identity I_2 or the swap

$$S = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

On bit-pair labels, a tiled product of these corners flips a fixed subset of bits. Apply it twice and every label is home. So every cycle has length one or two.

In the order $(00, 01, 10, 11)$, the product $S \otimes S$ sends

$$00 \leftrightarrow 11, \quad 01 \leftrightarrow 10.$$

It cannot walk $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$. It also cannot walk one four-cycle through all four streams. Those missing permutations are perfectly doubly stochastic. They are simply not products of two two-by-two permutations.

If one factor permutation has a cycle of length a and another has a cycle of length b , the pair returns after $\text{lcm}(a, b)$ steps. Larger tiles therefore allow longer cycles, and mixed sizes broaden the list. The action is still separable: simultaneous factor moves, not an arbitrary rearrangement of d streams.

The spectrum says the same thing. A three-cycle has eigenvalues

$$1, \quad e^{2\pi i/3}, \quad e^{-2\pi i/3},$$

which cannot appear as products of the real spectra of two-by-two doubly stochastic tiles. That does not say KromHC cannot learn useful routing. It says a cheap choice — many binary tiles — has a particular reach.

Suppose one factor is allowed to be 3×3 . Then a three-cycle can sit inside that factor, and the tiled product can carry it. The obstruction is not “Kronecker products cannot cycle.” It is “binary permutation tiles have order at most two.”

How much of the allowed set we cover

The set $\mathcal{B}_d$ of all $d \times d$ doubly stochastic matrices has dimension

$$(d - 1)^2.$$

A KromHC family with a fixed factorization $d = \prod_k i_k$ has at most

$$\sum_{k=1}^K (i_k - 1)^2$$

geometric degrees of freedom, because each factor U^k lives in $\mathcal{B}_{i_k}$ and that set has dimension $(i_k - 1)^2$. For $d = 4 = 2 \cdot 2$ this is $1 + 1 = 2$. The full set $\mathcal{B}_4$ has dimension 9.

The gap is not a failed training run. Softmax logits can be redundant, but they cannot invent a dimension the tiles do not have. A four-by-four Kronecker matrix also has a visible block test: all four two-by-two blocks must be scalar multiples of the same tile. Most doubly stochastic matrices fail that test.

Hence exact membership in $\mathcal{B}_d$ is not exhaustive coverage of $\mathcal{B}_d$.

Larger tiles trade savings for coverage. A single factor $i_1 = d$ can, in principle, recover the whole Birkhoff family, but its convex-mixture parameterization again lists $d!$ permutations. Small tiles are both the bargain and the fence.

What the paper spends

Let the paper's feature width be c , and flatten the residual input to $x'_l \in \mathbb{R}^{1 \times dc}$. The residual projection matrices are

$$W_l^{\text{res},k} \in \mathbb{R}^{dc \times i_k!}.$$

Including the unchanged pre and post projections, biases, and scalar gains, KromHC reports the per-layer count [R004]

$$2d^2c + (dc + 1) \sum_{k=1}^K i_k! + 2d + 3.$$

When every $i_k = 2$ and $K = \log_2 d$, the factor-router term uses $\sum_k i_k! = 2 \log_2 d$ outputs rather than d^2 . The common pre/post term $2d^2c$ remains, so the paper writes the total parameter complexity as $\mathcal{O}(d^2c)$, against its stated $\mathcal{O}(d^3c)$ for original mHC and $\mathcal{O}(dc d!)$ for mHC-lite. [R004]

There are two ways to apply the mixer. Materialize $H \in \mathbb{R}^{d \times d}$ and multiply, which is ordinary dense routing. Or reshape the stream axis into $i_1 \times \dots \times i_K$ and apply each tile along its own mode. A direct count for the second route is

$$\mathcal{O}\left(Bmc d \sum_{k=1}^K i_k\right),$$

or $\mathcal{O}(Bmc d \log d)$ for binary tiles, without storing the full $d \times d$ mixer. That is an algebraic opportunity, not a claim that every released kernel hits the bound. The paper emphasizes ordinary array ops and parameter count, not a specialized Sinkhorn kernel. [R004]

Empirical result. In the paper's $d = 4$ language-model experiments, KromHC reported fewer extra learned parameters than mHC and mHC-lite: 240K versus 462K and 609K for six Transformer blocks, and 959K versus 1,844K and 2,433K for twelve blocks. It reported similar training loss and validation bits-per-byte, and a higher CORE score in those configurations. Validation BPB improved as d ran through 4, 8, 16, and late-training gradient norms were lower in the comparison. [R004]

Interpretation. Those are author-reported numbers at two modest scales, not a universal ranking. Parameter complexity $\mathcal{O}(d^2c)$ counts learned numbers, dominated by the pre and post projections. It does not count runtime, and runtime still depends on whether H is materialized.

Where the tile sits

KromHC drops into the same update you already know:

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l),$$

with shapes

$$H_l^{\text{res}} \in \mathbb{R}^{d \times d}, \quad H_l^{\text{pre}}, H_l^{\text{post}} \in \mathbb{R}^{1 \times d}, \quad x_l, x_{l+1} \in \mathbb{R}^{B \times d \times m \times c}.$$

Only H_l^{res} is tiled. Applied on the stream axis, it keeps the state shape $B \times d \times m \times c$. The branch F still sees one routed tensor of shape $B \times m \times c$. KromHC does not mean d evaluations of F .

The later go-mHC paper takes KromHC's exactness and cheapness as a plus, and uses the spectral reach as a motivating limit. It reports broader spectral coverage from a generalized orthostochastic construction, and also proposes composing that construction with Kronecker tiles. [R005] Useful comparison. Another recent proposal. Not a final ranking.

KromHC chooses a small exact subset that is easy to walk. go-mHC looks for a larger exact subset at a different price. Neither sentence says which model wins tomorrow.

Suppose the tiles are applied to the token axis instead of the stream axis. Then the mixer rearranges positions, not leftover copies. The shapes can still multiply. The model is talking to the wrong house.

What we lose if we skip a piece

Keep arbitrary learned factors and drop their doubly stochastic parameterization, and the tiled product is no longer guaranteed to be doubly stochastic. One bad tile is enough: a row sum of two in A multiplies the matching row sums in $A \otimes B$.

Keep exact factors and drop the tiling, and you need another way to parameterize a full $d \times d$ doubly stochastic matrix. Finite Sinkhorn returns to an approximation tolerance. Listing every large permutation returns to factorial growth. Other exact parameterizations exist. They bring their own geometry and their own bill.

Pretend the tiling restriction is not there, and architectural reasoning breaks. A target three-cycle can sit inside $\mathcal{B}_d$ and still be unreachable with binary tiles. More training cannot buy a missing dimension.

Skipping KromHC just because it is restricted would also be a mistake. A model may not need arbitrary stream permutations. The repeated coordinate rule can be a useful bias. The paper's experiments are evidence that useful language-model routing lives inside this smaller family. [R004]

Ways to get this wrong

- Calling $A \otimes B$ an ordinary matrix product. Its shape is $pq \times pq$, not $p \times q$, and its entries live in scaled tiles.
- Proving only that rows sum to one. Double stochasticity also needs nonnegativity and column sums.
- Hearing "exact" and concluding there is no floating-point error. The construction has zero Sinkhorn-iteration gap in real arithmetic. Evaluation still rounds.
- Equating exactness with covering every doubly stochastic matrix. KromHC lands exactly inside $\mathcal{B}_d$. It generally does not fill $\mathcal{B}_d$.
- Ignoring factor order. $A \otimes B$ and $B \otimes A$ differ by a permutation of coordinates. Under a fixed stream order they are not the same matrix.
- Claiming KromHC can only ever make two-cycles. That claim is for permutation corners built entirely from two-by-two tiles. Larger tiles allow longer cycles, still with product structure.
- Quoting $\mathcal{O}(d^2c)$ as the cost of everything. In the paper that is the dominant parameter statement, including pre and post projections. Runtime depends on whether the large mixer is materialized or applied mode by mode.
- Routing tokens with the tiles. The tiles mix the residual-stream index. Batch, sequence, and feature stay distinct.

The factors can be expressive one by one. Their tiled product is still a product. Independence makes the construction cheap. It does not invent cross-coordinate dependencies.

Try it

1. Expand and check

Let

$$A = \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix}, \quad B = \begin{pmatrix} 0.3 & 0.7 \\ 0.7 & 0.3 \end{pmatrix}.$$

Glue $A \otimes B$ by the tile rule, then add the first column.

Answer

$$A \otimes B = \begin{pmatrix} 0.24 & 0.56 & 0.06 & 0.14 \\ 0.56 & 0.24 & 0.14 & 0.06 \\ 0.06 & 0.14 & 0.24 & 0.56 \\ 0.14 & 0.06 & 0.56 & 0.24 \end{pmatrix}.$$

The first column sums to $0.24 + 0.56 + 0.06 + 0.14 = 1$.

2. Find the stretch factors

For the same two tiles, what are the four eigenvalues of $A \otimes B$?

Answer

A matrix $U(t)$ has eigenvalues 1 and $2t - 1$. Thus A has $\{1, 0.6\}$, while B has $\{1, -0.4\}$. Pairwise products give

$$\{1, 0.6, -0.4, -0.24\}.$$

The negative modes alternate sign under repeated routing.

3. Test a cycle

Can the four-cycle $00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$ equal $P \otimes Q$ for two-by-two permutation matrices P and Q ?

Answer

No. Every 2×2 permutation is I_2 or S , so $(P \otimes Q)^2 = P^2 \otimes Q^2 = I_4$. A four-cycle has order four, not order at most two.

4. Count the room

Compare the dimension of $\mathcal{B}_8$ with the maximum dimension of the tiled family $\mathcal{B}_2 \otimes \mathcal{B}_2 \otimes \mathcal{B}_2$.

Answer

$$\dim \mathcal{B}_8 = (8 - 1)^2 = 49.$$

Each $\mathcal{B}_2$ factor has dimension one, so the product family has dimension at most $1 + 1 + 1 = 3$. It is therefore a very small structured subset of $\mathcal{B}_8$.

5. Pick tiles under a budget

For $d = 12$, compare $12 = 2 \cdot 2 \cdot 3$ with $12 = 3 \cdot 4$ using the residual-logit count $\sum_k i_k!$. Which is cheaper, and what do you give up?

Answer

For $2, 2, 3$, the count is $2! + 2! + 3! = 2 + 2 + 6 = 10$. For $3, 4$, it is $3! + 4! = 6 + 24 = 30$. The prime-factor choice is cheaper. The caveat is expressivity: a 4×4 factor can represent coordinated four-state mixing unavailable inside two independent binary factors.

Check yourself

1. **Show that** if A and B are doubly stochastic, then so is $A \otimes B$.
Entries are products $a_{ij}b_{rs}$, hence nonnegative. The all-ones vector tiles as $\mathbf{1}_{pq} = \mathbf{1}_p \otimes \mathbf{1}_q$, and each factor already fixes that vector, so row sums and column sums survive.
2. **What fails if** a single factor has a row summing to two?
That factor fails $A\mathbf{1} = \mathbf{1}$. The matching row sums of $A \otimes B$ fail with it. One bad tile is enough.
3. **Show that** if A has eigenvalues 1, 0.5 and B has eigenvalues 1, 0.2, then $\text{spec}(A \otimes B) = \{1, 0.5, 0.2, 0.1\}$.
Kronecker eigenvalues are all pairwise products.
4. **What fails if** you want a three-cycle of streams from binary permutation tiles?
Each binary permutation has order at most two, so the tiled permutation does too. A three-cycle does not.
5. KromHC is exact. Does that mean it covers all of $\mathcal{B}_d$?
No. Exact membership is not exhaustive coverage. For $d = 4 = 2 \cdot 2$ you have two degrees of freedom against nine.
6. The paper writes parameter complexity $\mathcal{O}(d^2c)$. What does that count, and what does it not count?
Learned parameters, dominated by the pre/post term $2d^2c$. Runtime depends on whether you materialize H or apply tiles mode by mode.

KromHC is not “approximate mHC.” It is not free full expressivity either. It is an exact, cheap, structured mixer with an explicit bias: the floor is tiled.

Chapter 12 asks whether we can stay exact without forcing the mixer to factor as tiles.

Sources

- [R004] W. Zhou, Y. Gu, G. Iacovides, and D. Mandic, “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices,” arXiv:2601.21579v2, 2026. <https://arxiv.org/abs/2601.21579>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R018] C. F. Van Loan, “The Ubiquitous Kronecker Product,” *Journal of Computational and Applied Mathematics* 123, 85–100, 2000. [https://doi.org/10.1016/S0377-0427\(00\)00393-9](https://doi.org/10.1016/S0377-0427(00)00393-9)
- [R019] G. Birkhoff, “Tres observaciones sobre el álgebra lineal,” *Universidad Nacional de Tucumán, Revista A* 5, 147–151, 1946. <https://www.ams.org/mathscinet-getitem?mr=0020547>

Chapter 12 — Twenty-eight numbers and a rotation

How do you get a rotation from an unconstrained list of numbers?

A rotation is fussy. Every column has to have length one, and every pair of columns has to sit at right angles. A network layer is not fussy. It will emit any real numbers you ask for, and the optimizer is happier when those numbers have no extra rules bolted on.

Those two habits collide. This chapter starts with a free list and ends with an eight-by-eight matrix that really does rotate. We will not ask the network to satisfy the angle rules by wishing.

Columns that stay at right angles

Take a square table Q with N rows and N columns. Multiply a vector v by that table. In general the vector comes out longer, shorter, or sheared.

There is a special case. If the columns of Q are unit length and pairwise at right angles, multiplication only turns v . Length is sacred:

$$\|Qv\|_2^2 = v^\top Q^\top Q v.$$

The middle product $Q^\top Q$ has to be the identity I_N , or the lengths will not match. When

$$Q^\top Q = I_N,$$

we call Q **orthogonal**. We take this as the definition: the whole table, in one equation, is the right-angle condition on its columns.

An orthogonal matrix can still flip a direction. Rotations are the ones that also keep orientation; we will meet that extra filter after we have a construction.

Asking a network for all N^2 entries of Q , then hoping the angle equations hold, is a bad job. For $N = 8$ that is 64 numbers and a pile of quadratic constraints. We need a list the optimizer may treat as free.

Suppose the columns are merely pairwise orthogonal but not unit length. Then $Q^\top Q$ is diagonal, not I_N , and lengths change. Right angles without unit length is not the definition we just accepted.

Opposite pairs above the diagonal

Here is a smaller object. Write a square table whose diagonal is zero and whose lower triangle is the upper triangle with every sign flipped:

$$A = \begin{pmatrix} 0 & a \\ -a & 0 \end{pmatrix}.$$

One real number a fills the whole 2×2 table. The diagonal had no choice: a number that equals its own negative is zero. Swap the two off-diagonal seats and you swap the sign.

That pattern is **skew-symmetric**. In symbols, a real square matrix A is skew-symmetric when

$$A^\top = -A.$$

A coordinate a_{ij} couples direction i to direction j . There is no self-coupling on the diagonal. Reverse the ordered pair and the sign reverses. Skew tables describe infinitesimal twists. They are not probabilities, and they are not leftover copies of a sentence.

Once the entries strictly above the diagonal are chosen, every entry below is forced. So you need one free number per unordered pair of indices $\{i, j\}$. There are

$$\binom{N}{2} = \frac{N(N-1)}{2}$$

such pairs.

The working size in this chapter is $N = 8$. That eight is 4×2 : four leftover copies of a sentence table, later cut into blocks of size two. For the count, eight is just the side of the square.

$$\binom{8}{2} = \frac{8 \cdot 7}{2} = 28.$$

A projection that wants to emit one of these tables only needs a vector of 28 unconstrained reals,

$$u_l = (u_1, \dots, u_{28}) \in \mathbb{R}^{28},$$

one list per routed instance. The subscript l is the network layer. If the mixer is allowed to change with the batch or the token, those extra axes sit in front of the 28. We suppress them while we study one instance. Do not quietly glue the batch axis onto the leftover-copy axis.

The vector space of real 8×8 skew-symmetric matrices is later called $\mathfrak{so}(8)$. The name does not add a twenty-ninth coordinate.

Suppose the diagonal is allowed a free number d . Then $A^\top = -A$ forces $d = -d$, so $d = 0$. The zero diagonal is not an extra convention. It is the definition on the diagonal.

Fill the upper triangle

Pack the 28 numbers into the strict upper triangle, row by row. Row 1 gets seven, row 2 gets six, and so on:

$$7 + 6 + 5 + 4 + 3 + 2 + 1 = 28.$$

Set every lower entry to the negative of its mirror. The zero diagonal stays zero. That is the whole table:

$$A_l = \begin{pmatrix} 0 & u_1 & u_2 & u_3 & u_4 & u_5 & u_6 & u_7 \\ -u_1 & 0 & u_8 & u_9 & u_{10} & u_{11} & u_{12} & u_{13} \\ -u_2 & -u_8 & 0 & u_{14} & u_{15} & u_{16} & u_{17} & u_{18} \\ -u_3 & -u_9 & -u_{14} & 0 & u_{19} & u_{20} & u_{21} & u_{22} \\ -u_4 & -u_{10} & -u_{15} & -u_{19} & 0 & u_{23} & u_{24} & u_{25} \\ -u_5 & -u_{11} & -u_{16} & -u_{20} & -u_{23} & 0 & u_{26} & u_{27} \\ -u_6 & -u_{12} & -u_{17} & -u_{21} & -u_{24} & -u_{26} & 0 & u_{28} \\ -u_7 & -u_{13} & -u_{18} & -u_{22} & -u_{25} & -u_{27} & -u_{28} & 0 \end{pmatrix} \in \mathbb{R}^{8 \times 8}.$$

Transpose the table and every u_k becomes $-u_k$, while the zeros stay put. Therefore $A_l^\top = -A_l$.

The map $u_l \mapsto A_l$ is linear and one-to-one for this ordering. There is no softmax. Each u_k may be any real number. A different code base may walk the triangle in a different order. That is fine, as long as packing and unpacking agree.

Suppose two different lists u and u' produce the same A_l . Then their upper triangles agree, so $u = u'$. The packing does not throw coordinates away. It also does not invent a twenty-ninth one.

A two-by-two twist

Before trusting an 8×8 recipe, finish the 2×2 table by hand. One coordinate a still gives

$$A = \begin{pmatrix} 0 & a \\ -a & 0 \end{pmatrix} \in \mathbb{R}^{2 \times 2}.$$

We want a rotation out of A . Form the identity I_2 , subtract A on the left, add A on the right, and multiply:

$$Q = (I_2 - A)(I_2 + A)^{-1}.$$

The inverse is elementary:

$$(I_2 + A)^{-1} = \frac{1}{1 + a^2} \begin{pmatrix} 1 & -a \\ a & 1 \end{pmatrix}.$$

Multiply through and you get

$$Q = \frac{1}{1 + a^2} \begin{pmatrix} 1 - a^2 & -2a \\ 2a & 1 - a^2 \end{pmatrix}.$$

For $a = 1$,

$$A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

The columns of Q have unit length and zero dot product, so $Q^\top Q = I_2$. Under this sign convention the table rotates vectors by $+90^\circ$. Its determinant is $+1$. Its eigenvalues are $+i$ and $-i$.

This recipe is the **Cayley transform**. It is a fixed, parameter-free map: the network never learns the transform, only the free list that feeds it. The go-mHC paper uses this same convention [R005].

In code, solve a linear system. Do not form $(I + A)^{-1}$ as an explicit matrix and then multiply. The inverse is how the mathematics is written, not how the kernel should run.

Suppose A is not skew. Then the same formula may still produce a matrix, but the next two sections' proofs die: they use $A^\top = -A$ twice, and a generic A can make $I + A$ singular.

The rotation you can approach but never land on

Send $|a|$ to infinity in the 2×2 formula. Then

$$Q \longrightarrow -I_2.$$

No finite a actually produces $-I_2$. The excluded matrix has eigenvalue -1 . You can walk toward that point as close as you like. You cannot sit on it.

The same hole will reappear for $N = 8$. It is not a rounding error. It is a missing chart coordinate.

Suppose someone claims that because $SO(2)$ is a circle, every rotation has a finite a . The circle is connected. The chart is not onto. Connectedness of the group is not a covering property of one coordinate.

Why the inverse exists

Back to eight dimensions. We want

$$Q_I = (I_8 - A_I)(I_8 + A_I)^{-1}.$$

Could $I_8 + A_I$ fail to invert? Not when A_I is real and skew-symmetric.

Suppose a real vector $v \in \mathbb{R}^8$ satisfies $(I_8 + A_I)v = 0$. Multiply on the left by $v^\top$:

$$0 = v^\top (I_8 + A_I)v = \|v\|_2^2 + v^\top A_I v.$$

The leftover scalar $v^\top A_I v$ equals its own transpose, but

$$(v^\top A_I v)^\top = v^\top A_I^\top v = -v^\top A_I v,$$

so it is zero. Hence $\|v\|_2^2 = 0$, which forces $v = 0$. The null space is empty, so $I_8 + A_I$ is invertible. The same argument works for $I_8 - A_I$.

Over $\mathbb{C}$, a real skew-symmetric matrix has purely imaginary eigenvalues $i\theta$. Then -1 cannot be an eigenvalue of A_I , and $1 + i\theta \neq 0$. Same conclusion, different language.

Drop skew-symmetry and the quadratic form $v^\top A v$ need not vanish. Then $I + A$ can kill a nonzero vector, and the Cayley formula is not even defined.

Why the columns stay at right angles

If $A \in \mathbb{R}^{N \times N}$ satisfies $A^\top = -A$, the Cayley table

$$Q = (I_N - A)(I_N + A)^{-1}$$

satisfies $Q^\top Q = I_N$.

Compute the transpose, reversing the product:

$$\begin{aligned} Q^\top &= [(I_N + A)^{-1}]^\top (I_N - A)^\top \\ &= (I_N + A^\top)^{-1} (I_N - A^\top) \\ &= (I_N - A)^{-1} (I_N + A). \end{aligned}$$

The matrices $I_N - A$ and $I_N + A$ commute, because both are polynomials in the same A . Their inverses commute with them too. Therefore

$$\begin{aligned} Q^\top Q &= (I_N - A)^{-1} (I_N + A) (I_N - A) (I_N + A)^{-1} \\ &= (I_N - A)^{-1} (I_N - A) (I_N + A) (I_N + A)^{-1} \\ &= I_N. \end{aligned}$$

Hence Q is orthogonal. That is exact algebra. In floating point a computed Q_I will hit the identity only to numerical precision.

The same calculation with a generic A stops at the second line: without $A^\top = -A$, the transpose does not swap $I - A$ with $I + A$. Orthogonality is not a gift of writing Cayley's formula. It is a gift of feeding that formula a skew table.

Eigenvalues on the circle, except minus one

Let $i\theta$, with $\theta \in \mathbb{R}$, be an eigenvalue of A . On the same complex eigenvector, Q has eigenvalue

$$\lambda_Q = \frac{1 - i\theta}{1 + i\theta}.$$

Its magnitude is

$$|\lambda_Q|^2 = \frac{1 + \theta^2}{1 + \theta^2} = 1.$$

Cayley sends imaginary-axis eigenvalues of a skew table to unit-circle eigenvalues of an orthogonal table. Yet

$$\frac{1 - i\theta}{1 + i\theta} = -1$$

has no finite solution: it would require $1 = -1$. Only the limit $|\theta| \rightarrow \infty$ reaches -1 .

The ordinary Cayley transform therefore lands on those special orthogonal matrices that do **not** have eigenvalue -1 . Conversely, if Q is a rotation matrix and -1 is not an eigenvalue of Q , the inverse coordinate

$$A = (I_N - Q)(I_N + Q)^{-1}$$

is the matching skew-symmetric table.

So Cayley is a useful chart, not a single global address system for every rotation. Large coordinate magnitudes are needed to approach the omitted locus. That can hurt numerical conditioning and gradients.

“Orthogonal” does not mean “every eigenvalue is $+1$.” Real orthogonal matrices can have eigenvalues $+1$, -1 , or complex-conjugate pairs on the unit circle. Finite Cayley coordinates exclude exactly the -1 case. They do not exclude ordinary complex rotations.

The counterexample is already in hand. $-I_8$ is orthogonal, and $\det(-I_8) = (-1)^8 = 1$, so it is a rotation. It has eigenvalue -1 . Inverse Cayley asks for $(I_8 + (-I_8))^{-1}$, which does not exist. Connectedness of $SO(8)$ describes paths inside the group. It does not make one chart cover the group.

Two islands of eight-by-eight tables

The set of all real orthogonal 8×8 tables is

$$O(8) = \{Q \in \mathbb{R}^{8 \times 8} : Q^\top Q = I_8\}.$$

It has two disconnected islands. Take determinants of $Q^\top Q = I_8$ and you get $\det(Q)^2 = 1$, so $\det(Q) = +1$ or -1 . The $+1$ island is the **special orthogonal** group $SO(8)$, the rotations. The -1 island includes reflections. The name is worth keeping because “special” is not praise. It means the determinant is $+1$. “Orthogonal” alone does not tell you which island you are on.

For a Cayley matrix,

$$\det Q = \frac{\det(I_8 - A)}{\det(I_8 + A)}.$$

But

$$\det(I_8 - A) = \det((I_8 - A)^\top) = \det(I_8 + A),$$

so $\det Q = +1$. Cayley never crosses into the reflection island.

That restriction is real at the level of Q . It is not a restriction on the mixer we will build next, because the next map uses squared sizes of blocks and throws the signs away [R005]. Concretely, let R be the diagonal table with one -1 and seven $+1$ entries. Then RQ has the opposite determinant, while multiplying a row by -1 does not change any block's squared Frobenius size. Both islands can produce the same mixer.

Do not mash this island argument into the missing-eigenvalue caveat. They are different holes:

- the determinant statement keeps the auxiliary Q off the entire $\det = -1$ island, and the later sign-blind block-size map erases that distinction;
- the -1 -eigenvalue statement omits a locus even inside $SO(8)$ from finite Cayley coordinates, though the locus sits in the closure as the coordinates run off to infinity.

This builds a router, not streams

The four leftover copies already exist. They live in

$$x_l \in \mathbb{R}^{B \times 4 \times L \times D},$$

where B is how many sentences ride together, the second axis holds the four copies, L is how many tokens sit in a sentence, and D is how wide each token is. Nothing in this chapter manufactured those four copies. Nothing in this chapter turned the eight auxiliary axes of Q_l into eight copies either.

What we still need is a 4×4 table H_l^{res} that mixes information *between* those copies. For one layer, each batch slice updates as

$$x_{l+1}[b] = H_l^{\text{res}} x_l[b] + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l[b], W_l),$$

with shapes

$$\begin{aligned} x_l &\in \mathbb{R}^{B \times 4 \times L \times D}, \\ H_l^{\text{pre}} &\in \mathbb{R}^{1 \times 4}, \\ H_l^{\text{post}} &\in \mathbb{R}^{1 \times 4}, \\ H_l^{\text{res}} &\in \mathbb{R}^{4 \times 4}. \end{aligned}$$

Earlier exact mixers face an awkward choice. A convex mixture over every permutation can cover the Birkhoff polytope but grows factorially with the number of copies. A Kronecker factorization is much cheaper and cannot reach every mixing pattern. The 2026 go-mHC paper proposes a third route: parameterize a larger orthogonal table continuously, then collapse its blocks to an exactly doubly stochastic mixer [R005].

Original mHC used a Sinkhorn-style map for H_l^{res} . go-mHC replaces the residual mixer with this pipeline [R005]:

$$\text{normalized activation} \xrightarrow{\text{learned projection}} u_l \in \mathbb{R}^{28} \xrightarrow{\text{skew}} A_l \in \mathbb{R}^{8 \times 8} \xrightarrow{\text{Cayley}} Q_l \in SO(8) \xrightarrow{\Phi_{4,2}} H_l^{\text{res}} \in \mathbb{R}^{4 \times 4}.$$

Equation (7) of that paper writes the first two arrows as a learned projection onto normalized activations, then skew. We have built and proved the middle two arrows. In the last arrow, Q_l is cut into a 4×4 grid of 2×2 blocks, and each mixer entry is a scaled squared Frobenius size. The next chapter proves that this produces nonnegative entries with unit row and column sums.

Here $d = 4$ is the number of leftover copies and $s = 2$ is that block size. Their product is the auxiliary side $N = ds = 8$. The 28 parameters belong to one routed instance of this auxiliary table. They are not 28 streams.

So: this builds a router, not streams. H_l^{res} mixes the four copies already sitting on axis 1 of batch-first x_l . The 8×8 table Q_l is scaffolding.

Skew coordinates let an ordinary optimizer emit unconstrained reals while the deterministic Cayley map enforces the right-angle invariant. Learning lives in the projection that emits u_l . The Cayley transform itself has no learned parameters.

Because go-mHC appeared as arXiv:2604.02309v1 in 2026, its architectural and training claims are recent research, not settled fact. The linear-algebra statements in this chapter are standard.

Empirical result. Dandachi and Diggs-Galligan report synthetic routing experiments and a 30M-parameter GPT-style experiment [R005].

Interpretation. Those results do not turn every proposed optimization story into a theorem, and they do not establish behavior at arbitrary scale. Claims about why this parameterization helps optimization remain paper evidence.

What breaks if we skip a step

Skip the antisymmetric packing, and 28 free numbers become 64 unrelated entries. The orthogonality proof dies immediately: it uses $A^\top = -A$ twice. A generic A can also make $I + A$ singular.

Skip Cayley and use A_l itself as the mixer, and the entries may be negative, the row and column sums are not one, and the table is 8×8 rather than the 4×4 we actually apply to the four copies. Skew-symmetry is not double stochasticity.

Use an arbitrary dense Q_l with no orthogonality, and the next chapter's block-size row and column proof loses its identities. Normalizing afterward would be a different method, with different numerical and exactness properties.

Forget the block-size map, and you have only an orthogonal scaffold, not H_l^{res} . Call Q_l “the streams,” and the shapes stop making sense: $Q_l \in \mathbb{R}^{8 \times 8}$, while the state is $x_l \in \mathbb{R}^{B \times 4 \times L \times D}$.

Ways to get this wrong

- Counting 64 learned coordinates. Skew-symmetry leaves $8 \cdot 7/2 = 28$ independent entries. The lower triangle and the diagonal are determined.
- Building streams from A_l . The streams already exist. This path builds residual routing between them.
- Equating $s = 2$ with two extra streams. s is the auxiliary block size. Stream count stays $d = 4$.
- Calling A_l orthogonal. A_l is skew-symmetric. Q_l is orthogonal.
- Claiming Cayley covers all of $O(8)$. It yields $\det = +1$ and misses matrices with eigenvalue -1 at finite coordinates.
- Claiming determinant $+1$ means every eigenvalue is positive. The determinant is the product of the eigenvalues. Rotations commonly have non-real unit-modulus eigenvalues.
- Writing an explicit inverse in production code. Use a linear solve. The equations may wear an inverse; the kernel need not.
- Claiming orthogonality already proves double stochasticity. It does not. The scaled block-squared-size map supplies nonnegativity and the stochastic sums.
- Saying the Cayley transform learns. The learned projection emits u_l . Cayley just constrains it.
- Overstating granularity. A suppressed leading axis on u_l has to be restored from the actual implementation. The paper's algebra alone does not say that every token, sequence, or layer shares — or does not share — the same router.

Try it

1. Count before you pack

For five leftover copies and blocks of size two, what is N ? How many independent skew coordinates does one routed instance need?

Answer

$N = ds = 10$. The strict upper triangle has $N(N-1)/2 = 10 \cdot 9/2 = 45$ entries, so the learned coordinate vector has shape $\mathbb{R}^{45}$ per routing instance.

2. Pack a smaller skew table

Let $u = (p, q, r) \in \mathbb{R}^3$. Fill a 3×3 skew-symmetric matrix from the upper triangle, row by row, and check the transpose relation.

Answer

$$A = \begin{pmatrix} 0 & p & q \\ -p & 0 & r \\ -q & -r & 0 \end{pmatrix}.$$

Transposition swaps each off-diagonal pair, producing $-A$; the zero diagonal is unchanged. Thus $A^\top = -A$.

3. Check the two-by-two recipe

Using

$$Q(a) = \frac{1}{1+a^2} \begin{pmatrix} 1-a^2 & -2a \\ 2a & 1-a^2 \end{pmatrix},$$

verify directly that $Q(a)^\top Q(a) = I_2$ and $\det Q(a) = 1$.

Answer

Each column has squared norm

$$\frac{(1-a^2)^2 + 4a^2}{(1+a^2)^2} = 1,$$

and their dot product is zero. Also

$$\det Q(a) = \frac{(1-a^2)^2 + 4a^2}{(1+a^2)^2} = 1.$$

4. A coverage claim that sounds right

A colleague says, “Every matrix in $SO(8)$ has a finite skew Cayley coordinate because $SO(8)$ is connected.” Give a counterexample and explain why connectedness does not save the claim.

Answer

$-I_8 \in SO(8)$ because it is orthogonal and $\det(-I_8) = (-1)^8 = 1$. It has eigenvalue -1 , so $I_8 + Q = 0$ is singular and inverse Cayley coordinates do not exist. Connectedness describes the existence of continuous paths inside $SO(8)$; it does not imply that one coordinate chart covers the whole group.

5. Signs on the scaffold, not on the mixer

Let $R = \text{diag}(-1, 1, \dots, 1) \in \mathbb{R}^{8 \times 8}$, and cut an orthogonal Q into 2×2 blocks. Why does RQ flip determinant island but leave every squared block Frobenius size unchanged?

Answer

$\det R = -1$, so $\det(RQ) = -\det Q$. Left multiplication by R changes the sign of only the first scalar row of Q . Every affected block contains squared entries in $\|Q_{ij}\|_F^2$, so the sign disappears; unaffected blocks are identical. Thus the later routing map cannot distinguish these two auxiliary matrices.

Check yourself

The first half of the go-mHC mixer, at $d = 4$, $s = 2$, is this ledger:

$$u_l \in \mathbb{R}^{28} \mapsto A_l \in \mathbb{R}^{8 \times 8}, A_l^\top = -A_l \mapsto Q_l \in \mathbb{R}^{8 \times 8}, Q_l^\top Q_l = I_8.$$

1. **Show that** an 8×8 skew table has twenty-eight independent entries.
The strict upper triangle has $8 \cdot 7/2 = 28$ seats. The lower triangle is determined by the sign flip, and the diagonal is zero.
2. **Show that** $I_8 + A_l$ is invertible when $A_l^\top = -A_l$.
If $(I + A)v = 0$, then $\|v\|_2^2 + v^\top Av = 0$. The quadratic form $v^\top Av$ equals its own negative, hence vanishes, so $v = 0$.
3. **What fails if** you drop skew-symmetry and still write $Q = (I - A)(I + A)^{-1}$?
The inverse need not exist, and the transpose calculation no longer swaps $I - A$ with $I + A$. Orthogonality dies.
4. Does $\det Q_l = +1$ mean every eigenvalue is positive?
No. It is the product of unit-modulus eigenvalues. Rotations commonly carry non-real conjugate pairs.
5. **What fails if** you claim every matrix in $SO(8)$ has a finite Cayley coordinate?
 $-I_8$ is a rotation with eigenvalue -1 . Inverse Cayley is undefined there. Connectedness of $SO(8)$ does not make one chart onto.
6. Who learns?
The projection that emits u_l . Cayley has no learned parameters.

Say the boundary aloud: A_l and Q_l are auxiliary tables used to build H_l^{res} . They are not leftover copies of the sentence. The construction produces routing, not streams.

We now have an 8×8 orthogonal table and we still need a 4×4 nonnegative mixer. The next chapter cuts Q_l into sixteen 2×2 blocks and sets

$$(H_l^{\text{res}})_{ij} = \frac{1}{2} \|Q_{ij}\|_F^2.$$

Orthogonality then does a second job: sums of block energies become exactly one along every mixer row and column. That is where exact double stochasticity is proved — and not one paragraph earlier.

Sources

- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R026] K. Helfrich, D. Willmott, and Q. Ye, “Orthogonal Recurrent Neural Networks with Scaled Cayley Transform,” arXiv:1707.09520, 2018. <https://arxiv.org/abs/1707.09520>
- [R027] M. Lezcano-Casado and D. Martínez-Rubio, “Cheap Orthogonal Constraints in Neural Networks: A Simple Parametrization of the Orthogonal and Unitary Group,” *Proceedings of Machine Learning Research* 97, 2019. <https://proceedings.mlr.press/v97/lezcano-casado19a.html>
- [R028] E. Gutkin, “On a Multi-Dimensional Generalization of the Notions of Orthostochastic and Unistochastic Matrices,” *Journal of Geometry and Physics* 74, 28–35, 2013. <https://doi.org/10.1016/j.geomphys.2013.06.009>
- [R029] I. Nechita, Z. Ouyang, and A. Szczepanek, “Generalized Unistochastic Matrices,” arXiv:2310.03436, 2023. <https://arxiv.org/abs/2310.03436>

Chapter 13 — Windows in the ledger

Last chapter left us an eight-by-eight orthogonal table. Call it Q . Every column has length one. Every pair of columns sits at right angles. It is a ledger of how eight coordinates move.

We do not have eight leftover copies of the sentence. We have four streams. Multiplying those four by an eight-by-eight table is illegal: the widths do not meet.

So we group. Bundle the first two coordinates into stream 1, the next two into stream 2, and so on. Do the same for the columns. The ledger now has sixteen windows. The window in block-row i and block-column j is the channel from stream j to stream i .

Each window still holds four signed numbers. A mixer for four streams needs one nonnegative number per channel. The rest of this chapter turns each window into that number, then checks — slowly — that the four-by-four table of numbers is an honest recipe both ways.

Orthogonality of Q is not that honesty. Different size, different rules, and a negative entry is still allowed in the ledger.

Pair the rows, pair the columns

Let d be the number of streams and s the number of ledger coordinates we bundle into one stream. Both are positive integers. The ledger is then square of side

$$N = ds.$$

The running choice in this book is $d = 4$ streams and $s = 2$ coordinates per stream, so $N = 8$. The same grouping works for any positive d and s .

Number the ordinary rows of Q from 1 to N . Stream group i owns the consecutive block

$$I_i = \{(i-1)s + 1, \dots, is\},$$

for $i = 1, \dots, d$. The same formula, with index j , names the column groups. The window from input group j to output group i is just the subtable

$$Q_{ij} = Q[I_i, I_j] \in \mathbb{R}^{s \times s}.$$

Drawn as blocks,

$$Q = \begin{pmatrix} Q_{11} & \cdots & Q_{1d} \\ \vdots & \ddots & \vdots \\ Q_{d1} & \cdots & Q_{dd} \end{pmatrix}.$$

That display has d block-rows and d block-columns. Every window is $s \times s$. For $d = 4$ and $s = 2$,

$$Q \in \mathbb{R}^{8 \times 8} = \begin{pmatrix} Q_{11} & Q_{12} & Q_{13} & Q_{14} \\ Q_{21} & Q_{22} & Q_{23} & Q_{24} \\ Q_{31} & Q_{32} & Q_{33} & Q_{34} \\ Q_{41} & Q_{42} & Q_{43} & Q_{44} \end{pmatrix}, \quad Q_{ij} \in \mathbb{R}^{2 \times 2}.$$

This is a **partition**, not a new multiplication. The same 64 scalar entries are now sixteen windows of four scalars each. The s coordinates inside a window are auxiliary routing coordinates. They are not extra tokens, extra features, or extra persistent streams.

Suppose you treat the eight ordinary rows as eight streams. Then you have invented four streams the state does not possess, and Q still has the wrong width for the leftover-copy axis. The grouping is how an 8×8 scaffold talks to four streams. It is not a fifth axis.

How much length sits in one window

A mixer entry cannot be allowed to go negative. The ledger entries can. The cheapest way to forget a sign is to square it.

Take one window Q_{ij} and add up every squared entry:

$$\|Q_{ij}\|_F^2 = \sum_{a=1}^s \sum_{b=1}^s (Q_{ij})_{ab}^2.$$

Here a and b are the local row and column seats inside that one $s \times s$ window. The double sum is never negative. It is also blind to every sign.

That sum of squares is the square of the **Frobenius norm**. The name is worth keeping because it is just the ordinary Euclidean length after you flatten the window into a list. For any real table M , it is also a trace:

$$\|M\|_F^2 = \text{tr}(MM^\top) = \text{tr}(M^\top M).$$

The first trace form will fit a block-row. The second will fit a block-column. We will need both.

The operation throws information away on purpose. Many different orthogonal ledgers can produce the same window sizes. That is not a bug in the proof below. A nonnegative mixer does not get to keep the signs.

Suppose you keep a single signed entry from each window instead of summing squares. Then H can go negative, and the mixer checks fail before they start. The Frobenius size is not a decoration on top of “pick a representative.” It is the whole channel number.

One number per window

Orthogonality says each full row of Q has total squared length one, and each full column does too. A window collects s of those ordinary rows against s of those ordinary columns. Summing the windows across one block-row should therefore recover s units of squared length, not one.

So divide by the bundle size. The candidate mixer entry from stream j to stream i is

$$H_{ij} = \frac{1}{s} \|Q_{ij}\|_F^2.$$

The letter H is the stream-to-stream table we actually multiply by. It is $d \times d$. Each entry is one channel.

We have a formula. We do not yet have honest recipes. Nonnegativity is already visible — squares and a positive s — but unit row sums and unit column sums are a claim about the whole ledger, not about one window.

Drop the factor $1/s$ and every row of H will later sum to s , not one. For $s = 2$ the router then carries twice the intended total mass. The denominator is forced by the bundle size. It is not a tunable gain.

A smaller table you can check by hand

Use $d = 2$ streams and $s = 2$. Then $N = 4$, so Q is 4×4 . Choose

$$Q = \begin{pmatrix} \frac{3}{5} & 0 & -\frac{4}{5} & 0 \\ 0 & \frac{3}{5} & 0 & -\frac{4}{5} \\ \frac{4}{5} & 0 & \frac{3}{5} & 0 \\ 0 & \frac{4}{5} & 0 & \frac{3}{5} \end{pmatrix} = \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{pmatrix},$$

where every window is 2×2 and

$$Q_{11} = Q_{22} = \begin{pmatrix} \frac{3}{5} & 0 \\ 0 & \frac{3}{5} \end{pmatrix}, \quad Q_{12} = \begin{pmatrix} -\frac{4}{5} & 0 \\ 0 & -\frac{4}{5} \end{pmatrix}, \quad Q_{21} = \begin{pmatrix} \frac{4}{5} & 0 \\ 0 & \frac{4}{5} \end{pmatrix}.$$

The columns of Q are orthonormal, so $Q^\top Q = I_4$. Because Q is square, also $QQ^\top = I_4$. It is two copies of the same planar rotation after a coordinate regrouping.

For a general 2×2 window

$$Q_{ij} = \begin{pmatrix} a & b \\ c & e \end{pmatrix},$$

the Frobenius size is just four squares:

$$\|Q_{ij}\|_F^2 = a^2 + b^2 + c^2 + e^2.$$

The first window gives

$$\|Q_{11}\|_F^2 = \left(\frac{3}{5}\right)^2 + 0^2 + 0^2 + \left(\frac{3}{5}\right)^2 = \frac{18}{25}.$$

With $s = 2$, the rule $H_{ij} = \frac{1}{2}\|Q_{ij}\|_F^2$ produces

$$H_{11} = H_{22} = \frac{1}{2} \cdot \frac{18}{25} = \frac{9}{25},$$

and

$$H_{12} = H_{21} = \frac{1}{2} \left[\left(\frac{4}{5}\right)^2 + 0^2 + 0^2 + \left(\frac{4}{5}\right)^2 \right] = \frac{16}{25}.$$

Thus

$$H = \begin{pmatrix} \frac{9}{25} & \frac{16}{25} \\ \frac{16}{25} & \frac{9}{25} \end{pmatrix} \in \mathbb{R}^{2 \times 2}.$$

Every entry is nonnegative. Each row sums to $9/25 + 16/25 = 1$, and each column does too. Notice that Q contains negative entries while H does not. Also notice that H is not orthogonal: the squared Euclidean length of its first row is

$$\left(\frac{9}{25}\right)^2 + \left(\frac{16}{25}\right)^2 = \frac{337}{625} \neq 1.$$

Hence orthogonality belongs to Q , not to H . One hand-checked table is not a proof for every orthogonal ledger. It is the arithmetic we are about to do without picking numbers.

The same rule for any d and s

Write $O(ds)$ for the set of real $ds \times ds$ orthogonal matrices. The map that sends a ledger to a mixer is

$$(\Phi_{d,s}(Q))_{ij} = \frac{1}{s} \|Q_{ij}\|_F^2 = \frac{1}{s} \sum_{a=1}^s \sum_{b=1}^s (Q_{ij})_{ab}^2,$$

and we write $H = \Phi_{d,s}(Q) \in \mathbb{R}^{d \times d}$.

That map is the **generalized orthostochastic** construction. The ordinary orthostochastic case is the same idea at $s = 1$: square every entry of an orthogonal table. Here each “entry” is a whole window, and the square is a Frobenius square [R005]; [R029].

For the running choice $d = 4$, $s = 2$, each mixer entry is four squares and a half:

$$H_{ij} = \frac{1}{2} ((Q_{ij})_{11}^2 + (Q_{ij})_{12}^2 + (Q_{ij})_{21}^2 + (Q_{ij})_{22}^2).$$

There are 16 such numbers, producing $H \in \mathbb{R}^{4 \times 4}$. No local window index survives in H : the two local axes are reduced by the sum of squares.

The map is not an iterative repair. Once Q is orthogonal, the window sizes will be forced by algebra. That exactness is the claim we still have to prove.

Suppose $s = 1$. Then each window is a scalar, $\Phi_{d,1}$ squares the entries of Q , and we recover the classical orthostochastic table. The general case is that construction with windows in place of scalars. It is not a different philosophy.

Three checks

We need three facts about H : every entry at least zero, every row summing to one, every column summing to one. The set of all $d \times d$ tables with those three properties is the Birkhoff polytope $\mathcal{B}_d$:

$$\mathcal{B}_d = \{M \in \mathbb{R}^{d \times d} : M_{ij} \geq 0, M \mathbf{1}_d = \mathbf{1}_d, M^\top \mathbf{1}_d = \mathbf{1}_d\}.$$

Here $\mathbf{1}_d$ is the column of d ones. The second condition is the row sums. The third is the column sums.

We claim: if $Q \in \mathbb{R}^{ds \times ds}$ is orthogonal, cut into windows $Q_{ij} \in \mathbb{R}^{s \times s}$, and $H_{ij} = \|Q_{ij}\|_F^2/s$, then $H \in \mathcal{B}_d$.

That is not true because Q is orthogonal and H is “the same idea, smaller.” Orthogonality and the three mixer checks live on different tables. The claim is true only if the checks go through. We verify them in order.

Nonnegative first

Fix any $i, j \in \{1, \dots, d\}$. Since $s > 0$ and every square of a real number is nonnegative,

$$H_{ij} = \frac{1}{s} \sum_{a=1}^s \sum_{b=1}^s (Q_{ij})_{ab}^2 \geq 0.$$

Every entry of H is nonnegative. They need not be strictly positive: a zero window gives $H_{ij} = 0$.

This check did not use orthogonality. A messy non-orthogonal ledger would still produce a nonnegative H . The next two checks will not be so generous.

Rows from $QQ^\top$

Because Q is orthogonal and square,

$$QQ^\top = I_{ds}.$$

Both sides can be viewed as $d \times d$ block tables with $s \times s$ windows. Fix a block-row index i . The (i, i) window of $QQ^\top$ has shape $s \times s$ and equals

$$(QQ^\top)_{ii} = \sum_{j=1}^d Q_{ij}(Q^\top)_{ji} = \sum_{j=1}^d Q_{ij}Q_{ij}^\top.$$

The (i, i) window of I_{ds} is $I_s \in \mathbb{R}^{s \times s}$. Therefore orthogonality gives the block identity

$$\sum_{j=1}^d Q_{ij}Q_{ij}^\top = I_s.$$

Take the trace of both sides. Trace is linear, so

$$\sum_{j=1}^d \text{tr}(Q_{ij}Q_{ij}^\top) = \text{tr}(I_s).$$

For each window Q_{ij} , the identity $\text{tr}(Q_{ij}Q_{ij}^\top) = \|Q_{ij}\|_F^2$ holds, while $\text{tr}(I_s) = s$. Hence

$$\sum_{j=1}^d \|Q_{ij}\|_F^2 = s.$$

Now use the definition of H :

$$\sum_{j=1}^d H_{ij} = \sum_{j=1}^d \frac{1}{s} \|Q_{ij}\|_F^2 = \frac{1}{s} \sum_{j=1}^d \|Q_{ij}\|_F^2 = \frac{s}{s} = 1.$$

This holds for every i , so $H\mathbf{1}_d = \mathbf{1}_d$.

Columns from $Q^\top Q$

Use the other orthogonality identity,

$$Q^\top Q = I_{ds}.$$

Fix a block-column index j . The (j, j) window of $Q^\top Q$, again of shape $s \times s$, is

$$(Q^\top Q)_{jj} = \sum_{i=1}^d (Q^\top)_{ji}Q_{ij} = \sum_{i=1}^d Q_{ij}^\top Q_{ij}.$$

Equating this window with the (j, j) window of I_{ds} gives

$$\sum_{i=1}^d Q_{ij}^\top Q_{ij} = I_s.$$

Taking traces yields

$$\sum_{i=1}^d \text{tr}(Q_{ij}^\top Q_{ij}) = \text{tr}(I_s) = s.$$

Since $\text{tr}(Q_{ij}^\top Q_{ij}) = \|Q_{ij}\|_F^2$,

$$\sum_{i=1}^d \|Q_{ij}\|_F^2 = s.$$

Divide by s through the definition of H :

$$\sum_{i=1}^d H_{ij} = \frac{1}{s} \sum_{i=1}^d \|Q_{ij}\|_F^2 = 1.$$

This holds for every j , so $H^\top \mathbf{1}_d = \mathbf{1}_d$.

Nonnegativity, all row sums, and all column sums. Therefore $H \in \mathcal{B}_d$: H is **doubly stochastic**. Those are exactly the three checks, not a leftover property of the orthogonal ledger.

The factor $1/s$ is forced by the trace. Without it, every row and column would sum to s , not one. It is normalization derived from the bundle size, not a tunable gain.

The two conservation laws that made the sums work are

$$\sum_j \|Q_{ij}\|_F^2 = s \quad \text{and} \quad \sum_i \|Q_{ij}\|_F^2 = s.$$

One came from $QQ^\top$. The other came from $Q^\top Q$. Checking rows twice is not a column proof.

What the proof does not say

The argument shows that every orthogonal input Q produces a doubly stochastic output H . In set notation,

$$\Phi_{d,s}(O(ds)) \subseteq \mathcal{B}_d.$$

The inclusion is exact in real arithmetic. There is no finite sequence of alternating row and column normalizations whose early stopping leaves a constraint residual. In floating-point arithmetic, the observed row and column sums inherit numerical error from constructing Q and adding squares. “Exact by construction” is an algebraic statement, not a promise that silicon has discovered infinite precision.

The argument does not prove any of the following:

- that H is orthogonal;
- that every $H \in \mathcal{B}_d$ is represented at a fixed finite s ;
- that the map $Q \mapsto H$ is one-to-one;
- that all entries of H are strictly positive;
- that a full neural network is automatically stable or accurate;
- that the choice $s = 2$ is mathematically optimal.

Finite- s coverage and the role of s belong to the next chapter. The go-mHC paper reports empirical evidence about expressivity and learning behavior. Those observations are not smuggled into this algebraic claim [R005].

The hand-checked 3/5–4/5 table is already a counterexample to “ H is orthogonal”: its first row has squared length $337/625 \neq 1$. Containment in $\mathcal{B}_d$ is the claim. Equality with $\mathcal{B}_d$ is a different claim, and this chapter does not make it.

Where the mixer sits

The Hyper-Connection state at layer l has shape

$$x_l \in \mathbb{R}^{B \times d \times L \times D}.$$

Here B is how many sentences travel together, d is the stream axis, L is sequence length, and D is feature width. The paper’s stream-first notation is the transpose of the first two bookkeeping axes. In either layout, $H \in \mathbb{R}^{d \times d}$ acts only on streams.

For each batch item b , the residual-routing term is

$$H_l^{\text{res}} x_l[b], \quad H_l^{\text{res}} \in \mathbb{R}^{d \times d},$$

where multiplication contracts the stream axis of the $d \times L \times D$ slice. The stored batch result keeps shape $B \times d \times L \times D$.

Original mHC uses an iterative Sinkhorn-based constraint map for this residual mixing table [R002]. go-mHC instead builds an orthogonal

$$Q_l \in \mathbb{R}^{ds \times ds}$$

from learned skew coordinates and then sets

$$H_l^{\text{res}} = \Phi_{d,s}(Q_l) \in \mathcal{B}_d \subset \mathbb{R}^{d \times d}.$$

Q_l is an auxiliary geometric parameterization of routing. The persistent streams are the d slices of x_l ; they already exist. Partitioning Q_l into windows does not partition tokens, split the feature width, or manufacture ds residual streams.

For the common explanatory case $d = 4$, $s = 2$, the learned geometric object has shape 8×8 , the window view has shape $4 \times 4 \times 2 \times 2$, and reducing the two local window axes produces H_l^{res} of shape 4×4 . That shape ledger prevents a surprisingly energetic family of bugs.

The exact constraint statement concerns H_l^{res} . The read and write routers H_l^{pre} and H_l^{post} are separate objects in the Hyper-Connection equation and are not made doubly stochastic by this proof. go-mHC changes the residual-routing constraint map and keeps the surrounding architecture [R005]; [R002]; [R001].

What breaks if we skip a step

If we skip the window reduction and try to use Q directly, the dimensions do not match: Q routes ds coordinates, while the state has d persistent streams. It can also have negative entries, so it is not a nonnegative stream-mixing table.

If we square window entries but forget to sum them, the output retains local axes and has shape $d \times s \times d \times s$, not $d \times d$. If we sum but forget the factor $1/s$, every row and column sums to s . For $s = 2$, the router carries twice the intended total mass. A tiny missing denominator can be remarkably ambitious.

If Q is not orthogonal, squared window norms remain nonnegative, but the unit-sum proof collapses. For a general Q ,

$$\sum_j Q_{ij} Q_{ij}^\top$$

need not equal I_s , and its trace need not equal s . Normalizing the resulting H afterward would be a different construction with different guarantees.

Finally, checking row sums alone is insufficient. A nonnegative row-stochastic table can duplicate some input streams and neglect others. The column proof requires $Q^\top Q = I_{ds}$ and is not decorative symmetry.

Ways to get this wrong

“ Q_{ij}^2 ” is easy to read as the matrix product $Q_{ij}Q_{ij}$. The construction uses the sum of *entrywise* squares, $\|Q_{ij}\|_F^2$.

1. **Confusing indices.** i, j index stream groups; a, b index seats inside an $s \times s$ window. Mixing them hides the shape argument.
2. **Using only one scalar from each window.** H_{ij} uses all s^2 entries of Q_{ij} .
3. **Dividing by s^2 .** A window has s^2 entries, but orthogonality gives total block-row squared length s . The correct divisor is s .
4. **Proving rows twice.** $QQ^\top$ yields block-row sums; $Q^\top Q$ yields block-column sums.
5. **Calling H orthogonal.** It is doubly stochastic. Those two sets overlap, but they are not the same set.
6. **Claiming every honest mixer appears.** The proof shows image containment in $\mathcal{B}_d$, not equality at a fixed s .
7. **Treating s as a model axis.** It is an auxiliary window size in the routing parameterization, not the batch, sequence, feature, or persistent-stream count.
8. **Equating algebraic exactness with bitwise exactness.** Floating-point computations have tolerances even when the mathematical construction has no approximation loop.

Try it

1. Expand a window

Let $s = 2$ and

$$Q_{23} = \begin{pmatrix} 1/2 & -1/2 \\ 1/4 & 3/4 \end{pmatrix}.$$

Compute H_{23} from this window alone.

Answer

$$\|Q_{23}\|_F^2 = \frac{1}{4} + \frac{1}{4} + \frac{1}{16} + \frac{9}{16} = \frac{9}{8}, \quad H_{23} = \frac{1}{2} \cdot \frac{9}{8} = \frac{9}{16}.$$

This calculation says nothing about the other entries or whether the full, unspecified Q is orthogonal.

2. Find the missing scale

Suppose $G_{ij} = c\|Q_{ij}\|_F^2$ for orthogonal $Q \in \mathbb{R}^{ds \times ds}$. Find the constant c that makes every row of G sum to one.

Answer

Orthogonality gives $\sum_j \|Q_{ij}\|_F^2 = s$. Hence $\sum_j G_{ij} = cs$, so $c = 1/s$.

3. Check both directions before the trace

For the 4×4 ledger in the hand-checked example, calculate the first block-row identity and first block-column identity before taking traces.

Answer

Using $Q_{11} = (3/5)I_2$, $Q_{12} = -(4/5)I_2$, and $Q_{21} = (4/5)I_2$,

$$Q_{11}Q_{11}^\top + Q_{12}Q_{12}^\top = \frac{9}{25}I_2 + \frac{16}{25}I_2 = I_2,$$

and

$$Q_{11}^\top Q_{11} + Q_{21}^\top Q_{21} = \frac{9}{25}I_2 + \frac{16}{25}I_2 = I_2.$$

Their traces are both $2 = s$, yielding the first row and first column sums of H .

4. Diagnose a non-orthogonal input

Take $d = 2$, $s = 1$ and

$$Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

Apply the same squared-entry map. Which doubly stochastic conditions survive?

Answer

Because $s = 1$,

$$H = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}^{\odot 2} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix},$$

where $\odot 2$ denotes entrywise squaring. Nonnegativity survives. Row sums are 2 and 1; column sums are 1 and 2. Neither unit-sum condition holds because Q is not orthogonal.

5. Prove the two trace forms

Let $M \in \mathbb{R}^{p \times q}$. Prove directly from indices that $\text{tr}(MM^\top) = \text{tr}(M^\top M) = \|M\|_F^2$.

Answer

The a th diagonal entry of $MM^\top$ is $\sum_{b=1}^q M_{ab}^2$, so

$$\text{tr}(MM^\top) = \sum_{a=1}^p \sum_{b=1}^q M_{ab}^2.$$

The b th diagonal entry of $M^\top M$ is $\sum_{a=1}^p M_{ab}^2$, giving the same double sum. By definition that sum is $\|M\|_F^2$.

6. State the strongest valid conclusion

A program constructs an exactly orthogonal $Q \in \mathbb{R}^{12 \times 12}$, views it as $d = 4$ blocks per axis, and applies $H_{ij} = \|Q_{ij}\|_F^2/s$. Determine s , the shapes of the windows and of H , and the strongest coverage claim justified by this chapter.

Answer

Since $ds = 12$ and $d = 4$, $s = 3$. Each window has shape 3×3 , and H has shape 4×4 . The justified claim is $H \in \mathcal{B}_4$: it is nonnegative with unit row and column sums. We may not conclude that every matrix in $\mathcal{B}_4$ is reachable at $s = 3$.

Check yourself

1. You start with an orthogonal 8×8 ledger and four streams. What are d , s , and the shape of H ?
 $d = 4$, $s = 2$, and $H \in \mathbb{R}^{4 \times 4}$. The ledger side is $N = ds = 8$.

2. **Show that** $H_{ij} \geq 0$ even if Q is not orthogonal.
Squares of real numbers are nonnegative and $s > 0$. Orthogonality is not used until the unit-sum checks.

3. **What fails if** you omit the factor $1/s$?

A block-row of Q holds s orthonormal ordinary rows, so the window Frobenius sizes along that block-row add to s . Without the division, every row and column of H sums to s , not one.

4. Which identity of Q gives the row sums of H , and which gives the column sums?
 $QQ^\top = I_{ds}$ after a trace gives the rows. $Q^\top Q = I_{ds}$ after a trace gives the columns.
5. In the 3/5–4/5 example, what is H , and why is H not orthogonal?
 $H = \begin{pmatrix} 9/25 & 16/25 \\ 16/25 & 9/25 \end{pmatrix}$. Its first row has squared length $337/625 \neq 1$. Orthogonality stayed with Q .
6. The proof shows $\Phi_{d,s}(O(ds)) \subseteq \mathcal{B}_d$. What stronger claim does it *not* justify?
 It does not say every table in $\mathcal{B}_d$ appears at a fixed finite s , and it does not say H is orthogonal.

An orthogonal ledger is not a doubly stochastic mixer. Group it into stream-to-stream windows, replace each window by its Frobenius size over s , and the three mixer checks become algebra. The next chapter asks a different question: which honest mixers can appear at a chosen finite s .

Sources

- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R029] I. Nechita, Z. Ouyang, and A. Szczepanek, “Generalized Unistochastic Matrices,” arXiv:2310.03436, 2023. <https://arxiv.org/abs/2310.03436>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024. <https://arxiv.org/abs/2409.19606>

Chapter 14 — What a bigger block buys

You want three leftover copies of a sentence to share equally. Each copy keeps a third of itself and sends a third to each of the others. The mixer is a three-by-three table of ones, every entry divided by three.

Chapter 13 already built a machine that turns a large orthogonal table into a mixer with those two honesty checks: cut the table into square blocks, keep only each block's energy, divide by the block size. Every output is nonnegative, and every row and every column adds to one.

That machine can refuse a perfectly honest target. With one-by-one blocks the equal three-stream table never appears. With two-by-two blocks it does.

The extra room did not add a leftover copy. It hid extra geometry behind each visible mixing number.

Legal is not the same as reachable

Let d be the number of leftover copies and s the side length of each hidden block. Then Q is orthogonal of shape $(ds) \times (ds)$. Partition it into a $d \times d$ grid of tiles $Q_{ij} \in \mathbb{R}^{s \times s}$, and set

$$H_{ij} = \frac{1}{s} \|Q_{ij}\|_F^2.$$

Chapter 13 proved that every such H lies in $\mathcal{B}_d$, the set of $d \times d$ doubly stochastic tables. The argument never used a special value of s . It gives nonnegativity and unit row and column sums. It is silent about preimages.

Write $\Phi_{d,s}$ for that block-energy map and $O(ds)$ for the real orthogonal tables of shape $(ds) \times (ds)$. The image is

$$\mathcal{S}_{d,s} := \Phi_{d,s}(O(ds)) \subseteq \mathcal{B}_d.$$

We take this as the definition. The symbol $\subseteq$ is the whole distinction: every generated mixer is legal, and not every legal mixer is generated.

Those are different jobs. **Exactness** says every output belongs to $\mathcal{B}_d$. **Coverage** says every target in $\mathcal{B}_d$ is hit. Exactness is already proved. Coverage has to be argued separately, and it depends on both d and s .

A row that adds to one does not magically supply a preimage. Drop the distinction and a containment proof gets advertised as a surjectivity theorem.

Square the entries

When $s = 1$, each tile is a single real number. The map just squares:

$$\Phi_{d,1}(Q)_{ij} = Q_{ij}^2.$$

A mixer that arises this way is **orthostochastic**. The name is worth keeping because the orthogonal table is still hiding inside the squares. The signs were thrown away, but they had to cancel first.

For $s = 1$ each leftover copy is one coordinate of Q . Orthogonality can cancel only with scalar signs. After squaring, those signs are gone. Their former obligation to cancel is not gone. It leaves scars on H .

For $d \geq 3$, the orthostochastic set is a proper, generally non-convex subset of $\mathcal{B}_d$ [R029]; [R030]. That subset is not a synonym for “the boundary.” An orthostochastic table can have every entry positive and still sit in the interior of the Birkhoff polytope.

Hence $s = 1$ is not “the edge of the polytope.” It is a thinner set, cut by hidden orthogonality, and it can occupy interior points while still missing others.

Two leftover copies already cover everything

Start with $d = 2$ and $s = 1$. Pick an angle θ and the plane rotation

$$Q = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \in O(2).$$

Every block is a scalar, so $\Phi_{2,1}$ just squares:

$$H = \Phi_{2,1}(Q) = \begin{pmatrix} \cos^2 \theta & \sin^2 \theta \\ \sin^2 \theta & \cos^2 \theta \end{pmatrix}.$$

Every table in $\mathcal{B}_2$ has the form

$$\begin{pmatrix} t & 1-t \\ 1-t & t \end{pmatrix}, \quad 0 \leq t \leq 1.$$

Set $t = \cos^2 \theta$. Then $\mathcal{S}_{2,1} = \mathcal{B}_2$. One-by-one blocks already hit every legal two-stream mixer.

The first missing mixer appears at three streams. Coverage at $d = 2$ is not a license to write $\mathcal{S}_{d,1} = \mathcal{B}_d$ for every d .

The missing equal mixer

The barycenter of $\mathcal{B}_3$ is the perfectly uniform table

$$J_3/3 = \frac{1}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

Suppose it were $\Phi_{3,1}(Q)$ for some real $Q \in O(3)$. Then every entry of Q would have magnitude $1/\sqrt{3}$. The inner product of two distinct rows would be

$$\frac{1}{3}(\pm 1 \pm 1 \pm 1).$$

A sum of three numbers from $\{-1, +1\}$ is odd, so that inner product can never be zero. Orthogonality dies. Hence

$$J_3/3 \notin \mathcal{S}_{3,1}, \quad \mathcal{S}_{3,1} \subsetneq \mathcal{B}_3.$$

Three leftover copies, one-by-one blocks, and the most boring fair mixer is already illegal for the parameterization — not because it fails the row-and-column test, but because no real orthogonal 3×3 table can hide underneath those squares.

Odd dimension is essential. An even number of ± 1 terms can cancel; three of them cannot. Drop that parity and the contradiction evaporates.

A rotation hiding in each tile

Now let $s = 2$. Cancellation can happen inside a matrix. Two 2×2 tiles may have the same Frobenius energy and still differ by a rotation, a reflection, or an entry pattern. The visible mixer H sees only one number per tile.

Write the plane rotation

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}.$$

The real form of the normalized 3×3 Fourier table is this $Q \in \mathbb{R}^{6 \times 6}$, drawn as a 3×3 grid of 2×2 tiles:

$$Q = \frac{1}{\sqrt{3}} \begin{pmatrix} I_2 & I_2 & I_2 \\ I_2 & R(2\pi/3) & R(4\pi/3) \\ I_2 & R(4\pi/3) & R(2\pi/3) \end{pmatrix}.$$

Turning a complex unitary table into real blocks this way preserves the unit-length, right-angle rules, so $Q^\top Q = I_6$. Every displayed tile is $R(\theta)/\sqrt{3}$, and

$$\left\| \frac{1}{\sqrt{3}} R(\theta) \right\|_F^2 = \frac{1}{3} \operatorname{tr}(R(\theta)^\top R(\theta)) = \frac{2}{3}.$$

Therefore

$$\Phi_{3,2}(Q)_{ij} = \frac{1}{2} \cdot \frac{2}{3} = \frac{1}{3},$$

so $J_3/3 \in \mathcal{S}_{3,2}$. That is a concrete gain: $s = 2$ reaches at least one important mixer that $s = 1$ cannot reach. It is not a proof that $\mathcal{S}_{3,2} = \mathcal{B}_3$.

The same construction is why two-by-two tiles can hide a complex phase as a real rotation. One-by-one blocks have nowhere to put that phase except a sign, and a sign is not enough when three of them must cancel.

One extra mixer is not a covering theorem. Hitting the barycenter does not fill the rest of the polytope.

What a fixed block can and cannot hit

For any positive integers d and s , partition $Q \in O(ds)$ into tiles $Q_{ij} \in \mathbb{R}^{s \times s}$ and set

$$\Phi_{d,s} : O(ds) \longrightarrow \mathcal{B}_d, \quad \Phi_{d,s}(Q)_{ij} = \frac{1}{s} \|Q_{ij}\|_F^2 = \frac{1}{s} \sum_{a=1}^s \sum_{b=1}^s (Q_{ij})_{ab}^2.$$

Chapter 13 already showed that every such image lands in $\mathcal{B}_d$. That is exact containment:

$$\mathcal{S}_{d,s} \subseteq \mathcal{B}_d$$

for every finite s . The proof does not replace $\subseteq$ by $=$.

The group $O(ds)$ is compact, and $\Phi_{d,s}$ is continuous because its coordinates are finite sums of squares. So $\mathcal{S}_{d,s}$ is compact, hence closed.

At $s = 1$ we have an explicit hole for every $d \geq 3$. At a larger fixed s the image contains more constructions. The general theorem cited by go-mHC does not establish that one particular choice such as $s = 2$ hits every point of $\mathcal{B}_d$ for every d [R029]. A spectral plot, a two-dimensional slice, or success on sampled targets also cannot certify that two high-dimensional sets are equal.

Hence a closed finite image that sits inside $\mathcal{B}_d$ is either the whole polytope or a proper closed subset. Sampling cannot tell you which.

Bigger by multiples, not automatically by one

There is one clean inclusion. Given $Q \in O(ds)$ and an integer $k \geq 1$, replace every scalar entry by that scalar times the $k \times k$ identity I_k . Equivalently, use $Q \otimes I_k \in O(dsk)$. The visible tiles now have size $sk \times sk$, and

$$\frac{1}{sk} \|Q_{ij} \otimes I_k\|_F^2 = \frac{1}{sk} (\|Q_{ij}\|_F^2 \|I_k\|_F^2) = \frac{1}{s} \|Q_{ij}\|_F^2.$$

Thus

$$\mathcal{S}_{d,s} \subseteq \mathcal{S}_{d,ks}.$$

Coverage cannot shrink when s is replaced by a multiple. That does not prove the consecutive nesting $\mathcal{S}_{d,s} \subseteq \mathcal{S}_{d,s+1}$. Claims of monotonic growth for every adjacent integer need a separate argument.

Drop the multiple and keep only “larger s .” Then the inclusion you actually proved has vanished, and adjacent images may fail to nest.

Average permutations with a hidden counter

A second construction explains why growing s eventually fills the polytope. Let $P_1, \dots, P_r$ be permutation tables. Choose s of them, repeats allowed, and write $P^{(a)}$ for the one attached to internal coordinate $a \in \{1, \dots, s\}$.

Build $Q \in \mathbb{R}^{ds \times ds}$ so that each internal coordinate is routed by its own permutation:

$$(Q_{ij})_{aa} = (P^{(a)})_{ij}, \quad (Q_{ij})_{ab} = 0 \quad (a \neq b).$$

Every tile Q_{ij} is $s \times s$, and the full Q is $(ds) \times (ds)$. Up to a reordering of coordinates, Q is a direct sum of permutation tables, hence orthogonal. The block map returns

$$\Phi_{d,s}(Q) = \frac{1}{s} \sum_{a=1}^s P^{(a)}.$$

So $\mathcal{S}_{d,s}$ contains every equal-weight average of s permutation tables. More generally, if the integers $k_r \geq 0$ satisfy $\sum_r k_r = s$, then

$$\sum_r \frac{k_r}{s} P_r \in \mathcal{S}_{d,s}.$$

The hidden counter a is not another leftover copy. It is an internal seat used once, then forgotten when the energies are taken. Confusing that seat with a new stream axis would add s to the wrong shape of x_i .

Grow the block until the target appears

The Birkhoff–von Neumann theorem says every $H \in \mathcal{B}_d$ is a convex combination of permutation tables [R019]:

$$H = \sum_{r=1}^R \alpha_r P_r, \quad \alpha_r \geq 0, \quad \sum_{r=1}^R \alpha_r = 1.$$

Approximate each real weight α_r by a fraction k_r/s , keeping the k_r nonnegative and $\sum_r k_r = s$. Then

$$H_s := \sum_{r=1}^R \frac{k_r}{s} P_r \in \mathcal{S}_{d,s}, \quad H_s \rightarrow H.$$

For fixed d ,

$$\overline{\bigcup_{s \geq 1} \mathcal{S}_{d,s}} = \mathcal{B}_d.$$

Equivalently: for every target $H \in \mathcal{B}_d$ and every tolerance $\varepsilon > 0$, there exist a finite block size s and a $Q \in O(ds)$ such that

$$\|\Phi_{d,s}(Q) - H\|_F < \varepsilon.$$

The same orthogonal statement follows from the density theorem for generalized unistochastic tables together with the usual real embedding $U(n) \hookrightarrow O(2n)$ [R029]. The permutation-average picture makes the quantifiers visible.

“Dense as $s \rightarrow \infty$ ” means you may grow the block until the target is as close as you like. It does not mean that one finite image $\mathcal{S}_{d,2}$ is dense. Each finite image is closed, so a fixed $\mathcal{S}_{d,2}$ could be dense in $\mathcal{B}_d$ only if it actually equaled $\mathcal{B}_d$.

The same rounding gives a rate in a simple existence sense. By Carathéodory’s theorem one may take $R \leq (d-1)^2 + 1$, the affine dimension of $\mathcal{B}_d$ plus one. Rounding the R weights to denominator s yields an $O(R\sqrt{d}/s)$ Frobenius-error bound. That is an approximation guarantee. It is not a claim about how easily gradient descent finds the corresponding Q .

Hence growing s can approximate every legal mixer, while a single named finite s remains a closed image that may still miss points.

The extra room is not another leftover copy

In a Hyper-Connection layer the persistent state has shape

$$x_l \in \mathbb{R}^{B \times d \times L \times D},$$

where B is the batch axis, d the leftover-copy axis, L the sequence axis, and D the feature axis. The residual mixer is applied independently to each batch slice along the second axis:

$$H_l^{\text{res}} \in \mathbb{R}^{d \times d}.$$

The auxiliary size s appears only while that mixer is being built:

$$A_l \in \mathbb{R}^{ds \times ds} \rightarrow Q_l \in \mathbb{R}^{ds \times ds} \rightarrow H_l^{\text{res}} = \Phi_{d,s}(Q_l) \in \mathbb{R}^{d \times d}.$$

It is not appended to x_l . After the block energies are taken, no s axis remains in the residual activation. Increasing s adds hidden geometric degrees of freedom behind each visible routing coefficient. It does not add leftover copies, tokens, or feature channels.

Batch, sequence, and feature ride along. They are not summed into s , and s is not summed into them. Drop that bookkeeping and a larger block looks like a wider model.

Why people stop at two

A real skew-symmetric table of order $N = ds$ has

$$p_s = \frac{ds(ds-1)}{2}$$

independent coordinates. With $d = 4$:

- $s = 1$: $N = 4$ and $p_1 = 6$;
- $s = 2$: $N = 8$ and $p_2 = 28$;
- $s = 3$: $N = 12$ and $p_3 = 66$.

A dense Cayley solve scales cubically in N , so doubling s multiplies its leading solve work by roughly eight at fixed d . Actual layer cost also includes the routing projections and the activation dimensions. The cubic comparison is not a full model FLOP count.

[**Experiment**] Dandachi and Diggs-Galligan use $s = 2$ as their main practical setting and report broader sampled spectral reach and broader sampled polytope slices than $s = 1$, together with synthetic and small-language-model experiments [R005]. Those measurements support $s = 2$ as a useful tested compromise in their regimes.

The choice is attractive for a second, exact reason: $s = 2$ is the first setting that can encode a complex phase as a real 2×2 rotation. The $J_3/3$ construction above is one such example. It adds substantial hidden freedom while keeping $N = 2d$.

It is not a theorem. The paper’s choice does not establish that $s = 2$ minimizes any stated objective, that $\mathcal{S}_{d,2} = \mathcal{B}_d$ for every d , or that two is the lowest-loss or lowest-latency choice for every hardware and model scale. Those claims would need a stated objective, a regime, and substantially broader evidence.

One more bookkeeping note, leftover from Chapter 12. The Cayley chart parameterizes the special-orthogonal component except tables with eigenvalue -1 . That missing locus does not affect exact double stochasticity of any produced H . Also, multiplying one scalar row of an orthogonal Q by -1 flips its determinant without changing any block energy, so $O(ds)$ and $SO(ds)$ have the same image under $\Phi_{d,s}$. The excluded-eigenvalue Cayley image is dense in $SO(ds)$. Exact reach through one chart should still not be silently identified with the full theoretical image.

If the block stays tiny, or grows without asking

Stay at $s = 1$ and, for $d \geq 3$, the mixer is confined to the orthostochastic subset. The perfectly uniform three-stream mixer is already unavailable. A target-dependent optimizer cannot learn a table the parameterization cannot represent, no matter how motivational the learning-rate schedule becomes.

Forget the finite-image distinction and exact double stochasticity gets advertised as full Birkhoff coverage. That is a category error. Exactness concerns every output. Coverage concerns every target.

Jump straight to a very large s and the skew-coordinate count and the dense Cayley solve grow. More coverage is not free, and some of it may be unnecessary for a particular task. The role of s is to expose a measurable tradeoff, not to command that a larger block always reduces loss or latency.

Finally, do not infer table coverage from eigenvalue coverage. Different mixers can share a spectrum. Spectral reach is informative about dynamics. It is only a surrogate for the image $\mathcal{S}_{d,s}$.

Ways to get this wrong

1. **Writing $\Phi_{d,s}(O(ds)) = \mathcal{B}_d$ after proving row and column sums.** The proof gives inclusion, not equality.
2. **Calling all $s = 1$ outputs boundary points.** Orthostochastic tables can lie in the interior of $\mathcal{B}_d$.
3. **Claiming $s = 1$ is always inadequate.** It covers all of $\mathcal{B}_2$; the proper-subset phenomenon starts at $d \geq 3$.
4. **Reading asymptotic density as fixed- s coverage.** “Some sufficiently expressive growing sequence” is not “ $s = 2$ covers everything.”

5. **Assuming consecutive nesting.** The simple theorem is $\mathcal{S}_{d,s} \subseteq \mathcal{S}_{d,ks}$ for multiples ks , not automatically $\mathcal{S}_{d,s} \subseteq \mathcal{S}_{d,s+1}$.
6. **Treating s as another activation axis.** It is internal to mixer construction; x_l still has d leftover copies.
7. **Calling algebraic exactness bitwise exactness.** Floating-point solves and norm reductions have numerical error even though the mathematical map lands exactly in $\mathcal{B}_d$.
8. **Declaring $s = 2$ optimal.** It is a paper-tested engineering choice, not an optimality theorem.

A larger sampled shadow in an eigenvalue plot is evidence of broader spectral reach. It is not a certificate that the full matrix image is larger in every direction.

The compact slogan is

exact membership for every s $\neq$ full coverage at that s .

Try it

1. Two leftover copies

Show that every $H \in \mathcal{B}_2$ is orthostochastic. Construct a corresponding $Q \in O(2)$ for

$$H = \begin{pmatrix} 0.36 & 0.64 \\ 0.64 & 0.36 \end{pmatrix}.$$

Answer

Every $H \in \mathcal{B}_2$ is $\begin{pmatrix} t & 1-t \\ 1-t & t \end{pmatrix}$. Choose $\cos \theta = \sqrt{t}$ and $\sin \theta = \sqrt{1-t}$. For $t = 0.36$, one choice is $Q = \begin{pmatrix} 0.6 & -0.8 \\ 0.8 & 0.6 \end{pmatrix}$. Entrywise squaring gives H .

2. Why the equal three-stream mixer fails at $s = 1$

Prove that $J_3/3$ is not orthostochastic, and identify the exact place where odd dimension matters in the proof.

Answer

If $Q_{ij}^2 = 1/3$, then $Q_{ij} = \pm 1/\sqrt{3}$. The inner product of two rows is $(\pm 1 \pm 1 \pm 1)/3$, never zero. Odd dimension matters because an odd number of ± 1 terms cannot sum to zero.

3. Check the two-by-two construction

For the 6×6 block table in the numerical example, verify (a) that each 2×2 tile contributes $1/3$ after normalization and (b) that each row of the resulting $H \in \mathbb{R}^{3 \times 3}$ sums to one.

Answer

For every tile, $Q_{ij} = R(\theta)/\sqrt{3}$, so $\|Q_{ij}\|_F^2 = \text{tr}(I_2)/3 = 2/3$. Dividing by $s = 2$ gives $H_{ij} = 1/3$. Each row has three entries, hence sum $3(1/3) = 1$; columns are identical.

4. Build a rational permutation mixture

Let $d = 3$, $s = 5$, and let P be the permutation table for the cycle $1 \mapsto 2 \mapsto 3 \mapsto 1$. Construct, at the block level, an orthogonal $Q \in \mathbb{R}^{15 \times 15}$ whose image is

$$H = \frac{3}{5}I_3 + \frac{2}{5}P.$$

Answer

Use five internal coordinates. Route the first three by I_3 and the last two by P . Set every tile $Q_{ij} \in \mathbb{R}^{5 \times 5}$ diagonal, with diagonal entry a equal to the (i, j) entry of the permutation selected for internal coordinate a . Up to coordinate reordering, $Q = I_3 \oplus I_3 \oplus I_3 \oplus P \oplus P$, so it is orthogonal. Counting squared ones in each tile gives $\Phi_{3,5}(Q) = 3I_3/5 + 2P/5$.

5. Separate three claims

Classify each statement as guaranteed, empirically supported in the go-mHC paper, or unsupported by the results in this chapter:

1. Every output at $s = 2$ is doubly stochastic.
2. Sampled $s = 2$ mixers showed broader spectral reach than sampled $s = 1$ mixers in the paper's setting.
3. $\mathcal{S}_{d,2} = \mathcal{B}_d$ for every d .
4. Allowing growing s permits arbitrary approximation of every $H \in \mathcal{B}_d$.

Answer

- (1) guaranteed by exact containment; (2) empirical result; (3) unsupported here and must not be inferred from (1) or (2); (4) guaranteed by asymptotic coverage.

Check yourself

1. **At $s = 1$, what is H_{ij} , and when is the image the whole of $\mathcal{B}_d$?**
 $H_{ij} = Q_{ij}^2$. The image is all of $\mathcal{B}_2$, and a proper subset for $d \geq 3$.
2. **Show that exact membership does not imply full coverage.**
 Chapter 13 proved every block-energy mixer is doubly stochastic. That is a statement about every output. Coverage is a statement about every target. $\mathcal{S}_{3,1} \subsetneq \mathcal{B}_3$ is the counterexample.
3. **What fails if the tiles are 1×1 and the target is $J_3/3$?**
 Every entry of Q would have magnitude $1/\sqrt{3}$, so two rows would have inner product $(\pm 1 \pm 1 \pm 1)/3$, which cannot vanish.
4. **What does $s = 2$ buy in the three-stream example, and what does it not prove?**
 It reaches $J_3/3$, which $s = 1$ cannot. It does not prove $\mathcal{S}_{3,2} = \mathcal{B}_3$, nor that $s = 2$ minimizes loss, latency, or parameter count.
5. **Show that growing s can approximate every legal mixer.**
 Every $H \in \mathcal{B}_d$ is a convex combination of permutations. Rounding the weights to denominator s produces a point of $\mathcal{S}_{d,s}$ whose Frobenius distance to H can be driven below any $\varepsilon > 0$. A fixed finite image is still closed and need not be the whole polytope.
6. **What fails if the go-mHC paper's choice $s = 2$ is read as an optimality theorem?**
 The paper tested a compromise in its regimes. That is not a theorem that two is lowest-loss, lowest-cost, or fully covering.

At $s = 1$, the mixer is orthostochastic: all of $\mathcal{B}_2$, a proper subset for $d \geq 3$. At every finite s , $\Phi_{d,s}$ produces exactly doubly stochastic tables. A fixed finite image is not automatically the whole Birkhoff polytope. When s is allowed to grow, the images approximate all of $\mathcal{B}_d$.

Sources

- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R029] I. Nechita, Z. Ouyang, and A. Szczepanek, “Generalized unistochastic matrices,” arXiv:2310.03436v2, 2023. <https://arxiv.org/abs/2310.03436>
- [R030] Y.-H. Au-Yeung and Y.-T. Poon, “ 3×3 orthostochastic matrices and the convexity of generalized numerical ranges,” *Linear Algebra and its Applications* **27**, 69–79 (1979). [https://doi.org/10.1016/0024-3795\(79\)90032-6](https://doi.org/10.1016/0024-3795(79)90032-6)
- [R028] E. Gutkin, “On a multi-dimensional generalization of the notions of orthostochastic and unistochastic matrices,” *Journal of Geometry and Physics* **74**, 28–35 (2013). <https://doi.org/10.1016/j.geomphys.2013.06.009>

-
- [R019] G. Birkhoff, “Three observations on linear algebra,” *Universidade Nacional de Tucumán, Revista A* **5**, 147–151 (1946). Bibliographic record: <https://www.ams.org/mathscinet-getitem?mr=0020547>

Chapter 15 — Five locks on the same door

The hallway has one door. Five locksmiths hang five different locks on it.

Same bolt. Same hallway. Same next room. Each lock still has its own key, its own forbidden turns, and its own bill. You do not crown a lock. You ask what it costs, what it forbids, and whether the key actually turns all the way.

The door is the residual update used since Hyper-Connections. The lock is the recipe that builds the residual routing matrix H^{res} . This chapter puts the five recipes on one ledger and refuses to pick a winner without units.

The same door

All five variants share the update

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l).$$

For one routed item there are d residual streams, each with C features. So $x_l \in \mathbb{R}^{d \times C}$, the read and write maps satisfy $H^{\text{pre}}, H^{\text{post}} \in \mathbb{R}^{1 \times d}$, and the residual lock is a square $H^{\text{res}} \in \mathbb{R}^{d \times d}$.

Batch and sequence can be restored in the usual order as $x_l \in \mathbb{R}^{B \times d \times L \times C}$. Neither batch nor sequence is a stream. We compare one hyper-connection module. A Transformer block commonly contains two such modules, so multiply a module-level parameter count by two and then by the number of blocks.

The papers use related but not identical routers, notation, and cost conventions. “The routing matrix has d^2 entries” is not a parameter count when an input-dependent linear projection must emit those entries. Likewise, $O(d^3)$ can mean a dense solve, a router projection, or both. Hiding C , the number of Sinkhorn steps, or the go-mHC block size changes the engineering conclusion.

Hence the shared update is not a shared implementation. The next sections keep the recipes, the produced sets, and the bills on separate lines.

Two questions that are not twins

A second trap is mixing two questions that look like one:

1. Is every produced matrix doubly stochastic?
2. Can the parameterization produce every doubly stochastic matrix?

The first is **exactness**. The second is **coverage**. We take the split as law for the rest of the chapter.

KromHC answers yes to the first and generally no to the second. For go-mHC, $s = 1$ is strictly non-covering when $d \geq 3$; equality or strictness at an arbitrary named larger finite s is not established here.

Multiples preserve earlier images, adjacent levels need not be nested, and the union over block sizes is dense in the full polytope. mHC-lite covers the whole Birkhoff polytope in the closure of its softmax parameterization,

but pays for all $d!$ permutation coordinates. Finite-step mHC aims at the full polytope but does not enforce both marginals exactly.

These are different bargains, not different adjectives. Drop the split and “doubly stochastic” starts meaning whatever the speaker wanted the next sentence to say.

Five ways to turn the key

Think of H^{res} as a rail switchyard for persistent state.

- **HC** learns every switch independently. It permits negative, amplifying, and non-conserving routes.
- **mHC** repeatedly rescales a positive switchboard until row and column traffic are nearly balanced.
- **mHC-lite** votes among every possible hard permutation of the tracks.
- **KromHC** builds a large switchboard as a Kronecker product of small balanced switchboards.
- **go-mHC** rotates an enlarged orthogonal space and converts block energies into nonnegative routes.

Exact constraint satisfaction is a safety rail, not a certificate that optimization will be easy or that a language model will be accurate. A rail can prevent one kind of fall while remaining impressively indifferent to your destination.

The analogy has done its work. From here the object is the matrix H^{res} and the map that produces it.

A two-by-two that all five can open

Take $d = 2$, $C = 1$, and target

$$M = \begin{pmatrix} 0.75 & 0.25 \\ 0.25 & 0.75 \end{pmatrix} \in \mathbb{R}^{2 \times 2}.$$

Every constrained variant can represent this target. mHC can approach it through Sinkhorn scaling. mHC-lite writes

$$M = 0.75I + 0.25P, \quad P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

A one-factor KromHC has the same two-permutation mixture. For go-mHC with $s = 1$, choose an orthogonal rotation

$$Q = \begin{pmatrix} \sqrt{0.75} & -\sqrt{0.25} \\ \sqrt{0.25} & \sqrt{0.75} \end{pmatrix}; \quad M_{ij} = Q_{ij}^2.$$

HC can also represent M , but it can equally represent

$$U = \begin{pmatrix} 1.1 & -0.2 \\ 0.3 & 0.7 \end{pmatrix},$$

whose negative entry and unequal row sums are forbidden in $\mathcal{B}_2$, the set of 2×2 doubly stochastic matrices. This tiny case conceals the coverage differences: $\mathcal{B}_2$ is only a line segment. At larger d , Kronecker structure and finite block size become genuine restrictions.

Hence agreement on one legal 2×2 table is not a comparison. The unconstrained recipe can leave $\mathcal{B}_2$; the constrained recipes cannot, and they start to disagree about which legal tables they can hit once d grows.

A shared bill for the later four

For the mHC-family formulas, flatten the routed features to $\hat{x} \in \mathbb{R}^q$ with $q = dC$. The shared input-dependent pre/post projections each map q coordinates to d outputs. Including their two biases and three scalar gains,

define

$$P_{\text{common}} = 2qd + 2d + 3 = 2d^2C + 2d + 3.$$

If a residual construction emits r coordinates using a weight matrix in $\mathbb{R}^{q \times r}$ and a bias in $\mathbb{R}^r$, its exact trainable count is

$$P(r) = (q + 1)r + P_{\text{common}}.$$

Substitute and you recover the primary-paper formulas:

$$\begin{aligned} P_{\text{mHC}} &= (dC + 1)d^2 + 2d^2C + 2d + 3, \\ P_{\text{lite}} &= (dC + 1)d! + 2d^2C + 2d + 3, \\ P_{\text{Krom}} &= (dC + 1) \sum_{k=1}^K i_k! + 2d^2C + 2d + 3, \\ P_{\text{go}} &= (dC + 1) \frac{ds(ds - 1)}{2} + 2d^2C + 2d + 3, \end{aligned}$$

where $d = \prod_{k=1}^K i_k$, each KromHC factor is $i_k \times i_k$, and s is the go-mHC block size [R002]; [R003]; [R004]; [R005]. These are per module, not per full Transformer block.

For fixed C , fixed s , and bounded KromHC factors, the respective asymptotics are

$$\Theta(Cd^3), \quad \Theta(Cd d!), \quad \Theta(Cd^2), \quad \Theta(Cs^2d^3).$$

The KromHC $\Theta(Cd^2)$ leading term comes from the shared pre/post projections; with binary factors, its residual-coordinate term is only $\Theta(Cd \log d)$.

Original dynamic HC does **not** use exactly this flattened-router ledger. Its primary paper reduces/normalizes the streams and reports

$$P_{\text{DHC}} = P_{\text{norm}} + C(d + 2) + d(d + 2) + 2,$$

while static HC has $P_{\text{SHC}} = d(d + 2)$ [R001]. This is why copying the later mHC count into the HC row would be false precision. Under the actual original DHC design, its router count is $\Theta(Cd + d^2)$; under a hypothetical flattened $q = dC$ unconstrained router, it would instead match the mHC coordinate count with $r = d^2$.

Hence a parameter is a learned weight, not a matrix entry. Hide C , s , or the factor list and the leading term changes.

Two ways to count arithmetic

A multiply-add may be counted as one MAC or two FLOPs. We use two FLOPs and count one length- q dense projection to r outputs as $r(2q - 1)$ FLOPs before bias and nonlinearities. Therefore the residual-logit projection alone is exact under this convention:

$$F_{\text{proj}}(r) = r(2dC - 1).$$

All variants also pay to apply the final dense route to $x \in \mathbb{R}^{d \times C}$:

$$F_{Hx} = Cd(2d - 1).$$

Read and write routing, normalization, gains, activations, backward recomputation, and memory traffic are common or implementation-dependent additions. They must not be smuggled into one row but omitted from another.

The go-mHC paper supplies a second, explicitly approximate convention for the *construction overhead*, multiplied by its symbol C , with S Sinkhorn iterations and equal KromHC factors i , $K = \log_i d$ [R005]:

$$\begin{aligned} \text{mHC:} \quad & 4CSd^2 = \Theta(CSd^2), \\ \text{mHC-lite:} \quad & 3C(d!) + 2Cd^2(d!) = \Theta(Cd^2d!), \\ \text{KromHC:} \quad & 3CK(i!) + 2CKi^2(i!) + C \frac{i^{2K+2} - i^4}{i^2 - 1}, \\ \text{go-mHC:} \quad & \frac{8}{3}C(ds)^3 + 4C(ds)^2 = \Theta(Cs^3d^3). \end{aligned}$$

The KromHC geometric term is $\Theta(Cd^2)$ because $i^{2K} = d^2$. These expressions are estimates from one paper, not hardware timings and not replacements for F_{proj} . They reconcile as follows: parameter generation is governed by r ; manifold construction is governed by Sinkhorn passes, permutation accumulation, Kronecker assembly, or a dense ds -solve; applying the resulting $d \times d$ route is always dense unless structure is exploited.

For $d = 4$, $S = 20$, $s = i = 2$, $K = 2$, and the cost-table convention $C = 1$, the four estimates are respectively 1,280, 840, 60, and about 1,621 FLOPs. Thus “factorial” does not mean slowest at $d = 4$; it means the curve becomes unusable as d grows.

Drop the convention and a MAC becomes two FLOPs or one, a construction estimate becomes a projection count, and a paper symbol C becomes a feature width. The arithmetic still runs. The comparison does not.

One table, no winner

“Exact” below means that exact real arithmetic places every produced H^{res} in $\mathcal{B}_d$. Floating-point roundoff remains. “Coverage” names the produced set or its closure, not a mood.

Metric	HC	mHC	mHC-lite	KromHC	go-mHC
Learned router	Static scalars or normal-ized/reduced activation followed by linear maps, tanh, gain, and static bias	Flattened normalized dC -vector projects dynamic pre/post gates and d^2 residual logits	Same pre/post router; residual projection emits $d!$ mixture logits	Same pre/post router; emits $i_k!$ logits for each of K factors	Same pre/post router; emits $ds(ds-1)/2$ skew coordinates
Constraint map	None on H^{res}	Exponentiation followed by $S = 20$ alternating Sinkhorn row/column normalizations in the paper	Softmax weights; weighted sum of all $d!$ permutation matrices	Softmax mixture for each small factor, then Kronecker product	Skew fill A ; Cayley solve $Q = (I - A)(I + A)^{-1}$; block squared Frobenius norms divided by s
Doubly stochastic exactness	No	No finite-step guarantee; the marginal normalized last is exact in real arithmetic, the other generally is not	Yes by convex combination	Yes by factor exactness and Kronecker closure	Yes by orthogonality and block-energy row/column identities

Metric	HC	mHC	mHC-lite	KromHC	go-mHC
Set coverage	Unconstrained real $d \times d$ routes; includes $\mathcal{B}_d$ but also amplifying and negative routes	Positive Sinkhorn limit can target the relative interior of $\mathcal{B}_d$; finite S is generally off the set; boundary points require limits	Closure is all $\mathcal{B}_d$. Finite softmax logits give strictly positive permutation weights, so vertices are approached as logits diverge	$\{\bigotimes_k U_k : U_k \in \mathcal{B}_{i_k}\}$, generally a strict subset of $\mathcal{B}_d$	Finite Cayley coordinates produce $\Phi_{d,s}(\text{cay}(\mathfrak{so}(ds)))$. After accounting for determinant signs and the missing -1 chart boundary, its closure can match the theoretical image $\Phi_{d,s}(O(ds)) \subseteq \mathcal{B}_d$. At $s = 1$ this orthostochastic image is strict for $d \geq 3$. For larger finite s , neither strictness nor surjectivity is asserted here: adjacent s -images need not be nested, although $\mathcal{S}_{d,s} \subseteq \mathcal{S}_{d,ks}$ for positive integers k , and the union is asymptotically dense.
Exact parameters, one module	SHC: $d(d+2)$. Original DHC: $P_{\text{norm}} + C(d+2) + d(d+2) + 2$	$(dC+1)d^2 + 2d^2C + 2d + 3$	$(dC+1)d! + 2d^2C + 2d + 3$	$(dC+1) \sum i_k! + 2d^2C + 2d + 3$	$(dC+1)ds(ds-1)/2 + 2d^2C + 2d + 3$
Construction FLOPs in paper convention	Original paper: routing application $O(Cd(d+1))$; measured 0.147% forward-FLOP increase for 7B DHC $\times 4$	Approx. $4CSd^2$, plus projection/application; repeated reductions favor fused kernels	Approx. $3Cd! + 2Cd^2d!$, plus shared work	Equal-factor estimate above; asymptotically $\Theta(Cd^2)$ for bounded i , plus shared work	Approx. $(8/3)C(ds)^3 + 4C(ds)^2$, plus shared work
Spectra of H^{res}	Unrestricted by stochasticity; eigenvalues may leave unit disk	If exactly doubly stochastic, 1 is an eigenvalue and all eigenvalues lie in the unit disk; finite SK weakens that guarantee	Full Birkhoff spectral possibilities in closure	$\text{spec}(\bigotimes U_k) = \{\prod_k \lambda_{j_k}(U_k)\}$; binary factors yield strongly restricted, real product spectra	Finite- s image only, but the paper's sampled spectra reach substantially more of the Birkhoff spectral region than tested binary KromHC

Metric	HC	mHC	mHC-lite	KromHC	go-mHC
Implementation	Native reductions, small projections, and dense stream mixing	Numerically careful exponentiation, 20 normalizations, fused forward/backward or recomputation for competitive throughput	Native softmax and matrix multiply, but materializing all permutations scales factorially	Native small softmaxes, permutation mixtures, Kronecker assembly; factorization bookkeeping	Native batched linear solve and tensor contractions; no explicit inverse should be formed
Large- d suitability	Quadratic routing/application and $\Theta(Cd + d^2)$ original router, but no conservation guarantee	Polynomial but $\Theta(Cd^3)$ parameters under flattened routing and S passes; large d is costly	Not suitable without changing the method: $d!$ logits and permutation basis	Parameter-efficient for bounded factors and power/composite d ; suitability is purchased with restricted routes	Polynomial and tunable in s , but $\Theta(Cs^2d^3)$ parameters and $\Theta(Cs^3d^3)$ solve estimate make very large d materially costlier than binary KromHC
Stability evidence	HC paper reports successful 1B/7B/MoE and vision training; mHC paper reports instability events and amplified signals when scaling HC. No theorem bounds products of unconstrained routes	Doubly stochastic target motivates norm/mean control; mHC reports stable large-scale training and 6.7% time overhead at $d = 4$, but finite SK is approximate	Exact construction theorem; paper reports lower/smooth gradients than HC/mHC in 6–24-layer, 45M–360M experiments. This is not yet evidence at mHC’s largest scale	Exact construction theorem; paper reports zero column-sum product error and competitive results around 60M/130M scales	Exact construction theorem; paper reports stable gradients in a 30M GPT-style model and synthetic mixing results. Large-scale stability remains untested in v1

At $d = 4$, mHC-lite’s 24-way mixture is modest and its native softmax and matrix multiply stay inside ordinary framework ops. At $d = 16$, $16! \approx 2.09 \times 10^{13}$ coordinates end the discussion before the optimizer gets a vote. For large composite d with a hard compute budget, KromHC has the smallest polynomial cost in this comparison, but cannot express arbitrary cycles or spectra. go-mHC spends cubic work for a finite- s image that is not forced to be a Kronecker product. mHC spends iterative normalization and accepts finite tolerance. HC retains the widest unconstrained algebraic freedom and the weakest conservation guarantee.

No row wins on every metric. A winner without units is just a mood.

One line that splits two locks

The exact distinction appears in one line. Given residual coordinates u_l , mHC computes

$$H_l^{\text{res}} = \text{SK}_S(\exp(\text{mat}(u_l))) \in \mathbb{R}^{d \times d},$$

whereas go-mHC sets $N = ds$, fills $A_l \in \mathbb{R}^{N \times N}$ with $A_l^\top = -A_l$, solves for orthogonal Q_l , partitions Q_l into blocks $Q_{ij} \in \mathbb{R}^{s \times s}$, and defines

$$(H_l^{\text{res}})_{ij} = \frac{1}{s} \|Q_{ij}\|_F^2.$$

In exact arithmetic, go-mHC's output is nonnegative and has every row and column sum equal to one [R005].

That statement proves membership, not full coverage, favorable conditioning of the Cayley solve, a lower validation loss, or large-scale systems efficiency. The v1 evidence for those questions is limited to the experiments it actually reports.

Omit orthogonality and the trace identities that prove the sums collapse. Omit the last Sinkhorn pass and one marginal of mHC is no longer exact even in real arithmetic.

What falls off

Omit the constraint from HC, and products $H_{L-1}^{\text{res}} \cdots H_l^{\text{res}}$ may amplify, attenuate, or sign-flip stream state without a conservation law. Omit enough Sinkhorn iterations from mHC, and the unnormalized marginal can deviate measurably from one. Omit permutations from mHC-lite, and full Birkhoff coverage no longer follows. Omit factors from KromHC, and the final matrix has the wrong dimension; enlarge factors, and factorial factor-mixture cost returns. Omit orthogonality from go-mHC, and the trace identities proving row and column sums collapse.

There is also a less dramatic failure: omit C, S, s , or i_k from a complexity statement, and a correct asymptotic comparison becomes incorrect procurement advice.

Ways to get this wrong

Calling finite-step Sinkhorn “exact enough” without reporting a row/column residual is the first one. Exactness is a mathematical property; tolerance is a measured number.

1. **Counting coordinates, not learned weights.** go-mHC emits $ds(ds-1)/2$ coordinates, but its dynamic projection contains $(dC+1)ds(ds-1)/2$ parameters.
2. **Claiming finite- s go-mHC covers—or necessarily misses— $\mathcal{B}_d$.** At $s=1, d \geq 3$, the image is strict. For a named larger finite s , neither equality nor strictness is asserted here. Images embed at integer multiples of s , adjacent levels need not nest, and the union over all block sizes is dense.
3. **Claiming KromHC has arbitrary Birkhoff spectra.** Kronecker eigenvalues are products of factor eigenvalues.
4. **Using $O(d^3)$ without saying what is fixed.** go-mHC is $O(Cs^2d^3)$ in parameters and its quoted solve estimate is $O(Cs^3d^3)$.
5. **Comparing original HC's reduced router to mHC's flattened router as if they were identical implementations.** They are not.
6. **Equating exact double stochasticity with identity.** A permutation matrix is doubly stochastic and may rearrange every stream.
7. **Treating paper FLOP estimates as wall-clock latency.** Kernel launches, memory movement, fusion, precision, and backward storage matter.

Try it

1. Count the keys on one module

For $d=4$, $C=512$, compute one-module parameter counts for mHC, mHC-lite, binary-factor KromHC with $(i_1, i_2) = (2, 2)$, and go-mHC with $s=2$.

Answer

Here $P_{\text{common}} = 2(4^2)(512) + 2(4) + 3 = 16,395$, $q+1 = 2,049$, and the residual coordinate counts are 16, 24, 4, and 28. Totals are respectively 49,179, 65,571, 24,591, and 73,767.

2. Exact is not the same as covering

Classify each statement as true or false: (a) every KromHC output is doubly stochastic; (b) every doubly stochastic matrix is a binary-factor KromHC output; (c) every finite- s go-mHC output is doubly stochastic; (d) every matrix in $\mathcal{B}_d$ has a finite-logit mHC-lite representation.

Answer

(a) true; (b) false in general; (c) true in exact arithmetic; (d) false as written because finite softmax logits assign positive weight to every permutation. The closure covers all of $\mathcal{B}_d$, including vertices.

3. A spectrum KromHC cannot hit

Let $d = 4$ and KromHC use two 2×2 doubly stochastic factors. Explain why it cannot represent a routing matrix whose spectrum contains i .

Answer

A 2×2 doubly stochastic factor has eigenvalues 1 and a real number $a \in [-1, 1]$. The Kronecker spectrum is $\{1, a, b, ab\}$, entirely real. Therefore i cannot occur.

4. A crossover you cannot invent

Using only the quoted asymptotics, can you prove that go-mHC is faster than mHC for all d ?

Answer

No. The estimates are $\Theta(Cs^3d^3)$ and $\Theta(CSd^2)$, so their ratio scales like $sd \cdot s^2/S = s^3d/S$, up to constants. Either can be smaller in a finite regime; hardware and the omitted shared projection/application also matter. Benchmark the intended (d, s, S, C) and implementation.

5. Pick a lock from a spec sheet

You require exact double stochasticity, native framework operations, $d = 32$, and the smallest parameter count among these methods. Full Birkhoff coverage is not required. Which row survives?

Answer

KromHC with small factors, assuming a suitable factorization of 32. mHC is not exact at finite S ; mHC-lite is factorial; go-mHC is exact and native but has cubic-in- d dynamic projection cost for fixed s ; HC is unconstrained. The choice explicitly accepts KromHC's restricted routing set.

Check yourself

1. What tensor emits the routing coordinates, and what is its input width?

Name the emitting map and its width. For the later flattened family the input width is $q = dC$; original DHC is a reduced/normalized router, not that flattened map.

2. Show that exact membership and full coverage are separate axes.

Finite-step mHC is approximate; mHC-lite, KromHC, and go-mHC are exact in real arithmetic; HC is unconstrained. Closure of mHC-lite is all of $\mathcal{B}_d$; KromHC is a Kronecker subset; finite- s go-mHC is an image. Exact KromHC still misses many targets.

3. What subset of $\mathcal{B}_d$ is covered at finite hyperparameters?

The produced set or its closure at the finite S , s , or factor list you actually use. Closure of mHC-lite is all of $\mathcal{B}_d$; KromHC is a Kronecker subset; finite- s go-mHC is an image, not a mood.

4. What fails if a paper construction-FLOP row is read as wall-clock latency?

Kernel launches, memory movement, fusion, precision, and backward storage are omitted. The row is also not the projection count F_{proj} .

5. At what model scale was stability actually tested?

Quote the paper's scale: HC at 1B/7B/MoE and vision; mHC at large scale with 6.7% time overhead at $d = 4$; mHC-lite at 45M–360M; KromHC around 60M/130M; go-mHC at 30M in v1.

The compact parameter rule is $P = (dC + 1)r + 2d^2C + 2d + 3$ for the later flattened-router family, with $r = d^2, d!, \sum i_k!$, or $ds(ds - 1)/2$. The compact mathematical rule is simpler: exact membership and full coverage are separate axes.

The table compares mechanisms. What those mechanisms *mean* as learned dataflow — which streams a block reads, where it writes, and how persistent state moves through depth — still has to stay tethered to the router granularity and constraint set established here.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” *ICLR 2025*, arXiv:2409.19606. <https://arxiv.org/abs/2409.19606>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025/2026. <https://arxiv.org/abs/2512.24880>
- [R003] Y. Yang and J. Gao, “mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations,” arXiv:2601.05732, 2026. <https://arxiv.org/abs/2601.05732>
- [R004] W. Zhou et al., “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices,” arXiv:2601.21579, 2026. <https://arxiv.org/abs/2601.21579>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309v1>
- [R021] R. Sinkhorn and P. Knopp, “Concerning Nonnegative Matrices and Doubly Stochastic Matrices,” *Pacific Journal of Mathematics* 21(2), 1967, pp. 343–348. <https://doi.org/10.2140/pjm.1967.21.343>
- [R019] G. Birkhoff, “Tres observaciones sobre el algebra lineal,” *Universidad Nacional de Tucumán Revista, Serie A* 5, 1946, pp. 147–151. <https://eudml.org/doc/130662>

Chapter 16 — Blend, pour, shuffle leftovers

Four labeled trays sit on a workbench. Each tray holds one leftover copy of the sentence table — a stream. A worker walks up.

The worker may scoop a blend from several trays, run one tool once, and pour the result back with different strengths. Meanwhile the leftover contents are shuffled so yesterday’s work is still there tomorrow. Then you add the pour to the leftover shuffle.

Those three moves are read, write, and persist. Together they are a *dataflow*: which state goes where. They do not decide which worker is standing at the bench.

We already wrote the one-line update in Chapter 4. Today we walk it with a pencil. Then we hire three more workers who do different jobs. Then we refuse a popular mix-up. Trays are not specialists, and a pretty heat map is not a job title.

Four trays, one worker

Let l be which layer we are on. Write

- d for how many leftover copies we carry;
- B for how many sentences ride together;
- L for how many tokens sit in one sentence;
- D for how many features each token has;
- $\mathbf{x}_l \in \mathbb{R}^{B \times d \times L \times D}$ for the workbench;
- $H_l^{\text{pre}} \in \mathbb{R}^{1 \times d}$ for the scoop;
- $H_l^{\text{post}} \in \mathbb{R}^{1 \times d}$ for the pour;
- $H_l^{\text{res}} \in \mathbb{R}^{d \times d}$ for the leftover shuffle;
- $F_l : \mathbb{R}^{B \times L \times D} \rightarrow \mathbb{R}^{B \times L \times D}$ for one worker, with parameters W_l .

The papers write the Hyper-Connection update in stream-first algebra [R001]; [R002]; [R005]. Under this book’s batch-first storage, apply that update independently to each $d \times L \times D$ slice of a sentence:

$$\mathbf{x}_{l+1}[b] := H_l^{\text{res}} \mathbf{x}_l[b] + (H_l^{\text{post}})^\top F_l(H_l^{\text{pre}} \mathbf{x}_l[b], W_l). \quad (16.1)$$

The multiplications act only on the stream axis, which is axis 1 in stored $\mathbf{x}_l$. The scoop therefore produces an ordinary table in $\mathbb{R}^{B \times L \times D}$. The leftover shuffle and the pour both come back in $\mathbb{R}^{B \times d \times L \times D}$.

Read the line from the inside out.

Innermost: $H_l^{\text{pre}} \mathbf{x}_l$ blends the trays into one table. That blend is the **read**. Next: F_l runs once on that blend. One call. Not d calls. Then $(H_l^{\text{post}})^\top$ fans the one answer back into the trays. That fan-out is the **write**. Meanwhile $H_l^{\text{res}} \mathbf{x}_l$ remixes the old trays without asking F_l . That remix is the leftover path: persist, then add.

Those three maps together are **state routing**. Changing H^{pre} , H^{post} , or H^{res} changes connections among streams. It does not, by itself, swap the worker.

A stream is a place where a representation persists. A worker is a parameterized function. A gate can point at either one. The noun after “route” is the whole lesson.

Batch, sequence, and feature ride along. They are not streams, and they are not contracted by these three maps.

Three numbers through the workshop

Hide batch, sequence, and feature axes by setting $B = L = D = 1$, and use $d = 3$. The workbench is then three scalars:

$$\mathbf{x}_l = \begin{bmatrix} 2 \\ 5 \\ -1 \end{bmatrix} \in \mathbb{R}^{3 \times 1 \times 1}.$$

Choose the scoop

$$H_l^{\text{pre}} = \begin{bmatrix} 0.5 & 0.3 & 0.2 \end{bmatrix} \in \mathbb{R}^{1 \times 3}.$$

The worker sees one number,

$$z_l = H_l^{\text{pre}} \mathbf{x}_l = 0.5(2) + 0.3(5) + 0.2(-1) = 2.3.$$

Let the worker be $F_l(z) = 2z$, so $y_l = 4.6$. That evaluates F_l once, not three times.

Choose the pour

$$H_l^{\text{post}} = \begin{bmatrix} 0.1 & 0.7 & 0.2 \end{bmatrix} \in \mathbb{R}^{1 \times 3}.$$

The new answer contributes

$$(H_l^{\text{post}})^\top y_l = \begin{bmatrix} 0.46 \\ 3.22 \\ 0.92 \end{bmatrix}.$$

Finally choose a leftover shuffle

$$H_l^{\text{res}} = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.2 & 0.6 & 0.2 \\ 0 & 0.2 & 0.8 \end{bmatrix} \in \mathbb{R}^{3 \times 3}.$$

Its rows and columns sum to one, so it is doubly stochastic. The carried state is

$$H_l^{\text{res}} \mathbf{x}_l = \begin{bmatrix} 2.6 \\ 3.2 \\ 0.2 \end{bmatrix}.$$

Add carry and write:

$$\mathbf{x}_{l+1} = \begin{bmatrix} 3.06 \\ 6.42 \\ 1.12 \end{bmatrix}.$$

Hence three different questions were answered: which state was read, where new work was written, and how old state persisted. No specialist was selected. There was only one function F_l .

Change only the scoop and the carry stays put. Change only the leftover shuffle and the worker still sees the same blend. Drop that separation and one matrix starts doing three jobs.

A second kind of routing

Neural-network English has reused “routing” with the enthusiasm of a programmer naming every temporary variable data. The trays have a gate. Mixture-of-Experts also has a gate. Therefore the two must be the same, yes? No.

Suppose you have several workers $E_1, \dots, E_K$, each with its own parameters. A second gate produces weights $g_1, \dots, g_K$ and either mixes their outputs or picks a sparse subset. That is **expert routing**. The name is worth keeping because the thing being chosen is a parameterized computation, not a leftover tray.

The original sparsely gated Mixture-of-Experts used a trainable gate to choose a sparse combination of feed-forward subnetworks [R014]. Switch Transformer, in its simplified formulation, sends an incoming item to one expert [R015]. Exact routing granularity belongs to the particular implementation. The letters “MoE” do not specify it.

A sparse expert gate normally decides **which parameter set processes an input**. An HC router decides **which persistent state feeds a branch, where the branch result is stored, and how state persists**. HC has been evaluated with dense and sparse computational blocks [R001]. That compatibility does not identify HC routing with MoE routing.

State routing alone does not skip an attention call. Four streams do not imply four experts. Four workers do not imply sparse conditional computation.

Question	State router	Expert router
What is routed?	activation state in persistent streams	input to parameterized functions
Typical destination	another stream, a branch input, or a stream write	one or more experts
Does routing alone change parameter choice?	no	yes
Does routing alone save branch FLOPs?	no	only if sparse dispatch actually skips experts
Representative coefficients	$H^{\text{pre}}, H^{\text{post}}, H^{\text{res}}$	g over experts
Main architectural level	residual/dataflow topology	conditional parameter activation

A coefficient close to zero is not automatically a computational skip. Dense multiplication by a nearly zero gate can spend the full FLOP budget with admirable precision.

Four different workers

A block has **multiple branches** when it contains $K > 1$ transformations $F_1, \dots, F_K$. That is a graph property: more than one computation is sitting there. They may all run, only some may run, or they may run at fixed positions. Branch multiplicity is not proof of conditional computation.

We now construct a speculative extension. It is a design exercise, not an architecture claimed by the HC, mHC, or go-mHC papers.

Let $K = 4$, and give every worker the same external signature

$$F_{l,k} : \mathbb{R}^{B \times L \times D} \rightarrow \mathbb{R}^{B \times L \times D}, \quad k \in \{1, 2, 3, 4\}.$$

Inside that interface, set

$$F_{l,1} = \text{attention}, \quad F_{l,2} = \text{WavePDE}, \quad F_{l,3} = \text{SSM}, \quad F_{l,4} = \text{MLP}.$$

Attention supplies content-dependent interaction across positions [R010]. A Wave-PDE layer is a recent proposal based on differentiable second-order wave-equation simulation [R016]. A selective SSM such as Mamba makes state-space parameters input-dependent and propagates state along the sequence [R017]. An MLP transforms each position’s feature vector independently unless another operation couples positions.

These descriptions concern the operators, not the HC stream axis. Branch-specific adapters may be needed to make widths, auxiliary state, causal conventions, and normalization agree. Declaring a shared signature is an interface contract. It is not evidence that four arbitrary library modules can be plugged together without adapters. Software remains stubbornly literal.

For each worker k , define a scoop $a_{l,k} \in \mathbb{R}^{1 \times d}$ and a pour $b_{l,k} \in \mathbb{R}^{1 \times d}$. Optionally define a worker coefficient $g_{l,k} \in \mathbb{R}$. This scalar is layer-level in our example. It is not a claim about the granularity used in any cited implementation.

The illustrative multi-branch update is

$$\mathbf{x}_{l+1} := H_l^{\text{res}} \mathbf{x}_l + \sum_{k=1}^4 g_{l,k} b_{l,k}^\top F_{l,k}(a_{l,k} \mathbf{x}_l, W_{l,k}). \quad (16.2)$$

Shape ledger for every k :

object	shape	role
$\mathbf{x}_l$	$B \times d \times L \times D$	persistent state
$a_{l,k}$	$1 \times d$	branch-specific read
$z_{l,k}$	$B \times L \times D$	branch input after contracting axis 1
$y_{l,k} = F_{l,k}(z_{l,k})$	$B \times L \times D$	branch output
$b_{l,k}^\top y_{l,k}$	$B \times d \times L \times D$	branch write
$H_l^{\text{res}} \mathbf{x}_l$	$B \times d \times L \times D$	carried state, applied batchwise

If all four $g_{l,k}$ are nonzero and all workers are evaluated, (16.2) is a dense multi-branch block. If a router makes some $g_{l,k} = 0$ **and the runtime does not dispatch those workers**, it becomes conditional branch computation. If the four $F_{l,k}$ are copies of the same architecture with separate parameters, they can be called experts. Here they are heterogeneous operators, so “branches” is the less misleading name.

Hence four workers are a graph, not a skip. Zeroing a coefficient after the call still spends the call.

A concrete routing plan

Use $d = 4$, $B = 1$, $L = 2$, and $D = 2$. Thus $\mathbf{x}_l \in \mathbb{R}^{1 \times 4 \times 2 \times 2}$. Stack the scoops into $A_l \in \mathbb{R}^{4 \times 4}$:

$$A_l = \begin{bmatrix} 0.7 & 0.1 & 0.1 & 0.1 \\ 0.1 & 0.7 & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.7 & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.7 \end{bmatrix}.$$

Rows correspond, in order, to attention, Wave-PDE, SSM, and MLP. Each worker receives a tensor in $\mathbb{R}^{1 \times 2 \times 2}$, but each receives a different mixture of the four streams.

Stack the pours into $C_l \in \mathbb{R}^{4 \times 4}$:

$$C_l = \begin{bmatrix} 0.1 & 0.6 & 0.2 & 0.1 \\ 0.2 & 0.1 & 0.6 & 0.1 \\ 0.1 & 0.2 & 0.1 & 0.6 \\ 0.6 & 0.1 & 0.2 & 0.1 \end{bmatrix}.$$

For example, the attention output is written most strongly to stream 2, while the MLP output is written most strongly to stream 1. This is not expert selection: all four workers may still run. It is also not semantic specialization. “Stream 2 receives much attention output” does not prove what stream 2 represents.

Choose the leftover map

$$H_l^{\text{res}} = 0.85I_4 + 0.15 \frac{\mathbf{1}\mathbf{1}^\top}{4} \in \mathbb{R}^{4 \times 4},$$

where I_4 is the 4×4 identity and $\mathbf{1} \in \mathbb{R}^4$ is all ones. This map keeps most of each stream and diffusively shares the remainder. It is doubly stochastic.

At layer l , $(A_l)_{kj}$ weights an edge from stream j to worker k ; $(C_l)_{ki}$ weights an edge from worker k to stream i ; and $(H_l^{\text{res}})_{ij}$ weights an edge from old stream j to new stream i . Changing these coefficients across layers creates a learned, weighted dataflow graph through depth.

That picture is an interpretation, not a theorem about discrete graph discovery. The workers are nonlinear, routes can be dense, and dynamic coefficients may depend on activations. A pretty graph drawing does not make the computation discrete.

The loop that still pays for everyone

The following is pedagogical pseudocode. Production code would batch contractions, fuse cheap operations, and handle branch dispatch explicitly.

```
input:
    x          [B, d, L, D]
    H_res      [d, d] or [B, d, d]
    A_read     [K, d]
    C_write    [K, d]
    gate       [K]
output:
    x_next     [B, d, L, D]

x_next = contract_axis1(H_res, x)          # [B, d, L, D]
for k in 0 .. K-1:
    z = contract_axis1(A_read[k], x)       # [B, L, D]
    y = branch[k](z)                      # [B, L, D]
    x_next += gate[k] * outer_stream(C_write[k], y)
return x_next
```

This loop evaluates every worker. Replacing `gate[k]` by zero after `branch[k](z)` saves no branch work. Sparse conditional computation requires deciding the active set first and dispatching only that set. For sequence-coupled operators such as attention, Wave-PDE, and SSMs, token-wise dispatch may also change or complicate the operator’s semantics. Routing granularity must therefore be designed, stated, and measured — not guessed from a gate tensor.

Hence a zero after the call is an output mask, not a computational skip. Drop the dispatch step and the FLOP budget does not notice your gate.

What a leftover mix is allowed to do

mHC constrains H_l^{res} toward the Birkhoff polytope of doubly stochastic matrices using finite Sinkhorn normalization [R002]. go-mHC replaces that leftover constraint map with a generalized orthostochastic construction derived from an orthogonal block matrix [R005]. In the go-mHC primary formulation, H^{pre} and H^{post} remain separately generated read/write gates, while the new construction modifies H^{res} [R005].

If H_l^{res} is doubly stochastic, it is nonnegative and its rows and columns sum to one. It therefore performs convex

state mixing and preserves the all-ones stream mode. This constrains leftover transport. It says nothing about whether attention or SSM is selected, and nothing about the semantic content of a stream.

[Experiment] The HC paper reports language and vision experiments, including a sparse OLMoE setting [R001]. The mHC paper reports large-scale language-model training and systems work [R002]. The go-mHC paper reports synthetic stream-mixing experiments and a 30M-parameter GPT-style validation [R005]. These are recent-paper results, not settled facts across architectures.

Training might make persistent streams carry systematically different kinds of information. That possible behavior is **stream specialization**. It is an empirical claim. It requires measurements across inputs, layers, checkpoints, and seeds. It does not follow from $d > 1$, unequal gates, or a readable visualization.

Combining exact or approximately constrained leftover routing with heterogeneous workers may yield useful learned dataflow. Equation (16.2) is one possible formulation. It has not been established by the cited results as an architecture with lower loss, lower latency, or higher accuracy in general.

What disappears if we omit a verb

Omit the **read**, and every worker sees a fixed stream or a fixed aggregate. The architecture loses the ability to choose which leftover state supplies each computation.

Omit the **write**, and worker output always lands in a fixed destination. Read choice can still vary, but storage choice cannot. This is like having flexible database queries and one hard-coded table for every update.

Omit the **leftover shuffle**, and old streams cannot mix directly between layers. Branch writes may still couple them, but there is no independent carry path. In mHC and go-mHC, this also removes the object to which the doubly stochastic constraint is applied.

Omit **branch routing** from the heterogeneous extension, and all workers follow a fixed schedule. That may be perfectly reasonable, but it is not conditional computation.

Omit **multiple branches**, and state routing remains useful. The original equation has one F_l , and the stream topology can still be dynamic.

Omit the evidence standard, and speculative stream roles quickly become “discoveries.” Architecture diagrams are unusually effective at converting hopes into past tense.

Ways to get this wrong

1. **“Each stream is an expert.”** A stream stores activations; an expert owns parameters. One worker can read all streams once.
2. **“Four streams mean four calls to F .”** Equation (16.1) has one call. Equation (16.2) has four because we deliberately added four workers.
3. **“A soft gate gives sparse-compute savings.”** Not unless dispatch avoids the gated computation.
4. **“ H^{res} chooses attention versus MLP.”** It routes carried state. A separate branch or expert gate would choose a computation.
5. **“Doubly stochastic means one-hot routing.”** It permits dense convex mixing; permutation matrices are only special cases.
6. **“Different read/write profiles prove specialization.”** They show different routing coefficients, not the information content or causal function of streams.
7. **“Token-wise MoE routing transfers directly to every heterogeneous operator.”** Attention, SSM, and Wave-PDE workers couple positions differently. A legal tensor shape is not yet a sound dispatch algorithm.
8. **“Latent computation graph means discrete program.”** Here it is a weighted, generally dense, differentiable graph interpretation.

The leftover mix is also a dynamical system. Repeat H_l^{res} and streams can share a stationary mode, smear into each other, or cycle. That is tomorrow’s algebra — what persist can do over depth, not which specialist gets the next token.

Try it

1. Separate the three state routes

Using the three-number workshop, recompute z_l if only the scoop changes to $H_l^{\text{pre}} = [0, 1, 0]$. Which other intermediate quantities change immediately?

Answer

The read becomes $z_l = 5$, so $y_l = F_l(5) = 10$ and the write becomes $[1, 7, 2]^\top$. The carried state $H_l^{\text{res}} \mathbf{x}_l = [2.6, 3.2, 0.2]^\top$ does not change because neither H^{res} nor $\mathbf{x}_l$ changed.

2. Classify the routers

Classify each operation as state routing, expert routing, both, or neither: (a) $H^{\text{res}} \mathbf{x}$; (b) top-1 selection among eight separately parameterized MLPs; (c) summing attention and SSM outputs with fixed weight $1/2$; (d) reading stream 3 into one fixed MLP.

Answer

(a) state routing; (b) expert routing; (c) neither routing operation is learned — there are merely two fixed branches; (d) state routing by a fixed one-hot read, but no expert selection because only one fixed MLP exists.

3. Audit the shapes

Let $d = 4$, $B = 8$, $L = 512$, $D = 1024$, and $K = 4$. Give the shapes of A_l , all branch inputs stacked over k , C_l , and the summed write term in (16.2).

Answer

$A_l \in \mathbb{R}^{4 \times 4}$. Stacking the four branch inputs gives $Z_l \in \mathbb{R}^{4 \times 8 \times 512 \times 1024}$, where the first axis indexes branches. $C_l \in \mathbb{R}^{4 \times 4}$. Each written output and their sum have canonical state shape $8 \times 4 \times 512 \times 1024$, matching $B \times d \times L \times D$.

4. Find the false compute saving

A program evaluates all four workers, multiplies three outputs by zero, and retains the attention output. Has it implemented top-1 conditional computation? Repair the algorithm in words.

Answer

It has top-1 output selection but not top-1 compute: all branches ran. Compute the gate first, form the active branch set, dispatch only active examples/sequences to compatible branches, run those branches, then scatter their outputs into the write accumulation. The dispatch granularity must respect each branch's sequence semantics.

5. Design a specialization test

Suppose stream 1 has a large attention read coefficient and stream 3 has a large SSM write coefficient. Propose evidence that would be stronger than naming stream 1 “global context” and stream 3 “memory.”

Answer

Measure stream contents and causal function across many inputs, layers, checkpoints, and random seeds. Examples include trained probes with held-out controls, interventions that ablate or swap one stream while measuring task-specific effects, routing consistency statistics, and comparisons against permuted-stream labels. A specialization claim should survive the fact that stream indices can be relabeled together with router coordinates.

Check yourself

1. In equation (16.1), what three jobs do H^{pre} , H^{post} , and H^{res} do?

Read which streams feed the worker; write the worker's answer back into streams; persist and remix leftover state.

2. Show that four leftover copies do not imply four experts.

A stream stores activations. An expert owns parameters. Equation (16.1) calls one F_l once. Four trays can feed one worker.

3. What fails if a program multiplies three of four branch outputs by zero after all four ran?

It does not save the skipped workers' FLOPs. A near-zero gate still spends the full branch budget unless dispatch skips the worker.

4. If H^{res} is doubly stochastic, what does that constrain, and what does it not decide?

It constrains leftover transport: nonnegative convex mixing that preserves the all-ones stream mode. It does not choose a worker or assign a semantic role to a stream.

5. What fails if a heat map is treated as a specialization theorem?

You still lack measurements of content and causal function across inputs, layers, checkpoints, and seeds. Specialization is a hypothesis, not a definition.

You should now be able to inspect a proposed routed architecture and ask four separate questions: which persistent states are read, written, and carried; which parameterized branch is selected, if any; how many branches actually execute; and what direct evidence supports any claimed semantic specialization.

For the heterogeneous example, A_l and C_l route state around attention, Wave-PDE, SSM, and MLP branches. The optional g_l routes among branches only if it affects dispatch. H_l^{res} carries and mixes old state. None of these definitions assigns human-readable roles to streams.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024/2025. <https://arxiv.org/abs/2409.19606>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309v1>
- [R014] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” arXiv:1701.06538, 2017. <https://arxiv.org/abs/1701.06538>
- [R015] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *JMLR* 23(120), 2022; arXiv:2101.03961. <https://arxiv.org/abs/2101.03961>
- [R010] A. Vaswani et al., “Attention Is All You Need,” arXiv:1706.03762, 2017. <https://arxiv.org/abs/1706.03762>
- [R016] H. Vejdla, “Wave-PDE Nets: Trainable Wave-Equation Layers as an Alternative to Attention,” arXiv:2510.04304, 2025. <https://arxiv.org/abs/2510.04304>
- [R017] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” arXiv:2312.00752, 2023. <https://arxiv.org/abs/2312.00752>

Chapter 17 — Leftover copies that refuse to settle

Three leftover copies of a sentence sit in a short column. An honest mixer can send them around a cycle, or blend them toward one gray. The same two sums that stop the total from exploding do not decide which of those walks you get.

This chapter watches one mixer, then another. The names for the motions come after the walks.

One leftover column

The residual mixer acts only on the leftover-copy axis. Batch, token, and feature sit still. Write d for the number of leftover copies — the **streams**. Freeze one coordinate (b, t, c) and stack the d leftover numbers into a short column $x_l \in \mathbb{R}^d$. Then the next layer is

$$x_{l+1} = H_l x_l.$$

The rest of the tensor $X_l \in \mathbb{R}^{B \times d \times T \times C}$ has not vanished. We have merely selected one vertical slice. B is batch size, T is sequence length, and C is feature width. Every other slice uses the same table.

Hence a spectral statement about H_l is a statement about leftover copies, not about tokens or features. Drop that axis name and you can still multiply a $d \times d$ table by a length- d vector. You will not know what walked.

Two honest tables, two walks

Take $d = 3$ and this permutation:

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \in \mathbb{R}^{3 \times 3}.$$

It sends $(a, b, c)^\top$ to $(c, a, b)^\top$. Start with all the mass in the first stream,

$$x_0 = e_1 = (1, 0, 0)^\top \in \mathbb{R}^3,$$

and apply P three times:

$$\begin{aligned} x_0 &= (1, 0, 0)^\top, \\ x_1 &= P x_0 = (0, 1, 0)^\top, \\ x_2 &= P^2 x_0 = (0, 0, 1)^\top, \\ x_3 &= P^3 x_0 = (1, 0, 0)^\top. \end{aligned}$$

The first stream is back. Another three applications would do the same walk. In matrix language, $P^3 = I_3$. Nothing mixed. The leftover copies just changed seats.

P is doubly stochastic. Every entry is nonnegative. Every row sums to one. Every column sums to one. Honesty did not force the copies to blend.

Now use a different honest table:

$$M = \begin{pmatrix} 1/2 & 1/4 & 1/4 \\ 1/4 & 1/2 & 1/4 \\ 1/4 & 1/4 & 1/2 \end{pmatrix} = \frac{1}{4}I_3 + \frac{1}{4}\mathbf{1}\mathbf{1}^\top \in \mathbb{R}^{3 \times 3}.$$

Each new stream keeps half of itself and takes a quarter from each neighbor. Start again with $x_0 = e_1$:

$$\begin{aligned} x_1 &= (0.5, 0.25, 0.25)^\top, \\ x_2 &= (0.375, 0.3125, 0.3125)^\top, \\ x_3 &= (0.34375, 0.328125, 0.328125)^\top, \\ x_4 &= (0.3359375, 0.33203125, 0.33203125)^\top. \end{aligned}$$

The three numbers add to one at every step. They are walking toward

$$u = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)^\top.$$

After four applications the first stream is already 0.3359..., a hair above one third. Keep going and the leftover copies become interchangeable.

Both P and M are doubly stochastic. One cycles forever. One blends. “The eigenvalues are bounded” is true of both and explains neither.

Why one cycles and one fades

A special direction of a table H is a nonzero vector v that comes out pointing the same way, only scaled:

$$Hv = \lambda v.$$

The scale factor λ is an **eigenvalue**. After k applications that direction is multiplied by λ^k .

Every row-stochastic table has $\lambda = 1$ with $v = \mathbf{1}$, the all-ones column. For a doubly stochastic table the uniform leftover $u = \mathbf{1}/d$ is therefore a fixed leftover, and the stream sum $\mathbf{1}^\top x_l$ never changes. That surviving direction is a **stationary mode**. Everything interesting happens to the leftover that disagrees with it.

For P , the characteristic equation is

$$\lambda^3 - 1 = 0.$$

Write

$$\omega = e^{2\pi i/3} = -\frac{1}{2} + i\frac{\sqrt{3}}{2}.$$

The three eigenvalues are the three cube roots of one:

$$\text{spec}(P) = \{1, \omega, \bar{\omega}\}, \quad \omega^3 = 1.$$

The eigenvalue 1 belongs to $\mathbf{1}$. The conjugate pair is rotation by $\pm 120^\circ$ on the plane of leftover copies that already add to zero,

$$\mathcal{D} = \{v \in \mathbb{R}^3 : \mathbf{1}^\top v = 0\}.$$

Call $\mathcal{D}$ the **disagreement plane**. A three-cycle does not mix that disagreement away. It walks it around the table.

A real table can have a nonreal eigenvalue. The partner is always the conjugate, and the two of them together describe a real two-dimensional wobble. Write $\lambda = re^{i\theta}$ with magnitude $r \geq 0$ and angle θ . After k applications the scale is r^k and the phase is $k\theta$. The leftover copies never become imaginary. The complex number is bookkeeping for a clock hand.

For M the eigenvalues are 1, $1/4$, and $1/4$. Split x_0 into the stationary leftover and the rest:

$$x_0 - u = \left(\frac{2}{3}, -\frac{1}{3}, -\frac{1}{3}\right)^\top.$$

Every vector in $\mathcal{D}$ is multiplied by $1/4$. So the whole walk has a closed form:

$$x_k = u + \left(\frac{1}{4}\right)^k (x_0 - u).$$

Hence x_4 already sits next to u : four factors of one fourth have almost erased the disagreement.

If a *fixed* stochastic table is irreducible and aperiodic, the leftover copies converge to a unique stationary distribution. For a doubly stochastic table that distribution is uniform. Our M is strictly positive, so it meets the conditions. P is periodic, so it does not. Both have spectral radius one. Radius one is a ceiling, not a personality.

A single table's eigenvalues describe the powers H^k only when that table stays put, and only after you include any Jordan blocks. A deep model generally uses a product $H_{L-1} \cdots H_1 H_0$. If the factors vary and do not commute, their individual spectra are not a complete analysis of that product.

Half stay, half rotate

A useful middle case is the lazy cycle

$$L = \frac{1}{2}(I_3 + P).$$

Half the leftover stays. Half is sent around the three-cycle. The eigenvalues are

$$1, \quad \frac{1}{2}(1 + \omega) = \frac{1}{4} + i\frac{\sqrt{3}}{4}, \quad \frac{1}{2}(1 + \bar{\omega}) = \frac{1}{4} - i\frac{\sqrt{3}}{4}.$$

The complex pair has magnitude $1/2$ and angles $\pm\pi/3$. Starting from e_1 ,

$$(1, 0, 0)^\top \rightarrow (0.5, 0.5, 0)^\top \rightarrow (0.25, 0.5, 0.25)^\top \rightarrow (0.25, 0.375, 0.375)^\top.$$

The disagreement rotates by 60° per step, in an appropriate real basis, and halves in size. That is damped circulation: the leftover chases itself while walking toward gray.

When the magnitude $r < 1$, rotation is damped. When $r = 1$, it can cycle without decay. When $r = 0$, that mode is erased in one step.

Drop the lazy half and you recover P : same angle family, magnitude one, no fade. Drop the cycle and you recover the identity: every leftover is fixed.

The average rides through

For any leftover column $x \in \mathbb{R}^d$, write its mean

$$\mu = \frac{1}{d} \mathbf{1}^\top x$$

and its centered part $z = x - \mu \mathbf{1}$. Then $\mathbf{1}^\top z = 0$. If H is doubly stochastic,

$$H(\mu \mathbf{1}) = \mu \mathbf{1}, \quad \mathbf{1}^\top H z = \mathbf{1}^\top z = 0.$$

The stationary line $\text{span}\{\mathbf{1}\}$ and the disagreement subspace $\mathcal{D}$ are both invariant. Spectral dynamics happen inside $\mathcal{D}$. The mean rides through unchanged.

That gives a practical diagnostic for a *fixed* H : ignore the guaranteed eigenvalue at one, and read the rest.

- Values near zero: leftover distinctions vanish quickly.
- Values near one: leftover distinctions linger.
- Values near a root of unity: nearly periodic motion.
- A nonreal pair: circulation in some two-dimensional leftover.

The diagnostic is incomplete for crooked tables. Eigenvectors can be badly conditioned. Jordan blocks can add polynomial factors. Layer-varying products can behave differently from any one layer. Spectra are a lens, not an MRI scanner.

They also isolate only the leftover route. In

$$X_{l+1} = H_l X_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} X_l, W_l),$$

the first term carries existing leftover copies. The second term injects a newly computed note. Eigenvalues of H_l do not, by themselves, predict the full nonlinear stack. The branch F , the read and write routers, data dependence, and the product of different H_l tables remain in play.

For a pedagogical, unbatched router the leftover multiply looks like this:

```
input:  H [d, d] or [B, d, d], X [B, d, T, C]
output: Y [B, d, T, C]
Y[b,i,t,c] = sum_j H[b or shared,i,j] * X[b,j,t,c]
```

This does not assert that a primary implementation shares one H across all batches or tokens. If a router is data-dependent or batched, analyze each realized $d \times d$ table and, for depth dynamics, the realized ordered product.

Corners that never mix

A permutation cycle of length q satisfies $P_q^q = I_q$, so every eigenvalue obeys $\lambda^q = 1$. Its spectrum is exactly the q th roots of unity:

$$\{e^{2\pi i j/q} : j = 0, 1, \dots, q-1\}.$$

A general permutation splits into disjoint cycles, and its spectrum is the union of those root sets. These tables sit at vertices of the Birkhoff polytope. Exact periodic leftover motion lives at some of the polytope's corners. Convex mixtures of permutations often pull the nonstationary eigenvalues inward and damp those periods. That is why $L = \frac{1}{2}(I_3 + P)$ blends while P does not.

Hence double stochasticity permits both blending and cycling. The corners are where the cycles sit in plain sight.

A map of allowed numbers

Which complex numbers can even appear as leftover stretch factors? For row-stochastic tables the answer is a region in the plane, not a list.

The **Karpelevič region** $\Theta_d \subset \mathbb{C}$ is the set of all numbers that occur as an eigenvalue of some $d \times d$ row-stochastic matrix. Karpelevič characterized its boundary by algebraic arcs joining particular roots of unity of orders at most d [R035]; [R036]. It lives in the closed unit disk and is symmetric across the real axis.

As d grows, more roots of unity become available and the feasible region grows more boundary arcs. Read the map as follows:

- 1 marks the stationary leftover;
- points near 0 mark leftover that is forgotten in a hurry;
- points near the unit circle mark leftover that persists;
- angle marks how fast a leftover oscillates or circulates.

Suppose a number lies outside Θ_d . Then no $d \times d$ row-stochastic table can have it as an eigenvalue, and therefore no doubly stochastic leftover mixer can either. The converse is false: a point inside Θ_d need not be an eigenvalue of any member of $\mathbf{B}_d$.

Θ_d describes all row-stochastic tables. Doubly stochastic tables form a subset, so their possible eigenvalues form a subset of Θ_d . A Karpelevič plot is a valid outer landscape for mHC routing, not automatically an exact picture of the Birkhoff polytope for every d .

Spectral reach is also only a surrogate for matrix reach. Distinct tables can share the same spectrum. Filling many eigenvalue bins does not prove coverage of all $(d-1)^2$ degrees of freedom in the Birkhoff polytope.

What the tiled mixer can hear

Can a cheap tiled mixer produce the three-cycle P ? The question is spectral, not aesthetic.

Original mHC maps residual-routing parameters toward the Birkhoff polytope using finite Sinkhorn normalization [R002]. Exact variants including KromHC and go-mHC instead construct doubly stochastic routers by design [R004]; [R005]. The stationary uniform leftover and the preservation of stream sums follow from double stochasticity, independent of how the router was written down.

KromHC builds a router from factor tables

$$H = A_1 \otimes A_2 \otimes \cdots \otimes A_K, \quad A_k \in \mathbf{B}_{i_k}, \quad \prod_{k=1}^K i_k = d,$$

where $\mathbf{B}_{i_k}$ is the set of $i_k \times i_k$ doubly stochastic matrices [R004]. The Kronecker identity for spectra is

$$\text{spec}(A \otimes B) = \{\lambda_i(A)\lambda_j(B)\}_{i,j}.$$

Every 2×2 doubly stochastic table has the form

$$A(a) = \begin{pmatrix} a & 1-a \\ 1-a & a \end{pmatrix}, \quad 0 \leq a \leq 1,$$

with eigenvalues 1 and $2a-1$, both real. Therefore a router made only from aligned 2×2 factors has only real eigenvalues: products of real numbers remain real. It cannot equal the three-cycle P , whose spectrum contains ω and $\bar{\omega}$.

That proof applies to the all- 2×2 -factor configuration emphasized in the go-mHC spectral comparison and commonly used to tile power-of-two stream counts. It does **not** prove that every KromHC configuration has only real eigenvalues. If some factor size $i_k \geq 3$, that factor can itself have complex eigenvalues and cycles up to its own coordinate structure. The broader restriction is separability: eigenvalues and eigenvectors inherit products of the chosen factor manifolds, and an arbitrary nonseparable d -stream router is generally unavailable.

The KromHC paper evaluated exact Kronecker-structured routing in language-model pretraining and emphasized parameter efficiency and exact double stochasticity [R004]. The spectral-reach comparison was reported by the later go-mHC paper: its Figure 4 optimized random spectral targets for $d = 4$ and found the tested 2×2 -factor KromHC baseline confined to a real, low-dimensional slice [R005]. That is evidence about the tested parameterization and optimizer, not evidence that complex routing is necessary for every language task. KromHC’s own pretraining results show that a restricted family can still be useful.

Drop the factor-size hypothesis and the real-only claim dies. Keep it, and P is simply not in the family.

What the rotation mixer can hear

Can the rotation construction hear P ? At one small size, yes. That is not a coverage theorem.

go-mHC forms an orthogonal matrix $Q \in \mathbb{R}^{ds \times ds}$, partitions it into $d \times d$ blocks $Q_{ij} \in \mathbb{R}^{s \times s}$, and defines

$$H_{ij} = \frac{1}{s} \|Q_{ij}\|_F^2.$$

This guarantees $H \in \mathcal{B}_d$ [R005]; [R029]. At fixed finite s , however, the image $\Phi_{d,s}(O(ds))$ is a subset of $\mathcal{B}_d$. Exact double stochasticity is not the same claim as exact finite- s coverage.

For $d = 3$, $s = 1$, the three-cycle P is itself orthogonal, has determinant $+1$, and has no eigenvalue -1 . It is therefore in the ordinary Cayley chart, and entrywise squaring leaves it unchanged. The go-mHC construction can represent this particular complex-spectrum router. That is a representability statement, not a claim that a trained network will select it.

The go-mHC paper reports toy spectral-target experiments at $d = 3$ for $s = 1, 2$, and a comparison at $d = 4$. In those experiments, go-mHC reached denser portions of the observed spectral region than the tested KromHC baseline, with $s = 2$ denser than $s = 1$ [R005]. The same paper used synthetic stream-mixing tasks and a roughly 30M-parameter GPT-style model, and explicitly lists the small model scale and synthetic emphasis as limitations [R005]. It did not demonstrate that every finite- s Birkhoff matrix is reachable, that learned large-model routers realize long cycles, or that broader spectral reach universally improves downstream accuracy.

go-mHC removes the all- 2×2 Kronecker product restriction and can expose complex, nonseparable leftover motion. Its reported reach is promising evidence for a parameterization. The architectural value of each spectral region remains a research question.

Drop the distinction between “this P is representable” and “every legal mixer is representable,” and a one-point existence proof becomes a surjectivity slogan it is not.

If we skip the pictures

If we skip the leftover motions and keep only the slogan “doubly stochastic is stable,” several different failures collapse into one sentence.

First, we confuse boundedness with mixing. The cycle P never explodes, but it never converges either. Second, we miss directed circulation: a real spectral interval cannot encode the same three-stream phase motion as $\omega, \bar{\omega}$. Third, we may compare parameterizations only by parameter count while overlooking which leftover walks they exclude.

The opposite omission is equally dangerous. If we use spectra without dynamics, we may infer behavior of $H_{L-1} \cdots H_0$ from separate layer spectra, ignore injected branch outputs, or mistake eigenvalue coverage for full matrix coverage. The correct tool is not “eigenvalues everywhere.” It is “eigenvalues where their assumptions fit.”

Ways to get this wrong

1. **Every doubly stochastic matrix mixes to uniform.** False. A permutation cycle is doubly stochastic and periodic. Fixed-matrix convergence needs conditions such as irreducibility and aperiodicity.
2. **Spectral radius one means all information is preserved.** False. The stationary mode has magnitude one while disagreement modes may contract to zero.
3. **Complex eigenvalues require complex activations.** False. They encode paired real modes.
4. **The Karpelevič region equals the spectrum of B_d .** Not in general; it is defined for row-stochastic matrices and provides an outer region for doubly stochastic ones.
5. **KromHC can never produce a complex eigenvalue.** Too broad. The real-only result follows for 2×2 doubly stochastic factors. Larger factors change that statement, though Kronecker separability remains.
6. **go-mHC with $s = 2$ covers the whole Birkhoff polytope.** Unsupported. The paper reports denser sampled spectral reach at small d , not finite- s surjectivity.
7. **A spectral histogram proves better language modeling.** No. It measures a surrogate on a specified experiment; task value requires task evidence.
8. **Average the eigenvalues across layers.** Eigenvalues do not compose that way. Ordered matrix products are less accommodating.

Try it

1. Send a leftover around the table

For the three-cycle P , compute $P^2(2, -1, 4)^\top$ and $P^3(2, -1, 4)^\top$.

Answer

Since $P(a, b, c)^\top = (c, a, b)^\top$, $P^2(2, -1, 4)^\top = (-1, 4, 2)^\top$, and $P^3(2, -1, 4)^\top = (2, -1, 4)^\top$.

2. Predict the blend without multiplying

For the matrix M above and $x_0 = (0, 1, 0)^\top$, find x_3 from the closed form.

Answer

Here $u = (1/3, 1/3, 1/3)^\top$ and $x_0 - u = (-1/3, 2/3, -1/3)^\top$. Thus

$$x_3 = u + 4^{-3}(x_0 - u) = (0.328125, 0.34375, 0.328125)^\top.$$

3. Read a lazy cycle

Let $L_\alpha = (1 - \alpha)I_3 + \alpha P$, with $0 \leq \alpha \leq 1$. Find its eigenvalues. What happens at $\alpha = 0$, $1/2$, and 1 ?

Answer

Because I_3 and P share eigenvectors,

$$\text{spec}(L_\alpha) = \{1, 1 - \alpha + \alpha\omega, 1 - \alpha + \alpha\bar{\omega}\}.$$

At $\alpha = 0$, every leftover is fixed. At $\alpha = 1/2$, disagreement rotates with magnitude $1/2$, so it decays. At $\alpha = 1$, the exact period-three cycle returns.

4. Listen to a tiled mixer

Let

$$A = \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix}, \quad B = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}.$$

Find the eigenvalues of $A \otimes B$. Can this router have a three-cycle eigenvalue?

Answer

A has eigenvalues 1, 0.6, and B has 1, 0.4. Pairwise products give

$$\text{spec}(A \otimes B) = \{1, 0.6, 0.4, 0.24\}.$$

All are real, so neither $e^{2\pi i/3}$ nor its conjugate is present. This particular all- 2×2 Kronecker router cannot implement a three-cycle.

5. Audit a reach claim

A paper samples 10^5 target eigenvalues for $d = 4$. Its router reaches more histogram bins than a baseline. Which of the following follow: (a) greater sampled spectral reach; (b) full coverage of B_d ; (c) better performance on every task; (d) evidence that the optimizer can reach more tested spectral targets?

Answer

Only (a) and (d) follow, under the reported protocol. Spectrum is not an injective description of a matrix, finite sampling cannot prove full polytope coverage, and a surrogate experiment cannot establish universal task performance.

Check yourself

- Show that both P and M preserve the stream sum, then say what each does to e_1 .**
Both are doubly stochastic, so $\mathbf{1}^\top Hx = \mathbf{1}^\top x$. $P^k e_1$ cycles forever through the three standard basis vectors. $M^k e_1 \rightarrow \frac{1}{3}\mathbf{1}$.
- Show that an honest mixer cannot move the mean.**
 $H(\mu\mathbf{1}) = \mu\mathbf{1}$. Spectral dynamics live in the disagreement subspace $\mathcal{D}$.
- What fails if you treat a complex eigenvalue as requiring complex leftover copies?**
Nonreal eigenvalues of a real table come in conjugate pairs and describe a real two-dimensional rotation-and-scale. The copies stay real.
- What fails if you read a Karpelevič plot as a picture of B_d ?**
 Θ_d is defined for all row-stochastic tables. Doubly stochastic tables are a subset, so their possible eigenvalues are a subset of Θ_d .
- Show that an all- 2×2 KromHC router cannot have ω as an eigenvalue.**
Each factor has spectrum $\{1, 2a - 1\} \subset \mathbb{R}$. Products of reals stay real. ω is not real.
- What fails if you promote sampled spectral reach to finite- s coverage of B_d ?**
The go-mHC experiments showed denser sampled spectral reach than the tested all- 2×2 KromHC baseline at small d and s . They did not show finite- s coverage, large-model long cycles, or a universal accuracy gain.

Look at a residual router and separate four questions: which leftover is stationary, which disagreements shrink, which leftovers alternate or rotate, and whether those conclusions concern one fixed table, an ordered depth product, or a reported experiment.

Both P and M are exactly doubly stochastic. Their different leftover walks are their different spectra.

For KromHC, say “real-only” only when the factor configuration justifies it, especially the all- 2×2 case. For go-mHC, say “broader observed spectral reach at tested small d and s ” rather than “full Birkhoff coverage at finite s .” Precision is shorter than the correction notice.

Sources

- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R004] W. Zhou, Y. Gu, G. Iacovides, and D. Mandic, “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices,” arXiv:2601.21579v2, 2026. <https://arxiv.org/abs/2601.21579>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>

-
- [R035] F. I. Karpelevich, “On the Characteristic Roots of Matrices with Nonnegative Elements,” *Izvestiya Akademii Nauk SSSR, Seriya Matematicheskaya* 15(4), 361–383, 1951. <https://m.mathnet.ru/eng/im3317>
 - [R036] C. R. Johnson and P. Paparella, “A Matricial View of the Karpelevič Theorem,” *Linear Algebra and its Applications* 520, 1–15, 2017, arXiv:1611.06970. <https://arxiv.org/abs/1611.06970>
 - [R029] I. Nechita, Z. Ouyang, and A. Szczepanek, “Generalized Unistochastic Matrices,” arXiv:2310.03436, 2023. <https://arxiv.org/abs/2310.03436>

Chapter 18 — A curved space of legal mixers

You have leftover copies of a sentence table — streams — and you want to remix them. The remixing table has to be honest both ways: no negative weights, each new stream a complete recipe, each old stream spent exactly once.

Those legal tables do not fill a flat box of numbers. Around any one of them the set still looks like a small flat neighborhood. Far away it may fold, stop at a face, or need another map. We will walk that surface: first the allowed velocities, then the particular path go-mHC uses.

Mixers that refuse to be a flat sheet

A $d \times d$ table of ordinary numbers has d^2 slots. Honesty both ways carves those slots down. The remaining set is locally Euclidean and globally not a single flat sheet.

A space that looks like ordinary Euclidean space around each point, even if the whole object curves or needs several coordinate maps, is a **manifold**. The name is worth keeping because the dimension that matters is the number of independent local directions, not the number of ambient coordinates used to store a point.

A car on a road can point along the road at an instant. A velocity straight through the asphalt is not allowed. That allowed instantaneous velocity is a **tangent vector**. All such velocities at one point form a **tangent space**. The collection is linear even when the road curves.

An orthogonal matrix $Q \in \mathbb{R}^{N \times N}$ must obey $Q^\top Q = I_N$. Its N^2 entries therefore cannot move independently. Tangent directions tell an optimizer how to move without immediately leaving the constraint.

A constraint can be satisfied in two rather different ways. We can optimize an arbitrary matrix and repeatedly repair it, or we can choose coordinates whose outputs obey the constraint automatically. mHC uses the first broad idea through finite Sinkhorn normalization. go-mHC uses the second: skew coordinates produce an orthogonal matrix, whose block energies produce a doubly stochastic matrix [R005]; [R029].

That switch changes more than numerical error. It changes which routers are reachable, how many parameter directions describe the same router, and how gradients are warped before they reach the router logits. “Both methods output nearly the same kind of matrix” is therefore not an optimization analysis.

Exact constraint satisfaction asks whether every produced H lies in $\mathcal{B}_d$. Coverage asks whether every member of $\mathcal{B}_d$ can be produced. go-mHC proves the first for every s ; it does not obtain the second at any fixed finite s when $d \geq 3$.

All claims below are labeled. Results about go-mHC are especially recent and should not be treated as settled architectural law.

A tiny turn, then a finished rotation

An orthogonal matrix preserves lengths. In two dimensions, rotations and mirror reflections both do so. Rotations have determinant $+1$; reflections have determinant -1 . These two types form disconnected pieces: a continuously

moving rigid frame cannot turn into its mirror image without becoming singular somewhere.

A tiny rotation is easier to describe than a finished one. In two dimensions, a single scalar t produces the skew-symmetric matrix

$$A(t) = \begin{bmatrix} 0 & -t \\ t & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Its transpose is its negative. The off-diagonal signs encode the opposition required for a turn: one coordinate leans into the other while the other leans back. Skew matrices are precisely the instantaneous directions of orthogonal motion at the identity.

The **Cayley transform** converts such a tiny-turn description into a finite orthogonal transformation using additions, multiplications, and a linear solve. It is a rational map from a flat coordinate sheet onto most, but not all, of the rotation surface. It is a map, not a learned module; the learned router emits its input coordinates.

Drop the “most but not all” and you will later claim that every rotation is a Cayley value. A half-turn is the counterexample.

Two streams, one knob

Take $d = 2$ streams and $s = 1$, so $N = ds = 2$. Set $t = 1$ in the skew matrix above. Define $I_2 \in \mathbb{R}^{2 \times 2}$ as the identity and use the go-mHC Cayley convention

$$Q = (I_2 - A)(I_2 + A)^{-1}.$$

A direct calculation gives

$$Q = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}, \quad Q^\top Q = I_2, \quad \det Q = 1.$$

For $s = 1$, each block $Q_{ij} \in \mathbb{R}^{1 \times 1}$ is one scalar. The generalized orthostochastic map squares it:

$$H = \Phi_{2,1}(Q) = \begin{bmatrix} Q_{11}^2 & Q_{12}^2 \\ Q_{21}^2 & Q_{22}^2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Thus a quarter-turn in the hidden orthogonal space becomes a stream swap. More generally,

$$Q(t) = \frac{1}{1+t^2} \begin{bmatrix} 1-t^2 & 2t \\ -2t & 1-t^2 \end{bmatrix},$$

and $H(t)$ has diagonal entry $((1-t^2)/(1+t^2))^2$ and off-diagonal entry $(2t/(1+t^2))^2$. Each row and column sums to one.

This example also warns us about optimization. At $t = 0$, $Q = I_2$ and $H = I_2$. For small t , the off-diagonal routing weight is approximately $4t^2$, not $4t$. The first derivative at zero vanishes. Exact geometry has not abolished flat spots; it has merely chosen rather elegant ones.

Neighborhoods that look like ordinary space

The local-flatness picture is enough to walk with. The next two sentences pin the words so later counts have something to attach to.

A smooth k -dimensional manifold is a space that can be covered by neighborhoods with smooth coordinates in $\mathbb{R}^k$, with smooth changes of coordinates where neighborhoods overlap. The number k is its intrinsic dimension.

Let M be a smooth manifold and $p \in M$. A tangent vector at p can be represented as the velocity $\gamma'(0)$ of a smooth curve γ lying in M with $\gamma(0) = p$. All such velocities form the tangent space $T_p M$.

The Birkhoff polytope is not a smooth manifold everywhere: vertices and faces have corners. Its relative interior is a smooth open subset of its affine hull, while its boundary is better treated as a manifold with corners or a stratified set. Calling the entire polytope “the Birkhoff manifold” hides this distinction.

The allowed velocities of a rotation

The orthogonal group is

$$O(N) = \{Q \in \mathbb{R}^{N \times N} : Q^\top Q = I_N\}.$$

The name “group” means that multiplying orthogonal matrices, taking an inverse, and using the identity all stay in the set. Its two components are separated by determinant. The special orthogonal group is

$$SO(N) = \{Q \in O(N) : \det Q = 1\}.$$

It contains rotations and is path-connected for $N \geq 2$. Both $O(N)$ and $SO(N)$ have intrinsic dimension

$$\dim O(N) = \dim SO(N) = \frac{N(N-1)}{2}.$$

Derivation. Let $Q(\tau) \in O(N)$ be a smooth curve with $Q(0) = I_N$, and write $V = Q'(0) \in \mathbb{R}^{N \times N}$. Differentiating $Q(\tau)^\top Q(\tau) = I_N$ at $\tau = 0$ gives

$$V^\top + V = 0.$$

Therefore the tangent space at the identity is the vector space of skew-symmetric matrices.

The Lie algebra of $SO(N)$ is

$$\mathfrak{so}(N) = \{A \in \mathbb{R}^{N \times N} : A^\top = -A\} = T_{I_N} SO(N).$$

A skew matrix has zero diagonal. Its strict upper triangle determines its strict lower triangle, so it has $N(N-1)/2$ independent coordinates. At a general $Q \in O(N)$,

$$T_Q O(N) = \{Q\Omega : \Omega \in \mathfrak{so}(N)\}.$$

This is the formal version of attaching a flat sheet of allowed velocities to every point on the curved set.

If you drop the first-order condition $V^\top + V = 0$, the curve immediately leaves $O(N)$. Skew symmetry is not decorative antisymmetry. It is the infinitesimal orthogonality constraint.

A map that misses a half-turn

For $A \in \mathfrak{so}(N)$, define

$$\text{cay}(A) = (I_N - A)(I_N + A)^{-1}.$$

In numerical code one solves a linear system rather than forming the inverse. For real skew A , $I_N + A$ is nonsingular: skew matrices have purely imaginary eigenvalues, so -1 cannot be one of them.

Theorem. $Q = \text{cay}(A)$ is in $SO(N)$.

Derivation. Because $A^\top = -A$, transposition swaps $I_N - A$ and $I_N + A$. These two polynomials in A commute, hence

$$Q^\top Q = (I_N - A)^{-1}(I_N + A)(I_N - A)(I_N + A)^{-1} = I_N.$$

Continuity from $A = 0$, where $Q = I_N$, places the result in the determinant-+1 component.

The derivative at the origin is $D \text{cay}_0[\Delta A] = -2\Delta A$, so the map supplies valid local coordinates near I_N . Globally, it reaches exactly those members of $SO(N)$ without eigenvalue -1 . A rotation by 180° therefore sits beyond this chart, approached only as coordinates grow without bound.

“Cayley parameterizes all of $SO(N)$ ” is false. Its range is dense in $SO(N)$, which is enough for approximation statements but not the same as exact coverage.

Drawers of squared energy

Before writing the map, picture $Q \in \mathbb{R}^{N \times N}$ as a $d \times d$ cabinet. Every drawer $Q_{ij} \in \mathbb{R}^{s \times s}$ contains s^2 numbers. We discard their signs and orientations and retain only each drawer’s normalized squared energy.

For $N = ds$, partition $Q \in O(N)$ into blocks $Q_{ij} \in \mathbb{R}^{s \times s}$, where $1 \leq i, j \leq d$. Define

$$\Phi_{d,s} : O(ds) \rightarrow \mathbb{R}^{d \times d}, \quad [\Phi_{d,s}(Q)]_{ij} := \frac{1}{s} \|Q_{ij}\|_F^2.$$

Let $\mathbf{1}_d \in \mathbb{R}^d$ be the all-ones vector. The Birkhoff polytope is

$$\mathcal{B}_d = \{H \in \mathbb{R}^{d \times d} : H_{ij} \geq 0, H\mathbf{1}_d = \mathbf{1}_d, H^\top \mathbf{1}_d = \mathbf{1}_d\}.$$

Theorem. For every $Q \in O(ds)$, $\Phi_{d,s}(Q) \in \mathcal{B}_d$ [R005]; [R029].

Derivation. Squared norms are nonnegative. Orthogonality gives, for each block row i ,

$$\sum_{j=1}^d Q_{ij} Q_{ij}^\top = I_s.$$

Taking traces and dividing by s makes row i sum to one. Applying the same argument to $Q^\top Q = I_N$ makes every column sum to one.

None of these $N \times N$ tables adds a token, a feature, or a stream. The stored stream state is still $x_l \in \mathbb{R}^{B \times d \times L \times D}$, with batch size B , stream axis d , sequence length L , and feature size D . The paper’s stream-first display is an algebraic view of each batch slice. The new matrices construct only the $d \times d$ persistent-state router.

Hence $\Phi_{d,s}$ is a printer of legal leftover mixers. It does not invent leftover copies, and it does not mix tokens.

Counting directions

A $d \times d$ matrix has d^2 entries. Its d row-sum and d column-sum equations contain one redundancy because total row mass equals total column mass. The affine hull of $\mathcal{B}_d$ therefore has dimension

$$\dim \mathcal{B}_d = d^2 - (2d - 1) = (d - 1)^2.$$

At an interior point, its tangent space is

$$\{Z \in \mathbb{R}^{d \times d} : Z\mathbf{1}_d = 0, Z^\top \mathbf{1}_d = 0\}.$$

Because $\Phi_{d,s}$ is smooth, the local dimension of its image cannot exceed either source or target:

$$\dim_{\text{local}} \text{im } \Phi_{d,s} \leq \min \left\{ \frac{ds(ds-1)}{2}, (d-1)^2 \right\}.$$

For $s = 1$, this yields $d(d-1)/2$ versus $(d-1)^2$. When $d \geq 3$, the deficit is

$$(d-1)^2 - \frac{d(d-1)}{2} = \frac{(d-1)(d-2)}{2} > 0.$$

Theorem. Thus $\Phi_{d,1}$ is not surjective onto $\mathcal{B}_d$ for $d \geq 3$. The exceptional case $d = 2$ is fully covered, as the numerical example already suggested.

For $s \geq 2$, the raw source dimension is usually larger than $(d-1)^2$, so parameter counting alone cannot prove coverage. It also overcounts meaningful directions. If $L = \text{diag}(L_1, \dots, L_d)$ and $R = \text{diag}(R_1, \dots, R_d)$, with every $L_i, R_j \in O(s)$, then

$$\Phi_{d,s}(LQR) = \Phi_{d,s}(Q).$$

These within-block changes of basis are **gauge symmetries**: different orthogonal matrices represent the same router. They create flat fibers and null directions in the differential. Away from stabilizers, the corresponding group has dimension $ds(s-1)$; this is a useful generic redundancy count, not a theorem that every fiber has exactly that dimension.

The differential makes the local geometry concrete. For a tangent perturbation $\Delta Q = Q\Omega \in T_Q O(N)$,

$$[D\Phi_{d,s}(Q)[\Delta Q]]_{ij} := \frac{2}{s} \langle Q_{ij}, \Delta Q_{ij} \rangle_F.$$

Its output automatically has zero row and column sums. Its rank can drop at singular points, so the image need not itself be one smooth manifold. It can have folds, self-overlaps, boundary strata, and changing fiber dimensions. “The s -orthostochastic manifold” is convenient shorthand, not a license to ignore those features.

Source dimension at least target dimension is necessary for a full-dimensional image. It is not sufficient for surjectivity. Drop that distinction and “enough parameters” becomes a coverage theorem it is not.

What a finite block size can cover

Theorem (ordinary level). At $s = 1$ and $d \geq 3$, the orthostochastic image $\Phi_{d,1}(O(d))$ is a proper subset of $\mathcal{B}_d$ [R030].

That statement must not be silently promoted to every larger finite block size. The generalized-unistochastic source used here establishes an asymptotic density result and explicitly leaves finer fixed-level structure as a separate problem [R029]. Thus, for an arbitrary named finite $s > 1$, these notes assert exact containment $\Phi_{d,s}(O(ds)) \subseteq \mathcal{B}_d$, not strictness and not surjectivity. Sampling, spectral plots, and favorable optimization runs cannot decide equality of these sets.

Theorem. The closure of the union over block sizes is the full polytope:

$$\overline{\bigcup_{s \geq 1} \Phi_{d,s}(O(ds))} = \mathcal{B}_d.$$

This follows from the density theorem for generalized unistochastic matrices together with their real embedding at doubled block size [R029]. It says that every doubly stochastic matrix can be approximated arbitrarily closely at some sufficiently large level. It does **not** say that the levels are nested, that approximation error decreases monotonically for every optimizer, or that $s = 2$ is universally sufficient.

What the network actually computes

For one go-mHC routing instance, the learned projection produces $N(N-1)/2$ coordinates, assembled into $A_l \in \mathfrak{so}(N)$. The architecture then computes

$$Q_l = \text{cay}(A_l) \in SO(N), \quad H_l^{\text{res}} = \Phi_{d,s}(Q_l) \in \mathcal{B}_d,$$

before applying

$$x_{l+1} := H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l).$$

The multiplication $H_l^{\text{res}} x_l[b]$ contracts only the stream axis of each $d \times L \times D$ batch slice. Stored together, the result retains shape $B \times d \times L \times D$.

Empirical result. The go-mHC paper reports faster convergence on its synthetic stream-mixing tasks and similar convergent behavior among constrained variants in a 30M-parameter TinyStories model; it explicitly limits broad performance conclusions because of experimental scale [R005]. These observations do not establish a general optimization theorem.

Derivation. At $Q = I_N$, every off-diagonal block is zero and every diagonal block is I_s . A tangent perturbation has skew diagonal blocks, orthogonal to I_s under the Frobenius inner product. Hence

$$D\Phi_{d,s}(I_N) = 0.$$

Near $A = 0$, $Q = I_N - 2A + O(\|A\|^2)$, so off-diagonal router entries grow quadratically:

$$H_{ij} = \frac{4}{s} \|A_{ij}\|_F^2 + O(\|A\|^3), \quad i \neq j.$$

A loss depending on A only through H therefore has zero first derivative at exactly $A = 0$. Initialization and other coupled paths matter.

Interpretation. The map’s gauge symmetries create many parameter-space directions that do not change H . The Cayley chart also compresses very large coordinates as it approaches rotations with eigenvalue -1 ; in the two-dimensional example, the rotation angle is $-2 \arctan t$, whose derivative has magnitude $2/(1+t^2)$.

Hypothesis. Compared with a softmax over permutation-mixture weights, Cayley coordinates may avoid some exponential saturation and may provide smoother paths between many routers. Conversely, their fibers, singular points, quadratic identity behavior, and chart boundary may slow particular transitions. Which effect dominates depends on initialization, target router, optimizer, d , s , and data. The reported synthetic advantage in [R005] is evidence for tested settings, not proof of this mechanism.

A permutation-mixture parameterization has different geometry. It maps simplex weights $\alpha_r \geq 0$, $\sum_r \alpha_r = 1$, to $H = \sum_r \alpha_r P_r$. This is linear in α , globally convex in router space, and surjective when all $d!$ permutations are available; but the representation is highly redundant and a softmax coordinate map can saturate near simplex faces. Neither coordinate system is simply “more geometric.” Both impose geometry; they impose different geometry.

What the slogans get wrong

Without the manifold picture, one may mistake N^2 stored entries for N^2 trainable freedoms, or mistake $N(N-1)/2$ skew coordinates for the dimension of the final router set. Without tangent spaces, the skew condition looks like decorative antisymmetry rather than the precise infinitesimal orthogonality constraint.

Without the distinction between $O(N)$ and $SO(N)$, the Cayley range is overstated. Without the chart caveat, an optimizer approaching a missing -1 eigenvalue can appear mysteriously ill-conditioned. Without studying Φ , exact membership in $\mathcal{B}_d$ is easily confused with finite- s surjectivity.

Most importantly, omitting image and fiber geometry invites causal stories unsupported by evidence: “manifold coordinates train faster,” “more source parameters mean full coverage,” or “no softmax means no vanishing gradients.” Mathematics is useful partly because it ruins excellent slogans before they reach production.

Ways to get this wrong

1. **Calling all of $\mathcal{B}_d$ a smooth manifold.** Its relative interior is smooth in an affine space; its boundary has faces and corners.
2. **Equating skew matrices with orthogonal matrices.** $A \in \mathfrak{so}(N)$ is a tangent coordinate; $Q = \text{cay}(A) \in SO(N)$ is the finite transformation.
3. **Saying Cayley covers $SO(N)$.** It misses matrices with eigenvalue -1 , although its range is dense.
4. **Counting $N(N-1)/2$ as output dimension.** Φ discards within-block orientation and lands in a space of dimension only $(d-1)^2$.
5. **Inferring surjectivity from enough parameters.** Source dimension at least target dimension is necessary for full-dimensional image, not sufficient for surjectivity.
6. **Confusing finite- s exactness with finite- s coverage.** Every output is exactly doubly stochastic; some doubly stochastic targets are absent.
7. **Claiming larger s gives nested images.** The density theorem concerns the closure of a union; it does not supply simple nesting for every pair of levels.
8. **Treating optimization explanations as theorems.** Convergence speed and gradient conditioning require analysis or experiments beyond set membership.

Try it

1. Allowed velocities at the identity

Let $V = \begin{bmatrix} a & b \\ c & e \end{bmatrix} \in \mathbb{R}^{2 \times 2}$. Impose the first-order orthogonality condition at I_2 and find all allowed V .

Answer

Differentiating $Q^\top Q = I_2$ gives $V^\top + V = 0$. Thus $a = e = 0$ and $c = -b$, so $V = \begin{bmatrix} 0 & b \\ -b & 0 \end{bmatrix}$. There is one independent tangent direction, matching $2(2-1)/2 = 1$.

2. A dimension ledger

Compute the dimensions of $SO(ds)$ and $\mathcal{B}_d$ for $(d, s) = (3, 1)$ and $(3, 2)$. What can dimension alone prove?

Answer

For $(3, 1)$, $N = 3$, so $\dim SO(3) = 3$ while $\dim \mathcal{B}_3 = 4$; surjectivity is impossible. For $(3, 2)$, $N = 6$, so $\dim SO(6) = 15$ while the target dimension remains 4. The obstruction disappears, but dimension alone cannot prove surjectivity.

3. A gauge that does not change the mixer

For $d = 2, s = 2$, let $L = \text{diag}(L_1, L_2) \in O(4)$, where $L_1, L_2 \in O(2)$. Show that $\Phi_{2,2}(LQ) = \Phi_{2,2}(Q)$.

Answer

Block (i, j) changes from $Q_{ij} \in \mathbb{R}^{2 \times 2}$ to $L_i Q_{ij}$. Orthogonal left multiplication preserves Frobenius norm: $\|L_i Q_{ij}\|_F = \|Q_{ij}\|_F$. Every output entry is therefore unchanged.

4. A missing mixer at a finite level

Let $d = 3$, let $P = I_3$, and let $R \in \mathbb{R}^{3 \times 3}$ be the permutation matrix of the cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. At $s = 2$, can $H = 0.7P + 0.3R$ belong to $\Phi_{3,2}(O(6))$?

Answer

No. On this segment, level- s generalized unistochastic points have mixture weight k/s . At $s = 2$, the only weights are $0, 1/2, 1$. Since generalized orthostochastic matrices form a subset, 0.3 is unreachable at this level.

5. Why the identity is flat

Use $[D\Phi(Q)[\Delta Q]]_{ij} = 2\langle Q_{ij}, \Delta Q_{ij} \rangle_F / s$ to prove $D\Phi(I_{ds}) = 0$ for tangent ΔQ .

Answer

At identity, off-diagonal blocks Q_{ij} vanish, so their inner products vanish. Diagonal blocks equal I_s . Tangency makes each diagonal block ΔQ_{ii} skew-symmetric, hence $\langle I_s, \Delta Q_{ii} \rangle_F = \text{tr}(\Delta Q_{ii}) = 0$. Every differential entry is zero.

Check yourself

1. Show that $\mathfrak{so}(N)$ is not an orthogonal matrix.

It is the tangent space of skew directions at the identity. A skew table is a coordinate; $Q = \text{cay}(A)$ is the finite rotation.

2. What fails if you say Cayley hits every rotation?

It maps smoothly into the part of $SO(N)$ without eigenvalue -1 . A 180° rotation sits beyond the chart.

3. Show that every output of $\Phi_{d,s}$ lies in $\mathcal{B}_d$, then say what a named finite $s > 1$ does not claim.

Squared norms are nonnegative; traces of the block-row and block-column identities give unit sums. At a named finite $s > 1$ these notes assert containment, not strictness and not surjectivity. At $s = 1$ and $d \geq 3$, the image is a proper subset.

4. What fails if you count $N(N-1)/2$ as the dimension of the final router set?

Φ discards within-block orientation. The target has dimension $(d-1)^2$, and the source coordinates contain gauge redundancy.

5. Show that the union-over- s density theorem does not nest the levels.

The closure of the union over all block sizes is the full polytope. It does not say the levels are nested, that approximation error decreases monotonically for every optimizer, or that $s = 2$ is universally sufficient.

6. What fails if you treat $D\Phi(I) = 0$ as an optimization theorem?

A loss that sees A only through H has zero first derivative at $A = 0$. That is geometry of this chart. Convergence speed and gradient conditioning still need analysis or experiments; the synthetic advantage in [R005] is evidence for tested settings, not a general theorem.

Sources

- [R002] Z. Xie et al., “Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R026] K. Helfrich, D. Willmott, and Q. Ye, “Orthogonal Recurrent Neural Networks with Scaled Cayley Transform,” arXiv:1707.09520, 2018. <https://arxiv.org/abs/1707.09520>
- [R027] M. Lezcano-Casado and D. Martínez-Rubio, “Cheap Orthogonal Constraints in Neural Networks: A Simple Parametrization of the Orthogonal and Unitary Group,” *Proceedings of Machine Learning Research* 97, 2019. <https://proceedings.mlr.press/v97/lezcano-casado19a.html>
- [R029] I. Nechita, Z. Ouyang, and A. Szczepanek, “Generalized unistochastic matrices,” arXiv:2310.03436v2, 2023. <https://arxiv.org/abs/2310.03436>
- [R031] P.-A. Absil, R. Mahony, and R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*, Princeton University Press, 2008. <https://press.princeton.edu/books/hardcover/9780691132983/optimization-algorithms-on-matrix-manifolds>

Chapter 19 — How many numbers, how much multiply

How many numbers, how much multiply. That is the whole question, and it is a question you can hold in one hand.

A short list of free numbers becomes a table that is nonnegative and sums to one both ways. The last few chapters named the path: pack the list into a skew table, turn it into a rotation, cut the rotation into blocks, and square the blocks.

That path is exact on paper. Software is not paper. Transpose the wrong two axes and the tensor still has the advertised size. Forget to square and you still get numbers. Forget to say *how many times* you solve the same little system, and an eight-by-eight solve that looked cheap becomes thousands of them.

Five names that are not the same

A batch of leftover copies is a four-axis box

$$x \in \mathbb{R}^{B \times d \times L \times D}.$$

B is how many examples sit in the batch. d is how many leftover copies of the sentence table you are carrying. L is how long the sentence is. D is how wide each word's feature list is.

The mixer we are about to build does not live on those four axes. It is cut from a larger rotation whose side is

$$N = ds,$$

where s is the block size from the last two chapters. N is not sequence length. In the usual teaching case $d = 4$, $s = 2$, the rotation is merely 8×8 . It is not a stealth attention matrix.

Keep the five names separate in every shape comment, or one of them will steal another's seat.

A triangle of free numbers

A skew-symmetric table $A \in \mathbb{R}^{N \times N}$ obeys $A^\top = -A$. The diagonal is forced to zero: a number that equals its own negative has nowhere else to go. Choose an entry above the diagonal and you have already chosen the entry below it, with the sign flipped.

So we do not store the whole square. We store a list

$$u \in \mathbb{R}^p, \quad p = \frac{N(N-1)}{2}.$$

The Cayley transform turns A into an orthogonal table $Q \in \mathbb{R}^{N \times N}$. We then view Q as a $d \times d$ grid of $s \times s$ blocks. Squared Frobenius sizes throw away the signs inside each block and keep only how much orthogonal energy moved between block rows and columns.

Orthogonality gives every block row and every block column the same total squared energy. Dividing by s turns that energy into unit row and column sums. No alternating row/column repair loop is required.

“Direct” is not “free.” The dense solve is cubic in $N = ds$. The projection that emits the p coordinates has its own feature-dependent cost. And a bug on the first arrow of

$$\text{coordinates} \longrightarrow A \longrightarrow Q \longrightarrow H^{\text{res}}$$

can still produce a tensor of the right size. A bug on the last arrow can still produce nonnegative numbers. Shape checks do not defend the contract. We will need tests that watch the invariants.

The go-mHC paper is a recent preprint. Its cost and training numbers are reported evidence, not settled architecture law [R005].

Two streams, one number

Take $d = 2$, $s = 1$, and hence $N = 2$. There is only

$$p = \frac{2(2-1)}{2} = 1$$

free coordinate. Choose $u = [1/2]$ and pack it:

$$A = \begin{pmatrix} 0 & 1/2 \\ -1/2 & 0 \end{pmatrix} \in \mathbb{R}^{2 \times 2}.$$

The Cayley transform is

$$Q = (I_N - A)(I_N + A)^{-1}.$$

In software we do not form that inverse. $I_N - A$ and $I_N + A$ are polynomials in the same table, so they commute, and we solve

$$(I_N + A)Q = I_N - A$$

instead. For this A the result is

$$Q = \begin{pmatrix} 0.6 & -0.8 \\ 0.8 & 0.6 \end{pmatrix}.$$

It is orthogonal: $Q^\top Q = I_2$. Since $s = 1$, every block is one scalar. Squaring those scalars gives the mixer

$$H^{\text{res}} = \begin{pmatrix} 0.6^2 & (-0.8)^2 \\ 0.8^2 & 0.6^2 \end{pmatrix} = \begin{pmatrix} 0.36 & 0.64 \\ 0.64 & 0.36 \end{pmatrix}.$$

Both rows and both columns sum to one. Q still contains signed rotations. H^{res} contains nonnegative leftover-copy weights. They are not interchangeable.

Drop the square and the mixer can contain negative weights. Drop the distinction between Q and H^{res} and you will later call a leftover router a rotation.

Why the mixer sums to one

If $A^\top = -A$, then in exact arithmetic the Cayley result Q is orthogonal, and

$$H_{ij}^{\text{res}} = \frac{1}{s} \|Q_{ij}\|_F^2$$

is doubly stochastic.

Skew symmetry gives $(I + A)^\top = I - A$. The Cayley factors commute, and multiplying through yields $Q^\top Q = I_N$. Cut Q into blocks $Q_{ij} \in \mathbb{R}^{s \times s}$. From $QQ^\top = I_N$, block row i obeys

$$\sum_{j=1}^d Q_{ij} Q_{ij}^\top = I_s.$$

Take traces:

$$\sum_{j=1}^d \|Q_{ij}\|_F^2 = s, \quad \sum_{j=1}^d H_{ij}^{\text{res}} = 1.$$

The same argument on $Q^\top Q = I_N$ gives

$$\sum_{i=1}^d H_{ij}^{\text{res}} = 1.$$

Every entry is nonnegative because it is a sum of squares. “Exact” describes the mathematical image. Floating-point tests still use tolerances. Silicon has declined the invitation to implement real arithmetic.

The recipe, with every shape named

A teaching summary first reduces the leftover copies

$$x \in \mathbb{R}^{B \times d \times L \times D}$$

to a per-example list

$$h \in \mathbb{R}^{B \times C_{\text{in}}}.$$

Averaging over the sentence axis and flattening leftover-copy and feature axes gives $C_{\text{in}} = dD$. That is a teaching choice, not a claim about every production recipe.

A linear map with

$$W_{\text{route}} \in \mathbb{R}^{p \times C_{\text{in}}}, \quad b_{\text{route}} \in \mathbb{R}^p$$

then emits

$$u = hW_{\text{route}}^\top + b_{\text{route}} \in \mathbb{R}^{B \times p}.$$

The rest of the box, with every axis written down:

```
GO_ROUTER(h [B, C_in], d, s) -> H [B, d, d]
  N <- d*s
  p <- N*(N-1)/2
  u [B, p] <- LINEAR(h [B, C_in])
```

```

A [B, N, N] <- zeros
scatter u into strict upper triangle of A
A <- A - transpose_last_two_axes(A)

I [B, N, N] <- broadcast(identity(N), batch=B)
# solve (I + A) Q = (I - A); never construct an inverse
Q [B, N, N] <- SOLVE(I + A, I - A)

blocks [B, d, d, s, s] <-
  reshape Q as [B, d, s, d, s], then permute to [B, d, d, s, s]
H [B, d, d] <- SUM(blocks * blocks, axes=(-2,-1)) / s
return H

```

Reshaping directly to $[B, d, d, s, s]$ is wrong. The matrix row index splits as (block_row, within_row) and the column index as (block_col, within_col), so the intermediate order is $[B, d, s, d, s]$ and needs a permutation.

The mixer that comes out is $B \times d \times d$. It still does not live on the token axis or the feature axis.

How many numbers, how much multiply

The strict upper triangle contains

$$p(N) = 1 + 2 + \dots + (N - 1) = \frac{N(N - 1)}{2}$$

coordinates. A dense factorization and solve costs $O(N^3) = O(d^3 s^3)$. Forming A , reducing block norms, and writing down Q cost $O(N^2)$. With R independent routing instances those become $O(RN^3)$ and $O(RN^2)$.

A bare $O(N^3)$ tells you the size of one solve. It does not tell you how many solves you asked for. One $N \times N$ solve per layer is different from one per batch example, and radically different from one per token.

For $s = 2$, the three teaching sizes are $N = 8, 16$, and 32 :

streams d	$N = ds$	skew coordinates p	N^2 entries	N^3 cubic units	paper's approximate go projection FLOPs
4	8	28	64	512	$\frac{4864}{3}C \approx 1,621.3C$
8	16	120	256	4,096	$\frac{35840}{3}C \approx 11,946.7C$
16	32	496	1,024	32,768	$\frac{274432}{3}C \approx 91,477.3C$

The last column evaluates the paper appendix's reported expression

$$\frac{8}{3}C(ds)^3 + 4C(ds)^2,$$

keeping its channel/count factor C rather than silently dropping it [R005]. The table is algebra, not a stopwatch. Kernel launch cost, batch occupancy, dtype, and hardware change elapsed time.

The coordinate projection itself has $(C_{\text{in}} + 1)p$ parameters and roughly $2BC_{\text{in}}p$ multiply-add FLOPs for a batch. In the paper's full per-layer accounting, the residual projection receives a flattened dC -wide activation summary, giving

$$(dC + 1) \frac{ds(ds - 1)}{2}$$

before the parameters shared with the other Hyper-Connection gates [R005]. So “the skew matrix has p parameters” is ambiguous. p is the number of **dynamic coordinates emitted per routing instance**. The learned linear map that emits them contains many more weights.

The rest of the layer is bigger

A dense Transformer projection over B sequences, L tokens, and width D costs on the order of BLD^2 . Attention score and value products add terms on the order of BL^2D . Leftover-copy mixing

$$H^{\text{res}}x, \quad H^{\text{res}} \in \mathbb{R}^{B \times d \times d}, \quad x \in \mathbb{R}^{B \times d \times L \times D},$$

costs $O(BLDd^2)$. When $d^2 \ll D$, that mixing term can be much smaller than a width- D projection. The dense Cayley solve is also small in absolute dimension at $N = 8, 16, 32$, but it is less friendly to accelerators than one giant matrix multiply. Small solves may be latency-bound or launch-bound.

Let R be the number of routers evaluated per layer. Layer-static routing gives $R = 1$. Per-example routing gives $R = B$. Token-wise routing gives $R = BL$. The cost is $O(RN^3)$. Do not infer token-wise routing merely because an equation contains token activations; look at the cited implementation. The code below implements per-example batching on purpose.

Cubic is a warning to count instances. It is not a bill, and it is not a discount.

A small Julia box

The following is pedagogical, executable-in-principle Julia. Production code may fuse the summary projection, cache indices more aggressively, pick a solve dtype, or use compiled graph capture. The mathematical path should stay the same.

```
using LinearAlgebra
using Random
using Statistics

"""x: (B,d,L,D) → h: (B, d*D), a pedagogical per-example summary."""
function summarize_streams(x; eps=1e-6)
    ndims(x) == 4 || throw(ArgumentError("x must have shape (B,d,L,D)"))
    x_norm = x ./ sqrt.(mean(x.^2; dims=4) .+ eps)
    return reshape(dropdims(mean(x_norm; dims=3); dims=3), size(x, 1), :)
end

"""Per-example go-mHC residual router: (B,C_in) → (B,d,d)."""
struct GoMHCRouter{T}
    d::Int
    s::Int
    N::Int
    p::Int
    weight::Matrix{T}    # (C_in, p)
    bias::Vector{T}      # (p,)
end

function GoMHCRouter(in_features::Integer, d::Integer, s::Integer;
    init_std=1e-3, T=Float64)
    in_features > 0 && d > 0 && s > 0 ||
```

```

        throw(ArgumentError("in_features, d, and s must be positive"))
    N = d * s
    p = N * (N - 1) ÷ 2
    # Near identity but not exactly at the square-map's stationary point.
    weight = (init_std / sqrt(in_features)) .* randn(T, in_features, p)
    bias = init_std .* randn(T, p)
    GoMHCRouter{T}(d, s, N, p, weight, bias)
end

to_coordinates(r::GoMHCRouter, h) = h * r.weight .+ r.bias'

function coordinates_to_router(r::GoMHCRouter, u)
    ndims(u) == 2 && size(u, 2) == r.p ||
        throw(ArgumentError("u must have shape (B,$(r.p))"))
    B = size(u, 1)
    T = eltype(u)
    A = zeros(T, B, r.N, r.N)
    k = 1
    for i in 1:r.N, j in (i + 1):r.N
        A[:, i, j] .= u[:, k]
        k += 1
    end
    A .= A .- permutedims(A, (1, 3, 2))

    Q = similar(A)
    I_N = Matrix{T}(I, r.N, r.N)
    for b in 1:B
        # (I+A) \ (I-A) is the Cayley transform. No explicit inverse.
        Q[b, :, :] = (I_N + A[b, :, :]) \ (I_N - A[b, :, :])
    end

    # Extract s×s blocks by index, not by a row-major reshape.
    H = zeros(T, B, r.d, r.d)
    for b in 1:B, i in 1:r.d, j in 1:r.d
        block = @view Q[b, (i-1)*r.s+1:i*r.s, (j-1)*r.s+1:j*r.s]
        H[b, i, j] = sum(abs2, block) / r.s
    end
    return H, Q, A
end

(r::GoMHCRouter)(h) = coordinates_to_router(r, to_coordinates(r, h))[1]

```

There is no explicit inverse. $A \setminus B$ solves $AX = B$ [R037]. A loop over a handful of $N \times N$ tables is the Julia way to write the batch; it is not a claim that a production kernel should launch one solve at a time on an accelerator.

A skew table has purely imaginary eigenvalues, so $I + A$ is nonsingular in exact real arithmetic. Large coordinate magnitudes can still worsen numerical conditioning and make Cayley gradients small in some directions. A practical box should watch coordinate scale and constraint residuals, and commonly performs this small solve in at least Float32 rather than trusting a Float16 solve with its life story.

Start near the copy, not on it

At $A = 0$, $Q = I_N$ and $H^{\text{res}} = I_d$. That is attractive if you want the leftover copies to pass through. But because H uses squared block entries, the first derivative of H with respect to A is zero exactly at $A = 0$.

The initialization above therefore gives both the projection weights and the bias tiny nonzero noise, scaling the weight standard deviation by $C_{\text{in}}^{-1/2}$ so unit-scale inputs emit coordinates of roughly `init_std` scale. It starts near the identity while avoiding the exact stationary point. That is a teaching initialization, not a claim that it reproduces an unpublished production recipe.

Automatic differentiation (Zygote, Enzyme, or a handwritten backward) differentiates through coordinate scatter, the solve, the block extraction, the squaring, and the reduction. If $G_H = \partial \mathcal{L} / \partial H \in \mathbb{R}^{B \times d \times d}$, the block-norm stage contributes

$$(G_Q)_{ij} = \frac{2}{s} (G_H)_{ij} Q_{ij}, \quad (G_Q)_{ij} \in \mathbb{R}^{B \times s \times s},$$

which matches the analytical block derivative reported by the paper [R005]. The solve's backward pass uses linear solves as well. Storing or recomputing its factorization is a framework decision.

Hence a loss that sees A only through H has a flat spot at the identity. Initialization is not a decorative random seed. It is how you step off that spot.

Tests that catch a lie

These tests watch observable invariants, not implementation text. Run them in Float64 so tight tolerances measure the construction rather than mixed-precision noise.

```
function test_go_router_shapes_and_invariants()
    Random.seed!(7)
    B, d, s, C_in = 3, 4, 2, 11
    router = GoMHCRouter(C_in, d, s; init_std=1e-2)
    h = randn(B, C_in)
    u = to_coordinates(router, h)
    H, Q, A = coordinates_to_router(router, u)

    @assert size(u) == (B, 28)
    @assert size(A) == size(Q) == (B, 8, 8)
    @assert size(H) == (B, 4, 4)
    @assert A .+ permutedims(A, (1, 3, 2)) ≈ zeros(size(A)) atol=1e-14

    QTQ = stack([Q[b, :, :]' * Q[b, :, :] for b in 1:B]; dims=1)
    @assert QTQ ≈ stack(fill(Matrix{Float64}(I, 8, 8), B); dims=1) rtol=1e-10 atol=1e-10
    @assert dropdims(sum(H; dims=3); dims=3) ≈ ones(B, d) rtol=1e-10 atol=1e-10
    @assert dropdims(sum(H; dims=2); dims=2) ≈ ones(B, d) rtol=1e-10 atol=1e-10
    @assert all(H .>= 0)
end

function test_zero_coordinates_give_identity_router()
    router = GoMHCRouter(5, 4, 2)
    u = zeros(2, router.p)
    H, Q, _ = coordinates_to_router(router, u)
    @assert Q == stack(fill(Matrix{Float64}(I, 8, 8), 2); dims=1)
    @assert H == stack(fill(Matrix{Float64}(I, 4, 4), 2); dims=1)
end

function test_loss_moves_when_projection_moves()
    Random.seed!(11)
    B, C_in, d, s = 2, 9, 4, 2
    router = GoMHCRouter(C_in, d, s; init_std=1e-2)
```

```

h = randn(B, C_in)
target = randn(B, d, d)
loss() = sum(router(h) .* target)
L0 = loss()
router.weight[1, 1] += 1e-3
L1 = loss()
@assert isfinite(L0) && isfinite(L1)
@assert L0 != L1
end

```

The zero-coordinate test checks exact identity on purpose. The random-coordinate test checks orthogonality and both stochastic axes. If a block permutation is wrong, at least one stochastic test should fail. If the Cayley solve is transposed incorrectly for a noncommuting replacement, orthogonality should fail. If norms are not squared or not divided by s , the sum tests fail.

Nonnegativity alone is not a contract. Any squared matrix is nonnegative.

Where this box sits

The full Hyper-Connection update is

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l, W_l),$$

where $x_l \in \mathbb{R}^{B \times d \times L \times D}$, $H_l^{\text{res}} \in \mathbb{R}^{B \times d \times d}$ in our per-example box, and the read/write gates broadcast over sentence and feature axes. The router above constructs only H_l^{res} . It does not create leftover copies, evaluate F , or define H^{pre} and H^{post} .

Original mHC applies finite Sinkhorn normalization to residual-routing logits. go-mHC replaces that residual constraint map with the skew-Cayley-block-norm map [R005]; [R002]. The recent go-mHC paper reports batched linear solves and tensor contractions without a custom CUDA kernel [R005]. That is an implementation-advantage claim from that preprint, not a guarantee that every workload or compiler will run faster.

The paper’s approximate comparison gives mHC $4CSd^2$ FLOPs for S Sinkhorn iterations and go-mHC $\frac{8}{3}C(ds)^3 + 4C(ds)^2$ [R005]. These formulas expose the regime change. Sinkhorn scales quadratically in d but repeats S times. The dense Cayley construction is cubic in d for fixed s , but it is not iterative. Measured crossover depends on actual batch granularity and kernels.

What you lose if you skip a step

Omit skew symmetrization, and Q is not guaranteed orthogonal. Omit the solve and substitute a first-order approximation $I - 2A$, and exact orthogonality disappears except at $A = 0$. Omit the square, and H can contain negative routing weights. Omit division by s , and every row and column sums to s , not one.

Omit batching from the design, and a serial loop may serialize thousands of tiny solves. Conversely, broadcast one H across a batch that was meant to be dynamic, and the model implements a different router. Performance bugs and semantic bugs occasionally share an apartment.

Omit gradient testing, and an apparently valid identity initialization can freeze the residual router at the square map’s stationary point. Omit column-sum testing, and a merely row-stochastic matrix can pass as doubly stochastic. Finally, omit explicit routing multiplicity from the cost estimate, and a reasonable $N = 8$ solve can become an unreasonable BL solves without anyone changing the asymptotic notation.

Ways to get this wrong

1. **Calling p the learned parameter count.** It is the emitted coordinate count. A dense projection from C_{in} features contains $(C_{\text{in}} + 1)p$ learned parameters.

2. **Using an explicit inverse in code.** Solve the linear system with `\`. It is clearer, more stable, and differentiable.
3. **Confusing N with sequence length.** $N = ds$; sequence length is L .
4. **Flattening blocks incorrectly.** Extract the $s \times s$ blocks by index. A row-major reshape from another language will scramble Julia's column-major layout.
5. **Testing only nonnegativity.** Any squared matrix is nonnegative. Orthogonality and row/column sums carry the contract.
6. **Testing exact real-arithmetic identities with exact floating equality.** Use tolerances except for deliberately exact cases such as zero coordinates.
7. **Initializing all coordinates exactly to zero.** It returns the identity but gives the squared block map a zero first derivative there.
8. **Claiming cubic automatically means expensive.** $8^3 = 512$ is small; multiplicity, kernels, and surrounding Transformer work decide runtime.
9. **Claiming cubic automatically means cheap.** Token-wise BL tiny solves can still be expensive. Count instances.
10. **Running the solve in low precision without checking residuals.** Validate $\|Q^\top Q - I\|$, row sums, and column sums under the actual dtype.

Try it

1. Count the coordinates

For $d = 8$, $s = 2$, compute N , p , and the coordinate-projection parameter count when $C_{\text{in}} = 256$.

Answer

$N = 16$, $p = 16 \cdot 15/2 = 120$, and the affine projection contains $(256 + 1)120 = 30,840$ parameters.

2. Find the missing scale

Suppose $Q \in \mathbb{R}^{12 \times 12}$ is split with $d = 4$, $s = 3$, and an implementation sets $H_{ij} = \|Q_{ij}\|_F^2$. What do rows and columns of H sum to, and how do you fix it?

Answer

They sum to $s = 3$, because each orthogonal block row/column has total squared Frobenius norm 3. Divide every entry by s .

3. Attach multiplicity to complexity

Compare per-layer solve work for per-example and token-wise routing when $B = 8$, $L = 2048$, $d = 4$, $s = 2$. Express both in cubic units RN^3 .

Answer

Here $N = 8$ and $N^3 = 512$. Per-example routing uses $R = B = 8$: 4,096 cubic units. Token-wise routing uses $R = BL = 16,384$: 8,388,608 units, a factor of $L = 2048$ larger.

4. Diagnose a failed invariant

A router produces nonnegative H whose rows sum to one, but columns are $[0.8, 1.1, 1.0, 1.1]$. Name two plausible implementation errors.

Answer

The source Q may not be orthogonal because A was not made skew-symmetric or the Cayley system was formed incorrectly. Alternatively, block axes may have been reshaped/permutated incorrectly, preserving accidental row normalization but destroying the block-column argument.

5. Repair initialization

Why can exact zero coordinates stall learning of H , and give one near-identity repair.

Answer

At $A = 0$, $Q = I$. Off-diagonal blocks start at zero and are squared; diagonal blocks change only at second order under an infinitesimal skew perturbation. Hence $\partial H / \partial A = 0$ there. Initialize the coordinate projection and bias with tiny nonzero noise, then test that gradients reaching both inputs and parameters are finite and nonzero.

Check yourself

1. **Show that N is not the sentence length.**

$N = ds$. It is the side of the Cayley table. Sequence length is L .

2. **What fails if you write down an inverse instead of solving $(I + A)Q = I - A$?**

$I - A$ and $I + A$ commute, the solve *is* the Cayley transform, and $\backslash$ is the right primitive [R037]. An explicit inverse is less stable and no clearer.

3. **Show the three teaching sizes of N for $s = 2$ and $d = 4, 8, 16$.**

$N = 8, 16$, and 32 .

4. **What fails if you quote a bare $O(d^3 s^3)$?**

Multiplicity R : one solve per layer, per example, or per token. The cost is $O(Rd^3 s^3)$.

5. **What fails if you initialize every coordinate exactly to zero?**

At $A = 0$, $H = I_d$ but $\partial H / \partial A = 0$. Start near the identity, not on it.

6. **Show that nonnegativity is not enough.**

Any squared matrix is nonnegative. A test must also check orthogonality of Q and both row sums and column sums of H .

The box is small. The surrounding layer is not. Name the tensor, name the axis, name how many solves you asked for, and test the sums both ways.

Sources

- [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024. <https://arxiv.org/abs/2409.19606>
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
- [R037] Julia documentation, LinearAlgebra left division $\backslash$, *The Julia Language*. <https://docs.julialang.org/en/v1/stdlib/LinearAlgebra/#LinearAlgebra.ldiv!>
- [R026] K. Helfrich, D. Willmott, and Q. Ye, “Orthogonal Recurrent Neural Networks with Scaled Cayley Transform,” arXiv:1707.09520, 2018. <https://arxiv.org/abs/1707.09520>
- [R038] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. <https://jhupbooks.press.jhu.edu/title/matrix-computations>

Chapter 20 — Ten mix-ups

You close the book and try to explain the last layer to a friend. The equation is still in your head. The story you tell about it is not.

You say the extra trays come from a skew matrix. You say each tray is last week’s layer. You say four trays means four Transformer blocks. You say the leftover router is a single frozen matrix. You say any legal leftover matrix can be written exactly, at any block size. You say “rows and columns sum to one” means “the matrix is a rotation.” You say leftover routing is the same gate that picks experts. You say cubic in a small auxiliary size means the method is too expensive to run. You say the routing map is a fat multilayer net. You say the Cayley transform is what learns.

Those ten sentences are the usual wreck. This chapter walks them back, then asks what a paper is still allowed to claim.

The ten sentences, in English

Here they are again, without the algebra. Each one names the wrong object.

1. The skew matrix is where the extra leftover copies come from.
2. Each leftover copy is one previous layer, sitting in a rolling cache.
3. Several leftover copies means several copies of the expensive block.
4. The leftover router is just one learned matrix, parked once and left alone.
5. The go-mHC construction can write every legal leftover matrix, at whatever finite block size you picked.
6. A matrix whose rows and columns sum to one is automatically a rotation.
7. Leftover routing is the same thing as expert routing.
8. An auxiliary solve that is cubic in a small size is automatically too expensive.
9. The map that emits routing numbers is a large multilayer net.
10. The Cayley transform itself is the learner.

A mathematically exact router can still be a poor model component. A useful measured result does not prove the story told about why it works. Almost every wreck in the list comes from mixing five objects:

- the leftover copies carried through depth;
- the small learned map that emits routing numbers;
- the constraint map that turns those numbers into a legal matrix;
- the branch that actually transforms token features;
- the experiment used to score the whole system.

Keep those five on separate hooks. The rest of the chapter is that habit, once with two numbers, once with the real equation, and once with a paper in your hand.

Two trays, one machine

Take one batch item, one token, one feature, and two leftover copies. Then $B = m = n = 1$, the stream count is $d = 2$, and the stored state has shape $B \times d \times m \times n = 1 \times 2 \times 1 \times 1$. Its only slice is

$$x_l[0, :, 0, 0] = \begin{bmatrix} 2 \\ 6 \end{bmatrix} \in \mathbb{R}^2.$$

Let the read gate be $H_l^{\text{pre}} = [0.75, 0.25] \in \mathbb{R}^{1 \times 2}$. Contracting only the leftover axis gives

$$z_l = H_l^{\text{pre}} x_l = 0.75(2) + 0.25(6) = 3 \in \mathbb{R}^{1 \times 1 \times 1}.$$

Let the single branch be $F(z) = z + 1$, so the note is $y_l = 4 \in \mathbb{R}^{1 \times 1 \times 1}$. Let the write gate be $H_l^{\text{post}} = [0.25, 0.75] \in \mathbb{R}^{1 \times 2}$. Its transposed write contributes $[1, 3]^\top$, stored with shape $1 \times 2 \times 1 \times 1$.

Choose a leftover router

$$H_l^{\text{res}} = \begin{bmatrix} 0.75 & 0.25 \\ 0.25 & 0.75 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Every entry is nonnegative, and every row and column sums to one. Persistent routing gives $[3, 5]^\top$. Therefore

$$x_{l+1} = H_l^{\text{res}} x_l + (H_l^{\text{post}})^\top y_l = \begin{bmatrix} 3 \\ 5 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 4 \\ 8 \end{bmatrix}.$$

That tiny sum already kills the first three mix-ups, and mix-up 6 as a bonus.

There was one evaluation of F . Neither leftover copy was “the previous layer.” And a matrix whose rows and columns sum to one need not be a rotation. Indeed

$$H_l^{\text{res}} (H_l^{\text{res}})^\top = \begin{bmatrix} 0.625 & 0.375 \\ 0.375 & 0.625 \end{bmatrix} \neq I_2.$$

The matrix averages leftover copies. It does not preserve every Euclidean length. Row and column mass is not an angle.

Mix-up 1 — the skew matrix is not the trays

Where do the extra leftover copies come from? Not from the skew table.

They are already there. They are axis 1 of the state $x_l \in \mathbb{R}^{B \times d \times m \times n}$, created when the architecture is built and then carried forward. The skew matrix $A_l \in \mathbb{R}^{ds \times ds}$ is an auxiliary pad of numbers used later to *build* the leftover router H_l^{res} . It does not invent the trays.

A layer reads a blend of leftover copies, runs one machine on that blend, writes the result into selected copies, and routes the old contents forward. The copies are residual streams. The machine is the branch F . The routing coefficients are not the copies.

A small learned projection emits coordinates. Those coordinates fill a skew-symmetric matrix. A Cayley transform produces an orthogonal auxiliary matrix. Squared Frobenius norms of its blocks produce the leftover router. The auxiliary orthogonal matrix is scaffolding. The network applies the resulting stream router to the persistent state.

Hence the constraint map is a printer of legal leftover mixers. Learned coordinates supply the contents. The printer enforces the format. The printer does not decide what to write, and it does not secretly become the document.

Drop the leftover copies and there is nothing to route. Drop the printer and the coordinates need not be legal. They are not the same object.

Mix-up 2 — a tray is not last week's layer

A leftover copy looks like “the thing we kept from before.” That is the wrong before.

Streams persist across every layer. They are parallel leftover copies of the whole table, not a rolling cache of the last d layer outputs.

If you want last week's layer, you already have it: it is x_l before this layer writes x_{l+1} . Extra leftover copies do not rename that fact. They add a new axis.

Drop that axis distinction and four leftover copies of a three-token sentence become a twelve-token sentence. The arithmetic may still run. The model is talking to the wrong house.

Mix-up 3 — several trays is not several blocks

Does $d = 4$ mean four Transformer blocks? Not in the central equation.

The read gate first combines d leftover copies into one branch input of shape $B \times m \times n$, so F can run once. A deliberately multi-branch extension is a separate design, with a separate bill.

Ordinary residual networks have one leftover copy. Hyper-Connections expose several and learn how to read, write, and propagate them. Unconstrained propagation can amplify or erase state through repeated matrix products. Manifold-constrained Hyper-Connections restrict persistent-state routing to doubly stochastic matrices: nonnegative matrices whose rows and columns each sum to one. mHC-lite, KromHC, and go-mHC use different constructions to obtain such matrices.

Hence several leftover copies change the routing graph, not the default number of expensive branches. Count F , not d , if you want the compute bill.

Mix-up 4 — the leftover router need not sit still

A leftover router can look like one learned matrix, parked once. That is one implementation, not the definition.

Hyper-Connection family routers can depend on the current activations. A learned projection reads a normalized or reduced copy of the state and emits routing coordinates. How often that happens — once per layer, once per token, once per channel — is an implementation fact. Do not silently promote a layer-wise router into a token-wise one, or the other way around.

The same warning applies to mix-up 9. The cited formulations use those reduced activations and small learned *linear* projections. A future implementation may choose a multilayer net. The name of the method does not summon hidden layers.

Mix-up 10 is the same object wearing a fancier hat. The Cayley transform is a deterministic differentiable map. Projection weights learn under backpropagation. Cayley converts their emitted skew coordinates into an orthogonal matrix and lets gradients pass. It does not itself decide what to learn.

Drop the multiplicity of the projection and a cheap layer-wise router becomes a token-wise storm of tiny solves. The equation did not change. The bill did.

Mix-up 5 — legal is not the same as everything

In go-mHC, pick an auxiliary block size $s \geq 1$ and set $N = ds$. Learned routing coordinates fill a skew-symmetric matrix

$$A_l \in \mathbb{R}^{N \times N}, \quad A_l^\top = -A_l.$$

There are $N(N-1)/2$ independent coordinates. The Cayley construction produces an auxiliary orthogonal matrix. One paper convention is

$$Q_l = (I_N - A_l)(I_N + A_l)^{-1} \in \mathbb{R}^{N \times N}.$$

Production code should compute the equivalent result with a linear solve, not an explicit inverse. Partition Q_l into $d \times d$ blocks $Q_{l,ij} \in \mathbb{R}^{s \times s}$ and define

$$(H_l^{\text{res}})_{ij} := \frac{1}{s} \|Q_{l,ij}\|_F^2.$$

Orthogonality of Q_l makes every entry of H_l^{res} nonnegative and makes every row and column sum to one [R005]; [R029]. In exact arithmetic the constructed router is exactly doubly stochastic.

That is the whole theorem. For a fixed finite s , the image

$$\Phi_{d,s}(O(ds)) \subseteq \mathcal{B}_d$$

need not equal the entire Birkhoff polytope $\mathcal{B}_d$. At $s = 1$, it is the orthostochastic subset. The generalized sets approach full coverage as s grows in the cited theory [R005]; [R029]. What is established is

$$\bigcup_{s \geq 1} \mathcal{S}_{d,s} = \mathcal{B}_d;$$

adjacent levels need not be nested [R029]. For $d \geq 3$, the $s = 1$ image is a proper subset. For an arbitrary named larger finite s , equality or strictness is not established here.

Every output is exactly doubly stochastic. That is not the claim that every doubly stochastic matrix appears. Exact constraint satisfaction and exact set coverage wear annoyingly similar hats.

“Exact” has an object. Ask: exact constraint, exact arithmetic, exact coverage, exact optimizer solution, or exact reproduction? If the noun is missing, so is half the claim.

Exact double stochasticity is a statement about the output of a mathematical construction, up to floating-point error in an implementation. It is not a theorem about final task accuracy, wall-clock speed, stream specialization, or optimization success.

Mix-up 6 — mass is not a rotation

The two-tray matrix above is already a counterexample. Rows and columns sum to one, yet

$$H_l^{\text{res}}(H_l^{\text{res}})^\top \neq I_2.$$

In go-mHC an auxiliary Q is orthogonal; block squared norms of Q produce a generally non-orthogonal doubly stochastic H^{res} . Double stochasticity controls row and column mass, not angles.

Drop that distinction and every honest leftover mixer is promoted to a rotation. The two-tray numbers refuse.

Mix-up 7 — two gates, two objects

Both gates are called routing. They do not route the same thing.

mHC routes persistent leftover copies. Mixture-of-experts routing typically assigns token representations to parameterized experts. The two gates act on different objects. They can sit in the same layer. Their axes, capacity constraints, and compute semantics still differ.

Compute balance, dropped-token behavior, and expert capacity remain mixture-of-experts concerns. Double stochasticity across leftover copies does not solve them.

Hence a legal leftover mixer is not an expert balancer. Confusing the two objects will send the capacity math to the wrong axis.

Mix-up 8 — cubic is a warning, not a bill

An $O((ds)^3)$ solve looks expensive on a slide. At $d = 4$, $s = 2$, it is an 8×8 system.

In dense go-mHC the auxiliary linear solve scales cubically in ds . Persistent activations scale as $dBmn$. The skew-coordinate count is $ds(ds - 1)/2$.

Actual overhead depends on those small matrix sizes, how often you emit coordinates, the hardware, memory traffic, kernel quality, and the much larger branch F . Cubic is a warning to measure. It is not a benchmark result.

Drop the multiplicity R and $8^3 = 512$ becomes either cheap or ruinous without anyone changing the big-O. Count instances.

The same equation, with the axes written down

Let d be the leftover-copy count, B the batch size, m the sequence length, and n the feature width. These notes use batch-first storage:

$$x_l \in \mathbb{R}^{B \times d \times m \times n}, \quad H_l^{\text{pre}}, H_l^{\text{post}} \in \mathbb{R}^{1 \times d}, \quad H_l^{\text{res}} \in \mathbb{R}^{d \times d}.$$

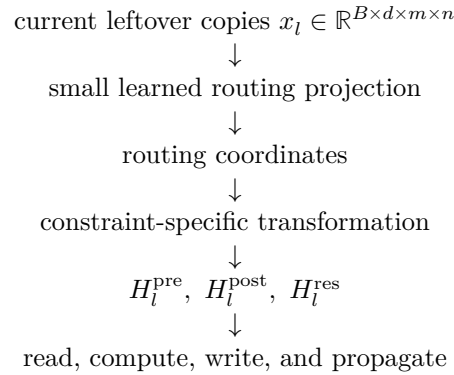
The papers often display a stream-first equation. Under batch-first storage, apply it to each batch slice:

$$x_{l+1}[b] = H_l^{\text{res}} x_l[b] + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l[b], W_l).$$

The matrices contract only the leftover axis, axis 1 in storage. They do not mix batch items, sequence positions, or feature coordinates. The branch input and output have shape $B \times m \times n$. The full persistent state has shape $B \times d \times m \times n$.

H^{pre} reads. F computes. H^{post} writes. H^{res} propagates old state. That four-verb split is more useful than memorizing variant names.

The path through one layer is then:



The exact constraint-specific transformation differs. Unconstrained Hyper-Connections use no Birkhoff projection. Original mHC uses finite Sinkhorn normalization. mHC-lite uses convex combinations of permutation matrices. KromHC uses Kronecker-structured factors. go-mHC uses Cayley plus generalized orthostochastic block norms.

What the papers actually measured

[Experiment] The original Hyper-Connections paper reports improvements over residual connections in language-model pretraining and vision experiments [R001].

[Experiment] The mHC paper reports that constraining residual mixing improves stability and scalability in its evaluated regimes [R002].

[**Experiment**] mHC-lite and KromHC report exact-by-construction alternatives with favorable results in their tested regimes [R003]; [R004].

[**Experiment**] The go-mHC paper reports synthetic leftover-mixing advantages and similar training behavior among exact mHC variants in a 30M-parameter, six-layer GPT-style experiment. It reports up to $10\times$ faster convergence on selected synthetic mixing tasks, wider sampled spectral reach than KromHC, and parity — not established superiority — among exact variants in that small language-model study [R005].

These papers are recent. None establishes a universal law of architecture design.

Two further sentences are still guesses:

- wider spectral reach is useful on large, deep, heterogeneous models;
- distinct leftover copies spontaneously acquire stable semantic jobs.

A heatmap of routing coefficients shows preference. It does not show a job.

More trays is not more width

The leftover-copy count d is a genuine architectural degree of freedom. It changes the number of persistent state channels and the space of learnable depth-wise routes. It is therefore reasonable to study it alongside depth, feature width, expert count, and context length.

It is not interchangeable with any of them.

Increasing depth adds sequential branch applications. Increasing width enlarges feature vectors and usually the expensive branch matrices. Increasing expert count adds alternative parameterized computations, often sparsely selected. Increasing context length enlarges the token axis. Increasing d enlarges persistent state and its routing graph. The standard equation can still evaluate F once per layer.

Useful scaling in d would mean that, under controlled compute and parameter budgets, increasing leftover copies improves a measured capability or efficiency frontier. A proper scaling study must sweep d , retune optimization, include activation memory and router time, and compare against spending the same budget on depth, width, context, or experts. The small go-mHC study makes d plausible as a research axis. It does not establish a scaling law [R005].

Mixing trays is not mixing tokens

When all layers use similar branches and routing stays near identity or gentle averaging, mHC acts primarily as a constrained residual stabilizer. Its main observable role may be preventing extreme propagation while retaining flexible reads and writes.

When different layers or parallel branches perform distinct operations, the same interface begins to look like a learned dataflow substrate: persistent state is routed among computations through depth. That phrase describes topology. It is not evidence of interpretable algorithms.

Possible combinations follow from the interface. They are architectural analyses, not product announcements.

Attention. Let F be self-attention with input and output $B \times m \times n$. Read routing selects a leftover mixture for contextual token interaction; write routing distributes the update. This is the closest to the GPT-style validation in [R005]. It does not make leftover routing into attention. Attention mixes token information. H^{res} mixes leftover copies.

A state-space model plus attention. Alternate F_l^{SSM} and F_l^{attn} , each preserving $B \times m \times n$. **Hypothesis** Leftover copies could preserve different mixtures useful to local recurrence and content-based retrieval. A fair test needs routing probes and interventions, not attractive heatmaps alone.

A WavePDE-like or convolutional operator. Use a sequence or spatial operator as F . **Hypothesis** Constrained leftover paths may carry information at different effective scales while the operator handles within-layer propagation. No result in the five primary Hyper-Connection papers establishes a general WavePDE advantage.

Mixture of experts. Put the experts inside F : that gate selects parameterized experts for tokens, while Hyper-Connection gates route persistent state. The two gates act on different objects and can coexist. Mix-up 7 again, wearing a wiring diagram.

Memory or Engram-like lookup. Let F retrieve from an external or learned lookup memory and write the returned $B \times m \times n$ update into leftover copies. **Hypothesis** Some leftover copies may become convenient carriers of retrieved state. Proving that requires causal ablation of leftover copies and memory reads.

Several different machines in one layer. A proposed extension can use branches F_r , each with its own read/write pair:

$$x_{l+1} = H_l^{\text{res}} x_l + \sum_{r=1}^R (H_{l,r}^{\text{post}})^{\top} F_r(H_{l,r}^{\text{pre}} x_l, W_{l,r}).$$

Here each branch input and output is $B \times m \times n$, and the sum restores shape $B \times d \times m \times n$. **Hypothesis** Attention, state-space, multilayer, and memory branches might learn compositional routes. This multi-branch equation is an architectural extrapolation, not the evaluated go-mHC block in [R005]. It also evaluates R branches, so its cost cannot be blamed on the leftover-copy count with a straight face.

What would count as a job

Stream specialization is a reproducible functional asymmetry among leftover copies: removing, swapping, or perturbing copy i has a systematically different causal effect from doing so to copy j .

Potential roles — local or lexical state, global context, retrieval features, intermediate reasoning state, memory, or modality-specific state — are hypotheses. Evidence should survive leftover-copy permutations, multiple seeds, and matched controls. Router coefficients alone show preference, not semantic function. Because leftover copies may have permutation symmetries, labels such as “stream 2 is memory” can change across training runs while the computation remains equivalent.

If we leave a piece out

Omit persistent leftover copies and the architecture collapses back toward a single residual state. There is no multi-copy route to study.

Omit read/write routing and the branch cannot selectively interface with that state.

Omit leftover routing and each copy mostly carries itself plus writes, reducing learned state-to-state topology.

Omit the constraint in deep unconstrained Hyper-Connections and repeated residual matrices may amplify, decay, or otherwise distort propagation. Learned flexibility alone does not guarantee stability.

Replace the go-mHC block-norm map with raw Cayley output and the router is orthogonal of shape $ds \times ds$, not a nonnegative $d \times d$ leftover mixer.

Omit the learned projection and the exact map still produces legal routers, but they are fixed by whatever coordinates remain.

Most importantly, omit the noun after “exact” and mathematical guarantees quietly mutate into empirical promises. “Rows and columns sum to one” becomes “training is stable,” then “reasoning improves,” then a slide deck acquires a rocket. The first implication is proved. The others require experiments.

Try it

1. Name the axes

For $d = 4$, $B = 8$, $m = 512$, and $n = 1024$, give the shapes of x_l , of $H^{\text{pre}} x_l$, of F ’s output, and of $(H^{\text{post}})^{\top} F(\dots)$.

Answer

x_l is $8 \times 4 \times 512 \times 1024$. Contracting axis 1 gives $8 \times 512 \times 1024$; a shape-preserving F returns the same; the transposed write restores $8 \times 4 \times 512 \times 1024$.

2. Legal is not a rotation

Verify that

$$M = \begin{bmatrix} 0.6 & 0.4 \\ 0.4 & 0.6 \end{bmatrix}$$

is doubly stochastic, then show it is not orthogonal.

Answer

All entries are nonnegative and every row and column sums to one. But

$$MM^\top = \begin{bmatrix} 0.52 & 0.48 \\ 0.48 & 0.52 \end{bmatrix} \neq I,$$

so it is not orthogonal.

3. Count the coordinates

For go-mHC with $d = 8$ and $s = 2$, compute N and the number of independent skew coordinates. State what this count does not prove.

Answer

$N = ds = 16$, so there are $N(N-1)/2 = 120$ coordinates. This does not prove full coverage of $\mathcal{B}_8$, low wall-clock overhead, or better task performance.

4. Label the claims

Label each statement: (a) the block-norm image of an orthogonal Q is doubly stochastic; (b) $s = 2$ is universally optimal; (c) the go-mHC paper found similar small-language-model training loss among exact variants; (d) leftover copies may separate retrieval from lexical state.

Answer

(a) Theorem. (b) Unsupported hypothesis — the paper uses $s = 2$ as an empirical compromise. (c) Empirical result in a limited regime. (d) Hypothesis until causal specialization evidence exists.

5. Design a test that could fail

Propose a minimal test of “leftover-copy count is a scaling dimension.”

Answer

Sweep several d values with matched training data and multiple seeds. Compare at matched parameter, FLOP, wall-clock, and activation-memory budgets against width and depth controls; retune learning rates; report task metrics, stability, router time, and uncertainty. A monotone unbudgeted curve is insufficient.

If you remember three versions

Beginner

A residual network carries information around a layer so the layer can modify rather than replace it. Hyper-Connections carry several persistent versions of that residual state.

At each layer, a read gate blends those leftover copies. One computation — perhaps attention or a multilayer net — processes the blend. A write gate distributes the new result back to the copies, while a leftover router moves old contents forward.

The leftover copies are not old layers. They are parallel containers that live through the whole network. Nor does having several copies automatically mean running several Transformer blocks.

Unrestricted routing matrices can repeatedly enlarge or erase information. mHC restricts the leftover router to a well-behaved family of nonnegative matrices whose rows and columns sum to one.

Different mHC variants construct those matrices differently. go-mHC uses learned numbers, a skew-symmetric matrix, an orthogonal intermediate matrix, and block norms. The intermediate machinery creates the router, not the leftover copies.

The exact mathematical guarantee is narrow but valuable: the leftover router satisfies the double-stochastic constraint. Whether a model trains faster or performs better remains an experimental question.

More leftover copies may give a model more ways to preserve and route intermediate state. Whether they become a reliable new scaling dimension or acquire human-readable jobs is not yet settled.

Intermediate

The persistent state is $x_l \in \mathbb{R}^{B \times d \times m \times n}$. For each batch slice, the layer update is

$$x_{l+1}[b] = H_l^{\text{res}} x_l[b] + (H_l^{\text{post}})^\top F(H_l^{\text{pre}} x_l[b], W_l),$$

with $H^{\text{pre}}, H^{\text{post}} \in \mathbb{R}^{1 \times d}$ and $H^{\text{res}} \in \mathbb{R}^{d \times d}$. The operators read, compute, write, and propagate, respectively.

In go-mHC, a learned projection emits $ds(ds-1)/2$ coordinates, which define skew $A_l \in \mathbb{R}^{ds \times ds}$. Cayley produces orthogonal Q_l ; after partitioning into $s \times s$ blocks,

$$(H_l^{\text{res}})_{ij} = \|Q_{l,ij}\|_F^2/s.$$

This is doubly stochastic by construction. Fixed finite s controls a representable subset, not necessarily all of $\mathcal{B}_d$. Increasing d expands state and routing but also activation memory, projection output, and solve cost. It is a distinct axis from width, depth, context, and experts.

Expert

go-mHC composes an activation-dependent coordinate map with a Lie-algebra parameterization, a Cayley chart into the special-orthogonal component, and the generalized orthostochastic map $\Phi_{d,s}$ into $\mathcal{B}_d$. This turns constrained optimization over residual mixing into unconstrained learning of coordinates followed by an exact feasible map.

The image geometry matters. Double stochasticity fixes a stationary all-ones mode and bounds induced one- and infinity-norm amplification for the residual matrices, but it does not make dynamic products orthogonal or settle nonnormal branch-coupled gradients. At $s = 1$, generalized orthostochastic reach is a strict subset of $\mathcal{B}_d$ for $d \geq 3$. Larger finite s can add reach, but full finite- s coverage is not established here and adjacent images need not be nested. Spectral reach is a useful diagnostic, not a complete expressivity metric.

Viewed architecturally, pre/post/res routing creates a learnable graph over persistent state and branch computations. With homogeneous branches it may function chiefly as residual stabilization. With causally verified heterogeneous routes it could become a learned dataflow substrate. Current evidence supports construction exactness and limited-regime performance claims more strongly than scaling laws, semantic specialization, or broad superiority.

How to read the paper

A checklist for arXiv:2604.02309v1. If a paper survives these questions, you understand more than its diagram. If it does not, you have found the next experiment rather than a reason to stop reading.

1. **Version and scope.** Is the claim from v1, a revision, code, or a later independent reproduction? The cited experiments are synthetic and small-model-heavy.
2. **Claim label.** Is a sentence a definition, theorem, derivation, paper-reported empirical result, interpretation, or hypothesis?
3. **Object and shape.** Does “routing” mean H^{pre} , H^{post} , or H^{res} ? Which axes are dynamic, and what are all leading batch and token axes in code?
4. **Exactness versus coverage.** Does “exact” mean every output lies in $\mathcal{B}_d$, or that every point of $\mathcal{B}_d$ is reachable? At finite s , do not exchange these claims.

5. **Cayley range.** Which orthogonal component and matrices are covered by the chosen Cayley chart? Why do block squared norms make some sign and reflection distinctions irrelevant?
6. **Numerics.** Is orthogonality obtained with a solve rather than an explicit inverse? What row and column errors appear in the actual dtype and hardware?
7. **Complexity.** Are coordinate projection, batching frequency, activation and channel factors, solve constants, memory traffic, and kernel launch costs included — not just $O(d^3)$?
8. **Baselines.** Are mHC, mHC-lite, and KromHC compared with matched parameters, compute, tuning, and implementation quality?
9. **Expressivity metric.** Spectral coverage is a surrogate. Could two router families have similar spectra but different reachable matrices, Jacobians, or task behavior?
10. **Synthetic task.** How are targets sampled? Does that distribution favor one parameterization’s geometry? Do alternative distributions preserve the conclusion?
11. **Language-model evidence.** A 30M-parameter, six-layer TinyStories-style result cannot by itself establish frontier-scale quality, long-context behavior, or reasoning gains.
12. **Statistics.** How many training seeds are used? Are error bars, stopping rules, and hyperparameter searches comparable? LLM-as-judge trials over generated stories are not a substitute for broad downstream evaluation.
13. **Stability.** Is stability measured by finite training traces, gradient norms, induced norms, or a theorem? Double stochasticity constrains residual routing but not every component of the nonlinear network.
14. **Scaling d .** Is increasing leftover copies compared with matched-budget increases in width, depth, experts, and context? Are memory and wall-clock reported?
15. **Specialization.** Are claims supported by causal leftover-copy interventions and permutation-aware analysis, or only by routing visualizations?
16. **Reproducibility and limits.** Can code reproduce the exact matrices and reported curves? Does the conclusion preserve the paper’s stated limits: small language models, cubic overhead relative to KromHC, and the dense-solve bottleneck?
17. **Terminology.** The Birkhoff polytope has boundaries and corners. Check whether informal “manifold” language is being used where a precise geometric statement is needed.

Check yourself

1. **Show that the leftover copies are not the skew matrix.**
They are axis 1 of $x_l \in \mathbb{R}^{B \times d \times m \times n}$. The skew matrix is an auxiliary coordinate pad used to build H^{res} .
2. **Show that F ran once in the two-tray example, and give x_{l+1} .**
Once. $x_{l+1} = [4, 8]^\top$.
3. **What fails if you treat “rows and columns sum to one” as making H^{res} a rotation?**
The two-tray matrix satisfies the mass constraints but $H^{\text{res}}(H^{\text{res}})^\top \neq I_2$. Mass is not an angle.
4. **What fails if you promote go-mHC’s exactness to finite- s coverage of $\mathcal{B}_d$?**
Every constructed H^{res} is doubly stochastic in exact arithmetic. Finite s need not cover all of $\mathcal{B}_d$.
5. **Show that leftover routing is not the expert gate.**
Leftover routing mixes persistent state. Expert routing assigns tokens to parameterized branches. Different objects, different bills.
6. **What fails if you call leftover-copy count a scaling dimension from an unbudgeted curve?**
You would need a sweep of d at matched parameter, FLOP, wall-clock, and activation-memory budgets, against width and depth controls, with retuned optimization.

The ten mix-ups all name the wrong object. Once the object is right, the remaining questions are ordinary: what was proved, what was measured, and what would count as a job.

Sources

- [R001] D. Zhu et al., “Hyper-Connections,” arXiv:2409.19606, 2024. <https://arxiv.org/abs/2409.19606>

-
- [R002] Z. Xie et al., “mHC: Manifold-Constrained Hyper-Connections,” arXiv:2512.24880, 2025. <https://arxiv.org/abs/2512.24880>
 - [R003] Y. Yang and J. Gao, “mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations,” arXiv:2601.05732, 2026. <https://arxiv.org/abs/2601.05732>
 - [R004] W. Zhou et al., “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices,” arXiv:2601.21579, 2026. <https://arxiv.org/abs/2601.21579>
 - [R005] T. Dandachi and S. Diggs-Galligan, “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices,” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309>
 - [R029] I. Nechita, Z. Ouyang, and A. Szczepanek, “Generalized Unistochastic Matrices,” arXiv:2310.03436, 2023. <https://arxiv.org/abs/2310.03436>

Glossary and notation guide

A memory aid, not a prerequisite. Terms appear in the order you meet them. If Chapter 1 has not happened yet, you do not need “stream.”

Shapes live in the chapters that introduce them. A row of the token table is never a leftover copy of the whole sentence.

Activation (*first developed: Chapter 1*) — A numerical representation produced while a network processes an input. $X \in \mathbb{R}^{(m \times n)}$ is an activation containing m token positions and n features per position. Activations are data-dependent; parameters are learned values retained between examples.

Axis (*first developed: Chapter 1*) — One independent indexing direction of a tensor. In these notes, batch, stream, token, and feature are different axes even when two happen to have the same numerical length. Canonical batched residual-state order is $B \times d \times m \times n$.

Batch / batch axis (*first developed: Chapter 1*) — A collection of B examples evaluated together. A leading batch dimension changes $X: m \times n$ to $X: B \times m \times n$ and $x_1: d \times m \times n$ to $x_1: B \times d \times m \times n$. It does not itself determine whether routers are shared across examples.

Broadcasting (*first developed: Chapter 1*) — Applying a value over axes it does not explicitly store. In $(H_1^{\text{post}})^T y_1$, each of the d write coefficients multiplies all $m \times n$ token-feature entries of y_1 . Broadcasting is convenient but dangerous when an unnamed axis is mistaken for the stream axis.

Computational branch F (*first developed: Chapter 1; HC use: Chapter 4*) — The learned transformation between read and write routing. With the book’s simplified shape contract, $F(\cdot, W_1): \mathbb{R}^{(m \times n)} \rightarrow \mathbb{R}^{(m \times n)}$ per example. It may be attention, an MLP, an SSM, a convolution, WavePDE, or another block. One central HC update contains one evaluation of F ; d streams do not imply d evaluations.

Feature / hidden feature (*first developed: Chapter 1*) — One coordinate in a token representation. The feature axis has length n . It is not the stream axis: several persistent streams can each contain the same n feature coordinates.

Feature width n (*first developed: Chapter 1*) — Number of features per token in one stream. Thus $X \in \mathbb{R}^{(m \times n)}$ and $x_1(i) \in \mathbb{R}^{(m \times n)}$.

Layer (*first developed: Chapter 1*) — One indexed stage of the network. l names the current layer and L the total layer count. A layer may read and update every persistent stream even though it evaluates one computational branch.

Layer history (*first distinguished: Chapter 3*) — The ordered record $x_0, x_1, \dots$ of states at different depths. It is indexed by l and must not be stored conceptually in the stream index i . At a fixed layer l , all $x_1(i)$ are current persistent states.

Parameter W_1 (*first developed: Chapter 1*) — A learned value associated with a layer. In $F(x, W_1)$, W_1 denotes branch parameters and its shape is branch-dependent. Router projection weights are written $W_1^{\text{route}} \in \mathbb{R}^{(r \times q)}$ when the distinction matters.

Sequence length m (*first developed: Chapter 1*) — Number of token or spatial positions represented in the sequence axis. It is independent of feature width n , stream count d , and layer count L .

Shape (*first developed: Chapter 1*) — The ordered lengths of a tensor’s axes. Shape equality is required before the persistent term and write term can be added: both have unbatched shape $d \times m \times n$.

Tensor (*first developed: Chapter 1*) — A multidimensional array with named semantic axes. A matrix is a rank-two tensor; the unbatched residual state $x_{-1} \in \mathbb{R}^{(d \times m \times n)}$ is rank three.

Token / token row (*first developed: Chapter 1*) — One sequence position and its n -dimensional representation. A token row $x_{-t} \in \mathbb{R}^n$ is not a residual stream. A stream contains all m token rows.

Ordinary layer (*first developed: Chapter 1*) — A transformation written $y = F(x, W)$. It replaces its input in the simplest feed-forward picture, before an identity/residual route is added.

Residual connection (*first developed: Chapter 2*) — An update $x_{-(l+1)} = x_{-l} + F(x_{-l}, W_{-l})$ that preserves an additive identity path while allowing the branch to modify the representation. Residual connections help optimization and representation preservation, but do not guarantee stability at arbitrary scales.

Identity path / gradient highway (*first developed: Chapter 2*) — The direct additive path from x_{-l} to $x_{-(l+1)}$ whose local derivative includes an identity contribution. “Highway” is an optimization interpretation, not a guarantee that all gradients remain bounded through any network.

Residual state (*first developed: Chapter 2; multi-stream form: Chapter 3*) — The persistent activation carried from one layer to the next. In an ordinary residual network it is typically one $m \times n$ array; in HC it is d such streams assembled as $x_{-l} \in \mathbb{R}^{(d \times m \times n)}$.

Persistent residual stream (*first developed: Chapter 3*) — One state slot $x_{-l}^{(i)} \in \mathbb{R}^{(m \times n)}$ that persists and is routed across depth. It is not a previous layer, token, feature channel, attention head, expert, or separate copy of F .

Stream count d (*first developed: Chapter 3*) — Number of persistent residual streams. It is the first axis of unbatched $x_{-l} \in \mathbb{R}^{(d \times m \times n)}$ and the side length of $H_{-l}^{res} \in \mathbb{R}^{(d \times d)}$. Whether increasing d is a useful scaling direction is an empirical and architectural question, not a theorem.

Copied-stream initialization (*first developed: Chapter 3*) — Initialization $x_{-0}^{(1)} = \dots = x_{-0}^{(d)} = X$. The streams initially contain the same values but are distinct state slots. Unequal writes and subsequent routing can make them diverge.

Hyper-Connections (HC) (*first developed: Chapter 3*) — The architecture family that carries multiple persistent residual streams and learns read, write, and state-to-state routing around a computational branch. In the family comparison, “HC” refers to the original unconstrained/general Hyper-Connections design; mHC-family variants add particular structure to residual routing.

Static Hyper-Connections (SHC) (*first developed: Chapter 3; compared: Chapter 15*) — The original HC paper’s input-independent form: learned routing coefficients are parameters shared across examples at a layer rather than functions of the current activation. “Static” describes router dependence, not frozen training; the coefficients themselves are learned.

Dynamic Hyper-Connections (DHC) (*first developed: Chapter 3; compared: Chapter 15*) — The original HC paper’s input-dependent form, which derives routing values from normalized/reduced activations using learned projections, gains, nonlinearities, and biases specified by that design. DHC is an HC subvariant, not another name for doubly stochastic mHC. $DHC \times d$ denotes expansion to d persistent streams, not d branch evaluations.

Expansion rate (*first developed: Chapter 3*) — The original HC paper’s term for the number of persistent streams. That paper and these notes reuse letters differently: these notes reserve d for expansion/stream count and n for feature width. Always translate the role of an axis, not merely its letter.

Depth-connection / width-connection (*first developed: Chapter 3*) — Terms used by the original HC paper for learned connectivity across layer depth and across the expanded persistent width/stream structure. In these notes the concrete, less ambiguous operators are read routing H^{pre} , write routing H^{post} , and persistent-state routing H^{res} ; the historical terms do not turn the stream axis into layer history.

Read routing H_{-l}^{pre} (*first developed: Chapter 4*) — A soft read operator of unbatched shape $1 \times d$. It contracts the stream axis:

$$(z_l)_{tf} = \sum_j (H_l^{pre})_{1j} (x_l)_{jtf}, \quad z_l \in \mathbb{R}^{m \times n}.$$

It selects a mixture of persistent states for the current branch without mixing tokens or features.

Branch input z_1 (*first developed: Chapter 4*) — The $m \times n$ activation read from streams, $z_1 = H_1^{pre} x_1$, before computation by F .

Branch output y_1 (*first developed: Chapter 4*) — The $m \times n$ activation $y_1 = F(z_1, W_1)$ that is distributed back to streams by write routing.

Write routing H_1^{post} (*first developed: Chapter 4*) — A soft write operator of unbatched shape $1 \times d$. Its transpose has shape $d \times 1$, and $(H_1^{post})^T y_1 \in \mathbb{R}^{(d \times m \times n)}$ gives destination stream i the scaled write $(H_1^{post})_{(1i)} y_1$.

Persistent-state routing H_1^{res} (*first developed: Chapter 4*) — The $d \times d$ router that transports old stream states into new stream slots:

$$(H_l^{res} x_l)_{itf} = \sum_j (H_l^{res})_{ij} (x_l)_{jtf}.$$

Row i describes destination stream i ; column j refers to source stream j . H_1^{res} is also abbreviated R_1 in path expansions.

Read–compute–write–persist update (*first developed: Chapter 4*) — The staged meaning of

$$x_{l+1} = H_l^{res} x_l + (H_l^{post})^T F(H_l^{pre} x_l, W_l).$$

Read and compute produce one $m \times n$ branch output; write and persist each produce $d \times m \times n$; the final addition is therefore shape-valid.

Path through depth (*first developed: Chapter 5*) — One algebraic route by which an earlier state or branch output reaches a later state through products of $R_1 = H_1^{res}$ and writes $p_1 = (H_1^{post})^T$. It is an interpretation of terms in the expansion, not necessarily a discrete path selected by the model.

Stream divergence (*first developed: Chapter 5*) — The process by which initially equal stream values become unequal, often beginning with unequal write coefficients. Only after divergence can a read mixture distinguish stream contents meaningfully.

Level 2 — Routers, constraints, and HC variants

Router / routing operator (*first developed: Chapter 4; generated routers: Chapter 6*) — A matrix or vector that moves information among persistent streams or between streams and the branch. It is an activation-dependent numerical operator when dynamically generated, not the projection parameters that generated it.

Learned router projection (*first developed: Chapter 6*) — A usually small learned linear map from an activation summary $\hat{x}_1 \in \mathbb{R}^r$ to coordinates $u_1 \in \mathbb{R}^q$, schematically $u_1 = \hat{x}_1 W_1^{route} + b_1^{route}$. Its weights, bias, and gain are trained by backpropagation. It does not alone enforce stochasticity, orthogonality, or another routing constraint.

Activation summary $\hat{x}_1$ (*first developed: Chapter 6*) — A normalized and/or reduced representation of the current activation used as router input. Its width r and reduction axes are implementation-specific; reduction over tokens must not be assumed unless the cited implementation does it.

Routing coordinates / routing logits u_1 (*first developed: Chapter 6*) — Unconstrained real values emitted by the learned router projection. Shape is q per routed item. A subsequent constraint map interprets them as read/write gates, matrix logits, skew coordinates, or mixture weights.

Router gain γ_l and bias b_l^{route} (*first developed: Chapter 6*) — Trainable scale and offset applied in the router projection. γ_l may be scalar or broadcastable; $b_l^{\text{route}} \in \mathbb{R}^q$. Their exact initialization and granularity belong to the cited implementation.

Constraint map / constraint mechanism (*first developed: Chapter 6*) — A deterministic differentiable transformation from unconstrained coordinates to a router in a desired set. Examples are finite Sinkhorn normalization, a softmax-weighted permutation mixture, Kronecker composition of constrained factors, and the go-mHC Cayley/block-norm map. The constraint map has no learned parameters unless parameters are explicitly added; learning occurs through coordinates fed into it.

Dynamic routing (*first developed: Chapter 6*) — Routing whose numerical values depend on current activations. “Dynamic” does not specify whether the router varies per token, sequence, batch item, or layer; that is the routing granularity.

Routing granularity (*first developed: Chapter 6*) — The axes over which one router is shared or varies: per layer, per batch item/sequence, or per token, for example. Shapes and primary implementations determine granularity. The phrase “input-dependent” alone does not imply token-wise routing.

Gate / soft gate (*first developed: Chapter 4; formal distinction: Chapter 6*) — An interpretation of coefficients that continuously weight reads or writes. A soft gate need not be a probability vector unless a normalization imposes that condition. HC gating should not be conflated with routing among MoE experts.

Unconstrained residual mixing (*first developed: Chapter 7*) — Repeated application of H_l^{res} without a structural constraint designed to control its action. Products can amplify, suppress, alternate, or rotate components. “Unconstrained” describes the operator class, not an assertion that training must fail.

Stochastic matrix (*first developed: Chapter 8*) — In these notes, a nonnegative square matrix with one specified set of sums equal to one. Always say row-stochastic or column-stochastic when only one condition is intended.

Row-stochastic matrix (*first developed: Chapter 8*) — A matrix $B \in \mathbb{R}^{(d \times d)}$ satisfying $B_{ij} \geq 0$ and $B \mathbf{1}_d = \mathbf{1}_d$; every row sums to one.

Column-stochastic matrix (*first developed: Chapter 8*) — A matrix satisfying $B_{ij} \geq 0$ and $B^T \mathbf{1}_d = \mathbf{1}_d$; every column sums to one.

Doubly stochastic matrix (*first developed: Chapter 8*) — A matrix $B \in \mathbb{R}^{(d \times d)}$ satisfying nonnegativity plus both unit row sums and unit column sums. It preserves the all-ones mode and total mass under the matching convention. It need not be orthogonal, symmetric, invertible, or immune to information mixing.

All-ones vector $\mathbf{1}_d$ (*first developed: Chapter 8*) — The length- d column vector of ones. Doubly stochasticity is compactly written $B \mathbf{1}_d = \mathbf{1}_d$ and $B^T \mathbf{1}_d = \mathbf{1}_d$.

Convex combination (*first developed: Chapter 8*) — A weighted sum $\sum_k \gamma_k V_k$ with $\gamma_k \geq 0$ and $\sum_k \gamma_k = 1$. It stays in the convex hull of the objects V_k .

Convex hull (*first developed: Chapter 8*) — The set of all convex combinations of a collection of points. The Birkhoff polytope is the convex hull of $d \times d$ permutation matrices.

Polytope (*first developed: Chapter 8*) — A bounded geometric set described as the convex hull of finitely many vertices, equivalently by finitely many linear constraints in this setting.

Birkhoff polytope $\mathcal{B}_d$ (*first developed: Chapter 8*) — The set of all $d \times d$ doubly stochastic matrices, a subset of $\mathbb{R}^{(d \times d)}$. Exact membership is different from the ability of a parameterization to reach every point in the set.

mHC (original matrix-constrained Hyper-Connections) (*first developed: Chapter 9*) — The HC variant that constrains residual mixing toward the Birkhoff polytope using the paper’s matrix normalization mechanism, presented here through finite Sinkhorn normalization. “mHC” is also used informally for the broader family; when a comparison could be ambiguous, these notes say **original mHC** or **mHC family**.

Matrix logits Z_l (*first developed: Chapter 9*) — An unconstrained $d \times d$ real matrix fed to a matrix constraint procedure. Positivity is obtained before alternating row/column normalizations in the Sinkhorn construction.

Sinkhorn normalization / Sinkhorn iteration (*first developed: Chapter 9*) — Repeated positive row and column normalization intended to approach a doubly stochastic matrix. In ordinary finite-precision code with finitely many iterations, unit sums are approximate within a measured tolerance, not algebraically exact.

Constraint residual / constraint error (*first developed: Chapter 9*) — A numerical measure of failed membership, such as $\max_i |\Sigma_j H_{ij} - 1|$, $\max_j |\Sigma_i H_{ij} - 1|$, and the magnitude of negative entries. It measures the routing constraint, not task loss or model quality.

Finite-iteration exactness (*first distinguished: Chapter 9*) — The question of whether the implemented finite computation satisfies a constraint exactly in exact arithmetic. Finite Sinkhorn iterations generally leave residual row/column error; an algebraic construction can satisfy the conditions by construction, while floating-point evaluation still has roundoff.

Permutation matrix P (*first developed: Chapter 10*) — A $d \times d$ binary matrix with one 1 in every row and column. It reorders streams, is doubly stochastic and orthogonal, and is a vertex of $\mathcal{B}_d$.

Birkhoff–von Neumann theorem (*first developed: Chapter 10*) — Every doubly stochastic matrix is a convex combination of permutation matrices. The theorem establishes existence; a parameterization that explicitly assigns a weight to all $d!$ permutations may be computationally impractical.

mHC-lite (*first developed: Chapter 10*) — The HC variant presented through exact convex mixtures of permutation routers. Nonnegative normalized mixture weights make the result exactly doubly stochastic in exact arithmetic. Full enumeration has a factorial burden ($d!$ candidates), so exactness does not imply scalable representation.

Softmax (*first developed: Chapter 10*) — A map from real logits to strictly positive weights summing to one. Applied over permutation logits, it supplies convex-mixture weights. With finite logits every enumerated weight is positive; an exact zero or a permutation vertex is reached only as a limit unless the implementation adds another mechanism.

Permutation bank (*first developed: Chapter 10*) — A stored or generated tensor of candidate permutation matrices, typically $M \times d \times d$ for M candidates. Combining routing logits of shape $\mathcal{G} \times M$ with that bank produces routers of shape $\mathcal{G} \times d \times d$; the batch/token axes in $\mathcal{G}$ are not permuted. For full mHC-lite enumeration, $M=d!$; K remains reserved for the KromHC factor count.

Factorial burden $d!$ (*first developed: Chapter 10*) — Growth in the number of permutations: $4!=24$, $8!=40,320$, and $16! \approx 2.09 \times 10^{13}$. It is a representation/parameterization burden, not a claim that every algorithm for a doubly stochastic matrix must enumerate all permutations.

Kronecker product $A \otimes B$ (*first developed: Chapter 11*) — A block matrix formed by replacing entry A_{ij} with the scaled block $A_{ij} B$. If A is $r \times c$ and B is $u \times v$, then $A \otimes B$ is $(ru) \times (cv)$. Eigenvalues of square factors combine pairwise as products.

KromHC (*first developed: Chapter 11*) — The HC variant whose residual router is built from a Kronecker product of smaller constrained factors. The structure can reduce construction cost and parameter burden, but restricts which full routers and spectra can be represented. Cost and expressivity must be compared for a named factorization and regime.

KromHC factorization (*first developed: Chapter 11*) — Choose K factor sizes $i_1, \dots, i_K \geq 2$ with $\prod_k i_k = d$, construct doubly stochastic factors $U_{1 \sim k} \in \mathbb{R}^{i_k \times i_k}$, and set $H_{1 \sim res} = U_{1 \sim K} \otimes \dots \otimes U_{1 \sim 1} \in \mathbb{R}^{(d \times d)}$. K counts factors; it is not the number of branches or layers.

Exact constraint satisfaction (*first developed: Chapter 10; central distinction: Chapter 14*) — Every output of a parameterization lies in the required set in exact arithmetic. It says nothing by itself about full coverage, optimization quality, floating-point tolerance, or predictive performance.

Set coverage / expressivity of a router map (*first developed: Chapter 10; central distinction: Chapter 14*) — The subset of valid routing matrices reachable by a parameterization. A map can be exact but cover only a strict or otherwise limited subset of $\mathcal{B}_d$.

Level 3 — The go-mHC construction

go-mHC (*first developed: Chapter 12*) — Generalized-orthostochastic mHC. A learned projection fills a skew-symmetric $N \times N$ matrix ($N=ds$); a Cayley transform produces an orthogonal matrix; squared Frobenius norms of its $s \times s$ blocks produce $H_1^{\text{res}} \in \mathcal{B}_d$. The machinery constructs a router between already-existing streams.

Block size s (*first developed: Chapter 12; interpreted: Chapter 14*) — Side length of each auxiliary block. With d streams, the auxiliary matrix side is $N=ds$; Q is partitioned into a $d \times d$ grid of $s \times s$ blocks. s is not sequence length, stream count, or feature width.

Auxiliary dimension $N=ds$ (*first developed: Chapter 12*) — Side length of $A_1, Q_1 \in \mathbb{R}^{(N \times N)}$. For $d=4, s=2, N=8$.

Skew-symmetric matrix A_1 (*first developed: Chapter 12*) — A square matrix satisfying $A_1^T = -A_1$. Its diagonal is zero and its lower triangle is fixed by its upper triangle. An $N \times N$ skew matrix has $p=N(N-1)/2$ independent real entries.

Free skew coordinates $p=N(N-1)/2$ (*first developed: Chapter 12*) — Number of independent strict-upper-triangle entries in A_1 . For $N=8, p=28$. The router projection emits these coordinates; they are not d stream tensors.

Identity matrix I_N (*first developed: Chapter 2; Cayley use: Chapter 12*) — The $N \times N$ matrix with ones on the diagonal and zeros elsewhere. It leaves an N -vector unchanged and appears in $Q = (I_N - A)(I_N + A)^{-1}$.

Transpose M^T (*first developed: Chapter 1*) — The matrix obtained by exchanging rows and columns. In the central equation it turns $H^{\text{post}}:1 \times d$ into $d \times 1$; in $A^T = -A$ it defines skew symmetry; in $Q^T Q = I$ it defines orthogonality.

Cayley transform (*first developed: Chapter 12*) — The rational map

$$Q = (I_N - A)(I_N + A)^{-1}$$

from a real skew-symmetric A to an orthogonal Q when the solve is defined. Implementations use a linear solve, not a materialized inverse. This transform deterministically maps coordinates; it does not learn without learned coordinates upstream.

Cayley -1 eigenvalue caveat (*first developed: Chapter 12*) — The standard Cayley chart does not cover orthogonal matrices having -1 as an eigenvalue. With the stated convention it lies in the identity/determinant- $+1$ component. Therefore “orthogonal output” must not be upgraded to “global parameterization of all $O(N)$.”

Orthogonal matrix Q_1 (*first developed: Chapter 12*) — A real $N \times N$ matrix satisfying $Q_1^T Q_1 = Q_1 Q_1^T = I_N$. Its rows and columns are orthonormal. Orthogonality is an auxiliary property used to prove sums of block energies; H_1^{res} itself is generally not orthogonal.

Orthogonal group $O(N)$ (*first developed formally: Chapter 18; first used: Chapter 12*) — The set of all real $N \times N$ orthogonal matrices under multiplication. It has determinant $+1$ and -1 components.

Special orthogonal group $SO(N)$ (*first developed formally: Chapter 18; first used: Chapter 12*) — The determinant- $+1$ component/subgroup of $O(N)$. Cayley transforms of finite real skew coordinates lie in this component, subject to the chart caveat.

Lie algebra $\mathfrak{so}(N)$ (*first developed intuitively: Chapter 12; formalized: Chapter 18*) — The vector space of real $N \times N$ skew-symmetric matrices, interpreted as tangent directions at the identity of $SO(N)$. It has dimension $N(N-1)/2$.

Block partition Q_{ij} (*first developed: Chapter 13*) — The decomposition of $Q \in \mathbb{R}^{(ds) \times (ds)}$ into a $d \times d$ grid of blocks $Q_{ij} \in \mathbb{R}^{(s \times s)}$. i indexes a destination block row and j a source block column.

Frobenius norm $\|M\|_F$ (*first developed: Chapter 13*) — The square root of the sum of squared entries, $\|M\|_F^2 = \sum (a, c) M_{ac}^2$. For an $s \times s$ block, it measures block energy and is always nonnegative after squaring.

Trace $\text{tr}(M)$ (*first developed: Chapter 13*) — Sum of a square matrix’s diagonal entries. The identity $\text{tr}(Q_{ij} Q_{ij}^T) = \|Q_{ij}\|_F^2$ turns block orthogonality equations into row/column energy sums.

Block-norm router (*first developed: Chapter 13*) — The go-mHC residual router defined by

$$(H_l^{res})_{ij} = \frac{1}{s} \|Q_{l,ij}\|_F^2.$$

Squared norms give nonnegativity. Orthogonality gives $\sum_j \|Q_{ij}\|_F^2 = s$ and $\sum_i \|Q_{ij}\|_F^2 = s$, hence unit row and column sums after division by s .

Exact by construction (*first developed: Chapter 13*) — A property that follows algebraically from how an object is formed rather than from an iterative penalty. For the ideal go-mHC map, nonnegativity and both unit-sum conditions follow from orthogonality. Floating-point code should still test tolerances.

Orthostochastic matrix (*first developed: Chapter 14*) — A doubly stochastic matrix whose entries are squared entries of an orthogonal matrix: $H_{ij} = Q_{ij}^2$. This is the $s=1$ case of the generalized block-norm construction. Orthostochastic matrices generally form only part of the Birkhoff polytope.

Generalized orthostochastic map $\Phi_{d,s}$ (*first developed: Chapter 14*) — The map

$$\Phi_{d,s} : O(ds) \rightarrow \mathcal{B}_d, \quad [\Phi_{d,s}(Q)]_{ij} = \frac{\|Q_{ij}\|_F^2}{s}.$$

Its image consists of routers reachable at the chosen finite block size. Membership of every output in $\mathcal{B}_d$ is proven; full surjectivity onto $\mathcal{B}_d$ is a separate question and must not be assumed.

Image of a map (*first developed: Chapter 14*) — The set of all outputs the map can produce. $\text{im } \Phi_{d,s} = \{\Phi_{d,s}(Q) : Q \in O(ds)\}$ is the actual go-mHC routing family before restrictions introduced by the Cayley chart or learned projection are considered.

Surjective / surjectivity (*first developed: Chapter 14*) — A map is surjective onto its target when every target point has at least one preimage. Exact doubly stochastic output does not prove that $\Phi_{d,s}$ is surjective onto $\mathcal{B}_d$ for a selected finite s .

Finite-s limitation (*first developed: Chapter 14*) — Any restriction associated with choosing a finite auxiliary block size, including limited image coverage and Cayley-coordinate reach. Increasing s can enlarge the parameterization's possibilities, but $s=2$ being useful is a paper-reported empirical design compromise, not a theorem of optimality.

Level 4 — Routing interpretations and spectral dynamics

mHC family (*first compared: Chapter 15*) — Collective name for HC, original mHC, mHC-lite, KromHC, and go-mHC when discussing persistent-stream routing. Each variant must still be named when exactness, coverage, cost, or evidence differs.

Learned dataflow (*first developed: Chapter 16*) — The interpretation that dynamic read, write, and persistent-state routers choose how information moves through state slots and depth. It becomes a stronger architectural claim when combined with heterogeneous branches; measured specialization is required before calling a particular latent computation graph an empirical fact.

Mixture of Experts (MoE) (*first compared: Chapter 16*) — An architecture that routes examples or tokens among parameterized expert computations. mHC primarily routes information among persistent state streams around a branch. Both use gates, but their routed objects and computational consequences differ.

Heterogeneous branch (*first developed: Chapter 16*) — One of several computational branches with different operators, such as attention, an SSM, WavePDE, or an MLP. Combining heterogeneous branches with HC routing is an architectural extension; it is not implied by the single-branch central equation.

Stream specialization (*first discussed: Chapter 16*) — A possible tendency for streams to carry different kinds of information. Unless a paper directly measures it with an appropriate protocol, specialization is a **Hypothesis** or **Interpretation**, not an architectural guarantee or empirical result.

Eigenvector and eigenvalue (*first developed: Chapter 7*) — For square matrix M , a nonzero vector v and scalar λ satisfying $Mv = \lambda v$. Repeating a fixed M scales that mode as λ^L : magnitude above one grows, below one decays, negative values alternate sign, and nonreal values rotate as well as scale.

Spectrum $\text{spec}(M)$ (*first developed: Chapter 7*) — The set or multiset of eigenvalues of M . For $A \otimes B$, eigenvalues are pairwise products $\lambda_i(A) \lambda_j(B)$.

Spectral radius $\rho(M)$ (*first developed: Chapter 7*) — $\max_{\lambda \in \text{spec}(M)} |\lambda|$. It is useful intuition for powers of one fixed matrix, but does not fully characterize transient growth or products of changing, nonnormal routers.

Nonnormal matrix (*first cautioned: Chapter 7; revisited: Chapter 17*) — A matrix that does not commute with its transpose/conjugate transpose. Its eigenvalues alone can miss large transient amplification. Dynamic products $H_{-1}^L \text{res} \cdots H_0^L \text{res}$ require more than a spectral-radius slogan.

Stationary mode (*first introduced: Chapter 8; developed: Chapter 17*) — A direction preserved with eigenvalue 1. A doubly stochastic matrix preserves the all-ones vector; additional stationary modes depend on reducibility and structure.

Mixing (*first developed: Chapter 17*) — Redistribution that reduces differences among stream values under repeated routing. Mixing can be desirable for controlled sharing yet can also erase distinctions; doubly stochasticity does not specify a mixing rate.

Diffusion (*first developed: Chapter 17*) — The dynamical interpretation of repeated local averaging or redistribution. It describes behavior of a chosen router/product, not every doubly stochastic matrix: permutation routers cycle without averaging.

Cycle / permutation cycle (*first developed: Chapter 17*) — A routing pattern that sends a sequence of stream indices around a closed loop. A length- k permutation cycle has eigenvalues among the k th roots of unity and can sustain oscillatory state movement.

Complex eigenvalue (*first introduced: Chapter 7; developed: Chapter 17*) — A nonreal eigenvalue $\lambda = re^{i\theta}$ whose magnitude controls scaling and phase controls rotation/oscillation in an invariant real plane. Real stochastic matrices may have complex-conjugate eigenvalue pairs.

Root of unity (*first developed: Chapter 17*) — A complex number λ satisfying $\lambda^k = 1$ for some positive integer k . Eigenvalues of permutation cycles are roots of unity.

Karpelevič region (*first developed: Chapter 17*) — The classical feasible region in the complex plane for eigenvalues of finite stochastic matrices of a given size. It supplies context for spectral reach; occupying more of that region is not by itself evidence of better task performance.

Spectral reach / spectral richness (*first developed: Chapter 17*) — The set or diversity of eigenvalue patterns a router parameterization can realize. Claims that KromHC is restricted or go-mHC is richer must retain the paper's matrix size, parameterization, and evidence scope.

Level 5 — Geometry, optimization, and implementation

Manifold (*first developed: Chapter 18*) — A space that looks locally like Euclidean space even if it has a curved global structure. $O(N)$ and $SO(N)$ are matrix manifolds defined by orthogonality constraints.

Tangent space (*first developed: Chapter 18*) — The linear space of infinitesimal directions at a point on a manifold. At the identity of $SO(N)$, tangent directions are skew-symmetric matrices in $\mathfrak{so}(N)$.

Local chart / chart-like parameterization (*first developed: Chapter 18*) — Coordinates that describe a neighborhood of a manifold point. The Cayley transform is chart-like around the identity but misses matrices with eigenvalue -1 , so it is not a global chart of all $O(N)$.

Dimension (*first developed: Chapter 18*) — Number of local degrees of freedom. $\mathfrak{so}(N)$ and $SO(N)$ have dimension $N(N-1)/2$. Dimension counting can rule out some surjectivity claims, but matching dimensions alone does not prove surjectivity.

Optimization geometry (*first developed: Chapter 18*) — How a parameterization's coordinates, curvature,

symmetries, singularities, and gradients shape optimization. A plausible geometric explanation for training behavior remains an **Interpretation** or **Hypothesis** unless evidence isolates it.

Differentiability (*first discussed: Chapter 9; implementation: Chapter 19*) — Existence of derivatives needed to propagate loss gradients through router generation. Sinkhorn steps, linear solves away from singularity, block reshapes, and squared norms are differentiable operations in standard autodiff systems; differentiability does not ensure well-conditioned gradients.

Automatic differentiation / autodiff (*first developed: Chapter 19*) — Library machinery that computes derivatives through recorded tensor operations. Correct gradients require using supported solves and shape operations without unintended detachments or in-place mutations.

Normalize and reduce (*first developed: Chapter 6; implementation: Chapter 19*) — Produce compact router input $\hat{x}_1$ by normalization and aggregation over explicitly named axes. The operation must state whether it reduces tokens, features, streams, or batch items; otherwise routing granularity is ambiguous.

Batched router construction (*first developed: Chapter 19*) — Building one or more routers with a leading batch-like axis, for example $A, Q: B \times N \times N$ and $H: B \times d \times d$. It should use batched linear solves and reductions rather than serial loops over examples when production efficiency matters.

Block reshape (*first developed: Chapter 19*) — A view/rearrangement from $Q: B \times (ds) \times (ds)$ to a layout exposing $d \times d$ blocks of $s \times s$, such as $B \times d \times s \times d \times s$, followed by a deliberate permutation before reducing the two within-block axes. A reshape alone may interleave the wrong axes.

Linear solve (*first developed: Chapter 12; implementation: Chapter 19*) — Computing X from $MX=R$ without explicitly forming M^{-1} . It is the numerically preferred implementation of the Cayley transform. Left-versus-right multiplication and transpose conventions must reproduce the displayed equation.

Explicit matrix inverse (*first warned: Chapter 12; implementation: Chapter 19*) — Materializing $(I+A)^{-1}$ and multiplying by it. Production code should avoid this in favor of a solve, which is generally more stable and efficient.

Dense solve cost (*first derived: Chapter 19*) — A conventional cubic arithmetic estimate $O(N^3)=O(d^3s^3)$ for a dense $N \times N$ solve, before batching, channels, hardware, and constant factors are accounted for. “Cubic” alone does not determine end-to-end expense relative to the model’s larger matrix multiplications.

Parameter count (*first calculated: Chapter 12; compared: Chapter 19*) — Number of trainable scalar values. The skew-coordinate output width $p=N(N-1)/2$ is not by itself the total router parameter count: a projection from width r also carries approximately $r \cdot p$ weights plus bias/gain under the chosen design.

FLOP count (*first compared: Chapter 15; developed: Chapter 19*) — Approximate number of floating-point arithmetic operations. It is not elapsed time; kernel fusion, memory traffic, batching, precision, and accelerator utilization also matter.

Asymptotic complexity (*first compared: Chapter 15; developed: Chapter 19*) — Growth rate as a size variable increases, suppressing constants and lower-order terms. It must name which variables are fixed: $O(d^3)$ for the dense go-mHC solve assumes fixed s .

Initialization near identity (*first discussed: Chapter 6; go-mHC implementation: Chapter 19*) — Choosing small skew coordinates makes the Cayley output near I_N , whose block-norm image is near the identity stream router. This can preserve an initial residual path; the actual bias/gain recipe must follow the implementation being reproduced.

Numerical stability (*first discussed: Chapter 9; developed: Chapter 19*) — Sensitivity of computed results and gradients to rounding, conditioning, scale, and algorithm choice. It is distinct from training stability, activation stability, or the architectural property of being doubly stochastic.

Numerical tolerance (*first developed: Chapter 9; tested: Chapter 19*) — A finite threshold used when checking equations in floating point. Tests should report dtype-appropriate tolerances for orthogonality, row sums, column sums, and nonnegativity rather than demand bitwise equality.

Orthogonality residual (*first developed: Chapter 19*) — A measure such as $\|Q^T Q - I_N\|$ used to test the numerical Cayley result. Small orthogonality residual supports the constraint calculation but is not a model-quality metric.

Row/column-sum test (*first developed: Chapter 19*) — A unit check that $H1_d \approx 1_d$ and $H^T T1_d \approx 1_d$ within stated tolerance, together with a nonnegativity check. Testing only the average sum can hide a bad row or column; use a maximum or elementwise assertion.

Production kernel (*first developed: Chapter 19*) — An implementation optimized for actual training constraints such as fusion, memory layout, precision, batching, and backward cost. Pedagogical code may be executable and correct without matching production performance.

Custom kernel (*first discussed: Chapter 9; compared: Chapter 19*) — A hardware-specific fused implementation used when composition of general tensor operations is too costly. Its existence or absence affects measured speed, so paper comparisons must not turn kernel maturity into a claim about mathematical complexity alone.

Empirical stability (*first discussed: Chapter 9; compared: Chapter 15*) — Observed behavior such as bounded activations, gradients, losses, or successful training in a named experimental setting. It is a paper-reported empirical result, not a consequence of double stochasticity alone.

Architecture fact (*label introduced: front matter; used throughout*) — A paper-specified component, shape, or transformation. These notes label it **Definition** and cite the primary architecture source where needed. An architecture fact states what was built, not whether it works better.

Paper-reported empirical result (*label introduced: front matter; first central use: Chapter 9*) — A measurement attributed to its source paper and experimental regime. It should not be restated as an unqualified fact or pooled with numbers from unmatched experiments.

Interpretation (*label introduced: front matter; used throughout*) — An explanatory view supported by the mechanism, such as read/write gating or diffusion over streams. It may guide reasoning without being uniquely established.

Hypothesis (*label introduced: front matter; first central use: Chapter 16*) — A testable but unestablished proposal, such as semantic stream specialization or a benefit from treating d as a scaling axis. It requires evidence before promotion to an empirical result.

Compact disambiguation checklist

When a symbol or claim feels ambiguous, ask:

1. **Which axis?** Batch B , stream d , token m , feature n , layer l , or auxiliary block s ?
2. **Which object?** Persistent state x_l , branch activation z_l/y_l , router H_l , coordinates u_l , constraint-map matrix A_l/Q_l , or layer history?
3. **Which stage?** Learned projection or deterministic constraint map?
4. **Which guarantee?** Valid output, exact arithmetic, floating-point tolerance, full set coverage, spectral property, or empirical training behavior?
5. **Which evidence label?** Definition/architecture fact, theorem, derivation, paper-reported empirical result, interpretation, or hypothesis?

If those five answers are explicit, residual streams, layer history, feature axes, routers, constraint maps, and empirical claims cannot quietly collapse into one another.

Global references

This bibliography resolves the 109 entries in the chapters’ **Essential references** sections to 38 unique works. Keys are global and stable: repeated citations of the same DOI, arXiv identifier, or work title share one key. The chapter map at the end preserves each chapter’s source order and provides an explicit key for every occurrence.

The five recent HC-family preprints are annotated as **Primary architecture source**. Their titles, author records, and current versions were checked against the arXiv records specified in `SCHEMA.md`; their architectural and empirical claims should be read at the cited version.

Foundations

- [R010] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30 (2017). arXiv:1706.03762v7.
- [R011] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3. Author-maintained book site.
- [R012] Julia documentation. “Broadcasting.” *The Julia Language Manual*. Stable documentation.
- [R013] Biao Zhang and Rico Sennrich. “Root Mean Square Layer Normalization.” *Advances in Neural Information Processing Systems* 32 (2019). arXiv:1910.07467v1.
- [R014] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.” 2017. arXiv:1701.06538v1.
- [R015] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.” *Journal of Machine Learning Research* 23(120), 1–39 (2022). arXiv:2101.03961v3.
- [R016] Harshil Vejjendla. “Wave-PDE Nets: Trainable Wave-Equation Layers as an Alternative to Attention.” 2025. arXiv:2510.04304v1.
- [R017] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.” 2023. arXiv:2312.00752v2.

Residual networks

- [R006] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition.” *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 770–778 (2016). doi:10.1109/CVPR.2016.90; arXiv:1512.03385v1.
- [R007] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Identity Mappings in Deep Residual Networks.” In *Computer Vision—ECCV 2016*, 630–645 (2016). doi:10.1007/978-3-319-46493-0_38; arXiv:1603.05027v3.
- [R008] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. “Learning Long-Term Dependencies with Gradient Descent Is Difficult.” *IEEE Transactions on Neural Networks* 5(2), 157–166 (1994). doi:10.1109/72.279181.
- [R009] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. “Densely Connected Convolutional Networks.” *2017 IEEE Conference on Computer Vision and Pattern Recognition* (2017). doi:10.1109/CVPR.2017.243; arXiv:1608.06993v5.

HC variants

- [R001] Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. “Hyper-Connections.” 2024; revised 2025. arXiv:2409.19606v3.
 - **Primary architecture source** — **HC**. Defines the static and dynamic Hyper-Connections designs and the expanded persistent-stream architecture. The arXiv record identifies v3 as the current version.
- [R002] Zhenda Xie et al. “mHC: Manifold-Constrained Hyper-Connections.” 2025; revised 2026. arXiv:2512.24880v2.
 - **Primary architecture source** — **original mHC**. Specifies the manifold-constrained HC design, its finite Sinkhorn-based residual routing, infrastructure work, and reported scaling experiments. The arXiv record identifies v2 as the current version.
- [R003] Yongyi Yang and Jianyang Gao. “mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations.” 2026. arXiv:2601.05732v1.
 - **Primary architecture source** — **mHC-lite**. Specifies exact convex mixtures of permutation matrices and reports the associated implementation and experimental comparisons. The cited record is v1.
- [R004] Wuyang Zhou, Yuxuan Gu, Giorgos Iacovides, and Danilo Mandic. “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices.” 2026. arXiv:2601.21579v2.
 - **Primary architecture source** — **KromHC**. Specifies Kronecker-factorized residual matrices and reports their parameter, expressivity, and empirical tradeoffs. The arXiv record identifies v2 as the current version.
- [R005] Torque Dandachi and Sophia Diggs-Galligan. “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices.” 2026. arXiv:2604.02309v1.
 - **Primary architecture source** — **go-mHC**. Specifies the skew-coordinate, Cayley-transform, and normalized block-Frobenius-norm construction and reports its expressivity, spectra, cost, and experiments. The cited record is v1.
- [R018] Charles F. Van Loan. “The Ubiquitous Kronecker Product.” *Journal of Computational and Applied Mathematics* 123, 85–100 (2000). doi:10.1016/S0377-0427(00)00393-9.

Stochastic matrices

- [R019] Garrett Birkhoff. “Tres observaciones sobre el algebra lineal.” *Universidad Nacional de Tucumán, Revista, Serie A* 5, 147–151 (1946). MR0020547. Catalog record.
- [R020] John von Neumann. “A Certain Zero-Sum Two-Person Game Equivalent to the Optimal Assignment Problem.” In *Contributions to the Theory of Games II*, 5–12 (1953). doi:10.1515/9781400881970-002.
- [R021] Richard Sinkhorn and Paul Knopp. “Concerning Nonnegative Matrices and Doubly Stochastic Matrices.” *Pacific Journal of Mathematics* 21(2), 343–348 (1967). doi:10.2140/pjm.1967.21.343.
- [R022] Philip A. Knight. “The Sinkhorn–Knopp Algorithm: Convergence and Applications.” *SIAM Journal on Matrix Analysis and Applications* 30(1), 261–275 (2008). doi:10.1137/060659624.
- [R023] Albert W. Marshall, Ingram Olkin, and Barry C. Arnold. *Inequalities: Theory of Majorization and Its Applications*. 2nd ed. Springer, 2011. doi:10.1007/978-0-387-68276-1.
- [R024] Gabriel Peyré and Marco Cuturi. “Computational Optimal Transport with Applications to Data Sciences.” *Foundations and Trends in Machine Learning* 11(5–6), 355–607 (2019). doi:10.1561/22000000073; arXiv:1803.00567v4.
- [R025] R. Tyrrell Rockafellar. *Convex Analysis*. Princeton University Press, 1970. ISBN 978-0-691-01586-6. Publisher record.

Geometry

- [R026] Kyle Helfrich, Devin Willmott, and Qiang Ye. “Orthogonal Recurrent Neural Networks with Scaled Cayley Transform.” 2018. arXiv:1707.09520v3.
- [R027] Mario Lezcano-Casado and David Martínez-Rubio. “Cheap Orthogonal Constraints in Neural Networks: A Simple Parametrization of the Orthogonal and Unitary Group.” *Proceedings of the 36th Interna-*

tional Conference on Machine Learning, PMLR 97, 3794–3803 (2019). PMLR record.

- [R028] Eugene Gutkin. “On a Multi-Dimensional Generalization of the Notions of Orthostochastic and Unistochastic Matrices.” *Journal of Geometry and Physics* 74, 28–35 (2013). doi:10.1016/j.geomphys.2013.06.009.
- [R029] Ion Nechita, Zikun Ouyang, and Anna Szczepanek. “Generalized Unistochastic Matrices.” 2023. arXiv:2310.03436v2.
- [R030] Yik-Hoi Au-Yeung and Yiu-Tung Poon. “ 3×3 Orthostochastic Matrices and the Convexity of Generalized Numerical Ranges.” *Linear Algebra and its Applications* 27, 69–79 (1979). doi:10.1016/0024-3795(79)90032-6.
- [R031] P.-A. Absil, Robert Mahony, and Rodolphe Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2008. ISBN 978-0-691-13298-3. Publisher record.

Spectra

- [R032] Lloyd N. Trefethen and Mark Embree. *Spectra and Pseudospectra: The Behavior of Nonnormal Matrices and Operators*. Princeton University Press, 2005. doi:10.1515/9780691213101.
- [R033] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. 2nd ed. Cambridge University Press, 2012. doi:10.1017/CBO9781139020411.
- [R034] Gian-Carlo Rota and W. Gilbert Strang. “A Note on the Joint Spectral Radius.” *Indagationes Mathematicae (Proceedings)* 22, 379–381 (1960). doi:10.1016/S1385-7258(60)50046-1.
- [R035] F. I. Karpelevich. “On the Characteristic Roots of Matrices with Nonnegative Elements.” *Izvestiya Akademii Nauk SSSR, Seriya Matematicheskaya* 15(4), 361–383 (1951). MR0043063. Math-Net record.
- [R036] Charles R. Johnson and Pietro Paparella. “A Matricial View of the Karpelevič Theorem.” *Linear Algebra and its Applications* 520, 1–15 (2017). doi:10.1016/j.laa.2017.01.009; arXiv:1611.06970v2.

Implementation

- [R037] Julia documentation. “LinearAlgebra: \ / ldiv!” *The Julia Language*. Stable API reference.
- [R038] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. 4th ed. Johns Hopkins University Press, 2013. ISBN 978-1-4214-0794-4. Publisher record.

Chapter occurrence map

Keys in each row correspond one-for-one, and in the same order, to that chapter’s numbered **Essential references** list. Repeated keys across chapters are intentional deduplication; no row contains an unmapped citation.

Chapter	Global keys in chapter order	Occurrences
1	[R010], [R012], [R001], [R006], [R011]	5
2	[R006], [R007], [R008], [R001]	4
3	[R006], [R001], [R002], [R005], [R009]	5
4	[R001], [R002], [R006], [R005], [R010]	5
5	[R001], [R002], [R006], [R005]	4
6	[R001], [R002], [R003], [R004], [R005], [R013]	6
7	[R001], [R002], [R032], [R033], [R034], [R005]	6
8	[R002], [R019], [R020], [R021], [R023]	5
9	[R002], [R021], [R022], [R024], [R001]	5

Chapter	Global keys in chapter order	Occurrences
10	[R003], [R019], [R020], [R025], [R002], [R021]	6
11	[R004], [R005], [R002], [R018], [R019]	5
12	[R005], [R026], [R027], [R028], [R029]	5
13	[R005], [R029], [R002], [R001]	4
14	[R005], [R029], [R030], [R028], [R019]	5
15	[R001], [R002], [R003], [R004], [R005], [R021], [R019]	7
16	[R001], [R002], [R005], [R014], [R015], [R010], [R016], [R017]	8
17	[R002], [R004], [R005], [R035], [R036], [R029]	6
18	[R005], [R029], [R030], [R002], [R031], [R027], [R026]	7
19	[R005], [R001], [R002], [R037], [R026], [R038]	6
20	[R001], [R002], [R003], [R004], [R005], [R029]	6
Total	38 unique works	109

Claim ledger

A scoreboard, not a second textbook. Each row says what kind of support a claim has. A paper’s architecture is a **Definition** of what that paper built. A measured accuracy bump stays an **Empirical result**. None of these notes promote a preprint number into a law of deep learning.

The six labels are exclusive: **Definition**, **Theorem**, **Derivation**, **Empirical result**, **Interpretation**, **Hypothesis**.

Definition

ID	Central claim	Exact scope and boundary	Source / chapter
DEF-01	A persistent residual stream is one current state slot $\mathbf{x}_l(i) \in \mathbb{R}^{(m \times n)}$ in $\mathbf{x}_l \in \mathbb{R}^{(d \times m \times n)}$.	It is not a token, feature partition, attention head, expert, branch copy, or output from an earlier layer. Copied initialization makes the slots numerically equal at $l=0$, not historically indexed.	[R001]; Chs. 3–4
DEF-02	The single-branch HC-family update is $\mathbf{x}_{l+1} = \mathbf{H}_l^{\text{res}} \mathbf{x}_l + (\mathbf{H}_l^{\text{post}})^T \mathbf{F}(\mathbf{H}_l^{\text{pre}} \mathbf{x}_l, \mathbf{W}_l)$.	$\mathbf{H}^{\text{pre}}: 1 \times d$ reads one $m \times n$ branch input, $\mathbf{F}$ is evaluated once in this equation, $\mathbf{H}^{\text{post}}: 1 \times d$ writes one result, and $\mathbf{H}^{\text{res}}: d \times d$ transports old state. Additional branches must be declared separately.	[R001], [R002], [R005]; Ch. 4
DEF-03	A learned router projection emits unconstrained coordinates; a constraint map converts them into an operator with specified structure.	Schematic form $\mathbf{u} = \hat{\mathbf{x}} \mathbf{W}^{\text{route}} + \mathbf{b}^{\text{route}}$. Projection weights learn values; Sinkhorn, permutation mixtures, Kronecker composition, or Cayley/block norms impose structure. “Dynamic” alone does not specify per-token, per-sequence, or shared routing.	[R001]–[R005]; Ch. 6
DEF-04	$\mathbf{B} \in \mathcal{B}_d$ means $B_{ij} \geq 0$, $B1=1$, and $B^T 1=1$; $\mathcal{B}_d$ is the Birkhoff polytope.	Double stochasticity does not imply orthogonality, symmetry, invertibility, absence of mixing, or stability of the complete nonlinear network.	[R019], [R020]; Ch. 8

ID	Central claim	Exact scope and boundary	Source / chapter
DEF-05	Original mHC constrains the residual router with finite Sinkhorn–Knopp normalization.	In the cited implementation, $d=4$, at most 20 iterations are used, routing is derived from the flattened per-position dc state, and Sinkhorn is applied to $\hat{H}_{\text{res}}$; pre/post gates use separate sigmoid maps. Finite iteration is approximate, not algebraically exact.	[R002], [R021]; Chs. 6, 9
DEF-06	mHC-lite constructs $\hat{H}_{\text{res}}$ as a softmax-weighted convex combination of permutation matrices.	Every produced mixture is doubly stochastic in exact arithmetic. Full Birkhoff coverage through explicit enumeration uses all $d!$ permutations; a restricted bank stays exact but loses coverage. Finite softmax logits lie in the relative interior of the enumerated hull.	[R003], [R019], [R020]; Ch. 10
DEF-07	KromHC constructs $\hat{H}_{\text{res}} = U^T K \otimes \dots \otimes U^1$ from smaller doubly stochastic factors, with each paper factor represented by a permutation mixture.	The cited KromHC changes residual routing while retaining mHC-style pre/post routing. Exactness applies to the product output; reachable matrices and spectra inherit the chosen factorization.	[R004], [R018]; Ch. 11
DEF-08	go-mHC sets $N=ds$, packs $N(N-1)/2$ coordinates into $A^T = -A$, forms a Cayley matrix Q , partitions it into $s \times s$ blocks, and defines $H_{ij} = Q_{ij} _F^2 / s$.	The construction makes a $d \times d$ residual router between already-existing streams. A and Q are auxiliary $N \times N$ matrices, not N streams and not branch activations.	[R005], [R026], [R029]; Chs. 12–13
DEF-09	Exact constraint satisfaction and full set coverage are different properties.	$\Phi_-(d, s)(Q) \in \mathcal{B}_d$ for every admissible Q proves validity of every output; it does not prove $\text{im } \Phi_-(d, s) = \mathcal{B}_d$, reach through one Cayley chart, favorable conditioning, or task performance.	[R005], [R029]; Ch. 14

Theorem

ID	Central claim	Assumptions and boundary	Source / chapter
THM-01	Every doubly stochastic matrix is a convex combination of permutation matrices, and the permutation matrices are exactly the vertices of $\mathcal{B}_d$.	Birkhoff-von Neumann theorem for finite square matrices. It is an existence theorem, not a scalable instruction to enumerate $d!$ vertices.	[R019], [R020]; Chs. 8, 10
THM-02	Products of doubly stochastic matrices remain doubly stochastic; for $B \in \mathcal{B}_d$, $\ B\ _1 = \ B\ _\infty = 1$ and $\ B\ _2 \leq 1$.	These are properties of the residual mixer or its product. They do not bound the branch injection, derivatives of an input-dependent router, normalization, or the complete HC update.	[R033]; Ch. 8
THM-03	Alternating row/column scaling of a strictly positive square matrix converges to a doubly stochastic diagonal scaling.	The conclusion is asymptotic as iteration count tends to infinity. Nonnegative matrices with zeros need additional support conditions; a chosen finite iteration count generally leaves residual error.	[R021]; Ch. 9
THM-04	If A and B are doubly stochastic, $A \otimes B$ is doubly stochastic; $\text{spec}(A \otimes B)$ consists of pairwise products of factor eigenvalues.	Extends to finitely many square factors. Exactness does not imply full Birkhoff coverage; spectral products impose factor-dependent restrictions.	[R018]; Ch. 11
THM-05	For real skew-symmetric A , $Q = (I - A)(I + A)^{-1}$ is orthogonal with determinant $+1$; conversely the ordinary Cayley chart covers the elements of $SO(N)$ without eigenvalue -1 .	$I + A$ is nonsingular in exact real arithmetic. The map does not cover the determinant -1 component or $SO(N)$ matrices with eigenvalue -1 ; implementations should use a solve rather than form an inverse.	[R026]; Ch. 12
THM-06	If orthogonal $Q \in \mathbb{R}^{(ds) \times (ds)}$ is partitioned into $s \times s$ blocks and $H_{ij} = \ Q_{ij}\ _F^2 / s$, then $H \in \mathcal{B}_d$.	This is the exact generalized-orthostochastic containment theorem. It proves nonnegativity and both unit-sum conditions only; it does not make H orthogonal or establish coverage.	[R005], [R029]; Ch. 13
THM-07	For $d \geq 3$, the ordinary orthostochastic image $S_{(d,1)}$ is a proper subset of $\mathcal{B}_d$; for $d=2$, $S_{(2,1)} = \mathcal{B}_2$.	This theorem is only the $s=1$ statement. Equality or strictness for an arbitrary named larger finite s is not established here; one Cayley chart can reach no more than the corresponding theoretical orthogonal image.	[R029], [R030]; Chs. 14, 18

ID	Central claim	Assumptions and boundary	Source / chapter
THM-08	For fixed d , the closure of the union of generalized block-norm images over all positive block sizes is $\mathcal{B}_d$.	This is an asymptotic density statement: it does not give full coverage at any named finite s , consecutive nesting in s , a useful error rate, or trainability.	[R029]; Ch. 14
THM-09	A fixed irreducible, aperiodic stochastic matrix has a unique stationary distribution and its iterates converge toward it; for a doubly stochastic matrix the stationary distribution is uniform.	Fixed-matrix Perron–Frobenius setting. Dynamic products require separate analysis, and permutation routers are periodic counterexamples to automatic mixing.	[R033]; Ch. 17
THM-10	A permutation cycle of length q has exactly the q th roots of unity as eigenvalues.	The result describes a specified cycle. It does not say a trained router will choose cycles or that cyclic spectra improve a task.	[R035], [R036]; Ch. 17

Derivation

ID	Central claim	Assumptions and boundary	Source / chapter
DER-01	Read and compute have shape $m \times n$; write and persist have shape $d \times m \times n$, so the final addition is shape-valid.	Unbatched canonical convention. With batching or dynamic routing, declared routing-instance axes precede the operator axes; all three routers still act only on the stream axis.	[R001]; Ch. 4
DER-02	Unrolling $x_{(l+1)} = R_{lx_l + p_{ly_l}}$ gives the initial state transported by every later R , plus each write transported only by routers after its insertion.	Exact algebra for the displayed recurrence. “Paths” are weighted contributions, not necessarily discrete selected wires.	[R001], [R002]; Chs. 5, 7
DER-03	Double stochasticity preserves the all-ones mode and stream sum and makes the residual-only transport non-expansive in the stream-vector Euclidean norm.	Applies pointwise in token/feature coordinates to H^{res} and products of valid routers. It does not prove that the full activation or gradient is bounded.	[R002]; Ch. 8
DER-04	A finite Sinkhorn computation generally repairs the last-normalized sums exactly in exact arithmetic while perturbing the other sums, leaving a measurable constraint residual.	Strictly positive input and the chapter’s alternating convention. Floating-point error adds another source of residual; tolerance is a certificate, not equality.	[R002], [R021]; Ch. 9

ID	Central claim	Assumptions and boundary	Source / chapter
DER-05	An $N \times N$ skew-symmetric matrix has $p = N(N-1)/2$ independent entries; for $d=4, s=2, N=8$ and $p=28$.	Counts coordinate outputs, not total projection parameters. An affine projection from width r also has roughly rp weights plus bias/gain under the selected design.	[R005]; Chs. 12, 19
DER-06	Orthogonality gives $\Sigma_j Q_{ij} _F^2 = s$ and $\Sigma_i Q_{ij} _F^2 = s$; division by s yields row and column sums one.	Requires correctly partitioned $s \times s$ blocks and exact orthogonality. A wrong reshape, missing $/s$, or nonorthogonal Q invalidates the conclusion.	[R005], [R029]; Ch. 13
DER-07	At $s=1$, $H_{ij} = Q_{ij}^2$, so the image is the orthostochastic subset; for $d \geq 3$ that subset does not fill $\mathcal{B}_d$.	Entrywise squares erase auxiliary signs but do not make arbitrary doubly stochastic matrices orthostochastic. The $d=2$ case is exceptional.	[R029], [R030]; Ch. 14
DER-08	One dense go-mHC Cayley solve costs $\mathcal{O}(N^3) = \mathcal{O}(d^3 s^3)$, while coordinate width is quadratic in N ; persistent activations scale as $d B m n$.	A complete cost must multiply by routing instances (shared, per example, or per token) and include projection width, branch cost, batching, memory traffic, precision, and kernels. $\mathcal{O}(d^3)$ assumes fixed s .	[R005]; Chs. 15, 19–20
DER-09	At $Q = I_-(ds)$, the differential of the block-norm map is zero along every orthogonal-group tangent direction; off-diagonal router entries begin quadratically in a small skew perturbation.	Concerns losses reaching coordinates only through H . Other coupled paths may contribute gradients. Exact-zero coordinate initialization can therefore stall this map even though near-identity initialization is useful.	[R005]; Chs. 18–19
DER-10	All- 2×2 KromHC factors have real factor spectra, so their Kronecker product cannot represent a router with a nonreal eigenvalue such as a three-cycle.	Scoped to the all- 2×2 factorization. A factor of size at least three can itself have complex eigenvalues; the general restriction is factor separability, not “all KromHC spectra are real.”	[R004], [R005], [R018]; Chs. 11, 17

Empirical result

Every row in this section is an **Empirical result** attributed to its source. None is claimed to be independently reproduced by these notes.

ID	Paper-reported result	Exact paper/platform scope and boundary	Source / chapter
EMP-01	Hyper-Connections reported improvements over residual baselines in language-model pretraining and vision experiments.	The HC paper includes dense and sparse language-model settings and vision tasks, and studies static HC and input-dependent DHC. The result is not a theorem that multiple streams help every architecture, scale, or dataset.	[R001]; Chs. 3, 16, 20
EMP-02	Original mHC reported stable loss/gradient profiles relative to its unconstrained HC comparison and a final training-loss reduction of 0.021 versus its baseline in the 27B run.	Paper used $d=4$ and at most 20 Sinkhorn iterations in reported 3B, 9B, and 27B experiments. In the 27B comparison, HC's loss surge occurred around step 12,000. This is one training system and comparison, not universal HC failure or mHC stability.	[R002]; Ch. 9
EMP-03	On eight reported downstream benchmarks for the 27B models, mHC exceeded the baseline on all eight and HC on seven; MATH was the exception (26.0 mHC, 26.4 HC).	Results inherit the paper's model, data, training budget, prompting/evaluation, and checkpoint choices. They are not controlled evidence against every HC configuration.	[R002]; Ch. 9
EMP-04	mHC Figure 7 reported maximum composite backward gain about 1.6, versus nearly 3000 for the compared HC analysis.	Same paper-specific $d=4$ analysis. Twenty Sinkhorn iterations still showed slight single-layer deviation; the plot supports reduced measured gain drift, not exact finite-Sinkhorn conservation or a proof of end-to-end stability.	[R002]; Ch. 9
EMP-05	The optimized mHC system reported 6.7% additional training time.	In-house large-scale $d=4$ setting using kernel fusion, custom recomputing backward, activation recomputation, and pipeline scheduling. It is whole-system overhead, not an isolated Sinkhorn cost and not a prediction for an unfused framework loop.	[R002]; Ch. 9

ID	Paper-reported result	Exact paper/platform scope and boundary	Source / chapter
EMP-06	mHC-lite reported final losses broadly comparable to mHC and smaller gradient-norm fluctuations than its mHC reimplementation.	d=4; 6-, 12-, and 24-layer language models on OpenWebText and FineWeb-Edu. It does not test factorial-scale d or establish universal throughput, stability, or quality superiority.	[R003]; Ch. 10
EMP-07	KromHC reported fewer added parameters than mHC and mHC-lite while obtaining similar training loss/validation BPB and higher CORE score in its compared configurations.	d=4 language models at six and twelve Transformer blocks: 240K vs 462K/609K added parameters, and 959K vs 1,844K/2,433K. These are two modest model scales and paper-selected comparisons.	[R004]; Ch. 11
EMP-08	KromHC reported improving validation BPB as d increased through 4, 8, 16, plus lower late-training gradient norms in its comparison.	Same paper’s language-model setup, factor choices, budgets, and optimizer. This does not establish a general stream-count scaling law or superiority of separable spectra.	[R004]; Ch. 11
EMP-09	go-mHC spectral-target experiments reached denser observed regions than the tested KromHC baseline, with s=2 denser than s=1.	Toy targets at d=3 for s=1, 2, and a d=4 comparison against the all-2×2-factor KromHC baseline. Sampled/optimized reach is not proof of full finite-s coverage or task utility.	[R005]; Chs. 14, 17
EMP-10	go-mHC reported convergence up to 10× faster on selected synthetic stream-mixing tasks.	Synthetic objectives and paper-selected configurations. “Up to” is not an average, and the result does not isolate a universal optimization mechanism or predict large-model training speed.	[R005]; Chs. 18, 20
EMP-11	go-mHC reported similar convergent behavior/parity among constrained variants in a roughly 30M-parameter, six-layer GPT-style TinyStories study.	Small-language-model validation in the v1 preprint; the authors explicitly limit broader conclusions because of scale and synthetic emphasis. It does not establish superiority, large-scale stability, or a scaling law in d.	[R005]; Chs. 17–18, 20

Interpretation

ID	Central interpretation	Boundary	Source / chapter
INT-01	H^{pre} , H^{post} , and H^{res} are usefully read as soft read, soft write, and persistent-state routing.	This is a dataflow lens, not a claim that gates are probabilities, discrete switches, MoE gates, or uniquely causal explanations.	[R001], [R002]; Chs. 4, 16
INT-02	HC widens persistent state while the displayed single-branch update can still evaluate F once.	State capacity and branch compute are distinct. A different multi-branch architecture may evaluate multiple F_r , but that cost is not implied by stream count alone.	[R001]; Chs. 3–4
INT-03	Double stochasticity is conservative redistribution over streams and a controlled residual-only highway.	It preserves mass/common mode and rules out gain above one for the mixer’s spectral norm, but may average away distinctions and does not stabilize every component of training.	[R002]; Chs. 8, 17
INT-04	HC-family variants share a dataflow skeleton but trade off constraint approximation, coordinate burden, separability, finite- s image, kernel needs, and cost.	No unqualified “better” ordering follows. A ranking requires named d, s , routing granularity, hardware, model, budget, metric, and implementation.	[R001]–[R005]; Ch. 15
INT-05	Spectral reach describes possible persistent-state dynamics such as mixing, cycles, and rotations.	Richer sampled spectra show parameterization reach, not semantic specialization, optimizer access, or higher downstream accuracy. Dynamic nonnormal products also exceed single-router eigenvalue analysis.	[R004], [R005], [R032], [R035], [R036]; Chs. 7, 17
INT-06	$s=2$ is a useful tested go-mHC compromise because real 2×2 blocks can encode complex phases while keeping $N=2d$.	It is not a theorem of optimality, surjectivity, best conditioning, or best hardware efficiency.	[R005], [R029]; Ch. 14
INT-07	Stream count d is a genuine architectural degree of freedom alongside, but distinct from, depth, width, context length, and expert count.	Calling d a design axis is not a scaling law. Its activation memory and router cost must be charged explicitly.	[R001], [R005]; Ch. 20

Hypothesis

ID	Testable hypothesis	Evidence required / present status	Source / chapter
HYP-01	Persistent streams may acquire reproducible semantic or functional specialization.	Requires causal interventions, swaps/ablations, probes across inputs, layers, checkpoints, seeds, and permutation-aligned controls. Unequal router weights or a heatmap are insufficient. Not established by the five architecture papers.	Chs. 5, 16, 20
HYP-02	Combining constrained persistent-state routing with heterogeneous attention, SSM, WavePDE/convolution, MLP, MoE, or memory branches may create useful learned dataflow graphs.	Must compare matched compute/parameters and distinguish state routing from expert/branch routing. The multi-branch equation in the notes is an extrapolation, not the evaluated go-mHC block.	Chs. 16, 20
HYP-03	Wider spectral reach and access to complex, nonseparable stream dynamics improve large, deep, or heterogeneous models.	Needs matched training where only reachable routing dynamics differ, plus router/spectrum measurements and downstream outcomes. Current support is synthetic and sampled parameterization evidence from [R005].	[R005]; Chs. 17, 20
HYP-04	Increasing d can improve a capability or efficiency frontier and become a model scaling dimension.	Requires sweeps over d , multiple seeds, optimizer retuning, and comparisons at matched parameters, FLOPs, wall time, and activation memory against spending the budget on depth, width, context, or experts. No scaling law is established.	[R004], [R005]; Ch. 20
HYP-05	Cayley coordinates may optimize more favorably than permutation-simplex or finite-Sinkhorn coordinates.	Fibers, chart compression, the zero differential at identity, softmax saturation, target router, initialization, d , s , and optimizer can favor either map. Synthetic go-mHC convergence is evidence only for tested settings, not the mechanism.	[R003], [R005], [R026], [R029]; Ch. 18

ID	Testable hypothesis	Evidence required / present status	Source / chapter
HYP-06	Large nonnormal or changing-router transient gains make training more sensitive to precision, initialization, and input-dependent routing.	Matrix products make the mechanism plausible, but validation requires direct transient/gain measurement and controlled interventions in named HC implementations. It is not a theorem that unconstrained HC always fails.	[R002], [R032], [R033]; Ch. 7

Provisional full source metadata for key coordination

These records pin down the works meant by the keys used above. The final grouped bibliography remains authoritative for house style, venue status, and later arXiv revisions. Architecture-paper metadata was checked against the arXiv records named in the schema; version suffixes preserve the versions used by the chapters.

- [R001] Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. “Hyper-Connections.” arXiv:2409.19606v3, initial submission 2024, cited revision 2025; ICLR 2025. <https://arxiv.org/abs/2409.19606>
- [R002] Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, Liang Zhao, Shangyan Zhou, Zhean Xu, Zhengyan Zhang, Wangding Zeng, Shengding Hu, Yuqing Wang, Jingyang Yuan, Lean Wang, and Wenfeng Liang. “mHC: Manifold-Constrained Hyper-Connections.” arXiv:2512.24880v2, initial submission 2025, cited revision 2026. <https://arxiv.org/abs/2512.24880>
- [R003] Yongyi Yang and Jianyang Gao. “mHC-lite: You Don’t Need 20 Sinkhorn-Knopp Iterations.” arXiv:2601.05732v1, 2026. <https://arxiv.org/abs/2601.05732v1>
- [R004] Wuyang Zhou, Yuxuan Gu, Giorgos Iacovides, and Danilo Mandic. “KromHC: Manifold-Constrained Hyper-Connections with Kronecker-Product Residual Matrices.” arXiv:2601.21579v2, 2026. <https://arxiv.org/abs/2601.21579>
- [R005] Torque Dandachi and Sophia Diggs-Galligan. “go-mHC: Direct Parameterization of Manifold-Constrained Hyper-Connections via Generalized Orthostochastic Matrices.” arXiv:2604.02309v1, 2026. <https://arxiv.org/abs/2604.02309v1>
- [R006] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition.” *Proceedings of CVPR*, 2016, pp. 770–778. arXiv:1512.03385. DOI: 10.1109/CVPR.2016.90. <https://doi.org/10.1109/CVPR.2016.90>
- [R018] Charles F. Van Loan. “The Ubiquitous Kronecker Product.” *Journal of Computational and Applied Mathematics* 123, 2000, pp. 85–100. DOI: 10.1016/S0377-0427(00)00393-9. [https://doi.org/10.1016/S0377-0427\(00\)00393-9](https://doi.org/10.1016/S0377-0427(00)00393-9)
- [R019] Garrett Birkhoff. “Tres observaciones sobre el álgebra lineal.” *Universidad Nacional de Tucumán, Revista, Serie A* 5, 1946, pp. 147–151. MathSciNet MR0020547. <https://cir.nii.ac.jp/crid/1570572699525842816?lang=en>
- [R020] John von Neumann. “A Certain Zero-Sum Two-Person Game Equivalent to the Optimal Assignment Problem.” In *Contributions to the Theory of Games II*, Princeton University Press, 1953, pp. 5–12. DOI: 10.1515/9781400881970-002. <https://doi.org/10.1515/9781400881970-002>
- [R021] Richard Sinkhorn and Paul Knopp. “Concerning Nonnegative Matrices and Doubly Stochastic Matrices.” *Pacific Journal of Mathematics* 21(2), 1967, pp. 343–348. DOI: 10.2140/pjm.1967.21.343. <https://doi.org/10.2140/pjm.1967.21.343>
- [R026] Kyle Helfrich, Devin Willmott, and Qiang Ye. “Orthogonal Recurrent Neural Networks with Scaled Cayley Transform.” arXiv:1707.09520, 2017; conference version 2018. <https://arxiv.org/abs/1707.09520>

- [R028] Eugene Gutkin. “On a Multi-Dimensional Generalization of the Notions of Orthostochastic and Unistochastic Matrices.” *Journal of Geometry and Physics* 74, 2013, pp. 28–35. DOI: 10.1016/j.geomphys.2013.06.009. <https://doi.org/10.1016/j.geomphys.2013.06.009>
- [R029] Ion Nechita, Zikun Ouyang, and Anna Szczepanek. “Generalized Unistochastic Matrices.” arXiv:2310.03436v2, 2023. <https://arxiv.org/abs/2310.03436>
- [R032] Lloyd N. Trefethen and Mark Embree. *Spectra and Pseudospectra: The Behavior of Nonnormal Matrices and Operators*. Princeton University Press, 2005. DOI: 10.1515/9780691213101. <https://doi.org/10.1515/9780691213101>
- [R033] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*, 2nd ed. Cambridge University Press, 2012. DOI: 10.1017/CBO9781139020411. <https://doi.org/10.1017/CBO9781139020411>
- [R035] F. I. Karpelevich. “On the Characteristic Roots of Matrices with Nonnegative Elements.” *Izvestiya Akademii Nauk SSSR, Seriya Matematicheskaya* 15(4), 1951, pp. 361–383. <https://m.mathnet.ru/eng/im3317>
- [R036] Charles R. Johnson and Pietro Paparella. “A Matricial View of the Karpelevič Theorem.” *Linear Algebra and its Applications* 520, 2017, pp. 1–15. arXiv:1611.06970. <https://arxiv.org/abs/1611.06970>

Cumulative exercise index

Every exercise in Chapters 1–20, once. Difficulty is local to the chapter: **1** is a warmup, **5** is a stretch. Answers stay in the chapter’s collapsible block.

Chapter 1 — Tokens, tensors, and ordinary layers

Exercise pointer	Difficulty	Concepts
Chapter 1 § “Exercise 1 — Read individual entries”	1	tensor indexing and shapes
Chapter 1 § “Exercise 2 — Change the output width”	2	affine-layer output shapes and parameter count
Chapter 1 § “Exercise 3 — Compute one new layer”	3	matrix multiplication, bias, and activation
Chapter 1 § “Exercise 4 — Diagnose an axis bug”	4	token/feature axis order
Chapter 1 § “Exercise 5 — Separate batch, token, and stream axes”	5	batch, token, stream, and feature axes

Chapter 2 — Residual connections and gradient highways

Exercise pointer	Difficulty	Concepts
Chapter 2 § “Exercise 1 — One forward update”	1	residual forward update
Chapter 2 § “Exercise 2 — Separate the two gradient routes”	2	identity and branch gradient routes
Chapter 2 § “Exercise 3 — Construct exact cancellation”	3	identity cancellation and Jacobian
Chapter 2 § “Exercise 4 — Depth changes the verdict”	4	depth-wise scalar residual dynamics
Chapter 2 § “Exercise 5 — A conditional bound”	5	end-to-end Jacobian bounds

Chapter 3 — Hyper-Connections and persistent streams

Exercise pointer	Difficulty	Concepts
Chapter 3 § “Exercise 1 — Shape ledger”	1	HC tensor shapes
Chapter 3 § “Exercise 2 — Copy, do not partition”	2	copied-stream initialization versus feature partition
Chapter 3 § “Exercise 3 — One evaluation, four writes”	3	one branch evaluation with multiple writes
Chapter 3 § “Exercise 4 — Refute the history claim”	4	persistent streams versus layer history
Chapter 3 § “Exercise 5 — Separate the three counts”	5	stream, feature-group, and branch counts

Chapter 4 — Read, compute, write, and persist

Exercise pointer	Difficulty	Concepts
Chapter 4 § “Exercise 1 — Read by hand”	1	read routing
Chapter 4 § “Exercise 2 — Write a branch result”	2	write broadcasting
Chapter 4 § “Exercise 3 — Route persistent state”	3	persistent-state routing
Chapter 4 § “Exercise 4 — Execute a complete scalar layer”	4	complete read–compute–write–persist update
Chapter 4 § “Exercise 5 — Diagnose the wrong program”	5	single-branch semantics and axis-safe code

Chapter 5 — Four layers by hand

Exercise pointer	Difficulty	Concepts
Chapter 5 § “1. Recompute one read”	1	read mixtures after divergence
Chapter 5 § “2. Verify stream divergence”	2	unequal writes and stream divergence
Chapter 5 § “3. Route a single contribution”	3	transport of one write contribution
Chapter 5 § “4. Repair a missing term”	4	path-expansion indexing
Chapter 5 § “5. Generalize the path formula”	5	general depth-path formula

Chapter 6 — Learned Dynamic Routers

Exercise pointer	Difficulty	Concepts
Chapter 6 § “Exercise 1 — Shape the projection”	1	router projection shapes and granularity

Exercise pointer	Difficulty	Concepts
Chapter 6 § “Exercise 2 — Separate proposal from validity”	2	projection coordinates versus constraint validity
Chapter 6 § “Exercise 3 — Diagnose a dead start”	3	router initialization and gradient flow
Chapter 6 § “Exercise 4 — Audit granularity”	4	routing-granularity audit
Chapter 6 § “Exercise 5 — Count go-mHC coordinates”	5	go-mHC coordinate count

Chapter 7 — Why unconstrained residual mixing can fail

Exercise pointer	Difficulty	Concepts
Chapter 7 § “Exercise 1 — Scalar depth”	1	scalar repeated mixing
Chapter 7 § “Exercise 2 — Diagonal directions”	2	eigendirections: persistence, alternation, decay
Chapter 7 § “Exercise 3 — Rotation versus scaled rotation”	3	rotation and spectral magnitude
Chapter 7 § “Exercise 4 — A nonnormal transient”	4	nonnormal transient growth
Chapter 7 § “Exercise 5 — Stable-looking layers, unstable pair”	5	products of changing routers

Chapter 8 — Doubly stochastic matrices and the Birkhoff polytope

Exercise pointer	Difficulty	Concepts
Chapter 8 § “Exercise 1 — Constraint check”	1	row/column/double stochasticity
Chapter 8 § “Exercise 2 — Route a tensor slice”	2	stream routing, conservation, convex range
Chapter 8 § “Exercise 3 — Prove products stay feasible”	3	closure under products
Chapter 8 § “Exercise 4 — A contraction counterexample”	3	double stochasticity versus orthogonality
Chapter 8 § “Exercise 5 — Geometry of $\mathcal{B}_2$ ”	4	geometry of the 2×2 Birkhoff polytope
Chapter 8 § “Exercise 6 — Full-update reasoning”	5	residual-path bounds versus the full HC update

Chapter 9 — Original mHC and finite Sinkhorn normalization

Exercise pointer	Difficulty	Concepts
Chapter 9 § “Exercise 1 — One repair damages another”	1	alternating row/column normalization

Exercise pointer	Difficulty	Concepts
Chapter 9 § “Exercise 2 — Compute a constraint certificate”	2	finite-Sinkhorn constraint certificate
Chapter 9 § “Exercise 3 — Decide whether to stop”	3	tolerance and stopping criteria
Chapter 9 § “Exercise 4 — Trace the shapes and cost”	4	routing granularity and Sinkhorn cost
Chapter 9 § “Exercise 5 — Differentiate one normalization”	5	derivatives and numerical conditioning

Chapter 10 — mHC-lite and exact permutation mixtures

Exercise pointer	Difficulty	Concepts
Chapter 10 § “Exercise 1 — Read a permutation”	1	permutation action and constraints
Chapter 10 § “Exercise 2 — Decompose a 2×2 router”	2	Birkhoff decomposition in 2×2
Chapter 10 § “Exercise 3 — Prove subset exactness”	3	exactness of restricted permutation mixtures
Chapter 10 § “Exercise 4 — Count the burden”	3	factorial parameter and storage burden
Chapter 10 § “Exercise 5 — Locate the lost coverage”	4	coverage loss from a restricted bank
Chapter 10 § “Exercise 6 — Boundary versus softmax”	5	softmax interior versus polytope boundary

Chapter 11 — KromHC and Kronecker structure

Exercise pointer	Difficulty	Concepts
Chapter 11 § “1. Expand and check”	1	Kronecker expansion and stochasticity
Chapter 11 § “2. Find the spectrum”	2	Kronecker spectrum
Chapter 11 § “3. Test a cycle”	3	cycle restrictions of product permutations
Chapter 11 § “4. Count geometric freedom”	4	dimension and coverage restriction
Chapter 11 § “5. Choose factors under a parameter budget”	5	factor choice under a parameter budget

Chapter 12 — go-mHC: skew coordinates to exact routing

Exercise pointer	Difficulty	Concepts
Chapter 12 § “Exercise 1 — Count before constructing”	1	auxiliary dimension and skew-coordinate count

Exercise pointer	Difficulty	Concepts
Chapter 12 § “Exercise 2 — Pack a smaller skew matrix”	2	packing skew-symmetric coordinates
Chapter 12 § “Exercise 3 — Verify the two-dimensional transform”	3	Cayley orthogonality and determinant
Chapter 12 § “Exercise 4 — Diagnose a false coverage claim”	4	Cayley chart coverage caveat
Chapter 12 § “Exercise 5 — Separate auxiliary signs from routing”	5	auxiliary determinant signs versus block-norm routing

Chapter 13 — Block Norms and Exact Double Stochasticity

Exercise pointer	Difficulty	Concepts
Chapter 13 § “Exercise 1 — Expand a block”	1	Frobenius block norm
Chapter 13 § “Exercise 2 — Find the missing scale”	2	normalization by block size
Chapter 13 § “Exercise 3 — Verify both directions numerically”	3	block-row and block-column orthogonality
Chapter 13 § “Exercise 4 — Diagnose a nonorthogonal input”	3	failure without orthogonality
Chapter 13 § “Exercise 5 — Prove the rectangular-block trace identity”	4	trace/Frobenius identity
Chapter 13 § “Exercise 6 — State the strongest valid conclusion”	5	strongest justified exactness and coverage claim

Chapter 14 — What the block size s buys

Exercise pointer	Difficulty	Concepts
Chapter 14 § “Exercise 1 — Two-stream coverage”	1	orthostochastic coverage for two streams
Chapter 14 § “Exercise 2 — Why the 3×3 barycenter fails at $s = 1$ ”	2	finite- s orthostochastic noncoverage
Chapter 14 § “Exercise 3 — Verify the $s = 2$ construction”	3	explicit $s=2$ block construction
Chapter 14 § “Exercise 4 — Build a rational permutation mixture”	4	rational permutation mixtures at finite block size
Chapter 14 § “Exercise 5 — Separate three claims”	5	constraint, empirical, and unsupported claims

Chapter 15 — Comparing HC, mHC, mHC-lite, KromHC, and go-mHC

Exercise pointer	Difficulty	Concepts
Chapter 15 § “Exercise 1 — Verify the shared parameter ledger”	1	cross-variant parameter counts
Chapter 15 § “Exercise 2 — Separate exactness from coverage”	2	exactness versus Birkhoff coverage
Chapter 15 § “Exercise 3 — Find a spectral exclusion”	3	KromHC spectral exclusion
Chapter 15 § “Exercise 4 — Locate a crossover without inventing one”	4	asymptotics versus measured crossover
Chapter 15 § “Exercise 5 — Choose a variant under explicit requirements”	5	variant selection under explicit constraints

Chapter 16 — Routing, gating, and learned dataflow

Exercise pointer	Difficulty	Concepts
Chapter 16 § “Exercise 1 — Separate the three state routes”	1	read/write/persistent state routes
Chapter 16 § “Exercise 2 — Classify routers”	2	state routing versus expert routing
Chapter 16 § “Exercise 3 — Audit shapes”	3	heterogeneous-branch tensor shapes
Chapter 16 § “Exercise 4 — Find the false compute saving”	4	conditional-compute semantics
Chapter 16 § “Exercise 5 — Design a specialization test”	5	causal tests of stream specialization

Chapter 17 — Spectral dynamics over streams

Exercise pointer	Difficulty	Concepts
Chapter 17 § “Exercise 1 — Complete the cycle”	1	permutation-cycle trajectory
Chapter 17 § “Exercise 2 — Predict mixing without multiplying”	2	closed-form mixing dynamics
Chapter 17 § “Exercise 3 — Read a damped cycle”	3	damped-cycle eigenvalues
Chapter 17 § “Exercise 4 — Audit a Kronecker spectrum”	4	Kronecker spectral restrictions
Chapter 17 § “Exercise 5 — Critique a reach claim”	5	scope of sampled spectral-reach evidence

Chapter 18 — Geometry and optimization

Exercise pointer	Difficulty	Concepts
Chapter 18 § “Exercise 1 — Tangent directions”	1	tangent space of the orthogonal group
Chapter 18 § “Exercise 2 — Dimension ledger”	2	domain/target dimension counting
Chapter 18 § “Exercise 3 — A gauge symmetry”	3	gauge symmetry of the block-norm map
Chapter 18 § “Exercise 4 — A finite-level missing point”	4	finite-block-size noncoverage
Chapter 18 § “Exercise 5 — The identity’s zero differential”	5	zero differential at identity

Chapter 19 — Complexity and implementation

Exercise pointer	Difficulty	Concepts
Chapter 19 § “Exercise 1 — Count the coordinates”	1	coordinate and projection parameter count
Chapter 19 § “Exercise 2 — Find the missing scale”	2	block-size normalization bug
Chapter 19 § “Exercise 3 — Attach multiplicity to complexity”	3	solve multiplicity and routing granularity
Chapter 19 § “Exercise 4 — Diagnose a failed invariant”	4	diagnosing failed stochastic invariants
Chapter 19 § “Exercise 5 — Repair initialization”	5	near-identity initialization and zero gradients

Chapter 20 — Misconceptions, Architecture Implications, and Synthesis

Exercise pointer	Difficulty	Concepts
Chapter 20 § “Exercise 1 — Identify the axes”	1	global axis and shape ledger
Chapter 20 § “Exercise 2 — Exactness is not orthogonality”	2	double stochasticity versus orthogonality
Chapter 20 § “Exercise 3 — Count coordinates”	3	go-mHC coordinate count and its limits
Chapter 20 § “Exercise 4 — Label the claims”	4	claim-label classification
Chapter 20 § “Exercise 5 — Design a falsifiable scaling test”	5	controlled, falsifiable stream-count scaling test

